Università degli studi di Perugia
Dipartimento di Ingegneria

Machine Learning and Data Analysis Notes

Gabriele Costante

Contents

1 Notation **8**

2 Introduction to Supervised Learning **12**
2.1 General Concepts . . . 12
2.2 Classification and Regression . . . 14
2.3 First example: Curve Fitting . . . 15
2.4 Overfitting . . . 18
2.5 Model Selection . . . 21

3 Probability Theory **25**
3.1 General Aspects . . . 25
3.2 The Bayes Theorem . . . 27
3.3 Example: the Monty Hall Problem . . . 29
3.4 Independency . . . 30
3.5 Continuous random variables . . . 31
3.5.1 Change of variable . . . 32
3.5.2 Fundamental rules for continuous random variables . . . 33
3.5.3 Expectation and Variance . . . 34
3.6 Curve Fitting re-visited . . . 34
3.6.1 Maximum Likelihood Estimation (ML) . . . 35
3.6.2 MLE for Curve Fitting . . . 41
3.6.3 Maximum a Posteriori (MAP) for Curve Fitting . . . 43

4 Linear Regression **45**
4.1 Relation to linear interpolation . . . 46
4.2 Basis Functions . . . 46
4.3 MLE for Linear Regression . . . 46
4.4 Hat Matrix . . . 50
4.5 Geometric interpretation of Linear Regression . . . 50
4.6 Gradient Descent . . . 50
4.6.1 Vectorization of Gradient Descent . . . 55
4.7 Feature Normalization . . . 55
4.8 Standard Error of LR optimal parameters . . . 58
4.9 Performance metrics for LR . . . 60
4.10 Data non-idealities . . . 63
4.10.1 Heteroscedasticity . . . 63
4.10.2 Influential Point, High-leverage points and Outliers . . . 64
4.11 Collinearity and Multi-collinearity . . . 70

5 Classification Algorithms 72
5.1 Loss function for classification . . . 72
5.1.1 Error Rate Lower Bound: Bayesian Error Rate . . . 73
5.2 K-NN Classifier . . . 74
5.3 Logistic Regression . . . 75
5.3.1 Maximum Likelihood Estimation for Logistic Regression . . . 77
5.3.2 Decision Function . . . 80
5.4 Classification Approaches . . . 81
5.5 Discriminant Function . . . 82
5.5.1 Perceptron Algorithm . . . 85
5.6 Quick Detour: Decision Boundaries for Multi-class Problems . . . 86
5.6.1 One vs One classifier . . . 86
5.6.2 One vs All Classifier . . . 86
5.7 Quick Detour: Classification Metrics . . . 87
5.7.1 Accuracy . . . 88
5.7.2 Precision and Recall . . . 89
5.7.3 F1-Score . . . 90
5.7.4 Confusion Matrix . . . 91
5.7.5 Receiver Operator Characteristic Curve - ROC Curve . . . 92

6 Generalized Linear Models (GLMs)* 94
6.1 Exponential Family formulation . . . 94
6.1.1 Bernoulli Distribution as special case of the exponential distribution 95
6.1.2 Gaussian Distribution as special case of the exponential distribution 96
6.2 Building GLM for Linear, Logistic and Softmax Regression . . . 96
6.2.1 Sufficient Statistic . . . 97
6.2.2 GLM assumption . . . 98
6.2.3 Softmax Regression . . . 99

7 Generative Models for Classification 103
7.1 Gaussian Class Conditional Densities . . . 104
7.1.1 Linear Discriminant Analysis (LDA) . . . 104
7.1.2 Quadratic Discriminant Analysis (QDA) . . . 108
7.1.3 MLE per LDA . . . 108
7.2 Class-Conditional Distributions for Discrete Features . . . 112
7.2.1 Naive Bayes for Multinomial Features* . . . 115
7.2.2 Laplace Smoothing . . . 119
7.2.3 Bag of Words (BoW) . . . 121

8 Lagrange Multiplier Method 124

9 Model Selection and Model Assessment 131
9.1 Bias-Variance Decomposition . . . 134
9.1.1 Example: K-NN Regressor . . . 136
9.1.2 Example: Linear Regression . . . 137
9.2 Approaches for Model Selection and Model Assessment . . . 138
9.2.1 Model Selection . . . 138
9.2.2 Model Assessment . . . 143

10 Ridge and Lasso Regression **145**
10.1 Ridge Regression . . . 145
10.1.1 Effective Number of Parameters . . . 150
10.1.2 Bias and Variance of the Ridge Regression model . . . 153
10.2 Lasso Regression . . . 154

11 Support Vector Machine (SVM) **156**
11.1 Functional and Geometric Margin . . . 157
11.2 Hard Margin Classifier . . . 160
11.2.1 Convex optimization notes . . . 160
11.2.2 Hard-margin classifier as a convex optimization problem . . . 165
11.2.3 Lagrange Duality . . . 167
11.3 Kernels . . . 173
11.4 Support Vector Machine (SVM) . . . 177

12 Neural Networks **181**
12.1 Loss Function and Stochastic Gradient Descent (SGD) . . . 182
12.2 Multi layer Neural Networks . . . 183
12.2.1 Two Layer MLP . . . 186
12.3 Multi Layer Perceptron . . . 188
12.4 Backpropagation . . . 190
12.4.1 Chain Rule . . . 191
12.4.2 Backpropagation of a NN with one neuron . . . 193
12.4.3 Backpropagation of a NN with two layers . . . 193
12.5 Backpropagation for MLP . . . 196
12.6 Loss functions for NN and Backpropagation . . . 197
12.6.1 Binary Classification Loss . . . 198
12.6.2 Softmax Classification Loss . . . 198
12.7 Model Selection and Regularization for NN . . . 200
12.7.1 L2 Regularization or Weight Decay . . . 200
12.7.2 Early Stopping . . . 201

13 Unsupervised Learning **202**
13.1 K-Means . . . 203
13.1.1 K-Medoids . . . 204
13.2 Gaussian Mixture Models (GMMs) . . . 206
13.3 Expectation Maximization (EM-Algorithm)* . . . 212
13.3.1 Jensen Inequality . . . 212
13.3.2 Kullback-Leibler Divergence (KL Divergence) . . . 212
13.3.3 EM Algorithm . . . 213

14 PCA and ICA **217**
14.1 Principal Component Analysis (PCA) . . . 217
14.2 Independent Component Analysis (ICA) . . . 220
14.2.1 Ambiguities . . . 221

List of Figures

1.1 Type of variables 9

2.1 28×28 image from the MNIST dataset depicting a handwritten digit. . . 14
2.2 What is the difference between the "3" drawn on the right and on the left? The digit is the same, but, in the right image, the pixels containing the 3 stroke are shifted to the left. A model-based approach should contain rules for all possible positions of the 3 in the image. 15
2.3 The graph represents, with the solid line the unknown true process $y = sin(2\pi x)$, while the dots indicate the available "noisy" samples. 16
2.4 Effect of hyper-parameter M on the hypothesis function. Increasing M makes the hypothesis function more flexible and better tracks the points in the dataset. 18
2.5 Training and Test Error for different values of M. 20
2.6 Training and Test error as lambda varies (in logarithmic scale). If we set $M = 9$ (high flexibility), if $log\lambda \ll 0$ we are not constraining the parameters and therefore we are not performing regularization: in this case the training error goes to zero but we experience overfitting. As $\log \lambda$ increases, training and test error get closer. 21
2.7 Example diagram of the cross-validation procedure. 24

3.1 Table depicting the joint distribution of X and Y. 25
3.2 The two boxes (red on the right and blue on the left) contain apples (in light blue) and oranges (in pink) 27
3.3 Monty Hall's problem. Behind two of the three doors are goats, and behind the third is a prize. 29
3.4 Change of variable). 33
3.5 The Gaussian distribution 37
3.6 The Gaussian predictive distribution 43

4.1 Geometric intepretation of the Linear Regression algorithm 51
4.2 Global minumim of $L(w)$, a loss function that depends on a single scalar parameter w. Note that this function does not have a unique minimum (it is not quadratic as in the LR case). 52
4.3 Initial value for the parameter w and derivative of the loss function computed at that point. 53
4.4 Iterative gradiente descent steps to reach w^*. 53
4.5 Effect of the hyper-parameter α on the gradient descent procedure. 54
4.6 Contour lines for features LA and NB. 56

4.7 Example of heteroscedasticity in a monodimensional case. In this example, the variance of the noise increases as x grows. 63
4.8 Examples of outliers and HLPs. The line in red represents the regression line computed including the potentially influent point. In green, instead, we depict the regression line computed without the red point. 64
4.9 Example of an HLP in the 2-dimensional case. 66
4.10 Loss function countours in the presence and in absence of collinearity. . . 70

5.1 Prediction line with the model $\mathbf{w}^T\mathbf{x}$. With the cross marks we highlight the class label, that are either 0 or 1. 75
5.2 Logistic function 76
5.3 Decision Boundary on a two-dimensional example. 83
5.4 Distance from the decision boundary. 84
5.5 One vs All uncertainty zone. 87
5.6 One vs One uncertainty zone. 88
5.7 Precision-recall curve. 89
5.8 Confusion matrix. 91
5.9 Confusion matrix for the multiclass case. In the cells with red dots the number of samples correctly classified for each class are reported. The blue dots indicate the cells containing the number of samples "confused" with other classes. 92
5.10 ROC Curve. 93

7.1 Decision boundary and class-conditional Gaussian distributions for the binary case. The plot depicts and example with two features. 106
7.2 Role of the sigmoid function for the example depicted in Figure 7.1. . . . 107
7.3 Creation of a BoW vocabulary by considering the most frequent words in a training set of documents.. 122

8.1 The gradient ∇h and the tangent $\boldsymbol{\epsilon}$ to the curve are orthogonal. 125
8.2 The gradient ∇f and the tangent $\boldsymbol{\epsilon}$ to the curve are not orthogonal when the motion has a component along the gradient of the function f. 126
8.3 The gradient ∇f and the gradient ∇h are parallel in an extrema point. . 127
8.4 In the case of a minimization problem, the optimal solution is the point that lies in the $h(x_1, x_2) = 0$ curve for which the gradient ∇h is orthogononal to the gradient ∇f. 127
8.5 Lagrange multiplier method with inequality constraints. 129

9.1 Training and test error curves varying the model complexity (degree of freedom). 133
9.2 Training, Validation and Test splits. 134
9.3 Configurations of DoF that result in underfitting, overfitting and Best Model. 139
9.4 Fixed Validation set, Cross-Validation and Leave-one-out. 140
9.5 Training and test error by varying the number of training samples 143

11.1 Hypothesis function for Logistic Regression 156
11.2 Relation between the confidence of the predictions and the distance from the decision hyper-plane (boundary). 157
11.3 There are many possible hyper-planes that separate the two classes. . . . 158

11.4 On the left, an example of a convex set. On the right a non-convex set. . 161
11.5 An example of a convex function. 162
11.6 An example of a non-convex function. 162
11.7 On the left the set $||\mathbf{w}||_2 = 1$, which includes all the points on the border of the circle (in red). It is non convex, since if we take two distinct of the set, the segment between them does not belong to the set. On the other hand, on the right, we depict the set $||\mathbf{w}||_2 \leq 1$, which includes all the points inside the circle. In this case the set is convex, since for each pair of points within the set, the segment between them belongs to the set as well. 166
11.8 Function $\frac{1}{\sqrt{x}}$. 167
11.9 Example of support vectors. 170
11.10Separable case. 173
11.11Non-separable case and feature transformation to obtaina separable scenario. 174
11.12Non separable case that it is difficult to address by using a kernel. 178
11.13Separable case with outliers. 178
11.14The role of slack variables. 179

12.1 Graphical representation of the Perceptron algorithm. 182
12.2 ReLU function. 184
12.3 Single neuron NN. 184
12.4 A more complex NN example. 185
12.5 Simple two-layer NN. 187
12.6 Early Stopping 201

13.1 K-means iteration example. 203
13.2 Example of a mixsture of gaussian in the one-dimensional case. The mixture (in red) is the result of the sum of three Gaussian (gree, blue and orange). The point $x^{(0)}$ is generated from the blue Gaussian with a higher probability than the green or the orange on (although these are non zero). Instead, for the sample $x^{(1)}$ the probability of being generated from the green Gaussian is the highest. 206
13.3 Example of the Jensen inequality in the monodimensional case. In this case X is a random variable with two values a and b, both associated with a 0.5 probability. Therefore, its expected value can be easily computed as $E[X] = \frac{a+b}{2}$. Similarly, the expecte value of $f(a)$ and $f(b)$ is $E[X] = \frac{f(a)+f(b)}{2}$ 212
13.4 Example of the EM-iterations 215

14.1 Direction of main variation in the helicopter example. 218
14.2 Projection of the sample along the direction of variation. 218

List of Tables

2.1 Dataset $\mathcal{D}_{training}$ "house prices". 13

Chapter 1

Notation

We will use the following notation during the course:

- Scalar variables will be indicated with lowercase letters and in italics, for example:

$$x, y \in \mathbb{R} \tag{1.1}$$

- vectors will instead be indicated with lowercase letters, in italics and bold, for example:

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_{D-1} \end{pmatrix} \in \mathbb{R}^{D \times 1} \tag{1.2}$$

  where D denotes the size of the vector. By convention, the components of the vector are indexed starting with 0 and ending with $D-1$. N.B. for convenience when using the blackboard we will use underlined italic letters instead of bold notation to indicate the vector:

$$\underline{x} = \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_{D-1} \end{pmatrix} \in \mathbb{R}^{D \times 1} \tag{1.3}$$

  The notations $\underline{x}$ and $\mathbf{x}$ are therefore equivalent and refer to **column** vectors. In case we need to use a row vector, we will use the transpose symbol:

$$\underline{x}^T \text{ or } \mathbf{x}^T = \begin{pmatrix} x_0 & x_1 & \dots & x_{D-1} \end{pmatrix} \in \mathbb{R}^{1 \times D} \tag{1.4}$$

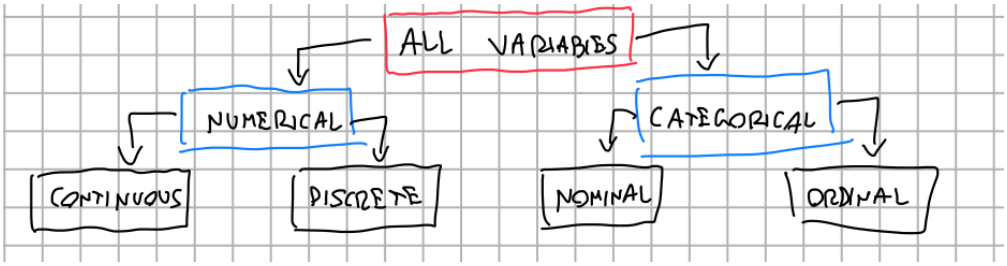


Figure 1.1: Type of variables

- Instead, we will use capital and bold letters for matrices:

$$\mathbf{X} = \begin{pmatrix} x_{0,0} & x_{0,1} & \cdots & x_{0,D-1} \\ x_{1,0} & x_{1,1} & \cdots & x_{1,D-1} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N-1,0} & x_{N-1,1} & \cdots & x_{N-1,D-1} \end{pmatrix} \in \mathbb{R}^{N \times D} \tag{1.5}$$

  N and D denote the number of rows and columns of the matrix, respectively (we will keep this convention with N and D where possible in the rest of the course).

In general, we will use w or θ to refer to the parameters that will be optimized by the algorithms. Depending on the case, w or θ will be in italics, bold or uppercase in case they refer to parameters in scalar, vector or matrix form.

We will use the letter x to indicate the independent input variables, also called *features*, to Machine Learning algorithms. Typically, the features provided as input to the algorithms are more than one and will, therefore, be organized in vector form (**feature vector**): $\mathbf{x} \in \mathbb{R}^{D \times 1}$.

The dependent variable, or *output variable*, or *label*, will instead be denoted by the letter y or the letter t, depending on the context. In particular, we typically use y to indicate the true output value, that is, in practice, inaccessible. We instead have access to noisy measurement of y, which we refer to as t. In this course, we will always be dealing with machine learning algorithms that output a single value (belonging to real numbers or a finite, discrete set of possible values):

$$y \in \mathbb{R} \text{ or } y \in S \tag{1.6}$$

where S is a finite discrete set.

Both features and labels can be classified based on the type of data they represent (see Figure 1.1). In particular, we can distinguish between:

- **Continuous Variables**: these are variables that take values in the set of real numbers $\mathbb{R}$.
- **Discrete Variables**: these are variables that can take values in a finite subset $S \subset \mathbb{N}$ or $S \subset \mathbb{Z}$.

- **Nominal Variables**: are variables typically of string type (or, in general, non-numerical), which can take on values within a predefined set of categories. They do **not** have an order associated.

  For example $x \in \{\text{Medic}, \text{Administrator}, \text{Manager}, \text{Engineer}, \dots\}$. In this case x is a nominal variable of string type representing a type of employment.

- **Ordinal Variables**: These are variables similar to nominal variables (so not numerical). However, in this case, the values they can take have an associated **ordering**.

  For example: $x \in \{\text{Low Risk}, \text{Medium Risk}, \text{High Risk}\}$. In this case, the value "Low Risk" has a value "conceptually" lower than "Medium Risk".

N.B. The algorithms of Machine Learning can take as input only variables of numeric type. In the case where features are instead encoded with non-numeric types, we will have to use strategies to convert them to numeric types.

We also introduce the concept of **Sample**. The sample is the element used by machine learning algorithms to perform the training and the subsequent testing phase. They are thus our "experience", which we provide to the algorithms in order to train them and to verify their performance after training.

In the case of **Supervised** algorithms, the sample consists of a pair of

$$(\mathbf{x}, y) \ , \ \mathbf{x} \in \mathbb{R}^{D \times 1} \text{ and } y \in \mathbb{R} \text{ or } y \in \mathbb{S} \subset \mathbb{Z} \tag{1.7}$$

where $\mathbf{x}$ is the input feature vector, and y is the supervisory label, i.e., the output we expect the algorithm to provide for that particular input. The labels are needed in the training phase to provide the algorithm with input-output relationships examples that can be used to learn. If training is successful, the algorithm will then be able to process new inputs $\mathbf{x}_{test}$ never seen in the training phase and predict the associated output of interest. By convention, we will distinguish between the supervision label provided in the training phase and the output provided by the algorithm with the following notation: in the former case, we will simply use the letter y, while for the prediction we will use $\hat{y}$ to indicate that it is an estimate of the unknown true value.

On the other hand, in the case of unsupervised algorithms, the supervision labels are not available and the sample consists only of the feature vector $\mathbf{x}$.

To train a Machine Learning algorithm, a certain number of samples are required. The set of samples used for training and testing the algorithms is defined **Dataset**:

$$\mathcal{D} = \{(\mathbf{x}^{(0)}, y^{(0)}), (\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(i)}, y^{(i)}), \dots, (\mathbf{x}^{(N-1)}, y^{(n-1)})\} \tag{1.8}$$

In this case N indicates the total number of samples in the dataset, while we will use the superscript (i) to indicate the i-th sample in the dataset.

We can also organize the feature vectors and their labels in matrix and vector form, respectively:

$$\mathbf{X} = \begin{pmatrix} x_{0,0} & x_{0,1} & \cdots & x_{0,D-1} \\ x_{1,0} & x_{1,1} & \cdots & x_{1,D-1} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N-1,0} & x_{N-1,1} & \cdots & x_{N-1,D-1} \end{pmatrix} \in \mathbb{R}^{N \times D}, \mathbf{y} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{pmatrix} \in \mathbb{R}^{N \times 1} \tag{1.9}$$

Chapter 2

Introduction to Supervised Learning

2.1 General Concepts

Unlike (*model-based*) approaches that rely on our knowledge about the physical phenomenon or process of interest, Machine Learning approaches (*data-driven*) aim to learn these models directly from data. The key element, in this case, is to provide the algorithm with *experience*, i.e., examples that the it can use in the training phase to learn the relationship between the input and the desired output. These "examples" are in fact the samples of the dataset that we intend to use to train the algorithm. In the case of *supervised learning*, the samples are pairs $(\mathbf{x}^{(i)}, y^{(i)})$ consisting of the feature vector $\mathbf{x}^{(i)}$ and the supervision label $y^{(i)}$. The presence of the label $y^{(i)}$ is what gives this paradigm its name "Supervised": in the training phase, the algorithm will not only have the features available, but also the expected output for that particular input. The label, in this case, is to be understood as information provided by an expert entity (such as a human operator) and can then be used as a true and reliable data. In other words, we are "supervising" the training of the algorithm by providing it with the outputs we expect for the samples in the dataset.

A useful parallelism to understand this concept is the one with the learning modalities of human beings: let's suppose that we want to train our brain to recognize the category "House" from the images that come from our visual system. To do this, we get help from another expert person, who has already learned this model. This person, in particular, will point us to houses and tell us that the objects we see fall into the category "House". We will observe them by generating an image of the scene (our feature vector) and provide our brain with both the acquired image and the label that the expert person has given us. Once we have acquired a number of these image-label pairs, we will begin the training phase and our brain will automatically generate a model describing the relationship between image and expected output. Once the training is complete, we will

Table 2.1: Dataset $\mathcal{D}_{training}$ "house prices".

features $\mathbf{x}$				label y
mq	n° rooms	distance from the city center	type	price
80	3	4km	seaside	100k€
120	5	1km	seaside	200k€
200	4	10km	seaside	250k€
...	...	...	...	...
...	...	...	...	...

be able to observe houses (even those never seen during the training phase) and associate the label "House" with them without the support of an expert.

Let's make a first example: let's assume that we have a dataset $\mathcal{D}_{training}$ of $n = 100$ samples describing houses with their selling prices. The input features vector, in particular, will contain some characteristics of the house (e.g. square meters, number of rooms, distance from the city center, type of house, etc...), while the label will indicate the selling price (Table 2.1).

The goal of the Machine Learning algorithm, in this case, is to use the available data to learn a predictive model capable of estimating the price y of a house not present in the dataset $\mathcal{D}_{training}$ used during training. That is, given the values $(\mathbf{X}, y) \in \mathcal{D}_{training}$, we would like our algorithm to be able to estimate $\hat{y}$ even for feature vectors $\mathbf{x} \notin \mathcal{D}_{training}$.

The dataset used for training the algorithm is called **training set** and the samples of which it is composed are called **training samples**. Instead, we define **test set**, containing the **test samples**, as the set of samples that were not "seen" during training and are instead used to evaluate the performance of the learned model on new data. The test set, in particular, is needed to evaluate the model's **generalization** capabilities against data not used during training.

More formally, then, our goal is, given a training set, to learn a function

$$h : \mathcal{X} \to \mathcal{Y} \tag{2.1}$$

such that, given a vector of features $\mathbf{x}$, $h(\mathbf{x})$ is a "good" predictor of the corresponding output value y. A good predictor means that, given a sample of the test set $(\mathbf{x}_{test}, y_{test}) \in \mathcal{D}_{test}$:

$$h(\mathbf{x}_{test}) = \hat{y}_{test} \approx y_{test} \tag{2.2}$$

which means that the value predicted from features "never" seen during training is as close as possible to the expected value.

The function $h(\mathbf{x})$ is called **predictor** or **hypothesis function**.

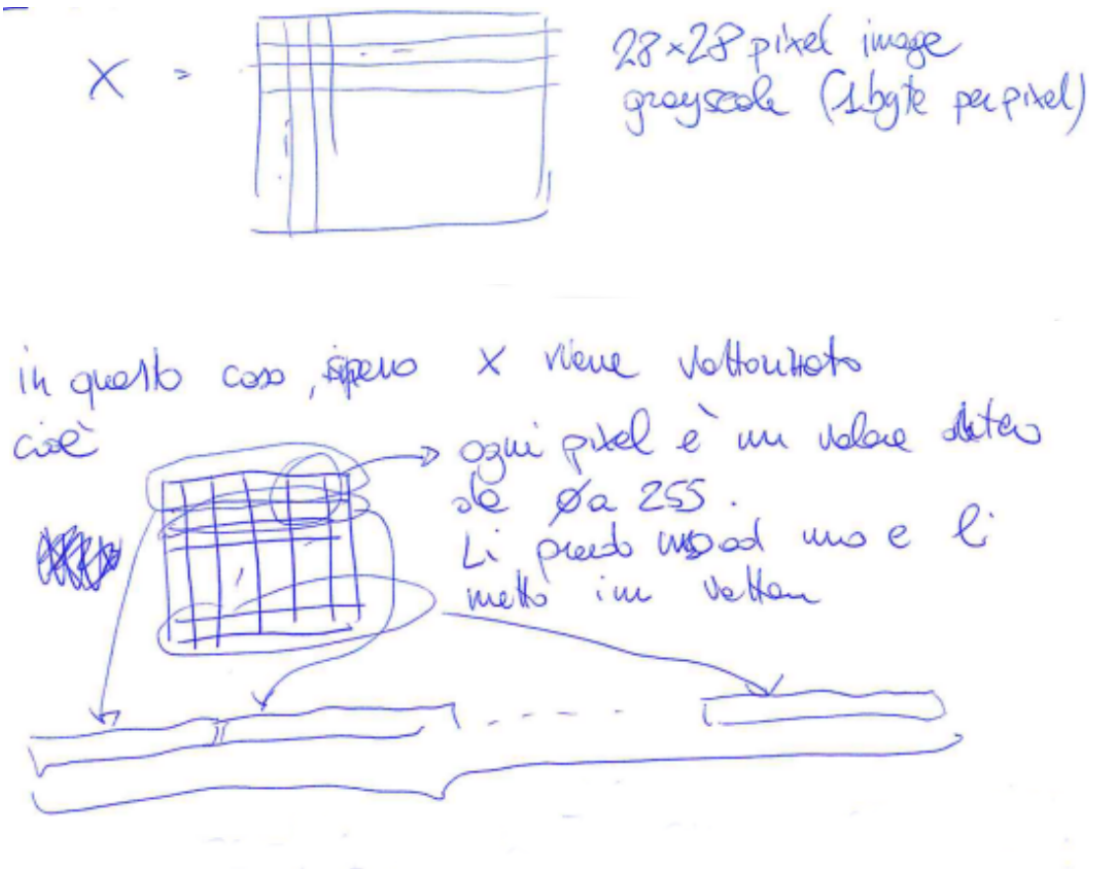


Figure 2.1: 28×28 image from the MNIST dataset depicting a handwritten digit.

2.2 Classification and Regression

Supervised learning algorithms can be distinguished into two categories, based on the type of output they are intended to estimate. In particular, a distinction is made between:

- **Classification Algorithms**: when the output variable belongs to a finite (typically small) and discrete set of elements, *i.e.,* $y \in S$, where S is a finite and discrete set.
- **Regression Algorithms**: when the output variable can take continuous values, *i.e.,* $y \in \mathbb{R}$.

The prediction of house prices is an example of a regression algorithm, since the output variable can take on any positive value.

An example of a classification problem, on the other hand, is **spam filtering**. In this case, given the text of an email, our goal is to determine whether that email belongs to the *spam* or *not spam* category.

$$y \in \{\text{spam}, \text{not spam}\} \tag{2.3}$$



Figure 2.2: What is the difference between the "3" drawn on the right and on the left? The digit is the same, but, in the right image, the pixels containing the 3 stroke are shifted to the left. A model-based approach should contain rules for all possible positions of the 3 in the image.

while typically the feature vector $x \in \mathbb{R}^d$ contains the number of occurrences of some keywords in the text useful to differentiate between spam and non-spam emails.

Another example of a classification problem is **handwritten digit recognition**. The goal is to recognize the digit inside an image that represents a handwritten number. In the scientific community, there exists the MNIST dataset that contains 28×28 images depicting handwritten digits.

In this case, the output variable takes a value between 0 and 9, while the feature vector consists of the pixels of the image (see Figure 2.1):

$$y \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} \tag{2.4}$$

Since the image is represented by an array of pixels, to construct the feature vector we "vectorize" the array by concatenating the rows of the pixel array:

$$\mathbf{x}^T = (x_0, x_1, \ldots, x_{d-1}) \tag{2.5}$$

with $d = 28 \times 28 = 756$ in the MNIST dataset case.

Why is this problem very complex to solve without machine learning algorithms?

This is because solving the problem with a model-based approach would require the definition of a set of mathematical relations and rules that, based on all possible combinations of pixel intensities in the image, are able to distinguish the 10 digits and recognize a digit even if drawn with a different rotation, in a different position in the image or with a different style (see Figure 2.2).

2.3 First example: Curve Fitting

Let's start with a very simple example to understand the basic principles behind training Machine Learning models.

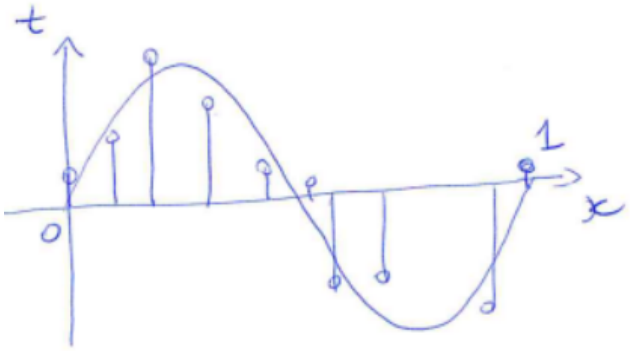


Figure 2.3: The graph represents, with the solid line the unknown true process $y = sin(2\pi x)$, while the dots indicate the available "noisy" samples.

Suppose we are interested in training a model to represent a process that is unknown to us. For simplicity, we will assume that this process is a sinusoidal function of type:

$$y = sin(2\pi x) \tag{2.6}$$

The only information we have access to relative to this unknown process are observations of pairs $(x^{(i)}, t^{(i)})$ (we assume we have n of them) that compose our dataset. In this case, we use the letter t for the measured output variable. This is because the measurements that we have access to (the supervision labels t) are affected by noise and therefore do not exactly correspond to the y that we would have access to knowing the real process that generates them. This situation is typical in Machine Learning problems: even the supervision labels, despite being provided by an expert entity, are not, in practice, an exact representation of the real output variable we are interested in, due to measurement noise or defects in the labeling process.

Our goal is to identify the "hidden" pattern among the data (i.e., the true process $y = sin(2\pi x)$) using "noisy" realizations of that process (see Figure 2.3).

Having no information about the mathematical structure of the real process that generated the data, how can we learn a model that approximates it as closely as possible without being affected by measurement noise?

In general, Machine Learning algorithms address this problem by defining a parametric hypothesis function that is generic enough to represent a good number of possible functions. We choose in this case a polynomial of degree M parameterized through the parameters $\mathbf{w}^T = (w_0, w_1, \ldots, w_M)$:

$$h_{\mathbf{w}}(x^{(i)}) = w_0 + w_1 x^{(i)} + w_2 (x^{(i)})^2 + \cdots + w_M (x^{(i)})^M = \sum_{j=0}^{M} w_j (x^{(i)})^j \tag{2.7}$$

Our hypothesis function then estimates the output $\hat{y}^{(i)} = h_{\mathbf{w}}(x^{(i)})$, relative to the i-th

input $x^{(i)}$ (N.B. samples consist of only one feature, so our input in this case is a scalar), by transforming the input feature through a polynomial expansion.

What we would like is that the estimate $\hat{y}^{(i)}$ is as close as possible to the supervision label $t^{(i)}$ associated with the input $x^{(i)}$, but without "overdoing it", i.e., without "chasing" the noise present in $t^{(i)}$. This behavior, of course, should occur for all samples used for training (and not only for the i-th). To achieve this goal, the Machine Learning algorithm has at its disposal the vector of parameters $\mathbf{w}$, which it can modify during the training phase. We note two aspects at this point:

- The model (the hypothesis function) is not linear with respect to x, but **linear** with respect to $\mathbf{w}$. Throughout the course we will be dealing with many such models, because they are easier to optimize.
- In order for the algorithm to learn the best values for the vector of parameters $\mathbf{w}$, we need some measure of the "quality" of a particular configuration. The algorithm will then need to choose the set of parameters $\mathbf{w}$ that maximizes this measure.

With respect to this second aspect, in practice a **error function** or **loss function** is defined to evaluate the quality of the estimates provided by the model:

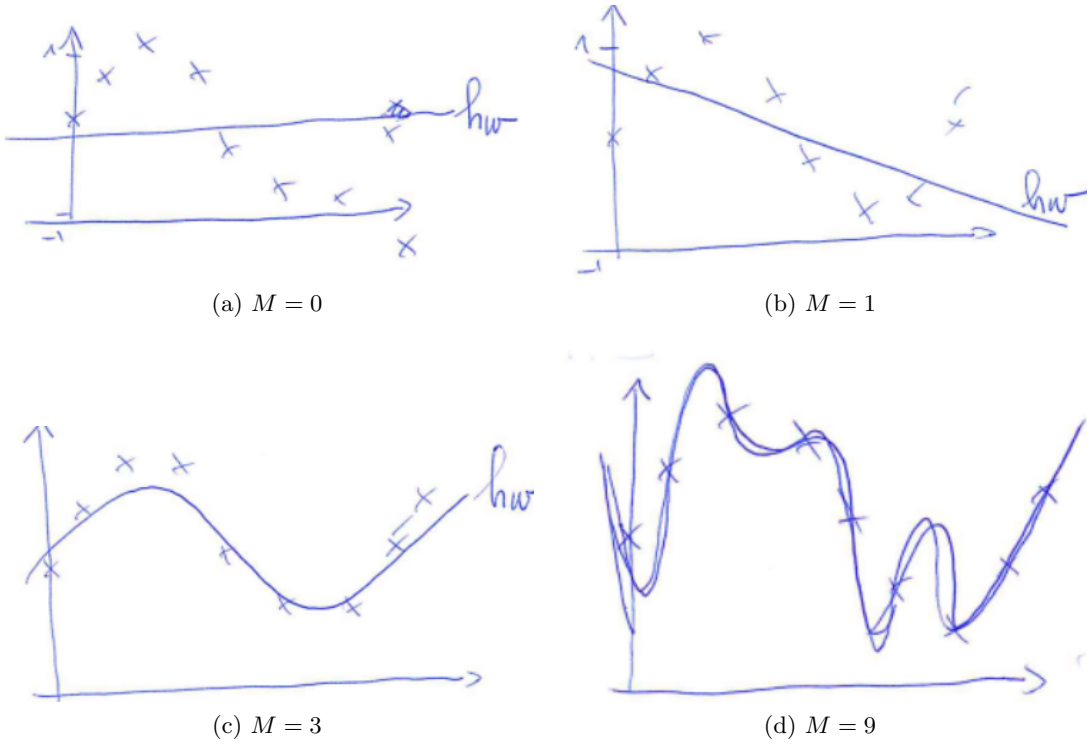
$$L(\mathbf{w}) = \frac{1}{2}\sum_{i=0}^{n-1}(h_{\mathbf{w}}(x^{(i)}) - t^{(i)})^2 \tag{2.8}$$

Where the summation iterates over all the samples in the dataset. The role of the function $L(\mathbf{w})$ is intuitive: its value will be larger the more the estimates of the hypothesis function deviate from the supervisory values. Minimizing this function with respect to the vector of parameters ($L(\mathbf{w})$ is a function of $\mathbf{w}$) is equivalent to "driving" the algorithm toward a choice of $\mathbf{w}$ that guarantees predictions as close as possible to the supervisory values. The optimal value of the parameters at the end of the optimization will be indicated by the symbol $\mathbf{w}^*$.

We chose to take the difference of squares in the first place to ensure that the function is non-negative. There are also other motivations that we will see later. Furthermore, it can be seen that this function is equal to 0 if:

$$h_{\mathbf{w}}(x^{(i)}) = t^{(i)} \ , \forall i = 0, \ldots, n-1 \tag{2.9}$$

N.B. Since $L(\mathbf{w})$ is a quadratic function of the parameter vector, when minimizing it we impose that its derivative is equal to 0. Since $h_{\mathbf{w}}(x^{(i)})$ is a linear function with respect to $\mathbf{w}$, we will see how the optimal solution $\mathbf{w}^*$ is unique.



(a) $M = 0$ (b) $M = 1$

(c) $M = 3$ (d) $M = 9$

Figure 2.4: Effect of hyper-parameter M on the hypothesis function. Increasing M makes the hypothesis function more flexible and better tracks the points in the dataset.

2.4 Overfitting

The algorithm is free to choose $\mathbf{w}$. However, when we defined the hypothesis function, we also said that its structure was a polynomial of degree M. How is the value of M set? We saw that the hypothesis function is optimized with respect to $\mathbf{w}$, but not with respect to M. Thus, the choice of M is delegated to the designer or to procedures not directly included in the algorithm optimization process. For this reason M is called **hyper-parameter** (because M is a parameter not optimized by the ML algorithm). The process of choosing the best value of M is called **Model Selection** and we will review it in detail later.

The choice of M has an effect on the complexity of the hypothesis function: the larger M is, the more "flexible" the polynomial will be. We can see the effect that different choices of M have on the shape of the hypothesis function in Figure 2.4.

In particular, we can see that for $M = 0$ the hypothesis function degenerates into a line parallel to the x axis (in this case $h_{\mathbf{w}}(x^{(i)}) = w_0$) and fails to "explain" the dataset samples well. For $M = 1$ we instead have a straight line, which certainly approximates the data pattern more closely than the previous case. However, its predictions are still associated with a high error for many points. $M = 3$ instead seems to approximate the hidden pattern well and the estimation error is small for all points in the dataset.

Finally, looking at $M = 9$, we would be inclined to think that this is the best value

for the degree of the polynomial: the hypothesis function is so flexible that it can chase all points in the dataset and guarantees $L(\mathbf{w}) = 0$. Yet, we quickly realize, intuitively, that the best value for M is 3. This is because it is evident that the model with $M = 9$ not only modeled the process, but also chased the data noise. On the other hand, the model with $M = 3$ is able to "ignore" the noise and focus on the true hidden pattern.

When the hypothesis function of an ML algorithm is so much flexible (or equivalently, has a high degree of **complexity**) that it also models the noise of the training data, we speak of **OVERFITTING**. This phenomenon corresponds to an overfitting of the training data.

Why is overfitting problematic? An excessive fit of the training data corresponds inevitably to a loss of the generalization capabilities of the model with respect to new data: the model has "memorized" the training data, but when it will process a sample never seen before, it will commit a very high estimation error.

How can we detect if we are in a situation of overfitting? The error function calculated on the training data can not help us in this sense. This is because our goal is to minimize it and clearly we want to make it as close as possible to zero. To detect this phenomenon we use instead the **test set**, that is that portion of the overall dataset that we did not use to optimize $L(\mathbf{w})$.

To detect overfitting, we compare the error obtained on the training set and the test set. Specifically, given a dataset $\mathcal{D}$ with n samples, we divide it into two disjoint sets $\mathcal{D}_{training}$ and $\mathcal{D}_{test}$ respectively of n_1 and n_2 samples, with $n_1+n_2 = n$ and, once we find the optimal parameters $\mathbf{w}^*$ we compute:

Training Error: measured by computing the error function $L(\mathbf{w}^*)$ (evaluated with respect to the optimal parameter set $\mathbf{w}^*$) on $\mathcal{D}_{training}$:

$$E_{training} = L(\mathbf{w}^*) = \frac{1}{2}\sum_{i=0}^{n_1-1}(h_{\mathbf{w}}(x^{(i)}) - t^{(i)})^2 \tag{2.10}$$

Test Error: measured by computing the error function $L(\mathbf{w}^*)$ (evaluated with respect to the optimal parameter set $\mathbf{w}^*$ **obtained by using** $\mathcal{D}_{training}$) on $\mathcal{D}_{test}$:

$$E_{test} = L(\mathbf{w}^*) = \frac{1}{2}\sum_{i=0}^{n_2-1}(h_{\mathbf{w}}(x^{(i)}) - t^{(i)})^2 \tag{2.11}$$

The test error actually measures the generalization ability of the calculated model.

Having defined Training and Test error, we can draw a graph at the variation of M to study the trend of these two quantities (see Figure 2.5). From the figure we can see that, for low values of M the error is high both on the training and on the test set: this

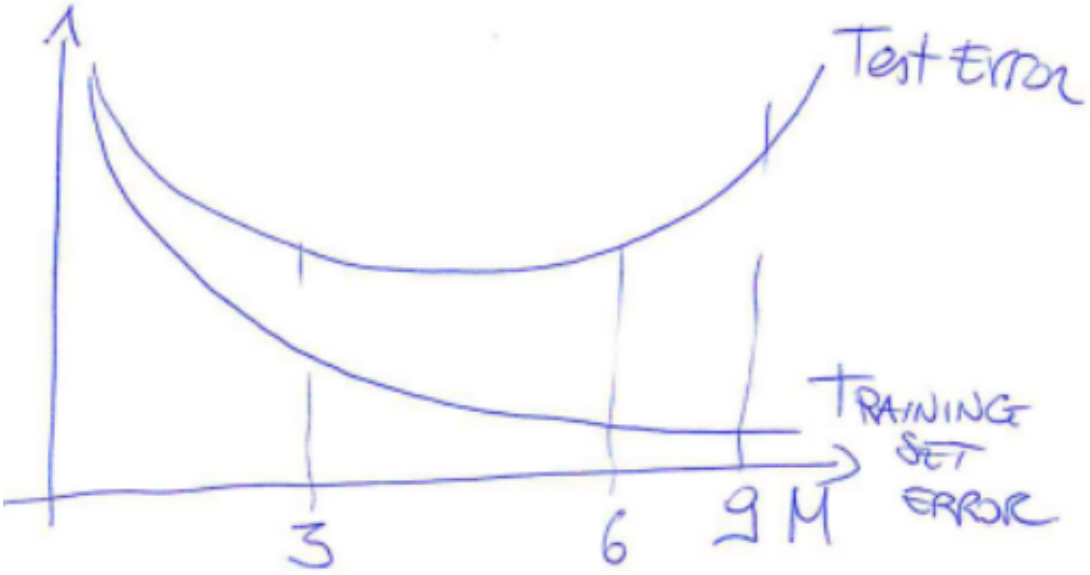


Figure 2.5: Training and Test Error for different values of M.

is because the model does not have enough representative capacity and is not able to explain the data. By increasing too much M, instead, the model explains the data too well and chases the noise, losing the generalization capabilities. The optimal value of M is the intermediate value that guarantees a reasonable level of complexity.

Another aspect that is possible to observe when we experience overfitting is the presence of very high values of the vector $\mathbf{w}^*$. We will see how a good model has values of the parameters $\mathbf{w}$ "controlled" and not too large. Intuitively, this is because a very flexible model, to chase the data to the best of its ability, will be characterized by a hypothesis function with particularly large derivatives (we can see this if we look at the model with $M = 9$), which are associated with high $h_{\mathbf{w}^*}(\mathbf{x})$ parameter values.

There are three main strategies to control overfitting:

- Increase the number of training data: Increasing the number of samples taken into account in the error function during the optimization clearly "complicates" the job of the algorithm that will need to explain more samples.
- Choose hyper-parameter values to reduce the complexity of the model.
- Use **Regularization** techniques that constrain the values of $\mathbf{w}$ to prevent them from becoming too large. This, in practice, reduces the flexibility of the function without directly acting on the hyper-parameters that control it.

For example, we can regularize the error function defined previously in the following way:

$$L(\mathbf{w}) = \frac{1}{2}\sum_{i=0}^{n-1}(h_{\mathbf{w}}(x^{(i)}) - t^{(i)})^2 + \frac{\lambda}{2}||\mathbf{w}||^2 \tag{2.12}$$

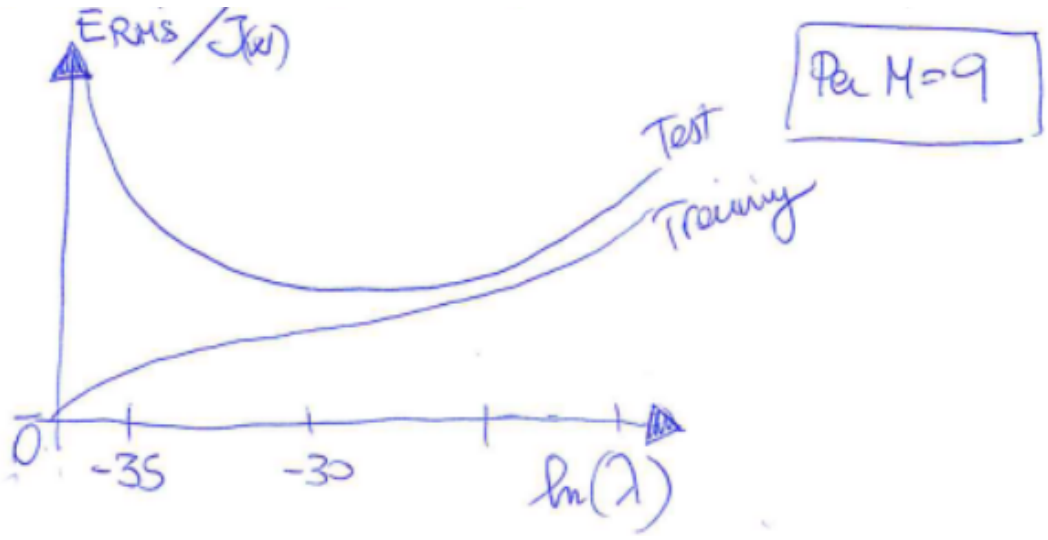


Figure 2.6: Training and Test error as lambda varies (in logarithmic scale). If we set $M = 9$ (high flexibility), if $log\lambda \ll 0$ we are not constraining the parameters and therefore we are not performing regularization: in this case the training error goes to zero but we experience overfitting. As $\log \lambda$ increases, training and test error get closer.

with

$$||\mathbf{w}|| = \sqrt{w_0^2 + w_1^2 + \cdots + w_M^2} \tag{2.13}$$

The second term of the 2.12 pushes the optimizer to keep the values of $\mathbf{w}$ low . λ is instead another hyper-parameter that balances the effect of regularization and must always be chosen using **model selection** techniques.

Methods that add a penalty term on the values of $\mathbf{w}$ are called **Shrinkage methods**. The case we saw in the equation 2.12 is called **Ridge Regression** and we will see it later in the course.

If we use a regularization technique to prevent overfitting, we can avoid to choose the best M, but rather set it to a value that generates a highly flexible hypothesis function: in particular, we will focus on choosing the regularization λ to indirectly control this complexity and prevent overfitting.

2.5 Model Selection

Regardless of whether we decide to focus on choosing the best hyper-parameter that controls complexity (M in the previous polynomial curve fitting example) or regularization (λ, as defined in the example of the previous section), we have seen that to determine it is necessary to check the error the model makes on a dataset not used during training, namely the test set.

However, employing the test set to choose the optimal value of M or λ invalidates the assumption that the test data should never be used to optimize our model, either with respect to parameters or hyper-parameters. This problem is not a simple formality.

Intuitively, if we optimize a parameter or a hyper-parameter on a dataset, we will clearly find the best value useful to explain that data and guarantee a low error with respect to it.

If we take the example of curve fitting and assume to intervene on the degree of the polynomial to handle the overfitting we must choose:

1. In the first instance the value M.
2. Once we have chosen the degree M, we can find the optimal parameters $\mathbf{w}^*$ for that particular polynomial

A possible scheme to find the best model is the following:

1. For every $M = 0, 1, \ldots, 9$
   (a) Train the model on $\mathcal{D}_{training}$ and find the optimal vector $\mathbf{w}$
   (b) Test on $\mathcal{D}_{test}$.
2. Choose the model that guarantees the least error on $\mathcal{D}_{test}$

However, this way we will find a model that performs well on the test data, since we used those to select it. As a result, we have no way to actually test the generalization capabilities of the model on data that is never seen or used in optimization.

To overcome this problem we use **Model Selection** procedures: the basic idea is to divide the dataset into training and test, but in addition, we also reserve an additional portion of the training set to identify the optimal hyper-parameters (M in our case). This portion, called the **Validation Set**, is not used to optimize the $\mathbf{w}$ parameters of the polynomial, but to evaluate a trained model and identify the best with respect to the hyper-parameters.

At this point the procedure for finding the best model becomes as follows:

1. divide the dataset into three subsets: $\mathcal{D}_{training}$, $\mathcal{D}_{validation}$ and $\mathcal{D}_{test}$. Typically, the proportions used are 70%-20%-10% or 60%-20%-20%.
2. For each $M = 0, 1, \ldots, 9$
   (a) train the model on $\mathcal{D}_{training}$ and find the optimal vector $\mathbf{w}$
   (b) Test on $\mathcal{D}_{validation}$.
3. choose the model that guarantees the smallest error on $\mathcal{D}_{validation}$
4. Using the best model emerged from the previous phase, perform the final test on $\mathcal{D}_{test}$. This phase, in which the performance of the optimal model is verified on unseen data, is called **Model Assessment**.

The procedure just seen is a special case of Model selection called **Model selection with Fixed Validation Set**. There are two other procedures, mainly used for datasets with a smaller number of samples: **K-fold Cross Validation** and **Leave-one-out**.

Cross-validation (see Figure 2.7), in particular, involves:

1. Dividing the dataset into two subsets $\mathcal{D}_{training+validation}$ and $\mathcal{D}_{test}$.
2. $\mathcal{D}_{training+validation}$ is, afterwards, divided into K subsets (typically $K = 3$ or $K = 5$).
3. For each $M = 0, 1, \ldots, 9$
   (a) For $k = 0, 1, \ldots, K-1$
      i. train on the $K-1$ subset excluding the k-th
      ii. test on the k-th subset
   (b) compute the average of the errors obtained in the K tests
4. choose the model with the lowest average error obtained in the K tests.
5. Using the best model that emerged from the previous step, perform the final test on $\mathcal{D}_{test}$.

We will revisit these procedures later in the course. Both K-fold Cross Validation and Leave-one-out are used when the number of available samples is small. This is because dividing the dataset into fixed sets may compromise the statistical significance of the various subsets and lead to incorrect test performance evaluations. Consequently, it is preferable to "iteratively" use all available data to train and test. For this reason, cross-validation performs multiple trainings for the same model by cycling on the K folds.

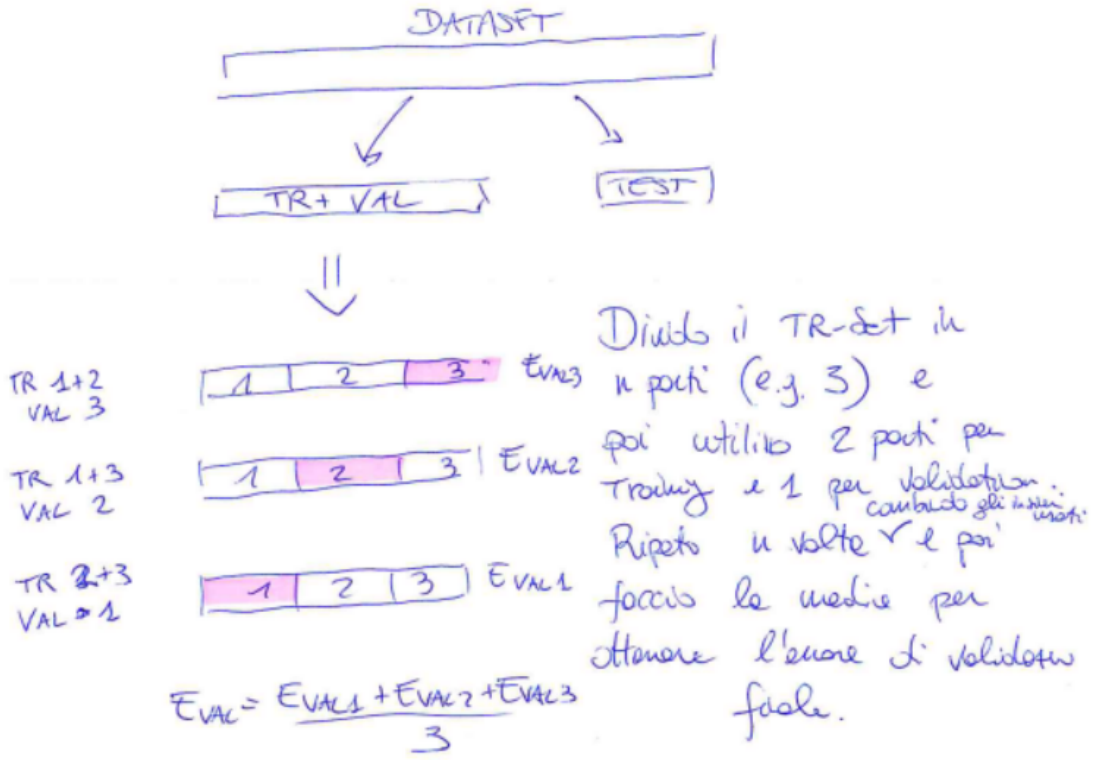


Figure 2.7: Example diagram of the cross-validation procedure.

Chapter 3

Probability Theory

3.1 General Aspects

Many of the ML algorithms we will see in the course can be formulated and interpreted from a probabilistic point of view. It is therefore appropriate to take up some concepts from probability theory.

We begin by considering two one-dimensional random variables Y and X and, for example, suppose that $Y \in \{A, B, C, D\}$ and $X \in \{1, 2, 3, 4, 5, 6\}$. Thus, we know what values the two random variables can take, but we do not know what their probability ditributions $P(X)$ and $P(Y)$ are. To estimate them, we can sample the two processes and estimate their **joint distribution** $P(X, Y)$ (see Figure 3.1).

If we sample N times $P(X, Y)$, we can determine in which box of Figure 3.1 each realization will fall, *i.e.,* , which values the random variables X and Y have taken for that particular sample. We call n_{ij} the number of occurrences of the event $P(X = x_i, Y = y_j)$ (*i.e.,* , the number of times a sample fell into the box (x_i, y_j). A first definition of the probability of the event $P(X = x_i, Y = y_j)$ can then be given by the following relation:

$$P(X = x_i, Y = y_j) = \frac{n_{i,j}}{N} \ , \ N \to \infty \tag{3.1}$$

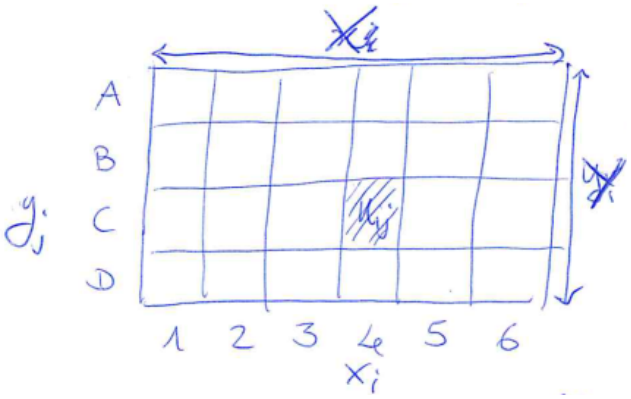


Figure 3.1: Table depicting the joint distribution of X and Y.

We then assume, with this strategy, that we calculate the probabilities of all possible events that characterize the joint distribution of X and Y. How can we, at this point, calculate the marginal probability (of the individual variables) $P(X)$ and $P(Y)$?

To compute $P(X = x_i)$ from $P(X, Y)$, we need to "marginalize" the joint with respect to all possible values of Y fixed at $X = x_i$. In particular, $P(X = x_i)$ is given by the sum of the values of all values of the x_i-th column of Figure 3.1):

$$\begin{aligned} &P(X = x_i) = \\ &= P(X = x_i, Y = A) + P(X = x_i, Y = B) + P(X = x_i, Y = C) + P(X = x_i, Y = D) \\ &= \frac{c_i}{N} \end{aligned} \tag{3.2}$$

where by c_i we denote the sum of the $n_{i,j}$, with $j = A, B, C, D$.

This relation is called **Sum Rule**. The probability $P(X = x_i)$ is called **Marginal Probability**, because it is obtained precisely by marginalizing with respect to Y.

We now introduce the concept of **Conditional Probability**. If we assume that we have measured the random variable X, we can calculate the probability of Y since we have observed the value x_i for X:

$$P(Y = y_j | X = x_i) = \frac{n_{i,j}}{c_i} \tag{3.3}$$

in this case we divide by c_i because having observed $X = x_i$ (*i.e.,* this event already occurred), this restricts my original probability set to only the values of the joint probability for which $X = x_i$.

With the above considerations, it is possible to define the **Product Rule**:

$$P(X = x_i, Y = y_j) = \frac{n_{i,j}}{N} = \frac{n_{i,j}}{c_i} \cdot \frac{c_i}{N} = P(Y = y_j | X = x_i) P(X = x_i) \tag{3.4}$$

To summarize, the probability rules we have seen are:

$$\text{SUM RULE} \qquad P(X) = \sum_Y P(X, Y) \tag{3.5}$$

$$\text{PRODUCT RULE} \qquad P(X, Y) = P(Y|X) P(X) \tag{3.6}$$

where we wrote $P(X)$ instead of $P(X = x_i)$ to avoid too cumbersome a notation (from now on we will use this more compact notation).

The product rule can be further extended to obtain the **Chain Rule** for probability:

$$P(X, Y, Z) = P(Z|X, Y) P(Y|X) P(X) \tag{3.7}$$

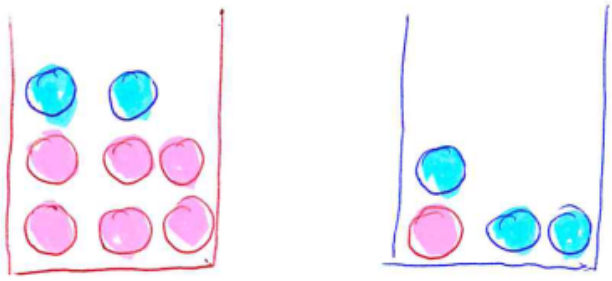


Figure 3.2: The two boxes (red on the right and blue on the left) contain apples (in light blue) and oranges (in pink)

3.2 The Bayes Theorem

All the rules previously introduced can be used to prove one of the most important theorems we will use: Bayes' theorem:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)} \tag{3.8}$$

and using the sum rule, it is possible to introduce the **Total Probability Theorem** to calculate $P(X)$:

$$P(X) = \sum_{Y} P(X|Y)P(Y) \tag{3.9}$$

The numerator of the term to the right of Bayes' theorem contains the product $P(X|Y = y_j)P(Y = y_j)$, *i.e.,* for the event y_j, while the denominator has the same product, but for all possible values of Y. In practice, the denominator acts as a normalizer, which is why Bayes' theorem is often expressed as:

$$P(Y|X) \propto P(X|Y)P(Y) \tag{3.10}$$

Let us now take a numerical example. Consider two boxes, as in Figure 3.2 The random process we consider consists in drawing a random fruit from a random box. Let us denote by b and r the events that correspond to the choice of the blue or red box, while by a and o we refer to the event "an apple (a) was drawn" and "an orange (o) was drawn", respectively. We then denote by $B \in \{b, r\}$ the random variable indicating the choice of the box and by $F \in \{a, o\}$ the one indicating the choice of the fruit.

Suppose we have done a large number of experiments to determine $P(B)$ and it turns out that the red box is selected 40% of the time. Consequently, we will have that:

$$P(B = r) = \frac{4}{10}\ ,\ P(B = b) = 1 - P(B = r) = \frac{6}{10} \tag{3.11}$$

The probability of the event b is automatically determined once we know the probability for r being the complementary event.

It is simple at this point to compute the conditional probabilities:

$$P(F = a|B = r) = \frac{1}{4} \tag{3.12}$$

$$P(F = a|B = b) = \frac{3}{4} \tag{3.13}$$

$$P(F = o|B = r) = 1 - P(F = a|B = r) = \frac{3}{4} \tag{3.14}$$

$$P(F = o|B = b) = 1 - P(F = a|B = b) = \frac{1}{4} \tag{3.15}$$

We can then use the total probability theorem to calculate, for example, the probability of drawing an apple:

$$P(F = a) = P(F = a|B = b)P(B = b) + P(F = a|B = r)P(B = r) \tag{3.16}$$

This example allows us to intuitively interpret the terms involved in Bayes' theorem. If we ask ourselves, without having any other knowledge, what is the probability of having chosen a certain box, we must answer $P(B)$: this unconditioned probability is called **PRIOR**, because it incorporates our a priori knowledge about the process. In this case, $P(B)$ in fact we set it by performing a campaign of observations and it is not computed with mathematical relations.

If, however, a further measurement gives us additional information and tells us that a certain fruit was drawn, we can then update our a priori hypothesis (our **Belief**, about the box, *i.e.,* $P(B|F)$. This is called **POSTERIOR**, precisely because is computed "a posteriori", *i.e.,* , after receiving information.

To calculate the posterior with Bayes' theorem we need $P(F|B)$, which represents the "generation" pattern of the fruit draws once the box is known. $P(F|B)$ is called **LIKELIHOOD**.

Using a numerical example, we know that the prior on the red box is $\frac{4}{10}$, however, if we know that it was drawn an orange, the probability that the red box was chosen increases:

$$P(B = r|F = o) = \frac{P(F = o|B = r)P(B = r)}{P(F = o)} = \frac{3/4 \cdot 4/10}{1/4 \cdot 6/10 + 3/4 \cdot 4/10} = \frac{12/40}{18/40} = \frac{2}{3} > \frac{4}{10} \tag{3.17}$$

Bayes' theorem is very useful because in practice it is often much easier to access the likelihood than the posterior. For example, modeling the distribution of an image that captures a door known to be open or closed ($P(I|X = 1)$ or $P(I|X = 0)$), is much more difficult than calculating the probability that the door is open or closed given the image (which can be done with computer vision algorithms), and multiplying it by the probability distribution of pixel intensities in the image (which can initially be assumed to be uniform).

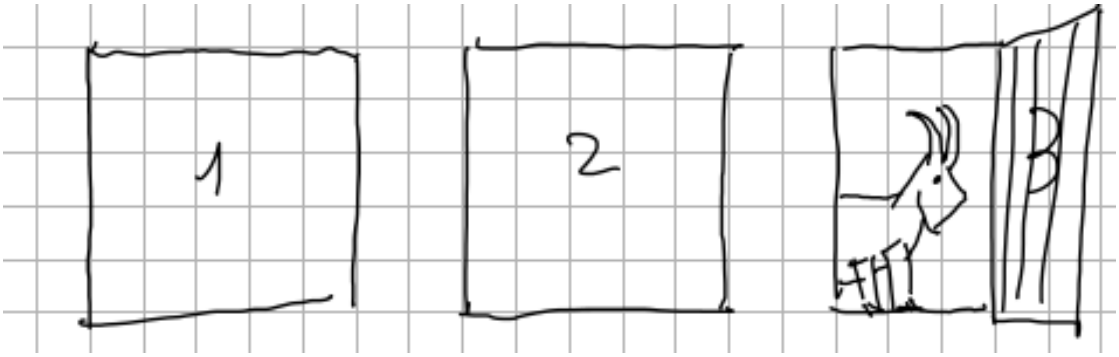


Figure 3.3: Monty Hall's problem. Behind two of the three doors are goats, and behind the third is a prize.

3.3 Example: the Monty Hall Problem

Now let's see how, thanks to Bayes' theorem, we can reach conclusions that are often not immediately intuitive.

Consider Monty Hall's problem (see Figure 3.3): suppose we have three doors that hide a prize in one case and goats in the other two. The contestant initially chooses a door, then the presenter reveals one of the two remaining doors without the prize. The contestant is then asked if he wants to change the door or not. What should he do?

Let's try to calculate it with the tools acquired so far. We have 3 random variables:

- $C : R_C = \{1, 2, 3\}$ which models the position of the prize with respect to the 3 doors. (R_C is a notation indicating the support of C. *i.e.,* , the set of values it can take);
- $H : R_H = \{1, 2, 3\}$ which models the presenter's choice;
- $X : R_X = \{1, 2, 3\}$ which models contestant's choice.

In the following we will, for example, write $P(C_1)$ instead of $P(C = 1)$ to compact the notation.

We assume that the a priori probability of C is described by a uniform distribution, since we have no information about the location of the prize:

$$P(C_1) = P(C_2) = P(C_3) = \frac{1}{3} \tag{3.18}$$

Suppose that initially the contestant chooses door 1 ($X = 1$) and then the presenter opens door 3. At this point we are interested in calculating the probability that, after seeing the door opened by the presenter, the prize is behind door 1 or 2, *i.e.,* :

$$P(C_1|H_3, X_1) \text{ and } P(C_2|H_3, X_1) \tag{3.19}$$

By applying Bayes we get:

$$P(C_1|H_3, X_1) = \frac{P(H_3|C_1, X_1)P(C_1|X_1)}{P(H_3|X_1)} \tag{3.20}$$

$$= \frac{P(H_3|C_1, X_1)P(C_1)}{P(H_3|C_1, X_1)P(C_1) + P(H_3|C_2, X_1)P(C_2) + P(H_3|C_3, X_1)P(C_3)} \tag{3.21}$$

$$= \frac{1/2 \cdot 1/3}{1/2 \cdot 1/3 + 1 \cdot 1/3 + 0 \cdot 1/3} = \frac{1/6}{1/2} = \frac{1}{3} \tag{3.22}$$

Where we replaced $P(C_1|X_1)$ with $P(C_1)$ since C and X are independent and used the total probability theorem to calculate the denominator.

Similarly, we can compute:

$$P(C_2|H_3, X_1) = \frac{P(H_3|C_2, X_1)P(C_2|X_1)}{P(H_3|X_1)} \tag{3.23}$$

$$= \frac{1 \cdot 1/3}{1/2} = \frac{2}{3} \tag{3.24}$$

So the best choice is to change the door after the presenter opens one with the goat!

3.4 Independency

Two random variables are said to be **independent** $X \perp\!\!\!\perp Y$ (*i.e.,* , the fact that one of them takes a certain value does not affect the distribution of the other and vice versa) if:

$$P(X, Y) = P(X)P(Y) \tag{3.25}$$

This is because by the product rule we know that $P(X, Y) = P(Y|X)P(X)$, but if $X \perp\!\!\!\perp Y$ then $P(Y|X) = P(Y)$.

Instead, we speak of **Conditional Independence** $X \perp\!\!\!\perp Y|Z$ if two variables are independent known a third variable:

$$P(X, Y|Z) = P(X|Z)P(Y|Z) \tag{3.26}$$

or, equivalently:

$$P(X|Y, Z) = P(X|Z) \tag{3.27}$$

N.B. Independence does not imply conditional independence. Consider, for example:

- Event A: coin toss $R_A = T, C$
- Event B: coin toss $R_B = T, C$
- Event C: the coins are both heads or both tails $R_C = \{TT, CC\}$.

In this case it is clear that $A \perp\!\!\!\perp B$, but $A \not\perp\!\!\!\perp B|C$, because if the event "both coins are heads" occurs, then if we get information about A, we will automatically know the value of B.
Similarly, conditional independence does not imply independence.

3.5 Continuous random variables

In the case of continuous random variables we define the probability density (**probability density function - pdf**). While for a discrete event it is possible to define the relative probability $P(X = x) = a$, $a \in [0, 1]$, in the case of pdf, $p_X(x)$ does not correspond to a defined probability value, but it is necessary to integrate it over a range of values:

$$P(X \in [x, x + dx]) \approx p_X(x)dx \tag{3.28}$$

where equality is obtained for $dx \to 0$.

In general, it is possible to calculate the probability of a certain interval as:

$$P(X \in [a, b]) = \int_a^b p_X(x)dx \tag{3.29}$$

Clearly, to have a valid probability distribution, it must satisfy:

$$p_X(x) \geq 0 \ , \forall x \tag{3.30}$$

$$\int_{-\infty}^{+\infty} p_X(x)dx = 1 \tag{3.31}$$

We additionally define the **Cumulative distribution function** $F_X(x)$ as:

$$P(X \leq x) = F_X(x) \tag{3.32}$$

Furthermore, it is clear that:

$$\frac{\partial F_X(x)}{\partial x} = \frac{\partial}{\partial x} \int_{-\infty}^{x} p_X(x)dx = p_X(x) \tag{3.33}$$

3.5.1 Change of variable

It can often happen that a random variable X is expressed as a deterministic function of another random variable Y:

$$x = g(y) \ , \ y \in S \tag{3.34}$$

with $g(\cdot)$ differentiable on S. It can then be shown that the relationship between $p_Y(y)$ and $p_X(x)$ is as follows:

$$p_Y(y) = p_X(g(y)) \left| \frac{\partial g(y)}{\partial y} \right| = p_X(x) \left| \frac{\partial x}{\partial y} \right| \tag{3.35}$$

Proof Consider two cases:

- Case $g^{-1}(X)$ strictly increasing (See Figure 3.4a):

$$F_Y(y) = P(Y \leq y) = P(g^{-1}(X) \leq y) = P(X \leq g(y)) = \int_0^{g(y)} p_X(x)dx = F_X(g(y)) \tag{3.36}$$

  The first equality is given by the definition of C.D.F. of Y. The second holds because $Y = g^{-1}(X)$. The third holds because, as it can be seen from the red segment in Figure 3.4a, for the portion of the function for which $g^{-1}(X) \leq y$ it is also true that $X \leq g(y)$. The penultimate equality holds by definition of probability of a continuous random variable X. The last one instead is given by the definition of C.D.F. At this point it is sufficient to calculate the derivative of $F_Y(y)$ to obtain the p.d.f. $p_Y(y)$:

$$\frac{dF_Y(y)}{dy} = \frac{dF_X(g(y))}{dy} \rightarrow p_Y(y) = p_X(g(y)) \frac{dg(y)}{dy} \tag{3.37}$$

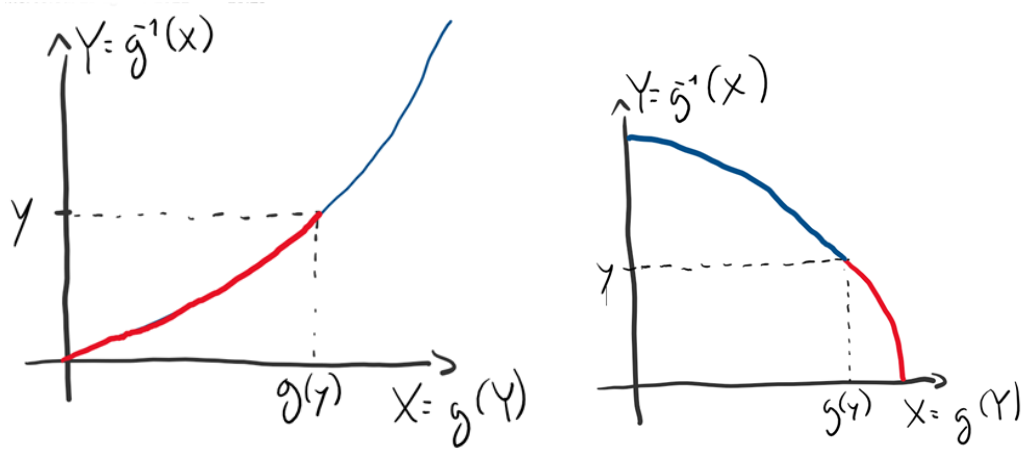
  Where, in the last step, we used the **chain rule** for the derivatives.

- Case $g^{-1}(X)$ strictly decreasing (See Figure 3.4b):

$$\begin{aligned} F_Y(y) = P(Y \leq y) = P(g^{-1}(X) \leq y) = P(X \geq g(y)) = \\ = 1 - P(X \leq g(y)) = 1 - \int_0^{g(y)} p_X(x)dx = 1 - F_X(g(y)) \end{aligned} \tag{3.38}$$

  In contrast to the previous case, since the function is decreasing, in the third equality we have that, as it can be seen from the red segment of Figure 3.4b, for the portion of the function where $g^{-1}(X) \leq y$ it holds that $(X \geq g(y)$. The fourth equality holds for the rule of complementary events. We can, as in the previous case, compute the derivative and obtain:

$$\frac{dF_Y(y)}{dy} = -\frac{dF_X(g(y))}{dy} \rightarrow p_Y(y) = -p_X(g(y)) \frac{dg(y)}{dy} \tag{3.39}$$



(a) case of $g^{-1}(y)$ strictly increasing (b) case $g^{-1}(y)$ strictly decreasing

Figure 3.4: Change of variable).

Combining the two cases, we finally get:

$$p_Y(y) = p_X(g(y)) \left| \frac{dg(y)}{dy} \right| \tag{3.40}$$

In the case where $g^{-1}(X)$ is not strictly increasing or decreasing, we can apply the same reasoning on the increasing or decreasing part of the function.

3.5.2 Fundamental rules for continuous random variables

Sum Rule

$$p_X(x) = \int_Y p_{X,Y}(x, y)dy \tag{3.41}$$

Product Rule

$$p_{X,Y}(x, y) = p_{Y|X}(y|x)p_X(x) \tag{3.42}$$

From now on, for simplicity of notation and when it is clear from the context, we will omit the subscripts of the pdfs:

$$p(x) = \int_Y p(x, y)dy \tag{3.43}$$

$$p(x, y) = p(y|x)p_X(x) \tag{3.44}$$

3.5.3 Expectation and Variance

Given a function $f(x)$ its mean with respect to a probability distribution $p(x)$ is called the **Expectation** of $f(x)$:

$$E_{p(x)}[f] = E[f] = \sum_x p(x)f(x) \text{ in the discrete case} \tag{3.45}$$

$$E_{p(x)}[f] = E[f] = \int_x p(x)f(x)dx \text{ in the continuous case} \tag{3.46}$$

In either case, if we have N points sampled by $p(x)$ available, it is possible to approximate the expectation in the following way:

$$E[f] \approx \frac{1}{N}\sum_{i=1}^{N} f(x^{(i)}) \tag{3.47}$$

where the approximation becomes exact for $N \to \infty$.

In the case of functions of more variables:

$$E_{p(x)}[f(x,y)] = \sum_x p(x)f(x,y) \tag{3.48}$$

and in the case of conditional distributions:

$$E_{p(x|y)}[f(x)] = E_{p(x|y)}[f(x)|y] = \sum_x p(x|y)f(x) \tag{3.49}$$

Instead, we define **Variance** as the variability around the expected value $E[f]$:

$$Var[f] = E[(f(x) - E[f(x)])^2] \tag{3.50}$$

and it can be demonstrated that:

$$Var[f] = E[f(x)^2] - E[f(x)]^2 \tag{3.51}$$

3.6 Curve Fitting re-visited

Having introduced the concepts of probability and, in particular, Bayes' theorem, we can reconsider the curve fitting problem in probabilistic terms. There are two ways, in general, to approach the problem:

- **Frequentist**: Assumes that the dataset D is a possible realization of a random distribution that depends on fixed $\mathbf{w}$ parameters. The problem is then to determine $\mathbf{w}$ from D.
- **Bayesian**: Assumes that the data D is fixed and that the uncertainty of the

problem is enclosed in the parameters $\mathbf{w}$, thus characterized by a distribution $p(\mathbf{w})$.

Either way er are interested in modeling:

$$\overset{posterior}{p(\mathbf{w}|D)} = \frac{\overset{likelihood}{p(D|\mathbf{w})}\ \overset{prior}{p(\mathbf{w})}}{\underset{marginal}{p(D)}} \tag{3.52}$$

The likelhood is interpreted differently in the frequentist and Bayesian case:

- **Frequentis**: there exist infinitely many possible D and only one $\mathbf{w}$ that maximizes $p(D|\mathbf{w})$.
- **Bayesian**: there is only one D (the observed one) and infinitely many possible $\mathbf{w}$.

The likelihood encodes the probability of observing a dataset D given the knowledge of a specific vector of parameters $\mathbf{w}$. In the frequentist approach, this representation is often sufficient (even though it can also work with the posterior distribution, as we will see). In addition, frequentist approaches are typically satisfied with point estimates and are not interested in modeling distributions. The Bayesian one, on the other hand, aims to describe the full probability distributions that characterize the models, and its predictions usually comes in the form of a pdf rather than a single value.

In general, we can write:

$$posterior \propto likelihood \times prior \tag{3.53}$$

this is because the denominator (the marginal) acts as a normalizer and can often be neglected if one is only interested in a relative comparison of probability values with different $\mathbf{w}$. Alternatively, the marginal can be computed as:

$$p(D) = \int p(D|\mathbf{w})p(\mathbf{w})d\mathbf{w} \tag{3.54}$$

3.6.1 Maximum Likelihood Estimation (ML)

The first strategy we will see to estimate the parameters $\mathbf{w}$ will be a frequentist one and, consequently, we will consider the likelihood. In particular, our goal will be to find the vector $\mathbf{w}$ that maximizes the likelihood. In other words, we are looking for the vector $\mathbf{w}$ that makes the dataset D the "most plausible", or, more formally, that maximizes the probability of observing that particular D. In the frequentist setting, it is common to estimate $\mathbf{w}$ as:

$$\mathbf{w}_{ML} = argmax_{\mathbf{w}} p(D|\mathbf{w}) \tag{3.55}$$

that is, as the vector $\mathbf{w}_{ML}$ that maximizes the likelihood (hence the name Maximum Likelihood).

N.B. If information about the prior $p(\mathbf{w})$ is available, frequentist approaches can also act on the posterior. In this case we speak of **Maximum a Posterior (MAP)**. We will see this later.

Before deriving the solution to this optimization problem, it is important to introduce some concepts that will be useful to us.

Log-likelihood

In many cases, instead of using likelihood, it is easier to work with **log-likelihood**:

$$\log p(D|\mathbf{w}) \tag{3.56}$$

Since the logarithm is a monotone function, using $\log p(D|\mathbf{w})$ or directly $p(D|\mathbf{w})$ makes no difference with respect to the maximization process. However, using log-likelihood offers two advantages:

- Many of the probability distributions we will be working with can be expressed in exponential form. Consequently, calculating their logarithm allows us to greatly simplify the derivations;
- In almost all cases considered in this course, we will assume that the samples of the dataset D used for training are **independent and identically distributed** or, more briefly **i.i.d.**. In this case, their joint distribution can be expressed as the product of the distributions of the individual samples. Applying the logarithm, we will then have a summation of probability logarithms, which is much easier to handle:

$$\log p(\{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N-1)}|\mathbf{w}\}) = \log \prod_{i=0}^{N-1} p(\mathbf{x}^{(i)}|\mathbf{w}) = \sum_{i=0}^{N-1} \log p(\mathbf{x}^{(i)}|\mathbf{w}) \tag{3.57}$$

N.B. In the Machine Learning literature, we often refer to the **Error Function** or **Loss Function**, defined as the **negative** log-likelihood:

$$L(\mathbf{w}) = -\text{log-likelihood} = -\log p(D|\mathbf{w}) \tag{3.58}$$

Gaussian Distribution

The Gaussian distribution (see Figure 3.5) is certainly one of the most used in the world of Machine Learning. In the one-dimensional case it is given by the following expression:

$$N(x|\mu, \sigma^2) = N(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{3.59}$$

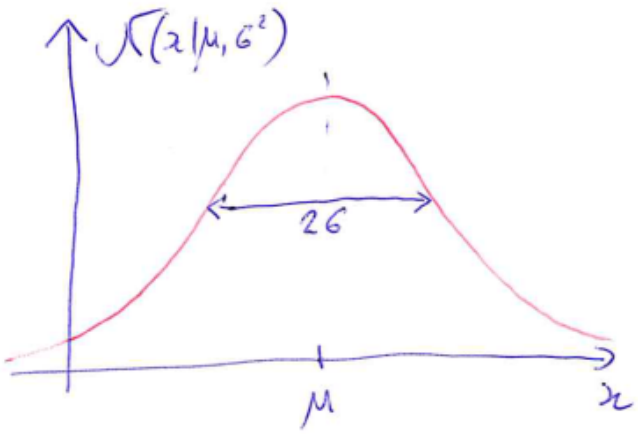


Figure 3.5: The Gaussian distribution

where μ, σ e $\sigma * 2$ are the mean, standard deviation and variance of the Gaussian, respectively. We often use the **precision** $\beta = \frac{1}{\sigma^2}$ in place of the variance. It can be demonstrated that:

$$E[x] = \int_{-\infty}^{\infty} N(x|\mu, \sigma^2) x dx = \mu \qquad (3.60)$$

$$\text{con } x \sim N(\mu, \sigma^2)$$

and since:

$$E[x^2] = \int_{-\infty}^{\infty} N(x|\mu, \sigma^2) x^2 dx = \mu^2 + \sigma^2 \qquad (3.61)$$

we can also derive:

$$Var[x] = E[x^2] - E^2[x] = \mu^2 + \sigma^2 - \mu^2 = \sigma^2 \text{con } x \sim N(\mu, \sigma^2)$$

In addition, the fundamental probability axioms hold:

$$\int_{-\infty}^{\infty} N(x|\mu, \sigma^2) dx = 1 \qquad (3.62)$$

$$N(x|\mu, \sigma^2) \geq 0 \ , \forall x \qquad (3.63)$$

In the multivariate case, the expression of the Gaussian becomes:

$$N(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma})) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})} \qquad (3.64)$$

where $\mathbf{x}$ and $\boldsymbol{\mu}$ are vectors of d dimensions and **Sigma** is the **Covariance** matrix of dimension $d \times d$, symmetric and positive semidefinite.

We now try to apply the Maximum Likelihood Estimation framework to estimate

the parameters of the Gaussian distribution from a dataset of N i.i.d. samples. Given $D = \{x^0, x^1, \ldots, x^{N-1}\}$ with $x^i \sim N(\mu, \sigma^2)$, we can write the likelihood as:

$$p(\mathbf{X}|\mu, \sigma^2) = \prod_{i=0}^{N-1} N(x^{(i)}|\mu, \sigma^2) \tag{3.65}$$
$$\text{con } \mathbf{X} = D = \{x^{(0)}, x^{(1)}, \ldots, x^{(N-1)}\}$$

We use the Maximum Likelihood Estimation approach to find the parameters $\mathbf{w}$, which in this case are μ and σ^2:

$$\begin{aligned} l(\mathbf{w}) = l(\mu, \sigma^2) = \log p(\mathbf{X}|\mu, \sigma^2) &= \sum_{i=0}^{N-1} \log N(x^{(i)}|\mu, \sigma^2) \\ &= -\frac{1}{2\sigma^2} \sum_{i=0}^{N-1} (x^{(i)} - \mu)^2 - \frac{N}{2} \log \sigma^2 - \frac{N}{2} \log(2\pi) \end{aligned} \tag{3.66}$$

After computing the log-likelihood $l(\mathbf{w})$ we can maximize it with respect to the parameters μ and σ^2. Specifically, we calculate:

$$\frac{\partial l(\mu, \sigma^2)}{\partial \mu} = 0 \ , \ \frac{\partial l(\mu, \sigma^2)}{\partial \sigma^2} = 0 \tag{3.67}$$

Let's start with the first equation:

$$\begin{aligned} \frac{\partial l(\mu, \sigma^2)}{\partial \mu} &= \frac{\partial}{\partial \mu} \left[-\frac{1}{2\sigma^2} \sum_{i=0}^{N-1} (x^{(i)} - \mu)^2 \right] = +\frac{1}{\sigma^2} \sum_{i=0}^{N-1} (x^{(i)} - \mu) = 0 \\ &\implies \sum_{i=0}^{N-1} x^{(i)} - \sum_{i=0}^{N-1} \mu = 0 \implies \mu \sum_{i=0}^{N-1} 1 = \sum_{i=0}^{N-1} x^{(i)} \\ &\implies \mu_{ML} = \frac{1}{N} \sum_{i=0}^{N-1} x^{(i)} \end{aligned} \tag{3.68}$$

That is, the Maximum Likelihood solution for the Gaussian mean is equal to the definition of sample mean caculated over the N samples of the D dataset.

For the variance, instead, we have:

$$\frac{\partial l(\mu, \sigma^2)}{\partial \sigma^2} = \frac{\partial}{\partial \sigma^2} \left[-\frac{1}{2\sigma^2} \sum_{i=0}^{N-1} (x^{(i)} - \mu)^2 - \frac{N}{2} \log \sigma^2 \right] \tag{3.69}$$

To calculate it, we use the estimated value for the mean μ_{ML}:

$$\frac{\partial l(\mu, \sigma^2)}{\partial \sigma^2} = -\frac{N}{2}\frac{1}{\sigma^2} + \frac{1}{2}\sum_{i=0}^{N-1}(x^{(i)} - \mu_{ML})^2\frac{1}{\sigma^4} = 0 \tag{3.70}$$

$$\implies \frac{1}{\cancel{2}}\sum_{i=0}^{N-1}(x^{(i)} - \mu_{ML})^2 = \frac{N}{\cancel{2}}\sigma^2$$

$$\implies \sigma^2_{ML} = \frac{1}{N}\sum_{i=0}^{N-1}(x^{(i)} - \mu_{ML})^2$$

which corresponds to the **Sample Variance**.

Important! The sample variance is actually an underestimate of the true variance. While the Maximum Likelihood estimator for the mean is correct (and is therefore called **Unbiased**), the estimator for the variance has a **Bias** instead:

$$E[\mu_{ML}] = \mu \tag{3.71}$$

$$E[\sigma^2_{ML}] = (\frac{N-1}{N})\sigma^2 \tag{3.72}$$

to correct for it, we must then estimate the variance as follows:

$$\tilde{\sigma^2} = \frac{N}{N-1}\sigma^2_{ML} = \frac{1}{N-1}\sum_{i=0}^{N-1}(x^{(i)} - \mu_{ML})^2 \tag{3.73}$$

N.B. 1 For very large N the estimator σ^2_{ML} approximates the true variance well without the need for corrections.

N.B. 2 This problem highlights the limitations of the Maximum Likelihood framework. The underestimation is indeed related to the use of μ_{ML} instead of the true mean μ (see demonstration below), but, more generally, this is an example of how the MLE framework is subject to **Overfitting** on the dataset. This problem becomes more significant in models with multiple parameters.

Proof of $E[\sigma^2_{ML}] = \frac{N-1}{N}\sigma^2$

$$E[\sigma^2_{ML}] = E[\frac{1}{N}\sum_{i=0}^{N-1}(x^{(i)} - \mu_{ML})^2] = \frac{1}{N}\left[\sum_{i=0}^{N-1}((x^{(i)})^2 - 2x^{(i)}\mu_{ML} + \mu^2_{ML})\right]$$

$$= \frac{1}{N}E\left[\sum_{i=0}^{N-1}(x^{(i)})^2 \underbrace{- \sum_{i=0}^{N-1} 2x^{(i)}\mu_{ML}}_{-2N\mu_{ML}\mu_{ML}} + \underbrace{\sum_{i=0}^{N-1}\mu^2_{ML}}_{N\sum_{i=0}^{N-1}\mu^2_{ML}}\right] \tag{3.74}$$

where the second term of the equation has been simplified by remembering that $\mu_{ML} = \frac{1}{N}\sum_{i=0}^{N-1} x^{(i)}$, per cui $\sum_{i=0}^{N-1} x^{(i)} = N\mu_{ML}$.We then have:

$$= \frac{1}{N}E[\sum_{i=0}^{N-1}(x^{(i)})^2] - E[\mu^2_{ML}] = \frac{1}{N}\sum_{i=0}^{N-1}E[(x^{(i)})^2] - E[\mu^2_{ML}]$$
$$= \frac{1}{N}NE[(x^{(i)})^2] - E[\mu^2_{ML}] = E[(x^{(i)})^2] - E[\mu^2_{ML}] \tag{3.75}$$

where $E[\sum_{i=0}^{N-1} = \sum_{i=0}^{N-1} E[(x^{(i)})^2]$ since the expected value of the sum of random variables is equal to the sum of the expected values of the individual variables. Recalling at this point that $Var[x] = E[x^2] - E^2[x]$ and substituting μ_{ML} for x we have:

$$Var[\mu_{ML}] = E[\mu^2_{ML}] - E^2[\mu_{ML}] = E[\mu^2_{ML}] - \mu^2 \tag{3.76}$$

where in the last step we used $E[\mu_{ML}] = \mu$ since μ_{ML} is an unbiased estimator. We therefore have:

$$E[\mu^2_{ML}] = Var[\mu_{ML}] + \mu^2 \tag{3.77}$$

Substituting the (3.77) and $E[(x^{(i)})^2] = \sigma^2 + \mu^2$ (N.B. σ and μ are the true parameters of the distribution, not known!) into the (3.75) we have:

$$E[\sigma^2_{ML}] = \sigma^2 + \mu^2 - Var[\mu_{ML}] - \mu^2 = \sigma^2 - Var[\mu_{ML}]$$
$$= \sigma^2 - Var[\frac{1}{N}\sum_{i=0}^{N-1} x^{(i)}] \tag{3.78}$$

Recalling that, in general, by properties of variance:

$$Var[aX] = a^2Var[X] \text{ con } a \text{ è una costante} \tag{3.79}$$
$$Var[X + Y] = Var[X] + Var[Y] \tag{3.80}$$

we finally have:

$$E[\sigma^2_{ML}] = \sigma^2 - \left(\frac{1}{N}\right) Var[\sum_{i=0}^{N-1} x^{(i)}] = \sigma^2 - \left(\frac{1}{N}\right) NVar[x^{(i)}]$$
$$= \sigma^2 - \frac{1}{N}\sigma^2 = \frac{N-1}{N}\sigma^2 \tag{3.81}$$

N.B. This result emerges from using μ_{ML} instead of μ. By redoing the calculations with μ it is easy to verify that the estimator would be unbiased.

3.6.2 MLE for Curve Fitting

We now have all the elements to define in probabilistic terms the polynomial interpolation problem (curve fitting). Suppose then that we have a Dataset $\mathcal{D}$ with N samples $\mathbf{X} = \{x^{(0)}, x^{(1)}, \ldots, x^{(N-1)}\}$. Each sample consists of a single feature and is associated with an output (label) $\mathbf{t} = \{t^{(0)}, t^{(1)}, \ldots, t^{(N-1)}\}$, with $\mathbf{t}$ vector.

In practice, we never have access to true $y^{(i)}$ outputs, but instead have available $t^{(i)}$ measures of $y^{(i)}$ affected by Gaussian noise with zero mean and variance σ^2_ϵ:

$$t^{(i)} = h_{\mathbf{w}}(x^{(i)}, \mathbf{w}) + \epsilon \tag{3.82}$$
$$\text{with } \epsilon \sim N(0, \sigma^2_\epsilon) = N(0, \beta^{-1})$$
$$\text{and } \beta = \frac{1}{\sigma^2_\epsilon}$$

where the function $h_{\mathbf{w}}(x^{(i)}, \mathbf{w})$ is our **hypothesis function** by which we attempt to estimate the true process that generates the data unknown to us $y(\cdot)$. The function $h_{\mathbf{w}}$ is parameterized with $\mathbf{w}$ and its form depends on the assumptions of the specific Machine Learning algorithm (as we will see throughout the course). Once we have found the optimal $\mathbf{w}$, we say that $h_{\mathbf{w}}$ is our **Model** describing the process we want to estimate. Clearly, the closer the choice of $h_{\mathbf{w}}$ is to reality, the more accurate the model will be.

N.B. We choose the form of the hypothesis function $h_{\mathbf{w}}$. Consequently, in Machine Learning, our a priori knowledge and experience about a certain process we intend to model are crucial. Thanks to them, we can choose the type of ML algorithm (and consequently the form of the hypothesis function $h_{\mathbf{w}}$) most appropriate for the problem at hand.

$t^{(i)}(x^{(i)})$ is given by the sum of a deterministic function $h_{\mathbf{w}}(x^{(i)}, \mathbf{w})$ and a random variable, and, therefore, is itself a random variable, whose mean is given precisely by $y(x^{(i)}, \mathbf{w})$ (since noise has zero mean) and whose precision (or variance) is that of noise

(β^{-1}):

$$p(t^{(i)}|x^{(i)}, \mathbf{w}, \beta) = N(t^{(i)}|h_{\mathbf{w}}(x^{(i)}, \mathbf{w}), \beta^{-1}) \tag{3.83}$$

This is the pdf for a single sample. We now use the entire dataset to express likelihood:

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) \underbrace{=}_{i.i.d.} \prod_{i=0}^{N-1} N(t^{(i)}|h_{\mathbf{w}}(x^{(i)}, \mathbf{w}), \beta^{-1}) \tag{3.84}$$

where the i.i.d assumption for samples allowed us to express the joint as the product of the individual pdfs. We now calculate the log-likelihood.

$$l(\mathbf{w}, \beta) = \log p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = -\frac{\beta}{2} \sum_{i=0}^{N-1} (h_{\mathbf{w}}(x^{(i)}, \mathbf{w}) - t^{(i)})^2 + \frac{N}{2} \log \beta - \frac{N}{2} \log(2\pi) \tag{3.85}$$

To find the ML solution, namely $\mathbf{w}_{ML}$, we maximize the log-likelihood with respect to $\mathbf{w}$:

$$\max_{\mathbf{w}} \left[\frac{\beta}{2} \sum_{i=0}^{N-1} (h_{\mathbf{w}}(x^{(i)}, \mathbf{w}) - t^{(i)})^2 + \underbrace{\cancel{\frac{N}{2} \log \beta} - \cancel{\frac{N}{2} \log(2\pi)}}_{\text{do not depend on } \mathbf{w}} \right] \tag{3.86}$$

$$\nabla_{\mathbf{w}} l(\mathbf{w}, \beta) = 0 \implies \nabla_{\mathbf{w}} \left[\frac{\beta}{2} \sum_{i=0}^{N-1} (h_{\mathbf{w}}(x^{(i)}, \mathbf{w}) - t^{(i)})^2 \right] = 0 \tag{3.87}$$

The calculation of the gradient depends on the form of $h_{\mathbf{w}}$ (our hypothesis function).

We can also note that:

$$\max_{\mathbf{w}} l(\mathbf{w}, \beta) = \min_{\mathbf{w}} \; -l(\mathbf{w}, \beta) = \min_{\mathbf{w}} L(\mathbf{w}) \tag{3.88}$$

$$\text{dove } L(\mathbf{w}) = \frac{1}{2} \sum_{i=0}^{N-1} (h_{\mathbf{w}}(x^{(i)}, \mathbf{w}) - t^{(i)})^2$$

i.e., , maximizing the log-likelihood is equivalent to minimizing **the error function**.

> The error function we have derived is the same one we chose in the previous chapter (see equation (2.8)). Consequently, the quadratic loss was not a random choice, but emerges from the use of the Maximum likelihood framework and the assumption that the labels in our dataset are affected by Gaussian noise.

The solution to the optimization problem in Eqn. (3.88) provides the optimal set of parameters $\mathbf{w}_{ML}$. With these, we can also compute β_{ML} by maximizing the (3.85) with

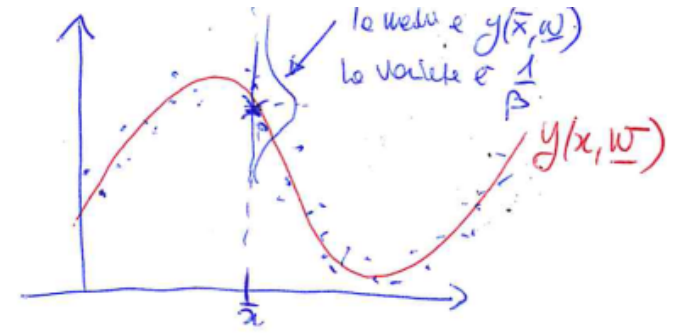


Figure 3.6: The Gaussian predictive distribution

respect to β.

$$\frac{\partial l(\mathbf{w}_{ML}, \beta)}{\partial \beta} = 0 \implies \frac{\partial}{\partial \beta}\left[\frac{\beta}{2}\sum_{i=0}^{N-1}(h_{\mathbf{w}}(x^{(i)}, \mathbf{w}_{ML}) - t^{(i)})^2 + \frac{N}{2}\log\beta\right] = 0 \tag{3.89}$$

from which we obtain:

$$\begin{aligned}\frac{N}{2}\frac{1}{\beta_{ML}} - \frac{1}{2}\sum_{i=0}^{N-1}(h_{\mathbf{w}}(x^{(i)}, \mathbf{w}_{ML}) - t^{(i)})^2 = 0 \\ \beta_{ML} = \left[\frac{1}{N}\sum_{i=0}^{N-1}(h_{\mathbf{w}}(x^{(i)}, \mathbf{w}_{ML}) - t^{(i)})^2\right]^{-1}\end{aligned} \tag{3.90}$$

Once we have identified $\mathbf{w}_{ML}$ and β_{ML} we can define the **Predictive Distribution**:

$$p(t|\mathbf{x}, \mathbf{w}_{ML}, \beta_{ML}) = N(t|h_{\mathbf{w}_{ML}}(\mathbf{x}, \mathbf{w}_{ML}), \beta_{ML}) \tag{3.91}$$

We are then able, at this point, to describe the output via a probability distribution. So we do not have a simple point estimate, but we know the probability of every possible value that the output can take on the basis of that particular input (see Figure 3.6). To obtain a point estimate, it is sufficient to calculate the expected value of this distribution, which is precisely the hypothesis function estimated with respect to the particular input under consideration and with the optimal parameter vector:

$$E[t] = E[N(t|h_{\mathbf{w}_{ML}}(\mathbf{x}, \mathbf{w}_{ML}), \beta_{ML})] = h_{\mathbf{w}_{ML}}(\mathbf{x}, \mathbf{w}_{ML}) \tag{3.92}$$

3.6.3 Maximum a Posteriori (MAP) for Curve Fitting

Let us now try to take a step towards a more Bayesian approach. We then assume that we have a priori knowledge about the parameters. In particular, we assume that we have

a Gaussian prior on $\mathbf{w}$:

$$p(\mathbf{w}|\alpha) = N(\mathbf{w}|0, \alpha^{-1}I) \tag{3.93}$$

$$= \left(\frac{\alpha}{2\pi}\right)^{\frac{D+1}{2}} \exp^{-\frac{\alpha}{2}\mathbf{w}^T\mathbf{w}} \tag{3.94}$$

The α parameter is a hyper-parameter that controls the accuracy of the prior.

We can at this point calculate the posterior using Bayes and maximize it (hence the name Maximum a Posteriori):

$$p(\mathbf{w}|\mathbf{X}, \mathbf{t}, \alpha, \beta) \propto p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta)p(\mathbf{w}|\alpha) \tag{3.95}$$

Always using the logarithm, in this case the function to be maximized is as follows:

$$\frac{\beta}{2}\sum_{i=0}^{N-1}(h_{\mathbf{w}}(x^{(i)}, \mathbf{w}) - t^{(i)})^2 + \frac{\alpha}{2}\mathbf{w}^T\mathbf{w} \tag{3.96}$$

From this expression we can see how taking the prior into account corresponds to performing an L2 regularization on the loss function. Intuitively, the regularizing effect of the prior is related to the constraint that our a priori knowledge imposes during optimization, preventing our model from chasing the observed data too much and imposing it, instead, to keep the weights small.

N.B. The regularization parameter λ corresponds to $\frac{\alpha}{\beta}$.

N.B. 2 The MAP result is **not** a distribution, but a point estimate, as with MLE. In this sense MAP is still within the more frequentist context and is not yet a fully Bayesian treatment, which instead involves modeling the output with a pdf. To this end, it is necessary to consistently apply the sum and product rule to compute al **posterior predictive distribution**, integrating with respect to all possible values of $\mathbf{w}$:

$$p(t|\mathbf{w}, \mathbf{X}, \mathbf{t}) = \int p(t|\mathbf{w}, \mathbf{w})p(\mathbf{w}|\mathbf{X}, \mathbf{t})d\mathbf{w} \tag{3.97}$$

Chapter 4

Linear Regression

We now begin to introduce supervised learning algorithms, starting with regression approaches. The polynomial curve fitting that we have seen in previous chapters is an example of a regression approach and, in particular, is a special case of a larger class of functions called **Linear Regression Models**, which share the characteristic of being linear functions with respect to the parameters $\mathbf{w}$.

In its simplest version, the **Linear Regression** is also linear with respect to the input variables and the hypothesis function, in this case, is defined as follows:

$$h_{\mathbf{w}}(\mathbf{x}^{(i)}, \mathbf{w}) = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} + \dots w_{R-1} x_{R-1}^{(i)} = w_0 + \sum_{j=1}^{R-1} w_j x_j^{(i)} \tag{4.1}$$

where $\mathbf{x}^T = [x_1, \dots, x_{R-1}]$ is the feature vector (the fact that we used R in the summation and not $D-1$ will become clear in a moment), while $\mathbf{w}$ is the vector of learnable parameters. w_0 is called the "bias" term.

We make two changes to this model to make it more general:

- Instead of considering the input features as they are, we consider a transformation of them $\phi_j(\mathbf{x})$ called **Basis Function**:

$$h_{\mathbf{w}}(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{D-1} w_j \phi_j(\mathbf{x}) \tag{4.2}$$

  In general, basis functions are functions of the entire input vector $\mathbf{x}$ and their number can be different from the number of features, *i.e.,* $D \neq R$. Moreover, basis functions are hand-engineered functions and their number is also defined by the designer.

- To make the formula (4.1) more compact, we assume that the bias term w_0 is

multiplied by the following basis function: $\phi_0(\mathbf{x}) = 1$. In this way, we can write:

$$h_{\mathbf{w}}(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{D-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) \tag{4.3}$$

where D refers to the number of features and $\boldsymbol{\phi}(\mathbf{x}) = [\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \ldots, \phi_{D-1}(\mathbf{x})]^T$

4.1 Relation to linear interpolation

The linear interpolation that we have seen in the previous chapters is a particular case of linear regression model, characterized by an input vector with a single feature and with basis functions defined as $\phi_j(x) = x^j$. It is possible to extend the polynomial interpolation also to multi-dimensional feature vectors, but we must be careful to increase the number of features as a result of the expansion:

$$\begin{aligned} \mathbf{x} = [x_1, x_2]^T &\overset{M=2}{\rightarrow} [x_1^2, x_2^2, x_1 x_2, x_1, x_2]^T \\ &\overset{M=3}{\rightarrow} [x_1^3, x_2^3, x_1^2 x_2^2, x_1^2 x_2, x_1 x_2^2, x_1 x_2, x_1, x_2]^T \end{aligned}$$

where each element of the vectors to the right of the arrows are basis functions.

4.2 Basis Functions

Depending on the problem at hand, there are many ways to choose the basis functions ϕ_j. Among the most used functions are sinusoidal functions (in this case we are actually doing a Fourier series expansion), or wavelet functions (similar to sinusoids, but limited in both time and frequency). In the course, we will often use polynomial expansions instead (as we have seen before). it is important to emphasize that the basis functions have a global character, *i.e.,* , their effect is applied over the entire domain of $\mathbf{x}$. If, however, it is necessary to model the nonlinear behavior of the output with respect to the input only locally, other arrangements must be used (*e.g.,* **Spline Interpolation**).

4.3 MLE for Linear Regression

We now apply the Maximum Likelihood Estimation framework to linear regression. To do so, we introduce the three assumptions that linear regression models make:

- the unknown process can be captured by a hypothesis function that is linear with respect to the parameters (and potentially non-linear with respect to the input via basis functions):

$$h_{\mathbf{w}}(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{D-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) \tag{4.4}$$

- The labels we have access to are a noisy version of the true output of $h_{\mathbf{w}}(\mathbf{x}, \mathbf{w})$ and, in particular, are affected by additive Gaussian-type noise with zero mean and β precision:
$$t = h_{\mathbf{w}}(\mathbf{x}, \mathbf{w}) + \epsilon \text{ with } \epsilon \sim N(0, \beta^{-}1) \tag{4.5}$$
- The samples in our $\mathcal{D}$ dataset are i.i.d.

As we have seen for the curve fitting case, t is a random variable described by the following Gaussian distribution:

$$p(t|\mathbf{x}, \mathbf{w}, \beta) = N(t|h_{\mathbf{w}}(\mathbf{x}, \mathbf{w}), \beta^{-1}) \tag{4.6}$$

Since we are within the MLE framework, we are only interested in a point estimate, which corresponds to the expected value of this distribution:

$$E[t|\mathbf{x}] = h_{\mathbf{w}}(\mathbf{x}, \mathbf{w}) \tag{4.7}$$

> Although we will consider the scalar t case (*i.e.,* linear regression models with a single output) it is possible to extend the reasoning to the **multi-output** case.

Let's suppose we have a dataset $\mathcal{D} = \{(\mathbf{x}^{(0)}, t^{(0)}), (\mathbf{x}^{(1)}, t^{(1)}), \ldots, (\mathbf{x}^{(N-1)}, t^{(N-1)})\}$ and represent the samples and their labels in matrix and vector form, respectively:

$$\mathbf{X} = \{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N-1)}\} = \begin{pmatrix} x_{0,0} & x_{0,1} & \cdots & x_{0,D-1} \\ x_{1,0} & x_{1,1} & \cdots & x_{1,D-1} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N-1,0} & x_{N-1,1} & \cdots & x_{N-1,D-1} \end{pmatrix} \in \mathbb{R}^{N \times D}$$

$$\mathbf{t} = \begin{pmatrix} t_0 \\ t_1 \\ \vdots \\ t_{N-1} \end{pmatrix} \in \mathbb{R}^{N \times 1} \tag{4.8}$$

$\mathbf{X}$ is called **Design Matrix** and has N rows (equal to the number of samples and D columns (equal to the number of features).

We can at this point write the likelihood:

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) \overset{i.i.d.}{=} \prod_{i=0}^{N-1} N(t^{(i)}|h_{\mathbf{w}}(\mathbf{x}^{(i)}, \mathbf{w}), \beta^{-1}) = \prod_{i=0}^{N-1} N(t^{(i)}|\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^{(i)}), \beta^{-1}) \tag{4.9}$$

Where for the first equality we used the assumption of i.i.d. samples. Calculating the

log-likelihood we obtain:

$$\log p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = \sum_{i=0}^{N-1} \log N(t^{(i)}|\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^{(i)}), \beta^{-1})$$
$$= \frac{N}{2} \log \beta - \frac{N}{2} \log(2\pi) - \beta L(\mathbf{w}) \tag{4.10}$$
$$\text{dove } L(\mathbf{w}) = \frac{1}{2} \sum_{i=0}^{N-1} (t^{(i)} - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^{(i)}))^2 \tag{4.11}$$

The problem expressed by 4.11 is called **Least square problem**.

To find the MLE solution we compute the gradient of this loss with respect to $\mathbf{w}$:

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = \nabla_{\mathbf{w}} \left[\frac{1}{2} \sum_{i=0}^{N-1} (t^{(i)} - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^{(i)}))^2 \right]$$
$$= \frac{1}{2} \sum_{i=0}^{N-1} \nabla_{\mathbf{w}} \left[(t^{(i)} - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^{(i)}))^2 \right] = \sum_{i=0}^{N-1} -(t^{(i)} - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^{(i)})) \cdot (\boldsymbol{\phi}(\mathbf{x}^{(i)})) \tag{4.12}$$

> The gradient of a function f with respect to a vector $\mathbf{w}$ is itself a vector of components $[\frac{\partial f}{\partial w_0}, \frac{\partial f}{\partial w_1}, \ldots, \frac{\partial f}{\partial w_{D-1}}]^T = \nabla_{\mathbf{w}} f$. As a consequence, $\nabla_{\mathbf{w}} L(\mathbf{w})$ is a vector.

From 4.12 it is possible to compute the optimal set of parameters $\mathbf{w}_{ML}$ by imposing $\nabla_{\mathbf{w}} L(\mathbf{w}) = 0$. However, it is simpler to do this, both as a proof and as an implementation, by expressing the loss in vector form:

$$L(\mathbf{w}) = \frac{1}{2} (\mathbf{t} - \boldsymbol{\Phi}\mathbf{w})^T (\mathbf{t} - \boldsymbol{\Phi}\mathbf{w}) \tag{4.13}$$

dove

$$\boldsymbol{\Phi} = \begin{pmatrix} \phi_0(\mathbf{x}^{(0)}) & \phi_1(\mathbf{x}^{(0)}) & \ldots & \phi_{D-1}(\mathbf{x}^{(0)}) \\ \phi_0(\mathbf{x}^{(1)}) & \phi_1(\mathbf{x}^{(1)}) & \ldots & \phi_{D-1}(\mathbf{x}^{(1)}) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(\mathbf{x}^{(N-1)}) & \phi_1(\mathbf{x}^{(N-1)}) & \ldots & \phi_{D-1}(\mathbf{x}^{(N-1)}) \end{pmatrix} \tag{4.14}$$

Where the fact that 4.13 corresponds to 4.11 can be seen by noting that:

$$\mathbf{t} - \mathbf{\Phi w} = \begin{pmatrix} t^{(0)} - \boldsymbol{\phi}(\mathbf{x}^{(0)})^T\mathbf{w} \\ t^{(1)} - \boldsymbol{\phi}(\mathbf{x}^{(1)})^T\mathbf{w} \\ \vdots \\ t^{(N-1)} - \boldsymbol{\phi}(\mathbf{x}^{(N-1)})^T\mathbf{w} \end{pmatrix} = \begin{pmatrix} t^{(0)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(0)}) \\ t^{(1)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(1)}) \\ \vdots \\ t^{(N-1)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(N-1)}) \end{pmatrix} \tag{4.15}$$

e quindi:

$$\begin{aligned} &\frac{1}{2}(\mathbf{t} - \mathbf{\Phi w})^T(\mathbf{t} - \mathbf{\Phi w}) = \\ &= \frac{1}{2}\left((t^{(0)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(0)})) \dots (t^{(N-1)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(N-1)})\right) \begin{pmatrix} t^{(0)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(0)}) \\ \vdots \\ t^{(N-1)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(N-1)}) \end{pmatrix} \\ &= \frac{1}{2}\left((t^{(0)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(0)}))^2 + \dots + (t^{(N-1)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(N-1)}))^2\right) \\ &= \frac{1}{2}\sum_{i=0}^{N-1}(t^{(i)} - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}^{(i)}))^2 \end{aligned} \tag{4.16}$$

We can then calculate the gradient of the 4.13 and equalize it to zero:

$$\begin{aligned} \nabla_{\mathbf{w}} L(\mathbf{w}) = \nabla_w \left[\frac{1}{2}(\mathbf{t} - \mathbf{\Phi w})^T(\mathbf{t} - \mathbf{\Phi w})\right] = 0 \\ \implies \nabla_w \left[\frac{1}{2}(\mathbf{t}^T\mathbf{t} \quad \underbrace{-\mathbf{t}^T\mathbf{\Phi w} - \mathbf{w}^T\mathbf{\Phi}^T\mathbf{t}}_{\text{sono scalari e sono uguali}} \quad +\mathbf{w}^T\mathbf{\Phi}^T\mathbf{\Phi w})\right] = 0 \\ \implies \frac{1}{2}\left[\underbrace{\cancel{\nabla_w(\mathbf{t}^T\mathbf{t})}}_{\text{non dipende da } \mathbf{w}} - 2\nabla_w(\mathbf{w}^T\mathbf{\Phi}^T\mathbf{t}) + \nabla_w(\mathbf{w}^T\mathbf{\Phi}^T\mathbf{\Phi w})\right] = 0 \\ \implies \frac{1}{2}\left[-2\mathbf{\Phi}^T\mathbf{t} + 2\mathbf{w}^T\mathbf{\Phi}^T\mathbf{\Phi}\right] = 0 \\ \implies \frac{1}{2}\left[-2\mathbf{\Phi}^T\mathbf{t} + 2\mathbf{\Phi}^T\mathbf{\Phi w}\right] = 0 \\ \implies \mathbf{\Phi}^T\mathbf{\Phi w} = \mathbf{\Phi}^T\mathbf{t} \\ \implies \mathbf{w}_{ML} = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathbf{t} \end{aligned} \tag{4.17}$$

where the 4.17 is called the **Normal Equation** and $(\mathbf{\Phi}^T\mathbf{\Phi})^{-1}$ is the inverse of $\mathbf{\Phi}^T\mathbf{\Phi}$, or the **pseudo-inverse** in case $\mathbf{\Phi}^T\mathbf{\Phi}$ is singular.

It can be seen that the normal equation is a closed form solution to the MLE problem applied to linear regression. it can also be seen that the computational complexity is dominated by the inverse of the matrix $\mathbf{\Phi}^T\mathbf{\Phi}$.

4.4 Hat Matrix

Since

$$\hat{t}^{(i)} = h_{\mathbf{w}_{ML}}(\mathbf{x}^{(i)}, \mathbf{w}_{ML}) = \mathbf{w}_{ML}^T \boldsymbol{\phi}(\mathbf{x}^{(i)}) \tag{4.18}$$

where by $\hat{t}^{(i)}$ we denote the estimate of $t^{(i)}$ provided by the model with the optimal weights $\mathbf{w}_{ML}$, we can write this relation in vector form for all samples in the training dataset:

$$\hat{\mathbf{t}} = \boldsymbol{\Phi}\mathbf{w}_{ML} = \underbrace{\boldsymbol{\Phi}(\boldsymbol{\Phi}^T\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^T}_{\mathbf{H}}\mathbf{t} \tag{4.19}$$

H is called the **Hat matrix** because it is the matrix that "caps" the labels **t**. In practice, it is the matrix that, given the labels **t**, allows us to compute $\hat{\mathbf{t}}$, i.e., the estimate on the training set of our model. As we will see, this will allow us to understand how well the hung model "explains" the traininig data.

4.5 Geometric interpretation of Linear Regression

The obtained MLE solution also has geometric significance. The vector **t** of labels lies on a vector space of dimension N (since $\mathbf{t} \in \mathbb{R}^{N\times 1}$). Each basis function $\phi_j(\mathbf{x}^{(i)})$, evaluated on each sample of the dataset, can be represented by a vector $\boldsymbol{\gamma}_j(\mathbf{x}^{(i)})$ (one of the columns of $\boldsymbol{\Phi}$), also composed of N components and thus belonging to the same vector space as **t**. If the basis functions are fewer in number than the samples, i.e. $D < N$, then the D vectors $\boldsymbol{\gamma}_j(\mathbf{x}^{(i)})$ span a subspace of the one where **t** lies of dimension D. The vector of estimates $\hat{\mathbf{t}}$ also has N components, but since it is a linear combination of the of the basis functions $\boldsymbol{\gamma}_j(\mathbf{x}^{(i)})$ it must necessarily lie in the subspace of the latter (see Figure 4.1). Consequently, what the MLE framework tries to do is to minimize the Euclidean distance between the vector **t**, free to move throughout N-dimensional space, and the estimate $\hat{\mathbf{t}}$, whose vector is bounded in the subspace of dimension D. In particular, the solution to the least square problem we have seen (given by the normal equation), is the orthogonal projection of **t** onto the subspace of basis functions (which guarantees the minimum distance). This last observation can be seen by noting that:

$$\mathbf{w}_{ML} = (\boldsymbol{\Phi}^T\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^T\mathbf{t} \implies \boldsymbol{\Phi}^T(\underbrace{\boldsymbol{\Phi}\mathbf{w}_{ML}}_{\hat{\mathbf{t}}} - \mathbf{t}) = 0 \tag{4.20}$$

as a consequence, $(\hat{\mathbf{t}} - \mathbf{t}) \perp \boldsymbol{\Phi}$

4.6 Gradient Descent

We have seen how the solution to the optimization problem arising from the MLE framework is provided by the normal equation in closed form. However, as we have already pointed out, the computational cost of this solution is highly dependent on the number

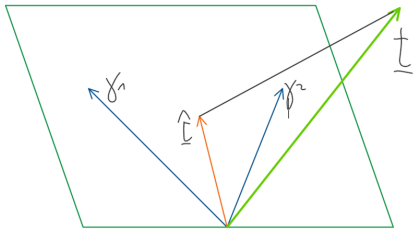


Figure 4.1: Geometric intepretation of the Linear Regression algorithm

of features (due to matrix inversion) and, for problems with a high number of features, it can be very expensive to compute. In addition, for other algorithms that we will see in the course, there is no closed form solution and it therefore makes sense to wonder if there are alternative procedures for minimizing loss functions. In particular we will see a method **iterative** called **Gradient descent** (in the case of minimum problems) or **Gradient Ascent** (in the case of maximum problems) for optimizing loss functions, applicable to linear regression, but also to many of the other algorithms we will see in the course.

In general, to find the optimal parameters of an ML model (hence also of Linear Regression (LR)), we minimize a loss function:

$$\min_{\mathbf{w}} L(h_{\mathbf{w}}(\mathbf{x}, \mathbf{w}), t) \tag{4.21}$$

This function is always positive or null and we have seen, as in the case of LR, that it can also be expressed as the opposite of log-likelihood in the case of a probabilistic formulation. Moreover, it usually depends on the samples in a dataset, so it is a function of the latter:

$$\min_{\mathbf{w}} \sum_{i=0}^{N-1} L(h_{\mathbf{w}}(\mathbf{x}^{(i)}, \mathbf{w}), t^{(i)}) \tag{4.22}$$

In the general case, it is not said that this optimization problem is convex and therefore there is a single global minimum. In the particular case of LR, however, we have seen that the form of the loss is quadratic:

$$\min_{\mathbf{w}} \sum_{i=0}^{N-1} \frac{1}{2}(t^{(i)} - h_{\mathbf{w}}(\mathbf{x}^{(i)}, \mathbf{w}))^2 \tag{4.23}$$

and can then be shown to have a single minimum (global minimum).

Instead of computing the solution in one step with the closed form, the alternative is to iteratively update the parameters $\mathbf{w}$ so that the loss function decreases with each

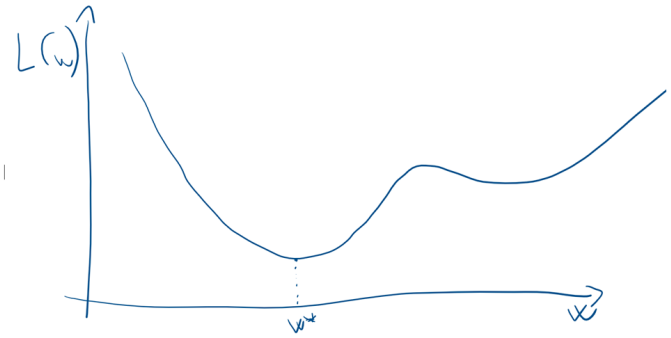


Figure 4.2: Global minumim of $L(w)$, a loss function that depends on a single scalar parameter w. Note that this function does not have a unique minimum (it is not quadratic as in the LR case).

iteration. We then look for a procedure that we can express as:

$$\mathbf{w}_{new} \leftarrow \mathbf{w}_{old} + \alpha\Delta\mathbf{w} \tag{4.24}$$

where α is hyper-parameter called **Learning Rate**, while $\Delta\mathbf{w}$ is a vector containing the increments or decrements to be applied to the current value of $\mathbf{w}$ to update it.

We must then choose how to compute $\Delta\mathbf{w}$. Since we know that the magnitude of the change in $L(\mathbf{w})$ with respect to $\mathbf{w}$ is measured by the gradient of $L(\mathbf{w})$ with respect to $\mathbf{w}$ ($\nabla_{\mathbf{w}}L(\mathbf{w})$) and since we want to minimize $L(\mathbf{w})$, it is logical to update the parameters as follows:

$$\mathbf{w}_{new} \leftarrow \mathbf{w}_{old} - \alpha\nabla_{\mathbf{w}}L(\mathbf{w}) \tag{4.25}$$

To understand the procedure graphically, we examine a loss function that depends on a single scalar parameter w, as shown in Figure 4.2 In the figure, w^* is the value to which corresponds the minimum of $L(w)$ that we would like to find (and which is therefore not known at the beginning of the optimization procedure). To iteratively find the optimal value, we choose a value for the parameter from which to randomly start $w_{init} \neq 0$ and compute the derivative $\frac{\partial L(w)}{\partial w}\Big|_{w_{init}}$ (see Figure 4.3). We then update the parameter according to the relationship:

$$\mathbf{w}_{new} \leftarrow \mathbf{w}_{old} - \alpha \left.\frac{\partial L(w)}{\partial w}\right|_{w_{init}} \tag{4.26}$$

Since the derivative at the point in the figure is negative and $\alpha > 0$, the value of the parameter will increase as we approach w^*. Proceeding iteratively with this process, ideally we will eventually reach w^* (see Figure 4.4).

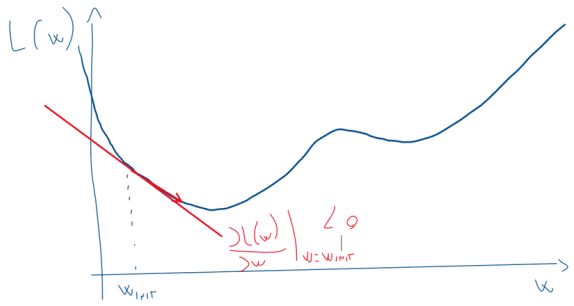


Figure 4.3: Initial value for the parameter w and derivative of the loss function computed at that point.

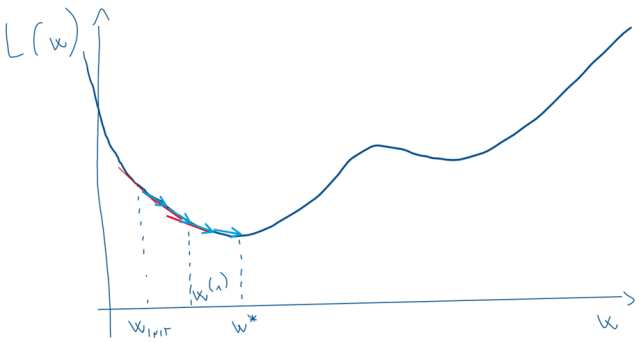


Figure 4.4: Iterative gradiente descent steps to reach w^*.



(a) Slow convergence

(b) Divergence

Figure 4.5: Effect of the hyper-parameter α on the gradient descent procedure.

The magnitude of the parameter shift is governed by the hyper-parameter α, so its role is very important. Since we would like to reach the minimum as soon as possible, we might be tempted to choose a very large α. However, with too high a learning rate we risk converging either very slowly, due to strong oscillations around the minimum (see Figure 4.5a), or diverging!(see Figure 4.5b). At learning rates that are too small, we risk instead taking a long time to reach convergence, or getting stuck in a local minimum with a very high error.

To evaluate the convergence of the optimization procedure, it is possible to compare the value of $L(\mathbf{w})$ during the various iterations. In particular, we can assume that the algorithm has reached convergence (or is stuck in a local minimum) if:

$$|L(\mathbf{w}_{new}) - L(\mathbf{w}_{old})| \leq \epsilon \tag{4.27}$$

with ϵ which constitutes a hyper-parameter, which can be set as small as desired.

Algorithm 1 Gradient Descent for Linear Regression

1: Repeat until convergence
2: $\quad \mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot GRAD_{L(\mathbf{w})}$
3: Where $GRAD_{L(\mathbf{w})}\{$
4: $\quad$ **for all** $i = 0, \ldots, N-1$ **do**
5: $\quad\quad grad_{L(\mathbf{w})} \leftarrow grad_{L(\mathbf{w})} + (-(t^{(i)} - \mathbf{w}^T \cdot \boldsymbol{\phi}(\mathbf{x}^{(i)})) \cdot \boldsymbol{\phi}(\mathbf{x}^{(i)}))$
6: $\quad$ **end for**
7: $\quad$ return $grad_{L(\mathbf{w})}$
8: $\}$

The gradient is accumulated for each sample in the dataset and the $\mathbf{w}$ update is performed using the cumulative gradient.

Gradient descent depends on 3 hyper-parameters: the learning rate α, the convergence threshold ϵ, and the number of maximum iterations T (used in case the convergence threshold is not reached).

4.6.1 Vectorization of Gradient Descent

Because gradient descent uses an iterative procedure, it is important that its implementation be as efficient as possible. In particular, if possible it is important to take advantage of the capabilities of computers to parallelize operations. Vector operations are an example of procedures that can be easily optimized and parallelized at a low level by linear algebra libraries. We therefore try to write the gradient update in vector form (avoiding the explicit cycle *for*).

Algorithm 2 Vectorized computation of gradient

1: $GRAD_{L(\mathbf{w})}\{$
2: $\quad grad_{L(\mathbf{w})} \leftarrow -\mathbf{\Phi}^T \cdot (\mathbf{t} - \mathbf{\Phi}\mathbf{w})$
3: $\}$

4.7 Feature Normalization

There is another fundamental difference between the closed-form solution with the normal equation and the iterative solution. In the first case, the solution is found correctly regardless of the values assumed by the features. In the second case, instead, if the features assume values in different intervals from each other, it is possible that the optimization procedure is negatively affected. In order to understand this aspect, let's consider, as an example, the dataset "House pricing prediction" and, in particular, let's consider the feature (Living Area) which expresses the metric of the house. (Living Area) that expresses the square footage of the house and the feature *NB* (number of bedrooms) which encodes the number of rooms in the house. Typical values for *LA* are between 500 and 2000 sqft, while *NB* typically varies between 2 and 6. If we draw the contour lines of the loss $L(\mathbf{w})$ with respect to the two features (i.e., the areas with the same loss value), we obtain very elongated shapes (see Figure 4.6).

The gradient descent algorithm has no knowledge of the meaning of the variables and numerically can only notice that the *LA* has much larger values than *NB* and therefore tends to give more weight to the former during descent. We see this effect by calculating the loss for a single sample:

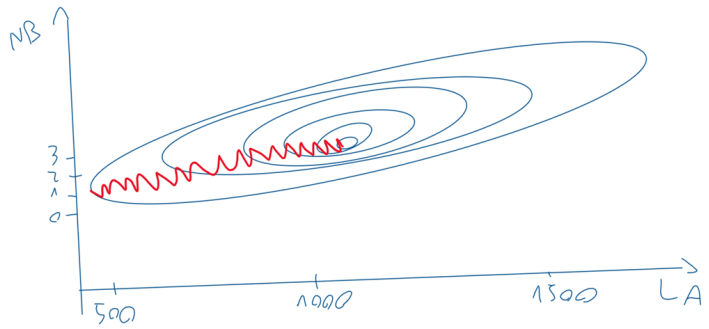


Figure 4.6: Contour lines for features *LA* and *NB*.

$$L(\mathbf{w}) = \frac{1}{2}(t - \mathbf{w}^T\mathbf{x})^2 = \frac{1}{2}\left(t - \underbrace{\begin{pmatrix} w_{LA} & w_{NB} \end{pmatrix}\begin{pmatrix} x_{LA} \\ x_{NB} \end{pmatrix}}_{w_{LA}x_{LA}+w_{NB}x_{NB}}\right)^2 \tag{4.28}$$

The first term of the scalar product weighs about 1000 times more than the second term, which is almost similar to noise. The consequence is that the convergence of the algorithm is very slow.

To solve this problem we perform the **Feature Normalization** before starting the optimization procedure. The purpose of this procedure is to transform the feature values to bring them into comparable ranges (in practice, approximately between -1 and 1 or between 0 and 1). There are several possible strategies, two of the most common are **Min-max Scaling** and **Standardization**. The former is applied as follows:

$$\tilde{x}_j^{(i)} = \frac{x_j^{(i)} - \bar{x}_j}{\max\limits_{i=0,\dots,N-1} x_j^{(i)} - \min\limits_{i=0,\dots,N-1} x_j^{(i)}} \tag{4.29}$$

where $x_j^{(i)}$ denotes the j-th feature of the i-th sample, while $\bar{x}_j = \frac{1}{N}\sum_{i=0}^{N-1} x_j^{(i)}$ is the average of that feature over the training dataset.

Instead, standardization is applied as follows:

$$\tilde{x}_j^{(i)} = \frac{x_j^{(i)} - \bar{x}_j}{\sigma_j} \tag{4.30}$$

where σ_j is the standard deviation of the j-th feature with respect to the training dataset.

With these choices, features will take on values largely between 0 and 1 or -1 and 1.

Normalizing the labels has no effect on the convergence of the algorithm.

With the normal equation it is not necessary to normalize since we calculate the value of optimal $\mathbf{w}$ in a single step in closed form and therefore we do not have numerical problems related to optimization iterations.

It is not always necessary to normalize, it depends on the range of feature values in the starting dataset.

Important! To calculate the statistics for normalization (mean, variance or maximum and minimum) we can not use the test set, because otherwise these values would also take into account the test samples, which in practice is not possible because we do not have access to them. The values for normalization must be calculated using only the training set and saved in memory. The features of the test samples are then normalized with the values calculated on the training set.

4.8 Standard Error of LR optimal parameters

In general, with **standard error (SE)** of a statistic (such as the mean, variance, or, in general, an estimate of any parameter of interest describing a set of samples, including the same parameters $\mathbf{w}_{ML}$ calculated via the normal equation of the LR), we mean the standard deviation of its **sample distribution**. The sample distribution of a statistic is the probability distribution of the statistic of interest (which is, in this sense, considered a random variable) computed from a finite sample of data.

For example, suppose we are interested in estimating the mean μ of an unknown distribution $p(x)$. To do so, let's assume we have available sets of samples $\mathcal{D}_0, \mathcal{D}_1, \ldots, \mathcal{D}_P - 1$, each containing a number of samples $\{x^{(i)}, i = 0, 1, \ldots, N-1 : x^{(i)} \sim p(x)\}$ representing realizations of $p(x)$. If we compute the sample mean for each of the sets $\mathcal{D}_p$ as $\hat{\mu}_p = \frac{1}{|\mathcal{D}_p|} \sum_{\mathcal{D}_p} x$ with $x \in \mathcal{D}_p$, we can construct a histogram of all calculated sample averages. This histogram is, in fact, an approximation of the sample distribution of $\hat{\mu}$. We are, in particular, interested in its mean (which we use to estimate the true unknown parameter μ) and its standard error (its standard deviation). In practice, we never have access to multiple sets of samples, but typically only to one. For this reason, the mean of the sample distribution is, in fact, estimated with the sample mean computed over the available data set, as is the standard error.

The parameters $\mathbf{w}_{ML}$ are themselves a statistic (they are estimated from the data using an MLE approach) and we are therefore interested in knowing the actual standard error to understand how reliable or uncertain the estimate provided by the normal equation is. We can then derive the SE by calculating the variance of $\mathbf{w}_{ML}$:

$$Var(\mathbf{w}_{ML}) = Var((\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathbf{t}) \tag{4.31}$$

By the properties of variance and noting that the only random variable in the above formula is $\mathbf{t}$, we can then write:

$$Var(\mathbf{w}_{ML}) = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T Var(\mathbf{t})\mathbf{\Phi}(\mathbf{\Phi}^T\mathbf{\Phi})^{-1} \tag{4.32}$$

Recalling that $\mathbf{t}$ follows a Gaussian distribution with mean $h_{\mathbf{w}}(\mathbf{x}, \mathbf{w})$ and variance $\sigma^2_{epsilon}$ (N.B. $\mathbf{w}$ denotes the true and unknown parameter vector, while $\mathbf{w}_{ML}$ is an estimate via normal equation):

$$Var(\mathbf{w}_{ML}) = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\sigma^2_\epsilon\mathbf{I}\mathbf{\Phi}(\mathbf{\Phi}^T\mathbf{\Phi})^{-1} = \sigma^2_\epsilon\mathbf{I}(\mathbf{\Phi}^T\mathbf{\Phi})^{-1} \tag{4.33}$$

i.e., , the variance (more precisely, the **covariance** of $\mathbf{w}_{ML}$), depends on the variance of the noise acting on the labels $\mathbf{t}$ and the inverse of the variance of the data $\mathbf{\Phi}$.

To verify that $\mathbf{\Phi}^T\mathbf{\Phi}$ represents the variance of the data, it is sufficient to recall the relation to the sample variance in the vector case. For simplicity of notation, we assume that no basis functions are used and thus work directly with the design matrix $\mathbf{X}$ (the derivation is identical in the general case involving $\mathbf{\Phi}$). We then prove that $\mathbf{X}^T\mathbf{X}$ is the variance of our samples. We start with the definition of variance for the vector case:

$$S = \frac{1}{N-1}\sum_{i=0}^{N-1}(\mathbf{x}^{(i)} - \boldsymbol{\mu})(\mathbf{x}^{(i)} - \boldsymbol{\mu})^T \tag{4.34}$$

(N.B. in this case we want to obtain a covariance matrix and it is therefore expected, from a dimensional point of view, that the transposed term is the second). Unfolding products:

$$S = \frac{1}{N-1}\sum_{i=0}^{N-1}(\mathbf{x}^{(i)}(\mathbf{x}^{(i)})^T - \mathbf{x}^{(i)}\boldsymbol{\mu}^T - \boldsymbol{\mu}(\mathbf{x}^{(i)})^T + \boldsymbol{\mu}\boldsymbol{\mu}^T) \tag{4.35}$$

Assuming the features have been normalized to have zero mean, the only term left is the first one:

$$S = \frac{1}{N-1}\sum_{i=0}^{N-1}\mathbf{x}^{(i)}(\mathbf{x}^{(i)})^T \tag{4.36}$$

Each of the products in the summation is of the type:

$$\mathbf{x}^{(i)}(\mathbf{x}^{(i)}) = \begin{pmatrix} x_0^{(i)} \\ \vdots \\ x_{D-1}^{(i)} \end{pmatrix}\begin{pmatrix} x_0^{(i)} & \dots & x_{D-1}^{(i)} \end{pmatrix} = \begin{pmatrix} x_0^{(i)}x_0^{(i)} & x_0^{(i)}x_1^{(i)} & \dots & x_0^{(i)}x_{D-1}^{(i)} \\ x_1^{(i)}x_0^{(i)} & x_1^{(i)}x_1^{(i)} & \dots & x_1^{(i)}x_{D-1}^{(i)} \\ \vdots & & & \\ x_{D-1}^{(i)}x_0^{(i)} & x_{D-1}^{(i)}x_1^{(i)} & \dots & x_{D-1}^{(i)}x_{D-1}^{(i)} \end{pmatrix} \tag{4.37}$$

summing for all samples we get:

$$
\begin{aligned}
S &= \sum_{i=0}^{N-1} \begin{pmatrix} x_0^{(i)}x_0^{(i)} & x_0^{(i)}x_1^{(i)} & \dots & x_0^{(i)}x_{D-1}^{(i)} \\ x_1^{(i)}x_0^{(i)} & x_1^{(i)}x_1^{(i)} & \dots & x_1^{(i)}x_{D-1}^{(i)} \\ \vdots & & & \\ x_{D-1}^{(i)}x_0^{(i)} & x_{D-1}^{(i)}x_1^{(i)} & \dots & x_{D-1}^{(i)}x_{D-1}^{(i)} \end{pmatrix} \\
&= \begin{pmatrix} \sum_{i=0}^{N-1} x_0^{(i)}x_0^{(i)} & \dots & \sum_{i=0}^{N-1} x_0^{(i)}x_{D-1}^{(i)} \\ \sum_{i=0}^{N-1} x_1^{(i)}x_0^{(i)} & \dots & \sum_{i=0}^{N-1} x_1^{(i)}x_{D-1}^{(i)} \\ \vdots & & \\ \sum_{i=0}^{N-1} x_{D-1}^{(i)}x_0^{(i)} & \dots & \sum_{i=0}^{N-1} x_{D-1}^{(i)}x_{D-1}^{(i)} \end{pmatrix} \\
&= \begin{pmatrix} x_0^{(0)} & x_0^{(1)} \dots & x_0^{(N-1)} \\ x_1^{(0)} & x_1^{(1)} \dots & x_1^{(N-1)} \\ \vdots & & \\ x_{D-1}^{(0)} & x_{D-1}^{(1)} \dots & x_{D-1}^{(N-1)} \end{pmatrix} \begin{pmatrix} x_0^{(0)} & x_1^{(0)} \dots & x_{D-1}^{(0)} \\ x_0^{(1)} & x_1^{(1)} \dots & x_{D-1}^{(1)} \\ \vdots & & \\ x_0^{(N-1)} & x_1^{(N-1)} \dots & x_{D-1}^{(N-1)} \end{pmatrix} \\
&= \mathbf{X}^T\mathbf{X} \qquad (4.38)
\end{aligned}
$$

We can then see that the variance (and hence the standard error minus a square root) of the parameter estimate $\mathbf{w}_{ML}$ depends directly on the standard deviation of the noise σ_ϵ^2 and inversely on the variance of the data. This is reasonable, since the more significant the noise in the labels, the more uncertain we will be about our estimate. On the other hand, the more "diffuse" our data is (it has more variance), the more reliable our estimate will be, because, intuitively, the algorithm will have seen, in the learning phase, a large number of possible scenarios (feature values), helping to find parameters with greater generalization capabilities.

N.B. In general, we do not even know the variance of the noise σ_ϵ^2 (or, equivalently, its precision β). This can be estimated with MLE, or, we can use the sample variance:

$$\hat{\sigma}_\epsilon^2 = \frac{1}{N-1}\sum_{i=0}^{N-1}(t^{(i)} - \bar{t})^2 \qquad (4.39)$$

$$\text{where } \bar{t} = \frac{1}{N}\sum_{i=0}^{N-1} t^{(i)} \qquad (4.40)$$

4.9 Performance metrics for LR

In order to evaluate the quality of the solution identified through the MLE framework for Linear Regression ($\mathbf{w}_{ML}$ weights), it is necessary to introduce performance metrics.

The first metric we might intuitively use is a measure of the difference between the values predicted through the optimal parameters $\mathbf{w}_{ML}$ and the true values provided by the ground-truth labels. This metric is called **Residual Sum of Square Errors (RSS)** and is analogous to the quadratic term of the loss function used in the previous sections:

$$RSS = \sum_{i=0}^{N-1} (t^{(i)} - \hat{t}^{(i)})^2 \tag{4.41}$$

where N is the number of samples in the set on which the metric is calculated (*e.g.,* the training or test set) and $\hat{t}^{(i)} = h_{\mathbf{w}_{ML}}(\mathbf{x}(i), \mathbf{w}_{ML}) = \mathbf{w}_{ML}^T \boldsymbol{\phi}(\mathbf{x}^{(i)})$.

When computed $\mathcal{D}_{training}$ gives us an indication of how well the optimal solution $\mathbf{w}_{ML}$ "explains" the data. Calculated instead on $\mathcal{D}_{test}$ it is useful to evaluate the generalization ability of the model.

Another very useful metric for valuater how close the optimal parameter set $\mathbf{w}_{ML}$ is to the true and unknown set $\mathbf{w}$ is the **Residual Standard Error (RSE)**:

$$RSE = \sqrt{\frac{1}{N-D-1} RSS} = \sqrt{\frac{1}{N-D-1} \sum_{i=0}^{N-1} (t^{(i)} - \hat{t}^{(i)})^2} \tag{4.42}$$

In this case we can see that as the number of samples N of the training set increases (N always refers to the number of samples of $\mathcal{D}_{training}$, even though the RSE is computed on $\mathcal{D}_{test}$), the RSE decreases, as we have more "experience" (data) that help us get closer to the "true" parameter set $\mathbf{w}$. On the other hand, if the dimensionality of the problem (the number of features D) increases, the RSE increases. This is because the problem becomes more complex.

Other useful metrics often used are:

Mean Square Error:

$$MSE = \frac{1}{N} RSS \tag{4.43}$$

Mean Absolute Error:

$$MAE = \frac{1}{N} \sum_{i=0}^{N-1} |t^{(i)} - \hat{t}^{(i)}| \tag{4.44}$$

Root Mean Square Error:

$$RMSE = \sqrt{MSE} \tag{4.45}$$

The RMSE, in particular, tends to penalize more very large errors.

The RSE provides an absolute measure of how well the model (i.e., the estimated parameters $\mathbf{w}_{ML}$) can fit the data. However, because the RSE is proportional to the scale of $t^{(i)}$ it is not straightforward to figure out which values are considered acceptable: if the labels have values in the order of hundreds, an RSE of the order of units may be acceptable, while such an error is surely too high if the labels take on values in the order of 10^{-2}.

To solve this problem, the metric R^2 called **R square** is used, which is usually between 0 and 1 and is independent of the scale of t: :

$$R^2 = \frac{TSS - RSS}{TSS} = 1 - \frac{RSS}{TSS} \tag{4.46}$$

Where TSS is the **Total Sum of Square** and is defined as:

$$TSS = \sum_{i=0}^{N-1} (t^{(i)} - \bar{t})^2 \tag{4.47}$$

and $\bar{t}$ is the average of the label values over all samples:

$$\bar{t} = \frac{1}{N} \sum_{i=0}^{N-1} t^{(i)} \tag{4.48}$$

R^2 can also assume negative values if, as an instance, the fitted line encodes a trend opposite to the data pattern: this case corresponds to a model whose performance are worse than those one would achieve with an algorithm that always predicts the same value (*e.g.,* the line fitted is parallel to the x axis in the one-dimensional case).

TSS measures the total variance of $\mathbf{t}$ and can be interpreted as the intrinsic variability of the labels of the (always training) dataset a priori. In contrast, RSS, which depends on the estimated value $\hat{t}^{(i)}$, measures the amount of information that is not "explained" by the learned regression model.

Consequently TSS-RSS measures the variability of the output explained by the model, The R^2 then measures the **proportion of variability** of $\mathbf{t}$ explained by the model:

- If $R^2 \to 1$, it means that a large variability in the data is explained by the model
- If $R^2 \to 0$, then the model fails to explain almost nothing of the data

The R^2 can be computed on either the training or the test set, and by comparing the values obtained on the two sets we can see if there is overfitting. In either case, $\bar{t}$ must be computed with data from the set on which the metric is computed, as we want to understand how well the model explains "that" data.

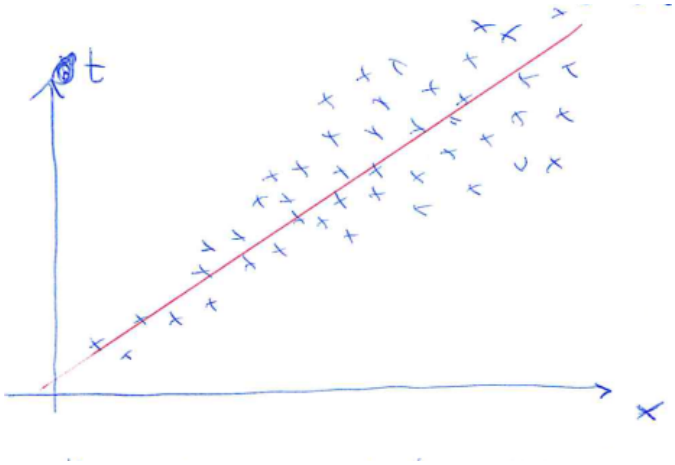


Figure 4.7: Example of heteroscedasticity in a monodimensional case. In this example, the variance of the noise increases as x grows.

4.10 Data non-idealities

When we defined the LR framework, we made some assumptions that we needed to derive the optimal solution through MLE. Some of these assumptions, however, are often too simplifying compared to practical cases. Let us now see what happens when the assumptions are violated and what are the possible remedies.

4.10.1 Heteroscedasticity

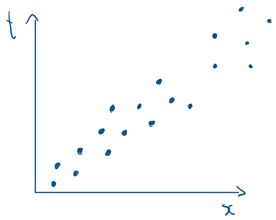
One of the assumptions we had made in defining the LR problem was that the labels were affected by zero mean Gaussian noise and β precision. Although we did not say so explicitly, the noise was assumed to be independent of the input $\mathbf{x}$ (i.e., the Gaussian parameters did not vary with the input). In this case we speak of **Homoscedasticity**. In the opposite case, i.e., when the noise depends on the input, we speak instead of **Heteroscedasticity**:

$$\textbf{Homoscedasticity} \qquad t = h_{\mathbf{w}}(\mathbf{x}, \mathbf{w}) + \epsilon \text{ con } \epsilon \sim N(0, \beta^{-1}) \tag{4.49}$$

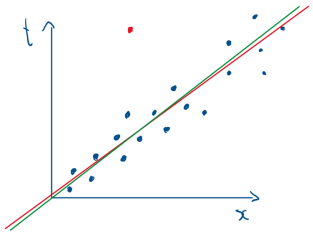
$$\textbf{Heteroscedasticity} \qquad t = h_{\mathbf{w}}(\mathbf{x}, \mathbf{w}) + \epsilon(\mathbf{x}) \text{ con } \epsilon \sim N(\mu(\mathbf{x}), \beta^{-1}(\mathbf{x})) \tag{4.50}$$

An example of heteroscedasticity is shown in Fig. 4.7.

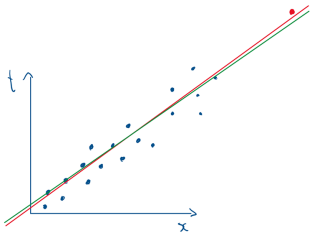
In these cases, in order to compensate the effect of heteroscedasticity and get back close to the ideal case of homoscedasticity one can consider to perform the regression not directly of t, but of a non-linear transformation (logarithmic or square root), which performs a compression and compensates the effect of heteroscedasticity.



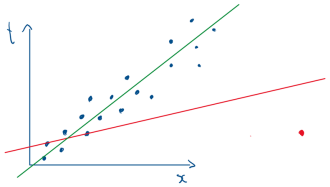
(a) No outliers or HLPs



(b) Presence of an outlier (in red)



(c) Presence of an HLP (in red)



(d) Presence of point that is both an outlier and an HLP.

Figure 4.8: Examples of outliers and HLPs. The line in red represents the regression line computed including the potentially influent point. In green, instead, we depict the regression line computed without the red point.

4.10.2 Influential Point, High-leverage points and Outliers

In general, if there are points in the dataset that behave very differently than everyone else, they have the "potential" to become **influential** points and have an impact (negative in most cases) on the regression.

We therefore distinguish between two types of points that have the **potential** to become influential:

- **Outlier**: An outlier is a sample whose output (label) t is very different from the general trend of the rest of the data
- **High Leverage Point (HLP)**: an HLP is a sample whose features have values that are very different from those of the other samples. By "very different" we can mean that a sample has a feature with "extreme" values compared to the others (very high or very low), or that a combination of features is "unusual", i.e. in contrast to the combinations observed in the rest of the data.

A sample can simultaneously be an outlier and an HLP. In general, however, the fact that a sample is an outlier or HLP or both does not necessarily imply that it has a significant impact in the regression. Only when the point has an impact on the regression result then the point is called **Influential** or **High Influence Point (HIP)**. Outliers and

HLPs have the potential to be HIPs, but to understand if they really are, investigations are needed. Let's look at an example (see Figure 4.8). Figure 4.8a shows a set of samples characterized by a single feature. In this case all samples are consistent and no HLPs or outliers are present.

If we consider instead the case in Figure 4.8b, the red point represents an outlier, as its t deviates from the trend of the other points. If we compute the regression line with and without the red point (the red line and the green line, respectively), we can see that the inclusion of the point does not have much impact on the solution. Consequently, if we compute the R^2 metric in the two cases we will have very similar values. However, the inclusion of the red dot does have an impact on the Standard Error of the solution and, in particular, increases it. We expect this result, since we have seen previously how the Standard Error of the optimal solution $\mathbf{w}_{ML}$ depends directly on the estimate $\hat{\sigma}_\epsilon^2$:

$$Var(\mathbf{w}_{ML}) = \hat{\sigma}_\epsilon^2 I(\Phi^T \Phi)^{-1} \tag{4.51}$$

with

$$\hat{\sigma}_\epsilon^2 = \frac{1}{N-1} \sum_{i=0}^{N-1} (t^{(i)} - \bar{t})^2 \tag{4.52}$$

$$\text{dove } \bar{t} = \frac{1}{N} \sum_{i=0}^{N-1} t^{(i)} \tag{4.53}$$

it is therefore clear that the presence of the outlier generates a very big term $(t^{(i)} - (t^{(i)})^2$, which increases $\hat{\sigma}_\epsilon^2$ and, consequently, the variance (and hence the standard error) of $\mathbf{w}_{ML}$. In this case the outlier is not influential with respect to the value of the optimal solution $\mathbf{w}_{ML}$, but it does have an impact on its standard error (and hence its uncertainty).

In contrast, the case in Figure 4.8c shows a high leverage point, as its value of x is extreme compared to the others. In this case, calculating the solution with and without the point shows how this HLP does not impact either the solution or its standard error. In general, we will see that whether HLP is actually HIP also depends on the value of its label (or observed response).

Finally, in Figure 4.8c, a point is shown that is both an outlier and an HLP. Calculating the solution with and without this point, we see that it is clearly influential, both with respect to the solution $\mathbf{w}_{ML}$ and with respect to its standard error.

It is therefore evident that it is necessary to introduce strategies to identify which points that are outliers and HLP are also influential with respect to the solution, in order to discard them.

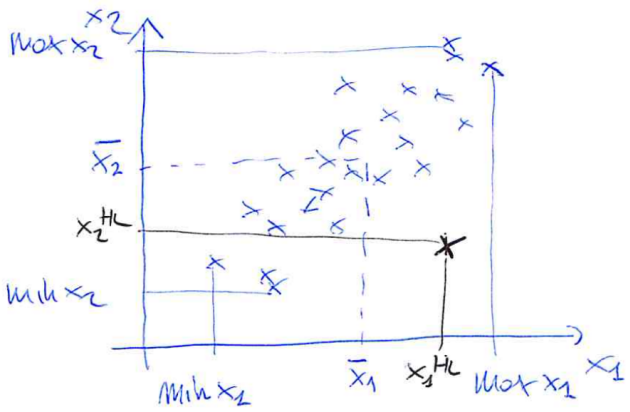


Figure 4.9: Example of an HLP in the 2-dimensional case.

Identifying HLP

Identifying HLPs is very simple in the one-dimensional case, where you only need to check if the feature value of the sample under consideration is "extreme" compared to the values of the other samples. However, when the input is characterized by several features, it is not immediate to understand if a point is an HLP, as shown in Figure 4.9: in this example, we see that the individual features of the point $\mathbf{x}^{HL}$, if taken individually, do not have extreme values, but taken together exhibit a behavior that differs from the values of the features of the other samples.

We then look for a more robust criterion to identify HLPs. In particular, we can use the notion of **Leverage**. To this end, recall that the estimated $\hat{\mathbf{t}}$ of the output of the training samples is given by the product between the **Hat Matrix** and the labels of the samples themselves:

$$\hat{\mathbf{t}} = \mathbf{H}\mathbf{t} \tag{4.54}$$

By expanding this expression, it is possible to see that:

$$\begin{pmatrix} t^{(0)} \\ t^{(1)} \\ \vdots \\ t^{(N-1)} \end{pmatrix} = \begin{pmatrix} h_{0,0}t^{(0)} + h_{0,1}t^{(1)} + \cdots + h_{0,N-1}t^{(N-1)} \\ h_{1,0}t^{(0)} + h_{1,1}t^{(1)} + \cdots + h_{1,N-1}t^{(N-1)} \\ \vdots \\ h_{N-1,0}t^{(0)} + h_{N-1,1}t^{(1)} + \cdots + h_{N-1,N-1}t^{(N-1)} \end{pmatrix} \tag{4.55}$$

In particular, we place attention on the terms of the diagonal $h_{i,i}$ that encode the influence that the label t^i has on the estimated output $\hat{t}^{(i)}$ of the same sample i (**Self-Influence**), in fact:

$$\frac{\partial \hat{t^{(i)}}}{\partial t^{(i)}} = h_{i,i} \tag{4.56}$$

If $h_{i,i}$ is large then the label of the i-th sample has a large influence on the prediction of the latter, while if it is small it does not have a large impact. As a result, $h_{i,i}$ is called

Leverage. Some properties of $h_{i,i}$ can be proved:

- $h_{i,i}$ measures the distance between the feature value $\mathbf{x}^{(i)}$ of the i-th sample and the average of the features over all N samples.
- $h_{i,i}$ is between 0 and 1.
- The sum of all $h_{i,i}$ (i.e., the sum of the diagonal of $\mathbf{H}$) corresponds to the number of parameters in the regression (the dimensionality of $\mathbf{w}_{ML}$), i.e., D.

To determine if a sample i is an HLP, it is then sufficient to see if the relative $h_{i,i}$ is to be considered large. Specifically, in practice a sample is identified as an HLP if its leverage $h_{i,i}$ greater than at least 3 times the average of all leverages:

$$\mathbf{x}^{(i)} \text{ is an HLP if } \quad h_{i,i} > 3\bar{h} \qquad \text{where } \bar{h} = \frac{\sum_{i=0}^{N-1} h_{i,i}}{N} = \frac{D}{N} \tag{4.57}$$

A word of caution! An HLP is a "potentially" influential point. Whether it becomes a **High Influence Point (HIP)** also depends on the value of its label $t^{(i)}$ (observed response).

Identifying Outliers

As mentioned in the example presented in Section 4.10.2, we have seen that outliers have an impact on the **standard error** of the solution and thus on its uncertainty. We have also seen that whether a point is an outlier depends on how much the associated label value deviates from the general trend in the data. We are therefore interested in evaluating the **residual** of that point:

$$e^{(i)} = t^{(i)} - \hat{t}^{(i)} \tag{4.58}$$

However, we have already mentioned that the absolute value of the residual depends on the unit of measurement of the labels, so it is not easy to identify an unambiguous criterion to understand if a point is an outlier or not. We therefore use **Studentized residuals** or **Internally studentized residuals**:

$$r^{(i)} = \frac{e^{(i)}}{SE(e^{(i)})} \tag{4.59}$$

where $SE(e^{(i)})$ denotes the standard error relative to the i-th sample. It is worth noting two aspects in this formula:

- The residuals are not errors, but **estimates** of the errors, since we have access to noisy labels $t^{(i)}$.

- The residuals, unlike the samples, are correlated with each other because they are obtained by optimizing a global function involving all samples.

Consequently, the standard error of the residuals is different for each sample and to calculate it we note that:

$$SE(e^{(i)}) = \sqrt{Var(e^{(i)})} = \sqrt{Var(t^{(i)} - \hat{t}^{(i)})} \tag{4.60}$$

The variance of the vector of residuals $Var(\mathbf{e})$ can be computed as:

$$Var(\mathbf{e}) = Var(\mathbf{t} - \hat{\mathbf{t}}) = Var(\mathbf{t} - \mathbf{H}\mathbf{t}) = Var((\mathbf{I} - \mathbf{H})\mathbf{t}) \tag{4.61}$$

since $\mathbf{H}$ is **Idempotent** (i.e. $\mathbf{H}^T\mathbf{H} = \mathbf{H}$) and symmetric, then:

$$(\mathbf{I} - \mathbf{H})^T(\mathbf{I} - \mathbf{H}) = \mathbf{I}^T\mathbf{I} - \mathbf{I}^T\mathbf{H} - \mathbf{H}^T\mathbf{I} + \mathbf{H}^T\mathbf{H} = \mathbf{I} - 2\mathbf{H} + \mathbf{H} = \mathbf{I} - \mathbf{H} \tag{4.62}$$

and the 4.61 then becomes:

$$Var((\mathbf{I} - \mathbf{H})\mathbf{t}) = (\mathbf{I} - \mathbf{H})Var(\mathbf{t})(\mathbf{I} - \mathbf{H})^T = \sigma_\epsilon^2(\mathbf{I} - \mathbf{H})(\mathbf{I} - \mathbf{H})^T = \sigma_\epsilon^2(\mathbf{I} - \mathbf{H}) \tag{4.63}$$

For the single sample we then have:

$$SE(e^{(i)}) = \sqrt{Var(e^{(i)})} = \sqrt{\sigma_\epsilon^2(1 - \underbrace{h_{i,i}}_{\text{leverage}})} \tag{4.64}$$

where σ_ϵ^2, as we previusly saw, can be estimated as:

$$\hat{\sigma}_\epsilon^2 = \frac{1}{N-1}\sum_{i=0}^{N-1}(t^{(i)} - \bar{t})^2 \tag{4.65}$$

$$\text{dove } \bar{t} = \frac{1}{N}\sum_{i=0}^{N-1} t^{(i)} \tag{4.66}$$

Therefore, the studentized residual is obtained as:

$$r^{(i)} = \frac{e^{(i)}}{SE(e^{(i)})} = \frac{e^{(i)}}{\sqrt{\hat{\sigma}_\epsilon^2(1 - h_{i,i})}} \tag{4.67}$$

At this point we could then identify a threshold against which to compare the residual to determine if the point is an outlier or not. There is still, however, a problem with internally studentized residuals: an outlier point could affect the regression to the point of "forcing" the model toward the outlier itself. In this way, the relative internally studentized residual would be small and the point would not be identified as an outlier. To overcome this problem we introduce **Deleted Residuals**, i.e., the residual for the i-th sample is computed with a model trained by excluding that very sample. The

standardized version of these residuals is known as **Studentized deleted residuals** or **Externally studentized residuals (ESR)**.

The deleted residuals are defined as:

$$d^{(i)} = t^{(i)} - \hat{t}^{(i)}_{(i)} \tag{4.68}$$

where with the subscript on $\hat{t}^{(i)}_{(i)}$ we indicate that the estimate is computed with a trained model excluding the i-th sample. Instead, externally studentized residuals are defined as:

$$d^{(i)}_{ESR} = \frac{d^{(i)}}{\sqrt{\hat{\sigma}^2_{\epsilon,(i)}(1 - h_{i,i})}} \tag{4.69}$$

where even for the variance $\hat{\sigma}^2_{\epsilon,(i)}$, the subscript (i) indicates that it is computed by excluding the i-th sample.

As a rule of thumb, identify a point as an outlier if the absolute value of its ESR is greater than 3:

$$\mathbf{x}^{(i)}\text{is an outlier if} \quad d^{(i)}_{ESR} > 3 \tag{4.70}$$

Identifying HIF

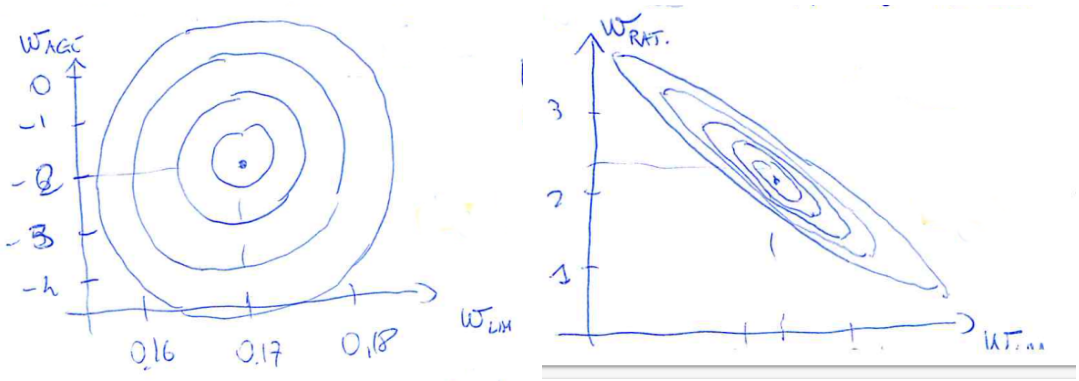
In general, a point identified both as an HLP and as an Outlier affects both the parameter estimate and the relative standard error, while a point identified only as an Outlier typically impacts only the standard error. A sample that is only an HLP instead, in most cases, has no impact on the solution. However, it is possible to introduce a criterion to directly identify a point as HIF, without first checking if it is HLP or outlier. We specifically introduce the metric **Difference in fits (DFFITS)**:

$$DFFITS_i = \frac{\hat{t}^{(i)} - \hat{t}^{(i)}_{(i)}}{\sqrt{\hat{\sigma}^2_{\epsilon,(i)} h_{i,i}}} \tag{4.71}$$

The numerator measures the difference between the estimate obtained by including and excluding the i-th sample. In practice, this metric measures how much the prediction for the i-th sample changes when it is included or not during training, and thus gives us a direct measure of its influence. A possible rule of thumb for determining whether a point is HIF is as follows:

$$\mathbf{x}^{(i)}\text{is an HIF if} \quad DFFITS_i > 2\sqrt{\frac{D+1}{N-D-1}} \tag{4.72}$$

This relationship gives us an indication of whether a sample is a HIF. In general, it is still good practice to try to figure out whether that point has the characteristics of an HLP, an outlier, or both, to ultimately decide whether to exclude or include it during optimization.



(a) No Collinearity between the features Age and Limit

(b) Collinearity between the features Limit and Rating

Figure 4.10: Loss function countours in the presence and in absence of collinearity.

A word of caution! The final decision should still be made by the designer, as these are empirical metrics and do not provide absolute criteria.

4.11 Collinearity and Multi-collinearity

Collinearity indicates the close correlation between two features as input to the model. If two features are collinear, it is difficult to distinguish the importance and effect of those on the output. In practice, this translates into a "hypersensitivity" to noise of the model parameters associated with the two collinear features. Let's see it with an example: we compare the contour lines of a cost function that depends on two features in the case of collinearity and non-collinearity. We consider three features: *Age*, *Limit* and *Rating*, which refer to a regression problem that tries to estimate the card credit that can be granted to a customer based on his age, credit rating and the limit imposed on the card.

Figure 4.10a depicts the loss contours as a function of the features *Age* and *Limit*. These two features are not collinear: this can be seen by calculating their correlation, but also intuitively we can say that the credit card limit is not strictly related to the person's age. In this case, the contour plot is characterized by curves close to the circumference as a shape.

The features *Limit* and *Rating*, on the other hand, are intuitively related, since a customer with a high rating will also have a larger limit on the credit card. The contour lines shown in Figure 4.10b, are more elongated and narrower. This means that as the parameter associated with *Limit* w_{LIM} varies, the parameter associated with *Rating* w_{RATING} can vary greatly without changing the cost function. Said differently, for small fluctuations of w_{LIM}, there are many possible values of w_{RATING} that guarantee the same loss. All this causes a large uncertainty in the determination of the parameters

of the LR (thus the standard error increases). The solution is very simple: if two features have a high direct or inverse correlation, typically greater in modulus than 0.85-0.9 (the correlation between pairs of features can be easily analyzed by looking at the **Correlation Matrix**), simply remove one of them. Intuitively, we can do this safely because, being strongly correlated, the two features bring, in practice, the same information for regression purposes.

Unfortunately, however, collinearity can also exist between more than two features, and in this case it cannot be identified with the correlation matrix (this does not tell us whether two features are correlated with a third one). This phenomenon is called **Multicollinearity** and one technique to figure out if one feature is multicollinear with the others is to use the **Variance Inflation Factor (VIF)**:

$$VIF(w_j) = \frac{1}{1 - R^2_{x_j|x_{-j}}} \tag{4.73}$$

The VIF is computed with respect to the j-th parameter relative to the j-th feature and depends on the R^2 score computed by regressing the j-th feature (considering this as the output of our model), using all the others (thus excluding the j-th feature). A high R^2 then indicates that the other features "explain" well the j-th feature. Therefore, it is collinear or multicollinear with the others and can be excluded. As a rule of thumb, a feature is collinear or multicollinear to the others if the VIF is greater than 4-5.

Chapter 5

Classification Algorithms

ML algorithms for classification aim to optimize models that can estimate **categorical** output variables, i.e., t that take values in a discrete, finite set. For example, an algorithm for object recognition allows to classify the content of an image with respect to the category of object it depicts (*e.g.,* chair, table, person, bottle, ...). In practice, regardless of the meaning associated with a given category, labels are provided to ML algorithms in numerical form, specifically as integers. It is important to note that, although numerical encoding naturally creates an order between classes (*e.g.,* "Chair" $\rightarrow 1$, "Person" $\rightarrow 2$, "Bottle" $\rightarrow 3$), this should **not** be understood as such. The fact that $t \in \{1, 2, 3, \ldots, C\}$ only indicates that the output can take one value among the possible C.

Often throughout the course, we will consider **binary** problems, *i.e.,* scenarios where t can only take on two values, *e.g.,* $t \in \{\text{Object detected}, \text{Object not detected}\}$ or $t \in \{\text{spam}, \text{not spam}\}$. In these cases, it is common to encode t with the values $\{0, 1\}$ or $\{-1, 1\}$. Assume, for the discussion that follows, that our dataset, in the case of classification problems, is defined as:

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, t^{(i)}) : t^{(i)} \in \{0, 1\}, \mathbf{x}^{(i)} \in \mathbb{R}^D, i = 0, 1, \ldots, N-1\} \tag{5.1}$$

5.1 Loss function for classification

In the case of classification problems, the quadratic loss we used for LR is not appropriate. The computation of residuals in the case of regression problems, in fact, has the form $(t^{(i)} - \hat{t}^{(i)})^2$, which measures the error based on the Euclidean distance between the prediction and the supervision label. This difference has little significance in classification problems. Assuming, for example, that the correct class for the i-th sample is $t^{(i)} = 3$, if the prediction of the model were $\hat{t}^{(i)} = 4$ we would have a smaller error than if the prediction were $\hat{t}^{(i)} = 5$: in fact we are saying that an incorrect prediction that estimates class 4 as the output is less severe than being confused with class 5, which does not necessarily reflect reality.

We then introduce a loss that simply detects if there has been a misclassification,

regardless of which class was incorrectly predicted. This loss is called **Error Rate** and is defined as follows:

$$ER = \frac{1}{N}\sum_{i=0}^{N-1} I(t^{(i)} \neq \hat{t}^{(i)}) \tag{5.2}$$

where $I(\cdot,\cdot)$ is the **Indicator Function**, which is 1 if the condition in parentheses is verified and 0 otherwise. The error function then simply sums all misclassified or misclassified samples. Intuitively, our classification algorithm (or **classifier**) will need to minimize the expected value of the error function with respect to the distribution of the output:

$$\min E_{t\sim p(y)}[I(t^{(i)} \neq \hat{t}^{(i)})] \tag{5.3}$$

5.1.1 Error Rate Lower Bound: Bayesian Error Rate

From a probabilistic point of view, the goal of the classifier is to model the (unknown) distribution of the output variables (the classes) of our dataset given their respective input feature vectors, i.e.:

$$p(\mathbf{t}|\mathbf{X}) \tag{5.4}$$

If we consider the distribution for the single sample, i.e., $p(t^{(i)} = j|\mathbf{x}^{(i)})$, which models the probability that the class of the i-th sample, given the features, is the j-th (among the C possible classes), the classifier is simply the model that chooses the j class that guarantees the highest $p(t^{(i)} = j|\mathbf{x})$:

$$\underset{j}{\text{argmax}} \quad p(t^{(i)} = j|\mathbf{x}^{(i)}) \tag{5.5}$$

It is the clear that, given the probabilistic nature of the model, even if we know the true distribution $p(\mathbf{t}|\mathbf{X})$, there is always the probability of making an error: the correct class will have the highest probability, but the others still have non-zero probability and therefore, in a sampling process, errors will be made.

In practice, we do not have access to $p(\mathbf{t}|\mathbf{X})$, but instead we try to approximate it with a $\hat{p}(\mathbf{t}|\mathbf{X})$. However, it is clear that, at best, we will have $\hat{p}(\mathbf{t}|\mathbf{X}) = p(\mathbf{t}|\mathbf{X})$ and, therefore, we always commit an error at least as large as that of the ideal classifier. Consequently, it is reasonable to introduce the concept of **lower bound** on the classification error, *i.e.,* , the minimum error we expect to experience:

$$\text{Bayesian Error Rate (BER)} \quad = 1 - E_{\mathbf{x}\sim p(\mathbf{x})}[\max_j p(t = j|\mathbf{x})] \tag{5.6}$$

or, more explicitly:

$$\begin{aligned}\text{BER} &= 1 - \sum_{j=1}^{C} \underbrace{\int_{\mathbf{x}\in C_j} p(\mathbf{x}, t=j)d\mathbf{x}}_{\text{prob. of correct classification i.e. } p(t=j)} \\ &= 1 - \sum_{j=1}^{C} \int_{\mathbf{x}\in C_j} p(\mathbf{x}|t=j)p(t=j)d\mathbf{x} \end{aligned} \tag{5.7}$$

where with C we denote the number of classes and with C_j the portion of the space of $\mathbf{x}$ in which assigning to the sample the class j is correct. In the binary case we can write:

$$\text{BER} = 1 - \int_{\mathbf{x}\in C_1} p(\mathbf{x}|t=1)p(t=1)d\mathbf{x} - \int_{\mathbf{x}\in C_2} p(\mathbf{x}|t=2)p(t=2)d\mathbf{x} \tag{5.8}$$

Or, more compactly:

$$\text{BER} = 1 - \int_{\mathbf{x}\in C_1} p(\mathbf{x}|C_1)p(C_1)d\mathbf{x} - \int_{\mathbf{x}\in C_2} p(\mathbf{x}|C_2)p(C_2)d\mathbf{x} \tag{5.9}$$

5.2 K-NN Classifier

Let us now look at the first classification algorithm of the course: **K-Nearest Neighbor**. The model assumed by this algorithm, *i.e.,* , the assumption it makes about the distribution of the output variable of the sample under analysis known the dataset and features of that sample ($p(t^{(i)} = j|\mathbf{X}, \mathbf{x}^{(i)})$), is as follows:

$$p(t^{(i)} = j|\mathbf{X}, \mathbf{x}^{(i)}) = \frac{1}{K} \sum_{t\in\mathcal{N}_{\mathbf{x}^{(i)}}} I(t=j) \tag{5.10}$$

Specifically, given a point in the feature space $\mathbf{x}^{(i)}$ and a dataset $\mathcal{D} = \{\mathbf{t}, \mathbf{X}\}$, and denoting by $\mathcal{N}_{\mathbf{x}^{(i)}}$ the neighborhood of $\mathbf{x}^{(i)}$ that contains the K points in the dataset closest to $\mathbf{x}^{(i)}$, the probability that the class $t^{(i)}$ of the sample under consideration is equal to j is given by the number of samples in $\mathcal{N}_{\mathbf{x}^{(i)}}$ associated with class j, divided by the number of total samples in the neighborhood (i.e., K). K-NN in fact uses a **voting** mechanism to compute the probability of a certain class.

> This algorithm assumes that there is a metric with which to compute distances in the feature space $\mathbf{x}$.

> The algorithm is computationally expensive in the case of very large datasets, since, in order to determine $\mathcal{N}_{\mathbf{x}^{(i)}}$, without exploiting more efficient data structures, it is necessary to compute N distance functions (with N size of the dataset).

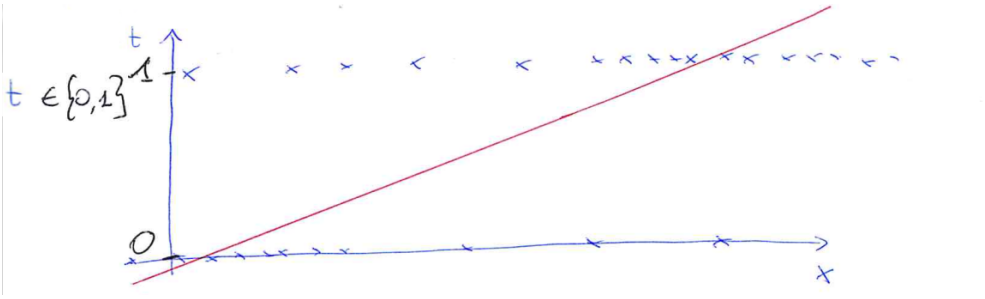


Figure 5.1: Prediction line with the model $\mathbf{w}^T\mathbf{x}$. With the cross marks we highlight the class label, that are either 0 or 1.

K-NN does not have a real training phase, as there are no parameters to learn. In fact the training consists in "saving" the dataset. All the computation takes place in the test phase, where for each new sample it is necessary to calculate the samples near the point for which you want to estimate the class.

5.3 Logistic Regression

Logistic regression is one of the most popular learning algortims for classification. In its basic form (which we will see below) it solves binary classification problems (nonetheless, it can be extended for multi-class problems).

At first, we might think of using a model analogous to Linear Regression, where the output is obtained as a linear combination of the input features with respect to the parameters: $\mathbf{w}^T\mathbf{x}$. However, this strategy is not appropriate for classification problems. From Figure 5.1, two problems can indeed immediately observed:

- The output of a linear regression model is continuous and can take on values in $]-\infty, +\infty[$, whereas the output of a binary classification problem only takes on values 0 and 1.
- The output of the linear regression model does not provide probability values and therefore we cannot use it to estimate the $p(t^{(i)} = j|\mathbf{x}^{(i)}$.

  N.B. In linear regression, to have a probability distribution in fact we needed to also estimate the precision of the noise on the labels and compute the predictive distribution (using the MLE solution), or estimate both the precision of the noise and the precision of the prior on the parameters and compute the posterior predictive distribution (using the full Bayesian formulation).

The first problem could be solved simply by imposing a threshold on the model output $\mathbf{w}^T\mathbf{x}$, i.e., if $\mathbf{w}^T\mathbf{x} > \gamma \implies t = 1$, otherwise $t = 0$. In this way we would have a binary output, but without an estimate of the relative confidence (probability). To overcome

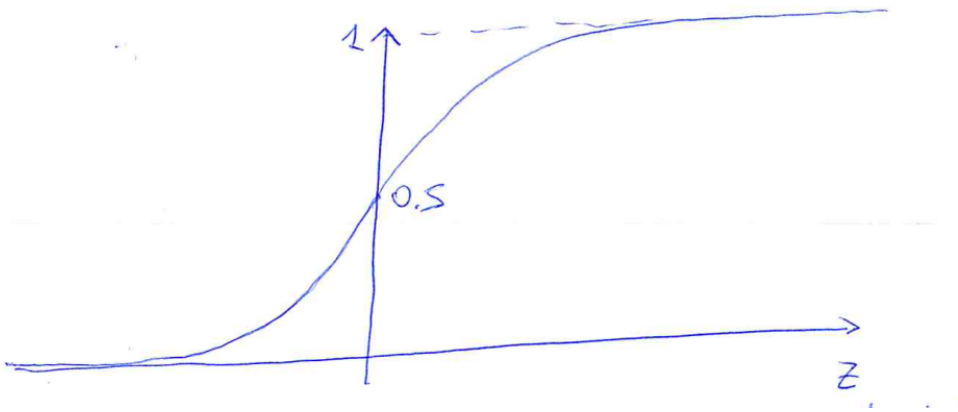


Figure 5.2: Logistic function

this problem and thus also solve the second point, instead of the threshold strategy we could directly use a nonlinear transformation $f(\cdot)$ of $\mathbf{w}^T\mathbf{x}$ so that:

$$f(\mathbf{w}^T\mathbf{x}) \in [0, 1] \tag{5.11}$$

and in this case, the decision strategy might be:

$$f(\mathbf{w}^T\mathbf{x}) \geq 0.5 \implies t = 1 \tag{5.12}$$

What function can guarantee this effect? One possible choice is the function **Sigmoid**, also called, **Logistic Function** (see Figure 5.2):

$$g(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z} \tag{5.13}$$

With this transformation, we can introduce the **hypothesis function of logistic regression** as the application of the sigmid function to the linear combination of the input features:

$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}} \in [0, 1] \tag{5.14}$$

Because of the way it is constructed and its properties, it is therefore reasonable to use $h_{\mathbf{w}}(\mathbf{x})$ as the conditional probability estimate of the classes:

$$p(t = 1|\mathbf{x}) = h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T\mathbf{x}) \tag{5.15}$$

N.B. $h_{\mathbf{w}}(\mathbf{x})$ estimates the probability that the class is equal to 1. The probability that the class is 0 is the complementary event, so $p(t=0|\mathbf{x}) = 1 - p(t=1|\mathbf{x}) = 1 - h_{\mathbf{w}}(\mathbf{x})$. We must then distinguish between the hypothesis function (which returns a probability) and the decision function, who "chooses" the class. The decision function chooses by comparing the probability provided by $h_{\mathbf{w}}(\mathbf{x})$ with a threshold (which is often set to 0.5, but can also take different values depending on the "uncertainty" one wants to have in predicting class 1 or class 0).

With this choice of $h_{\mathbf{w}}(\mathbf{x})$ we have lost the direct linear relationship with the features. However, it is possible to see that:

$$\frac{p(t=1|\mathbf{x})}{p(t=0|\mathbf{x})} = \frac{\frac{e^{\mathbf{w}^T\mathbf{x}}}{1+e^{\mathbf{w}^T\mathbf{x}}}}{1 - \frac{e^{\mathbf{w}^T\mathbf{x}}}{1+e^{\mathbf{w}^T\mathbf{x}}}} = e^{\mathbf{w}^T\mathbf{x}} \tag{5.16}$$

The previous relation is known as **Odds** and is another way of measuring probability (specifically as the ratio of the probabilities of the two complementary events). If we calculate its logarithm, we get the **Log-odds**:

$$\text{Log-odds} = \log(e^{\mathbf{w}^T\mathbf{x}}) = \mathbf{w}^T\mathbf{x} \tag{5.17}$$

We then see that there is still a linear relationship between the features and a probability measure (in this case log-odds).

We have therefore defined our hypothesis function. However, we must now determine the optimal $\mathbf{w}$ parameters with respect to the classification task.

5.3.1 Maximum Likelihood Estimation for Logistic Regression

In the case of linear regression, to find the optimal parameters we had identified the function $\frac{1}{2}\sum_{i=0}^{N-1}(t^{(i)} - h_{\mathbf{w}}(\mathbf{x}^{(i)}))$ as the loss function for training. However, at the beginning of this chapter we saw how this function is not adequate for classification problems and introduced the error rate:

$$ER = \frac{1}{N}\sum_{i=0}^{N-1} I(t^{(i)} \neq \hat{t}^{(i)}) \tag{5.18}$$

We could then consider using this function in the optimization procedure for logistic regression. However, this function is not easily derivable and is therefore not appropriate for parameter optimization. We therefore follow the maxmum likelihood estimation framework to find a suitable procedure for the logistic regression case. To do so, we need to define the shape of the probability distributions involved and, in particular, of the likelihood.

Let's assume we have a dataset

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, t^{(i)}) \ , \ i = 0, 1, \dots, N-1 \ , \ \mathbf{x}^{(i)} \in \mathbb{R}^D \ , \ t^{(i)} \in \{0, 1\}\} \tag{5.19}$$

it is reasonable to assume that the class distribution of the output variable given the features is a **Bernoulli** random variable. In fact, the bernoulli distribution is adequate for modeling binary random variables.

Bernoulli distribution
It models binary random variables:

$$p(y = 1) = \mu \tag{5.20}$$
$$p(y = 0) = 1 - \mu \tag{5.21}$$

The distribution depends on only one parameter μ. Moreover, by encoding the two classes with 0 and 1, it is possible to express the distribution in a compact form:

$$p(y) = Ber(y) = \mu^y (1 - \mu)^{1-y} \tag{5.22}$$

It can also be shown that:

$$E_{Ber}[y] = \mu \tag{5.23}$$
$$Var_{Ber}[y] = \mu(1 - \mu) \tag{5.24}$$

Given the considerations made earlier with respect to the logistic hypothesis function, it is reasonable to assume that the latter corresponds to the parameter μ of the Bernoulli distribution: in fact, we have seen how $h_{\mathbf{w}}(\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ expresses the probability that the output class is equal to 1. Similarly, the parameter μ of the Bernoulli distribution encodes the proability that the described random variable is equal to 1. Consequently, for the i-th sample:

$$\mu^{(i)} = h_{\mathbf{w}}(\mathbf{x}^{(i)}) = \frac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}^{(i)}}} \tag{5.25}$$

At this point we can write the likelihood. Assuming, as with linear regression, that the samples in the dataset are i.i.d. we have:

$$\begin{aligned} p(\mathbf{t}|\mathbf{X}) = \prod_{i=0}^{N-1} p(t^{(i)}|\mathbf{x}^{(i)}) &= \prod_{i=0}^{N-1} (\mu^{(i)})^{t^{(i)}} (1 - \mu^{(i)}))^{1-t^{(i)}} = \\ &= \prod_{i=0}^{N-1} (h_{\mathbf{w}}(\mathbf{x}^{(i)}))^{t^{(i)}} (1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})))^{1-t^{(i)}} \end{aligned} \tag{5.26}$$

We then calculate the log-likelihood:

$$l(\mathbf{w}) = \log p(\mathbf{t}|\mathbf{X}) = \sum_{i=0}^{N-1} t^{(i)} \log h_{\mathbf{w}}(\mathbf{x}^{(i)}) + (1 - t^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \tag{5.27}$$

To find the optimal parameters $\mathbf{w}_{ML}$, we compute the gradient of the log-likelihood with respect to $\mathbf{w}$ and use it to update the weights iteratively through, as an instance, gradient ascent (or descent):

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \nabla_{\mathbf{w}} l(\mathbf{w}) \tag{5.28}$$

N.B. for logistic regression there is no closed form solution, but it is necessary to use iterative methods for training.

N.B.2 To calculate the gradient, it is useful to first calculate the derivative of the sigmoid:

$$\begin{aligned} g'(z) = \frac{d}{dz}\left(\frac{e^z}{1+e^z}\right) = \frac{e^z}{1+e^z} - \frac{e^z}{(1+e^z)^2}e^z = \\ = \frac{e^z}{1+e^z}(1 - \frac{e^z}{1+e^z}) = g(z)(1-g(z)) \end{aligned} \tag{5.29}$$

We then calculate the individual elements of the gradient $\nabla_{\mathbf{w}} l(\mathbf{w})$:

$$\begin{aligned} \frac{\partial l(\mathbf{w})}{\partial w_j} &= \frac{\partial}{\partial w_j}\left(\sum_{i=0}^{N-1} t^{(i)} \log h_{\mathbf{w}}(\mathbf{x}^{(i)}) + (1 - t^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)}))\right) \\ &= \frac{\partial}{\partial w_j}\left(\sum_{i=0}^{N-1} t^{(i)} \log g(\mathbf{w}^T\mathbf{x}^{(i)}) + (1 - t^{(i)}) \log(1 - g(\mathbf{w}^T\mathbf{x}^{(i)}))\right) \\ &= \sum_{i=0}^{N-1}\left(t^{(i)}\frac{1}{g(\mathbf{w}^T\mathbf{x}^{(i)})} - (1-t^{(i)})\frac{1}{1-g(\mathbf{w}^T\mathbf{x}^{(i)})}\right) \underbrace{g'(\mathbf{w}^T\mathbf{x}^{(i)})}_{g(\mathbf{w}^T\mathbf{x}^{(i)}))(1-g(\mathbf{w}^T\mathbf{x}^{(i)})))} \underbrace{\frac{\partial \mathbf{w}^T\mathbf{x}^{(i)}}{\partial w_j}}_{x_j^{(i)}} \\ &= \sum_{i=0}^{N-1}\left(t^{(i)}(1 - g(\mathbf{w}^T\mathbf{x}^{(i)})) - (1-t^{(i)})g(\mathbf{w}^T\mathbf{x}^{(i)})\right)x_j^{(i)} \\ &= \sum_{i=0}^{N-1}\left(t^{(i)} - g(\mathbf{w}^T\mathbf{x}^{(i)})\right)x_j^{(i)} = \sum_{i=0}^{N-1}\left(t^{(i)} - h_{\mathbf{w}}(\mathbf{x}^{(i)})\right)x_j^{(i)} \end{aligned} \tag{5.30}$$

from which we see that the gradient to update the j-th parameter is proportional to the difference between the supervised and estimated output values. The update with gradient ascent can then be written as:

$$w_j \leftarrow w_j + \alpha\left(\sum_{i=0}^{N-1}\left(t^{(i)} - h_{\mathbf{w}}(\mathbf{x}^{(i)})\right)x_j^{(i)}\right) \quad \forall j \tag{5.31}$$

N.B. The final expression for the gradient is analogous to that of linear regression. The only thing that changes is the hypothesis function.

5.3.2 Decision Function

As anticipated earlier, $h_{\mathbf{w}}(\mathbf{x}^{(i)})$ represents the estimate of $p(t^{(i)} = 1|\mathbf{x}^{(i)})$. The decision function can then be derived simply by setting:

$$p(t^{(i)} = 1|\mathbf{x}^{(i)}) > p(t^{(i)} = 0|\mathbf{x}^{(i)}) \implies \hat{t}^{(i)} = 1 \quad \text{otherwise} \quad \hat{t}^{(i)} = 0 \tag{5.32}$$

which corresponds to:

$$p(t^{(i)} = 1|\mathbf{x}^{(i)}) = h_{\mathbf{w}}(\mathbf{x}^{(i)}) \geq 0.5 \implies \hat{t}^{(i)} = 1 \quad \text{otherwise} \quad \hat{t}^{(i)} = 0 \tag{5.33}$$

Depending on the specific case we could choose a threshold different from 0.5: for example, if we want to be more conservative with respect to class 1 we could raise the threshold and estimate as belonging to that class only samples for which the hypothesis function gives a high probability (a similar reasoning can be made with respect to class 0 by decreasing the threshold).

In any case, imposing a threshold on the hypothesis function translates into a threshold also with respect to its argument. This can be seen as follows: first consider that the inverse of the sigmoid function is:

$$g^{-1}(\sigma) = \log(\frac{\sigma}{1-\sigma}) \tag{5.34}$$

Therefore, if we impose the threshold on the hypothesis function:

$$h_{\mathbf{w}}(\mathbf{x}^{(i)}) = g(\mathbf{w}^T\mathbf{x}^{(i)}) \geq 0.5 \tag{5.35}$$

this implies that:

$$g^{-1}(h_{\mathbf{w}}(\mathbf{x}^{(i)})) = g^{-1}(g(\mathbf{w}^T\mathbf{x}^{(i)})) \geq g^{-1}(0.5) \implies \mathbf{w}^T\mathbf{x}(i) \geq log\frac{0.5}{1-0.5} \implies \mathbf{w}^T\mathbf{x}(i) \geq 0 \tag{5.36}$$

Which means that in the feature space we still have a linear separator (decision function).

5.4 Classification Approaches

In the previous chapter, we studied K-NN classifier and Logistic regression as possible solutions to classification problems. We now take a step back to study the more general categories of classification approaches. In general, three basic families of approaches can be distinguished, each of which may be more or less suitable depending on our design needs. The choice of one category of approaches over another is related to the interest we have in modeling certain process features, in the amount of data available and in the information needed to train the models.

To study the characteristics of these families of approaches, we more formally define the concept of **decision function**. In particular, the decision function is a function $f(\cdot) : \mathbb{R}^D \rightarrow S_t$ which, given an input feature vector $\mathbf{x}$, provides as output the class estimate associated with $\mathbf{x}$, i.e., $\hat{t} = f(\mathbf{x})$, with $\hat{t} \in S_t = \{1, \ldots, K\}$. In some situations, the decision function is sufficient for us. However, sometimes we may be interested not only in the predicted class, but also in the probability of that prediction, i.e. $p(t|\mathbf{x})$. Finally, in some situations it is important to model not only the probability of the class given the input, but also the joint probability $p(t, \mathbf{x})$. In general, if we are interested in estimating the statistical properties of the unknown distribution of interest from the data, we aim to perform **Inference**, and, with respect to what we want to infer, three families of approaches can be distinguished:

- **Discriminant Function**: This family of approaches make use of discriminant functions. They do not aim to infer the statistical characteristics of the unknown distribution, but merely identify a decision function $f(\mathbf{x})$ directly capable of predicting the category associated with the input. The decision function is clearly designed with respect to optimality criteria (loss functions).

- **Discriminative Model**: Discriminative model approaches instead attempt to make inference about the a posteriori probability distribution $p(t|\mathbf{x})$, i.e., the statistical process that describes how labels are generated from features. Logistic regression and K-NN classifier are two approaches that belong to this category.

  In the case of Logistic Regression, we modeled $p(t|\mathbf{x})$ as:

  $$p(t|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})^t(1 - \sigma(\mathbf{w}^T\mathbf{x}))^{1-t} \tag{5.37}$$

  where $\sigma(\mathbf{w}^T\mathbf{x}) = g(\mathbf{w}^T\mathbf{x})$ (we used σ to make the sigmoid function explicit). Thus, in Logistic regression, once we have determined the optimal parameter vector, we are able to obtain an estimate of the probability of the output via the Bernoulli distribution. Furthermore, once we have estimated $p(t|\mathbf{x})$, we can obtain the decision function by simply finding a threshold with which to compare the class probability, e.g.:

$$p(t = 1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \tag{5.38}$$

N.B. In this case, having available the posterior distribution of the classes, the decision function is easily modifiable simply choosing a new threshold. Instead, in the case of the discriminant functions, to change the decision function we need to perform the training from scratch.

- **Generative Model**: Instead, these approaches aim at inferring the joint distribution $p(t, \mathbf{x})$ by estimating the **class-conditional distribution** $p(\mathbf{x}|t)$ and the prior over classes $p(t)$. Once the joint distribution is modeled, it is possible to obtain the posterior with Bayes' theorem and, consequently, determine a decision function:

$$p(t|\mathbf{x}) = \frac{p(\mathbf{x}|t)p(t)}{p(\mathbf{x})} \propto p(\mathbf{x}|t)p(t) \tag{5.39}$$

  Clearly, in this case, we have to make a higher modeling effort, since we have to model the data generation process from the joint distribution. In particular, since $p(t, \mathbf{x}) = p(\mathbf{x}|t)p(t)$, in this case we need to model both $p(\mathbf{x}|t)$ and $p(t)$.

  Modeling the joint distribution means having the **generation model** of the data available. Consequently, this family of approaches is the most general possible and includes the other two categories. In addition, generative models make additional assumptions about the process that generates the features $\mathbf{x}$ given the classes and about the prior distribution over the classes. They are, thus, closer to Bayesian formulations and, in general, are more robust to overfitting.

5.5 Discriminant Function

We now consider discriminant functions for binary classification. As we have already seen in the previous chapters, the simple linear model is not adequate for classification problems, since the output variable assumes only the values 0 and 1. For this reason, it is necessary to use a nonlinear function, called, in general, **Activation Function** (for the Logistic Regression, for example, we used the sigmoid):

$$\hat{t} = f(\mathbf{w}^T\mathbf{x} + w_0) \tag{5.40}$$

where $f : \mathbb{R}^D \to \{0, 1\}$ or, more generally, $f : \mathbb{R}^D \to [0, 1]$.

Indicating for convenience with $\hat{t} = y(\mathbf{x}) = f(\mathbf{w}^T\mathbf{x} + w_0)$, to make a decision, we saw how, if the nonlinear function returns values in the range $[0, 1]$, it is sufficient to choose

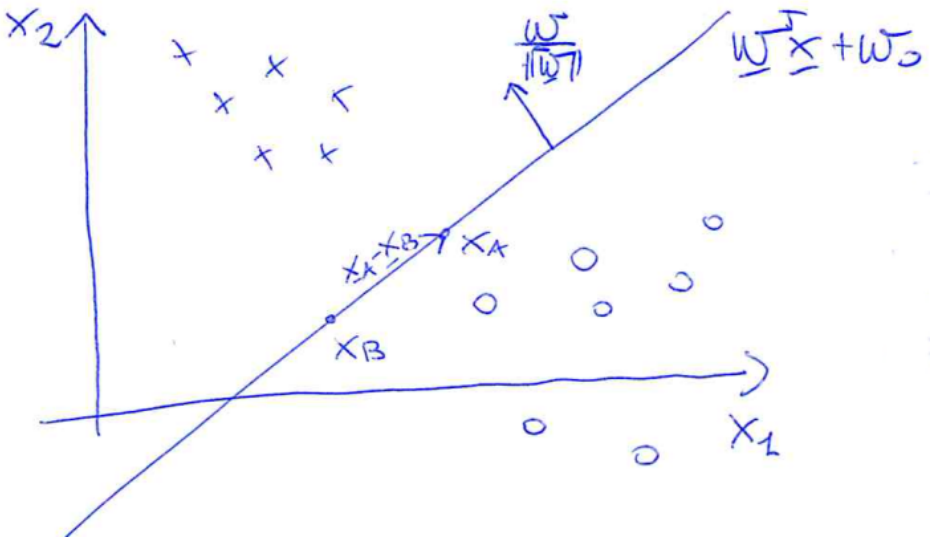


Figure 5.3: Decision Boundary on a two-dimensional example.

a threshold γ:

$$\begin{aligned} y(\mathbf{x}) \geq \gamma \rightarrow \hat{t} = 1 \\ y(\mathbf{x}) < \gamma \rightarrow \hat{t} = 0 \end{aligned} \tag{5.41}$$

From this relation, we can see that $y(\mathbf{x}) = \gamma$ is a surface, or curve, in the space of $\mathbf{x}$ for which $y(\mathbf{x})$ has a constant value. Furthermore, if $f(\cdot)$ is invertible, it is possible to translate this relation into a linear constraint (linear decision function):

$$\mathbf{w}^T\mathbf{x} + w_0 = f^{-1}(\gamma) \tag{5.42}$$

Let us now try to contextualize these aspects in the case of discriminant functions. Consider the surface $\mathbf{w}^T\mathbf{x} + w_0 = 0$. This surface is called **Decision Boundary**, since, all points for which $\mathbf{w}^T\mathbf{x} + w_0 \geq 0$ are points that belong to one class, while points for which $\mathbf{w}^T\mathbf{x} + w_0 < 0$ belong to the other class (see Figure 5.3). In general, if $\mathbf{x} \in \mathbb{R}^D$, then the plane identified by $\mathbf{w}^T\mathbf{x} + w_0 = 0$ is a hyperplane of dimension $D - 1$ and $\mathbf{w}/||\mathbf{w}||_2$ is its versor. This is because if a point $\mathbf{x}_A$ belongs to the hyperplane, then $\mathbf{w}^T\mathbf{x}_A + w_0 = 0$, so given two points $\mathbf{x}_A$ and $\mathbf{x}_B$ belonging to the hyperplane, then:

$$\mathbf{w}^T\mathbf{x}_A + w_0 = \mathbf{w}^T\mathbf{x}_B + w_0 = 0 \implies \mathbf{w}^T(\mathbf{x}_A - \mathbf{x}_B) = 0 \implies \mathbf{w} \perp \vec{\mathbf{x}_{AB}} \tag{5.43}$$

Furthermore, one can easily see that w_0 is a measure of how far the hyperplane is from the origin. In fact, since the projection of the origin onto the hyperplane is identified by a vector $\mathbf{x}$ that belongs to the hyperplane itself, then $\mathbf{w}^T\mathbf{x} + w_0 = 0$, furthermore, since the vector is perpendicular to the hyperplane, it is also proportional to its versor,

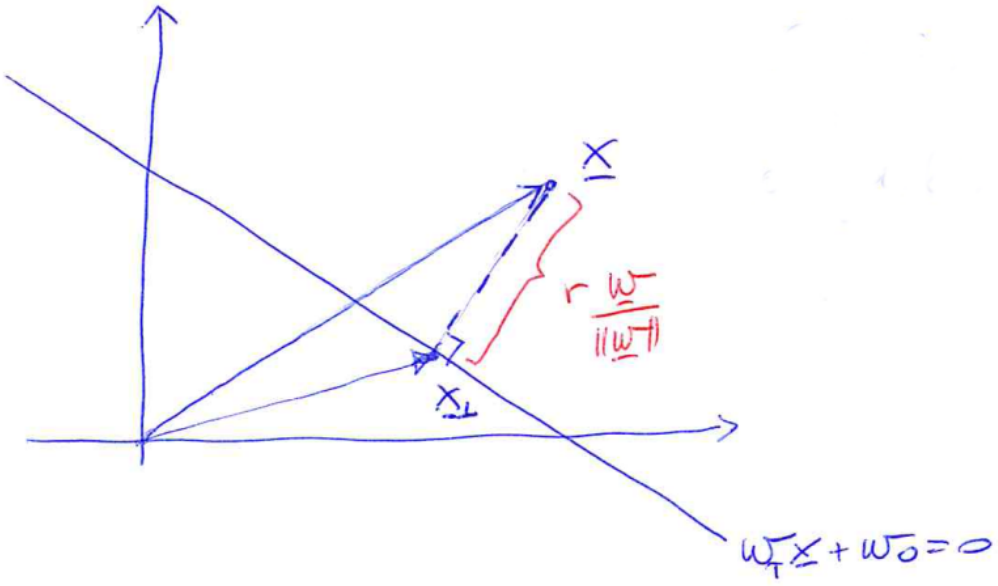


Figure 5.4: Distance from the decision boundary.

so $\alpha\frac{\mathbf{w}}{||\mathbf{w}||} = \mathbf{x}$. Consequently:

$$\alpha\mathbf{w}^T\frac{\mathbf{w}}{||\mathbf{w}||} + w_0 = 0 \implies \alpha\frac{||\mathbf{w}||^2}{||\mathbf{w}||} + w_0 = 0 \implies \alpha = -\frac{w_0}{||\mathbf{w}||} \tag{5.44}$$

Finally, even though the value of the function $y(\mathbf{x})$ is not a probability measure (we are considering dicriminant functions that do not make inference), it still gives us information about the "confidence" we have in classifying a point as belonging to a class, since it encodes the distance to the hyperplane. In fact, considering Figure 5.4, we can see that:

$$\mathbf{x} = \mathbf{x}_\perp + r\frac{\mathbf{w}}{||\mathbf{w}||} \tag{5.45}$$

multiplying both sides by $\mathbf{w}^T$ and adding up w_0:

$$\underbrace{\mathbf{w}^T\mathbf{x} + w_0}_{y(\mathbf{x})} = \underbrace{\mathbf{w}^T\mathbf{x}\perp + w_0}_{=0} + r\frac{\overbrace{\mathbf{w}^T\mathbf{w}}^{||\mathbf{w}||^2}}{||\mathbf{w}||} \implies r = \frac{y(\mathbf{x})}{||\mathbf{w}||} \tag{5.46}$$

In summary, if we have no interest in making inferences about the posterior distribution of the classes or the joint distribution, the linear separator seems to be a good strategy for classifying a point as belonging to one class or the other.

5.5.1 Perceptron Algorithm

The perceptron algorithm belongs to the Discriminant Function approaches and assumes a decision function as follows:

$$y(\mathbf{x}) = f(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})) \tag{5.47}$$

where

- $\mathbf{x} \rightarrow \boldsymbol{\phi}(\mathbf{x})$ is a feature transformation obtained via the **basis function** and where we assume, for convenience, that $\phi_0(\mathbf{x}) = 1$.
- The activation function $f(\cdot)$ in the perceptron case is defined as follows:

$$f(a) = \begin{cases} +1 \text{ if } a \geq 0 \\ -1 \text{ if } a < 0 \end{cases} \tag{5.48}$$

By encoding the output with +1 and -1, we are able to write the membership conditions on the two sides of the hyperplane as follows:

$$\begin{cases} \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) \geq 0 \implies t = +1 \\ \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) < 0 \implies t = -1 \end{cases} = t(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})) \geq 0 \tag{5.49}$$

How can we train this decision function and identify the optimal parameters of $\mathbf{w}$? As we have already mentioned, the loss $\sum_{i=0}^{N-1} I(t^{(i)} \neq \hat{t}^{(i)})$ is not easy to use since it has null gradient in most of the parameter space and has discontinuity points. To solve this problem, the perceptron algorithm uses the **Perceptron Criterion**: we assign zero error to each correctly classified sample and an error equal to

$$-\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^{(i)}) t^{(i)} \tag{5.50}$$

to each misclassified sample. The error to be minimized is therefore:

$$E_p(\mathbf{w}) = -\sum_{i \in \mathcal{M}} \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^{(i)}) t^{(i)} \tag{5.51}$$

Where $\mathcal{M}$ denotes the misclassified samples. $E_p(\mathbf{w})$ is a function of the parameters $\mathbf{w}$ that is piecewise linear, since it is the sum of linear functions whose number varies as $\mathbf{w}$ changes (as the number of samples in $\mathcal{M}$ changes). In this way, given a point in the parameter space, it is possible to calculate a non-zero gradient and then use gradient descent in its SGD (stochastic gradient descent) version. SGD updates the parameter values considering, at each iteration, only one sample and not the whole dataset as GD does:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla E_p(\mathbf{w}) = \mathbf{w} + \alpha \boldsymbol{\phi}(\mathbf{x}^{(i)}) t^{(i)} \tag{5.52}$$

If the classes are linearly separable it is possible to show that the perceptron algorithm

always converges, even if it is not possible to guarantee the time of convergence (although if there is an upper bound). If, on the contrary, the classes are not linearly separable, then the algortim does not converge.

5.6 Quick Detour: Decision Boundaries for Multi-class Problems

So far we have considered binary problems, where having a linear separator is intuitive and effective. What happens if the output can take more than two values? That is, how can we handle multiclass problems?

We now see two very common and simple approaches to handling multiclass problems as a combination of binary classifiers. However, it is important to say that there are approaches that inherently handle multiclass problems.

5.6.1 One vs One classifier

If $t \in \{C_1, C_2, \ldots, C_K\}$, we can think of training a binary classifier for each possible pair of classes, for a total of $K(K-1)/2$ classifiers. For example:

$$\underbrace{C_1 \text{ vs } C_2 \text{ , } C_1 \text{ vs } C_3 \text{ , } \ldots C_1 \text{ vs } C_K}_{K-1 \text{ classifiers}}$$
$$\underbrace{C_2 \text{ vs } C_3 \text{ , } C_2 \text{ vs } C_4 \text{ , } \ldots C_2 \text{ vs } C_K}_{K-2 \text{ classifiers}}$$
$$\ldots$$
$$\underbrace{C_{K-1} \text{ vs } C_K}_{1 \text{ classifier}}$$

In the testing phase, to estimate the class associated to a given sample, we process the latter with all $K(K-1)/2$ classifiers and a "voting" system is adopted: each class will receive a vote equal to the number of classifiers that predicted that class. The most voted class will then be the one selected at the end.

5.6.2 One vs All Classifier

Unlike One vs One, in this case a binary classifier is trained for each class, considering the set of remaining classes as the alternative class. Thus, in this case we have a smaller number of classifiers (we have K of them). However, how do we, given these K classifiers, choose the class to associate with a sample? If we refer to the example in Figure 5.5, we can see that there are uncertainty zones, *i.e.,* parts of the feature space shared among multiple classifiers. To resolve the ambiguity, it is sufficient to select as the final

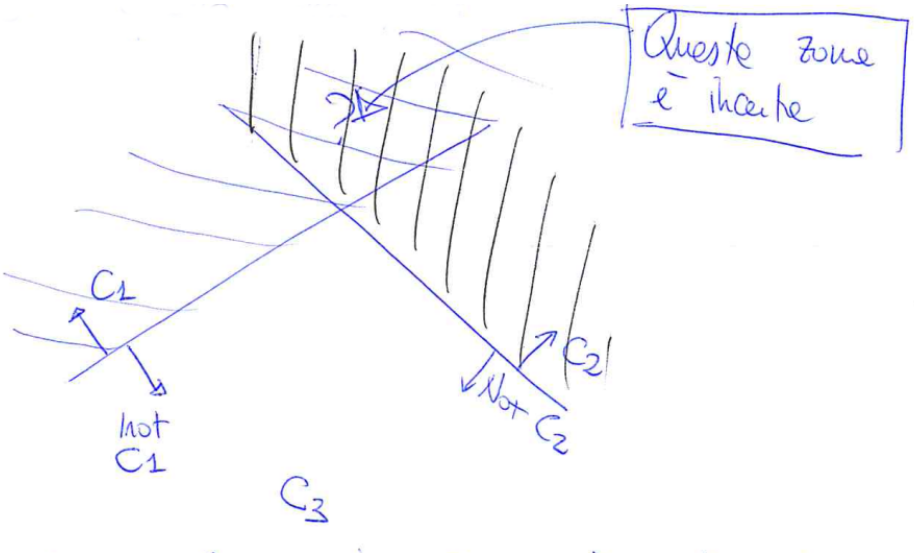


Figure 5.5: One vs All uncertainty zone.

class the one that guarantees a greater distance from the decision boundary (separating hyperplane):

$$\text{if } \mathbf{w}_k^T \mathbf{x} + w_{0,k} \geq \mathbf{w}_j^T \mathbf{x} + w_{0,j} \quad \forall j \implies \hat{t} = C_k \tag{5.53}$$

N.B. A similar ambiguity is also present for the One vs One classifier (see Figure 5.6). However, in this case it is not possible to resolve it as for the One vs All case.

5.7 Quick Detour: Classification Metrics

As with the loss function, the metrics introduced for regression problems are not adequate for classification problems and we must therefore introduce new ones.

If we consider the case of binary classification, the output of the algorithm can be categorized with respect to 4 possible scenarios depending on the correctness of the classification. If we consider class "1" as **positive** and class "0" as **negative**, we can define the following 4 scenarios:

- **True Positive TP**: In the case where the class estimated by the algorithm is the positive one and the sample is associated with a positive label ("1");
- **True Negative TN**: In the case where the class estimated by the algorithm is the negative class and the sample is associated with a negative label ("0");
- **False Positive FP**: In the case where the estimated class is positive, but the sample is actually associated with a negative label;

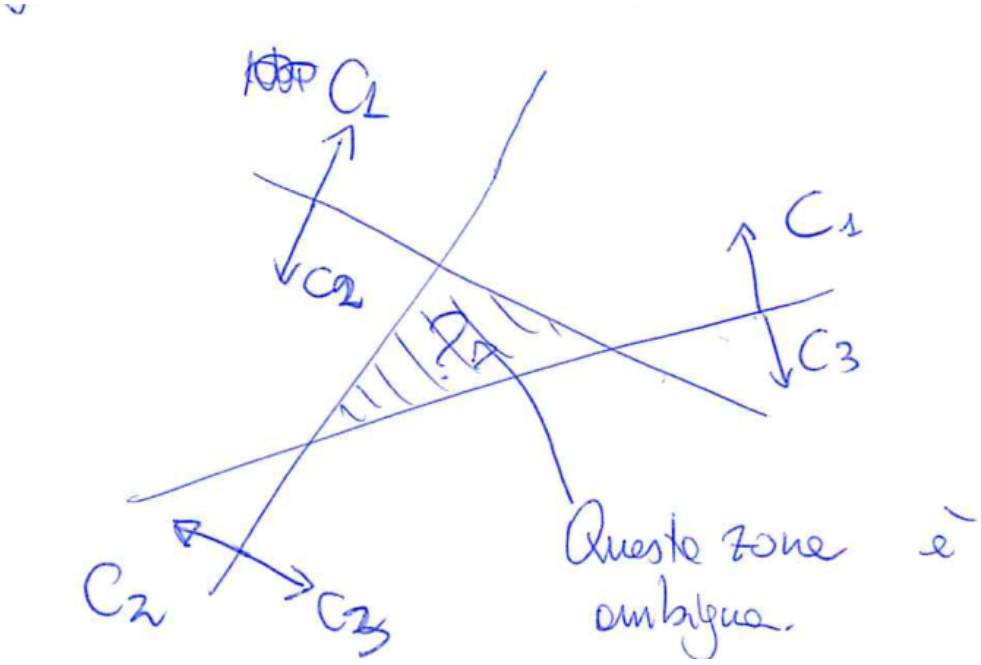


Figure 5.6: One vs One uncertainty zone.

- **False Negative FN**: in the case where the estimated class is negative, but the sample is actually associated with a positive label.

Clearly, we would prefer a model whose outputs fall in the true positive and true negative cases.

By calculating the number of TP, TN, FP and FN it is possible to introduce some useful metrics to evaluate the performance of the classification algorithms:

5.7.1 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\#\text{ correct predictions}}{\#\text{ total predictions}} \tag{5.54}$$

Accuracy is the simplest metric used to evaluate classification algorithms. However, it is not always the most appropriate. Imagine we have a classifier that must predict the presence or absence of a tumor. We consider 100 samples, of which 90 are not tumors (positive class), while 10 are (we consider the class "tumor" as the negative class). We further assume that the model trained with our algorithm predicts that all 100 samples are not tumors. If we calculate the accuracy we get: $TP = 90$, $TN = 0$, $FP = 10$ and $FN = 0$, so the accuracy is:

$$\text{Accuracy} = \frac{90}{100} = 90\% \tag{5.55}$$

The accuracy value is very high and we might therefore think that our model is working well. In reality, the model was never able to detect a tumor when present, a very dangerous behavior in a real application!

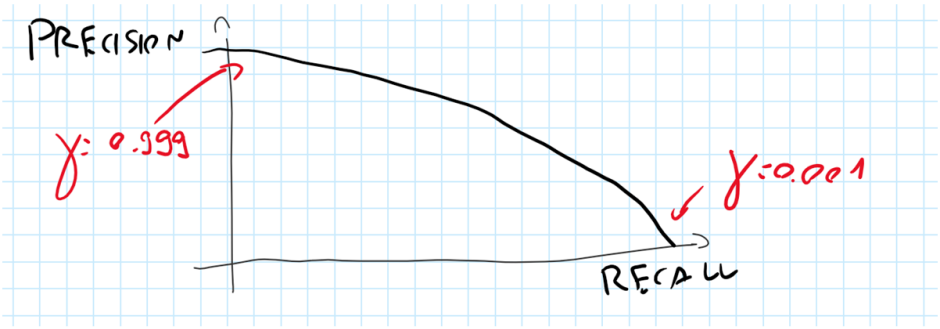


Figure 5.7: Precision-recall curve.

This effect occurred because we had only 10 samples related to the negative class and 90 related to the positive class. Our dataset is therefore said to be **Unbalanced** and accuracy is not a useful metric in this case. In unbalanced dataset situations, it is appropriate to use other metrics. In particular, it is very common to use **Precision** and **Recall**.

5.7.2 Precision and Recall

Precision answers the question *What is the proportion of correct positive predictions to all positive predictions?*:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5.56}$$

while recall answers the question *What is the proportion of correct positive predictions to all samples that are actually positive*?

$$\text{Recall} = \frac{TP}{TP + FN} \tag{5.57}$$

We often need to trade-off between precision and recall: in general, if we want to increase precision (by raising the decision threshold to be "more certain" that the sample is actually positive) we will inevitably classify as negative many "more uncertain" samples that are actually positive. Conversely, if we lower the decision threshold to increase recall, we will have more incorrect positive classifications.

This is why we often draw the **precision-recall curve** (see Figure 5.7) as the decision threshold varies (*e.g.*, $\gamma = 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9$). With a threshold close to 1, the decision function will only estimate the positive class with very high probabilities $p(t = 1|\mathbf{x})$. We will then have very few TPs, but virtually no FPs, so the precision will

tend to 1. On the other hand, if the threshold is very low, there will be many FP, but also TP will increase compared to FN and consequently the recall tends to 1.

The P-R curve allows to choose the optimal threshold value with respect to the problem at hand. It is, in fact, always important to ask ourselves if it is more problematic to make a mistake in predicting the positive class or if it is more important to "find" as many positive samples as possible.

5.7.3 F1-Score

With the precision (P) and recall (R) it is possible to define another metric that does not suffer from the problems of the accuracy in the case of unbalanced dataset: the **F1-Score**:

$$\text{F1-score } = 2\frac{P \cdot R}{P + R} = \frac{TP}{TP + \frac{1}{2}(FP + FN)} \tag{5.58}$$

The F1-score calculates a harmonic average between precision and recall and by maximizing it we are actually looking for a good compromise between the two (N.B. the compromise depends also on the choice of the decision threshold γ).

Important!: The F1-score takes into account the unbalance of the dataset, but it is calculated with respect to the class considered positive! Its value will be high or low depending on the performance of the classifier with respect to the positive class, even if it is unbalanced. There is no information about the negative class! To see this effect, let's take the example of the cancer dataset: we had 100 samples, 90 of which were cases without tumor and 10 with tumor. Suppose that our classifier predicts 99 samples as belonging to the class "without tumor" and 1 sample as belonging to the class "tumor". Clearly, of the 99 samples predicted as "without tumor", 9 are incorrect and instead belong to the "tumor" class.

If we assume that the positive class is "without tumor" we have:

$$TP = 90 \qquad TN = 1 \qquad FP = 9 \qquad FN = 0 \tag{5.59}$$

hence, we obtain:

$$\text{Precision } = \frac{TP}{TP + FP} = 0.909 \qquad \text{Recall } = \frac{TP}{TP + FN} = 1.0 \tag{5.60}$$

while the F1-Score is:

$$\text{F1-score } = 2\frac{P \cdot R}{P + R} = 2\frac{0.909 \cdot 1.0}{0.909 + 1.0} = 0.9524 \tag{5.61}$$

Thus we have a very high F1-Score and very close to accuracy:

$$\text{Accuracy } = \frac{\#\text{ correct predictions}}{\#\text{ total predictions}} = \frac{91}{100} = 0.91 \tag{5.62}$$

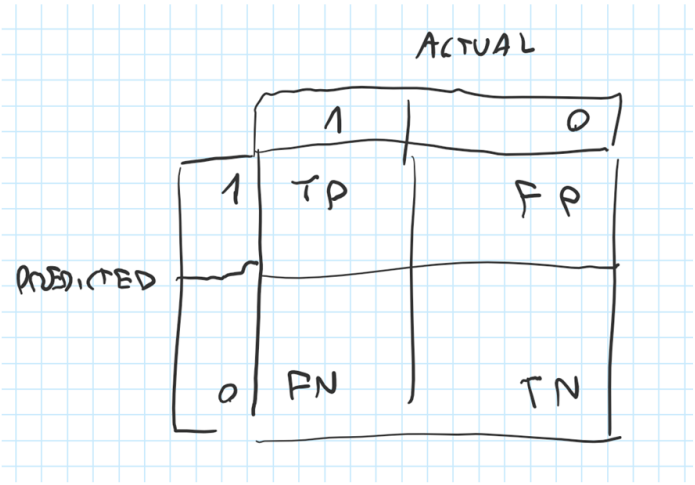


Figure 5.8: Confusion matrix.

Therefore, it seems that the F1-score is not able to capture the poor performance of the model with respect to the tumor class. However, if instead we calculate it considering the class "tumor" as the positive one we obtain:

$$TP = 1 \qquad TN = 90 \qquad FP = 0 \qquad FN = 9 \tag{5.63}$$

$$\text{Precision } = 1.0 \qquad \text{Recall } = 0.1 \tag{5.64}$$

$$\text{F1-score } = 2\frac{1 \cdot 0.1}{1.0 + 0.1} = 0.1818 \tag{5.65}$$

In this case we see how the F1-score detects the fact that the classifier is not performing well with respect to the class "tumor".

In general, the F1-score should be computed with respect to the class deemed most important. If both classes are of interest, then it is appropriate to consider the average of the calculated F1-scores by first considering one of the classes as positive and then recalculating it by considering the other as such.

5.7.4 Confusion Matrix

Accuracy and recall can be visualized through the **Confusion Matrix** (see Figure 5.8).

The confusion matrix can also be calculated in the multiclass case (see Figure 5.9), where, however, the concepts of TP, TN, FP and FN must be slightly revised: it is in fact necessary to calculate for each class the 4 values (and possibly the F1-score if of interest) considering the latter as positive and all the others as negative.

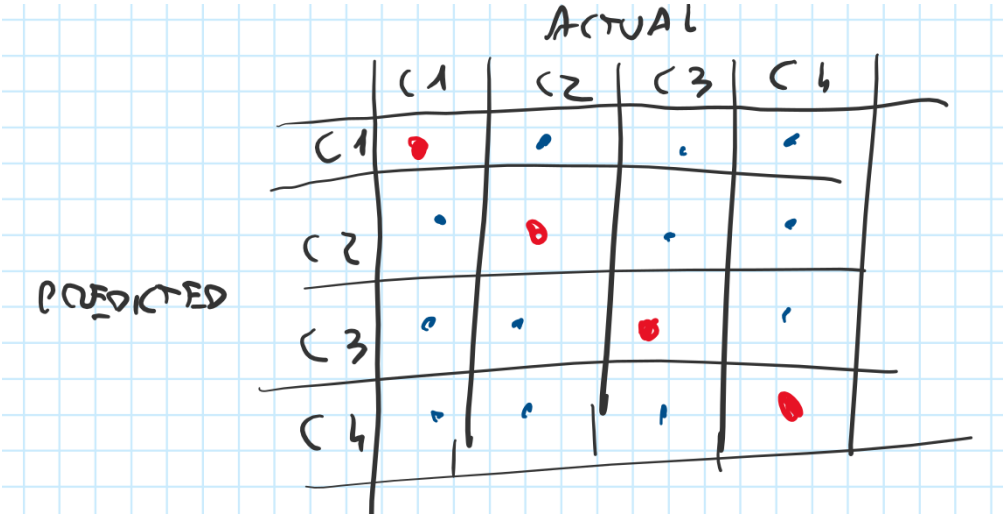


Figure 5.9: Confusion matrix for the multiclass case. In the cells with red dots the number of samples correctly classified for each class are reported. The blue dots indicate the cells containing the number of samples "confused" with other classes.

5.7.5 Receiver Operator Characteristic Curve - ROC Curve

The ROC curve depends on two quantities:

$$\text{True Positive Rate (TPR)} = \frac{TP}{TP + FN} \tag{5.66}$$

$$\text{False Positive Rate (FPR)} = \frac{FP}{FP + FN} \tag{5.67}$$

The ROC then shows the trend of TPR and FPR as the decision threshold changes (see Figure 5.10). To have a metric associated with the ROC curve, we consider the **Area Under the Curve (AUC)**. In general, the AUC can also be calculated for the P-R Curve.

Normally we use the AUC of the P-R curve when we are more interested in the number of false positives with respect to the least represented class in the dataset, while the ROC curve is more useful to analyze the overall FP number. As a rule of thumb, it is good to use the P-R Curve on unbalanced datasets and where the class of interest is the one with fewer samples. On **Balanced** datasets, on the other hand, it is more appropriate to use the AUC of the ROC Curve.

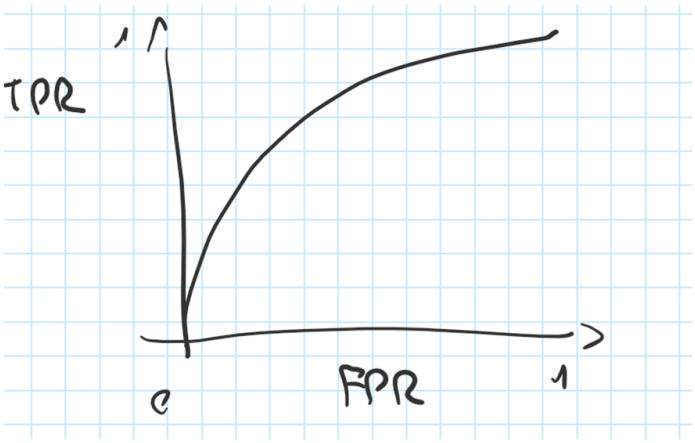


Figure 5.10: ROC Curve.

Chapter 6

Generalized Linear Models (GLMs)*

Before introducing generative models, let us pause for a moment to reflect on the relationship between linear regression and logistic regression. We have seen how the former solves regression problems, while, for classification problems, we have introduced logistic regression by making some changes to the linear regression hypothesis function.

In fact, as we will see in this chapter, these two approaches refer to a common framework from which it is possible to derive both (and derive other models as well): the **Generalized Linear Models (GLMs)**.

6.1 Exponential Family formulation

All models that belong to the GLM family are characterized by distributions of the output variable of exponential type (below we express the distribution with respect to a single sample $(\mathbf{x}, t)$):

$$p(t|\eta(\mathbf{x}, \mathbf{w})) = b(t) \exp(\eta^T T(t) - a(\eta)) \tag{6.1}$$

In this formula, η is called **natural parameter** or **canonical parameter** of the distribution and binds the input and trainable parameters $\mathbf{w}$ to the output. In the formula, the relationship between η and $\mathbf{x}$ and $\mathbf{w}$ is not made explicit to keep the notation compact (for completeness, we should write $\eta(\mathbf{x}, \mathbf{w})$). We will see in the next sections how η is obtained from the features and trainable parameters $\mathbf{w}$.

$T(t)$ is a function of the random variable called **sufficient statistic** (we will clarify what we mean by sufficient statistic shortly). $a(\eta)$ on the other hand is called **log partition function**, whose role is to act as a normalizer to ensure that $p(t|\eta)$ is a valid pdf.

By choosing T, a and b we can identify a family of distributions parameterized by the canonical parameter η and by varying the latter we can obtain specific distributions

within this family.

We now see how the Gaussian and Bernoulli distributions are families of exponential distributions.

6.1.1 Bernoulli Distribution as special case of the exponential distribution

Consider the Bernoulli distribution that we used in the previous chapter for Logistic Regression. This distribution models a binary random variable and, recalling the expression we saw in the previous chapter, the distribution is characterized by the parameter ϕ:

$$\begin{aligned} p(t=1|\phi) &= \phi \\ p(t=0|\phi) &= 1-\phi \end{aligned} \tag{6.2}$$

By varying ϕ we get different instances of the family of Bernoulli distributions (each with a different mean).

What is the connection between the Bernoulli distribution and the exponential family? Also, we just said that by varying ϕ we get specific instances of the Bernoulli family. But what is the connection between the Bernoulli family-specific parameter ϕ and the canonical parameter η?

To answer these questions, let's see if there is an appropriate choice of T, a and b for which the Bernoulli distribution can be expressed in the exponential form:

$$\begin{aligned} p(t|\phi(\mathbf{x},\mathbf{w})) &= \phi(\mathbf{x},\mathbf{w})^t(1-\phi(\mathbf{x},\mathbf{w}))^{1-t} = \exp(t\log\phi(\mathbf{x},\mathbf{w}) + (1-t)\log(1-\phi(\mathbf{x},\mathbf{w}))) \\ &= \exp\left(\left(\log\frac{\phi(\mathbf{x},\mathbf{w})}{1-\phi(\mathbf{x},\mathbf{w})}\right)t + \log(1-\phi(\mathbf{x},\mathbf{w}))\right) \end{aligned} \tag{6.3}$$

where we see that, by substituting

$$\begin{aligned} &T(t) = t \\ &a(\eta(\mathbf{x},\mathbf{w})) = -\log(1-\phi(\mathbf{x},\mathbf{w})) = \log(1+e^{\eta}(\mathbf{x},\mathbf{w})) \\ &b(t) = 1 \\ &\eta(\mathbf{x},\mathbf{w}) = \log(\frac{\phi(\mathbf{x},\mathbf{w})}{1-\phi(\mathbf{x},\mathbf{w})}) \end{aligned} \tag{6.4}$$

we get the exponential form 6.1. Thus, we have answered the first question. To answer the second, it is sufficient to refer to the last equation of 6.4 (the one expressing the canonical parameter as a function of the parameter ϕ of the Bernoulli distribution). it is also interesting to see that by inverting the latter (*i.e.,* expressing ϕ as a function of η) we get:

$$\phi = \frac{1}{1+e^{-\eta}} \tag{6.5}$$

that is, the parameter ϕ of the Bernoulli distribution is given by the sigmoid applied to the canonical parameter (we will see shortly how the canonical parameter binds to the input features). The choice of the sigmoid for the logistic regression is, therefore, not an arbitrary choice of design, but derives from the fact that the Bernoulli distribution belongs to the exponential family.

6.1.2 Gaussian Distribution as special case of the exponential distribution

An analogous reasoning to that made for the Bernoulli distribution can be made for the Gaussian distribution. To do so, we first assume that its variance is $\sigma^2 = 1$. We set this arbitrary value to simplify the derivation and, in this way, we only refer to the parameter μ of the Gaussian. We can perform this simplification since, as we saw when deriving the MLE solution for linear regression, the optimal set of $\mathbf{w}$ does not depend on σ^2. In any case, it is possible to show that the Gaussian belongs to the exponential family also considering σ^2 as a variable.

$$\begin{aligned} p(t|\mu) &= \frac{1}{2\pi} \exp\left(-\frac{1}{2}(t-\mu)^2\right) \\ &= \frac{1}{2\pi} \exp\left(-\frac{1}{2}t^2\right) \exp\left(\mu t - \frac{1}{2}\mu^2\right) \end{aligned} \tag{6.6}$$

Equivalence with the general form of the exponential distribution can be obtained by choosing:

$$\begin{aligned} T(t) &= t \\ a(\eta) &= \frac{\mu^2}{2} = \frac{\eta^2}{2} \\ b(t) &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{t^2}{2}\right) \\ \eta &= \mu \end{aligned} \tag{6.7}$$

In this case we see that the canonical parameter is just the same as the parameter controlling the Gaussian family μ. As already mentioned, the dependence between η and the input features and the trainable parameters $\mathbf{w}$ is not yet made explicit. We will see it in the next section.

6.2 Building GLM for Linear, Logistic and Softmax Regression

Let's see now how it is possible to build the models we introduced for linear and logistic regression from the general formulation of GLM. We will also see how, starting from this

general formulation, it is possible to introduce the **Softmax Regression**, the extension of logistic regression for the multiclass case.

To do so, we first need to provide some more details about the concept of sufficient statistic.

6.2.1 Sufficient Statistic

Definition 6.1. Sufficient Statistic: Consider a set $X_0, X_1, \ldots, X_{N-1}$ of random samples obtained from a distribution characterized by an unknown parameter w. Then the statistic:

$$T = u(X_0, X_1, \ldots, X_{N-1}) \tag{6.8}$$

is called **sufficient** for w if $p(X_0, X_1, \ldots, X_{N-1}|T) \perp w$.

This means that if the conditional distribution of the samples $X_0, X_1, \ldots, X_{N-1}$ given the statistic T does not depend on w then T is sufficient. In other words, T contains all the information needed to estimate the parameter w, i.e., T is a sufficient "summary" of the data for the purpose of inference about w. If T is sufficient, in fact we have $p(w|X_0, X_1, \ldots, X_{N-1}, T) = p(w|T)$.

A sufficient statistic is therefore an excellent tool to "condense" the data without losing information for the subsequent inference step through, for example, MLE. However, following the definition, to figure out if a statistic is sufficient we need to compute the conditional distribution of $X_0, X_1, \ldots, X_{N-1}$ given T, which in practical cases, may not be a viable route. An easier alternative to figure out if a statistic is sufficient is to use the **Factorization Theorem**:

Theorem 6.2. ***Factorization Theorem****: Consider a set $X_0, X_1, \ldots, X_{N-1}$ with joint distribution $p(X_0, X_1, \ldots, X_{N-1}|w)$ conditioned on parameter w. Then the statistic $T = u(X_0, X_1, \ldots, X_{N-1})$ is sufficient for w if and only if $p(X_0, X_1, \ldots, X_{N-1}|w)$ can be factorized into two components, i.e., :*

$$\begin{aligned} p(X_0, X_1, \ldots, X_{N-1}|w) &= \psi[u(X_0, X_1, \ldots, X_{N-1})|w]h(X_0, X_1, \ldots, X_{N-1}) \\ &= \psi[T|w]h(X_0, X_1, \ldots, X_{N-1}) \end{aligned} \tag{6.9}$$

where

- *ψ is a function that depends on data only through $u(X_0, X_1, \ldots, X_{N-1})$ and*
- *the function $h(X_0, X_1, \ldots, X_{N-1})$ does not depend on w*

Looking at the expression of the exponential family 6.1:

$$p(t|\eta(\mathbf{x}, \mathbf{w})) = \underbrace{b(t)}_{h(X_0, X_1, \ldots, X_{N-1})} \underbrace{\exp(\eta(\mathbf{x}, \mathbf{w})^T T(t) - a(\eta(\mathbf{x}, \mathbf{w})))}_{\psi[T|\eta(\mathbf{x}, \mathbf{w})]} \tag{6.10}$$

it is possible to notice that it is already in factorized form, consequently, $T(t)$ is a sufficient statistic for the parameter $\eta(\mathbf{x}, \mathbf{w})$.

6.2.2 GLM assumption

In general, a classification or regression problem in which we want to estimate an output variable t given the feature vector $\mathbf{x}$, can be solved through the GLM formulation. To construct the GLM model, it is necessary to make 3 assumptions:

- **Assumption 1 - Randomness Factor:** The function $p(t|\mathbf{x}, \gamma)$ must belong to the exponential family. γ denotes the generic parameter controlling the particular exponential distribution family chosen (*e.g.,* $\gamma = \phi$ in the Bernoulli case, or $\gamma = \mu$ in the Gaussian case). Furthermore, the hypothesis function is equal to the expected value of the sufficient statistic conditional on the features:

$$h_{\mathbf{w}}(\mathbf{x}) = E[T(t)|\mathbf{x}] \tag{6.11}$$

  This assumption is reasonable because the sufficient statistic "summarizes" all the information necessary to estimate the unknown parameter γ of the distribution. The link between the expected value, the features and the parameters of the model $\mathbf{w}$ is made explicit through the following two assumptions;

- **Assumption 2**: The link between the expected value $E[T(t)|\mathbf{x}]$ and the canonical parameter is obtained through the **Canonical Response Function**:

$$\gamma = g(\eta) = E[T(t)|\mathbf{x}] \tag{6.12}$$

  The inverse of this function is known as **Canonical Link Function**:

$$\eta = g^{-1}g(\eta) = g^{-1}E[T(t)|\mathbf{x}] \tag{6.13}$$

- **Assumption 3 - Systematic Factor:** The canonical parameter is a linear combination of the features with respect to the $\mathbf{w}$ parameters being trained:

$$\eta = \mathbf{w}^T\mathbf{x} \tag{6.14}$$

These three assumptions create a chain of relationships between the hypothesis function (with which we estimate the expected value of the sufficient statistic), the parameter γ that controls the exponential family, the canonical parameter and, finally, the input features.

Let's see, as an example, the case of Linear Regression. In this case, we had modeled the $p(t|\mathbf{x})$ as a Gaussian, so the first assumption is indeed met. In the Gaussian case, the parameter γ corresponds to the mean μ of the Gaussian, so we have $\gamma = \mu$. Furthermore,

from the 6.7, we see that the canonical response function is $\mu = \eta$ so:

$$\begin{aligned} h_{\mathbf{w}}(\mathbf{x}) &= E[T(t)|\mathbf{x}] \leftarrow \text{Since the Gaussian distribution belongs to the exponential family.} \\ &= E[t|\mathbf{x}] \leftarrow T(t) = t \text{ (see 6.7)} \\ &= E[N(t|\mu, \sigma^2)] = \mu \leftarrow \mu \text{ is the mean of the Gaussian distribution} \\ &= \eta \leftarrow \text{using the canonical response function for the Gaussian} \\ &= \mathbf{w}^T\mathbf{x} \leftarrow \text{using Assumption 3} \end{aligned} \tag{6.15}$$

We then found the Linear Regression hypothesis function that we introduced in previous chapters. The Linear Regression model is then a GLM with Gaussian distribution assumptions for $p(t|\mathbf{x})$.

Similarly, for Logitic regression, remembering that the parameter γ, in this case, corresponds to the mean ϕ of the Bernoulli distribution, we have $\gamma = \phi$. Furthermore, from the 6.5, we see that the canonical response function is $\phi = \frac{1}{1+e^{-\eta}}$ so:

$$\begin{aligned} h_{\mathbf{w}}(\mathbf{x}) &= E[T(t)|\mathbf{x})] \leftarrow \text{Since the Bernoulli distribution belongs to the exponential family.} \\ &= E[t|\mathbf{x}] \leftarrow T(t) = t \text{ (see 6.4)} \\ &= E[Bernoulli(t|\phi)] = \phi \leftarrow \phi \text{ is the mean of the Bernoulli distribution} \\ &= \frac{1}{1+e^{-\eta}} \leftarrow \text{using the canonical response function for the Bernoulli} \\ &= \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}} \leftarrow \text{using Assumption 3} \end{aligned} \tag{6.16}$$

We have therefore seen that both Linear and Logistic regression derive from the more general formulation of GLMs, in which the canonical parameter assumes a fundamental role to "abstract" with respect to the particular form of the family of distributions chosen to solve the specific task (Gaussian for regression and Bernoulli for classification).

6.2.3 Softmax Regression

Now that we understand how GLMs work, we use this framework to build a new learning model. In particular, we will derive the **Softmax Regression**, which extends Logistic Regression for multiclass classification problems.

We then consider the scenario where we want to estimate an output variable t that takes values in the set $\{1, 2, \ldots, K\}$ from a feature vector $\mathbf{x}$. We have already seen that multiclass problems can be handled with binary classifiers and One vs One or One vs All approaches. Softmax Regression, on the other hand, is a model that explicitly accounts for the multiclass nature of the output and thus directly provides predictions in the set $\{1, 2, \ldots, K\}$.

Because the output variable can take on more than two values, the Bernoulli distribution is not appropriate; instead, we will need to use a multinomial distribution. We

begin by verifying that the multinomial distribution is a special case of the family of exponential distributions in order to use the GLM framework.

In order to parameterize a multinomial distribution over K possible values of the output we could think of using K parameters, one describing the probability of each output: $\phi_1, \phi_2, \ldots, \phi_K$. However, since the sum of all these parameters must equal 1 (the sum of the probabilities of all possible events must be 1):

$$\sum_{i=1}^{K} \phi_i = 1 \tag{6.17}$$

one of these parameters is redundant because it is directly determined by the others. We therefore have $K-1$ parameters $\phi_1, \phi_2, \ldots, \phi_{K-1}$ such that:

$$\phi_i = p(t = i|\boldsymbol{\phi}) \tag{6.18}$$

$$\text{and } p(t = K|\boldsymbol{\phi}) = 1 - \sum_{i=1}^{K-1} \phi_i \tag{6.19}$$

The sufficient statistic must also be revised slightly in the multinomial case. In this case we do not have $T(t) = t$, but to express the multinomial distribution in the exponential form, we need to define $T(t) \in \mathbb{R}^{K-1}$ as follows:

$$T(1) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, T(2) = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \ldots, T(K-1) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}, T(K) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \tag{6.20}$$

where by $(T(t))_i$ we will denote the i-th element of the vector and, by using the **indicator function**, we can write for convenience $(T(t))_i = I\{t = i\}$. We also have that :

$$\begin{aligned} E[(T(t))_i] &= 1 \cdot p((T(t))_i = 1|\boldsymbol{\phi}) + 0 \cdot p((T(t))_i = 0|\boldsymbol{\phi}) \\ &= p((T(t))_i = 1|\boldsymbol{\phi}) = p(t = i|\boldsymbol{\phi}) = \phi_i \end{aligned} \tag{6.21}$$

At this point we write the standard form of the multinomial distribution and starting

from this we modify it to obtain the exponential form:

$$\begin{aligned}
p(t|\boldsymbol{\phi}) &= \phi_1^{I\{t=1\}}\phi_2^{I\{t=2\}}\dots\phi_K^{I\{t=K\}} \\
&= \phi_1^{I\{t=1\}}\phi_2^{I\{t=2\}}\dots\phi_K^{1-\sum_{i=1}^{K-1}I\{t=i\}} \\
&= \phi_1^{(T(t))_1}\phi_2^{(T(t))_2}\dots\phi_K^{1-\sum_{i=1}^{K-1}(T(t))_i} \\
&= \exp\left[(T(t))_1\log(\phi_1) + (T(t))_2\log(\phi_2) + \dots + (1-\sum_{i=1}^{K-1}(T(t))_i)\log(\phi_K)\right] \\
&= \exp\left[(T(t))_1\log(\phi_1/\phi_K) + (T(t))_2\log(\phi_2/\phi_K) + \dots + \log(\phi_{K-1}/\phi_K) + \log(\phi_K)\right]
\end{aligned} \tag{6.22}$$

The last equation is equivalent to the 6.1 if we define the canonical parameter, $a(\boldsymbol{\eta})$ e $b(t)$ as follows:

$$\begin{aligned}
\boldsymbol{\eta} &= \begin{bmatrix} \log(\phi_1/\phi_K) \\ \log(\phi_2/\phi_K) \\ \vdots \\ \log(\phi_{K-1}/\phi_K) \end{bmatrix} \\
a(\boldsymbol{\eta}) &= -\log(\phi_K) \\
b(t) &= 1
\end{aligned} \tag{6.23}$$

So the canonical link function is given by:

$$\begin{aligned}
\eta_i &= \log\frac{\phi_i}{\phi_K} \\
\eta_K &= \log\frac{\phi_K}{\phi_K} = 0
\end{aligned} \tag{6.24}$$

while to get the canonical response function, we first note that:

$$e^{\eta_i} = \frac{\phi_i}{\phi_K} \implies \phi_K e^{\eta_i} = \phi_i \implies \phi_K\sum_{i=1}^{K}e^{\eta_i} = \sum_{i=1}^{K}\phi_i = 1 \implies \phi_K = \frac{1}{\sum_{i=1}^{K}e^{\eta_i}} \tag{6.25}$$

hence, substituting in $e^{\eta_i} = \frac{\phi_i}{\phi_K}$ we get:

$$\phi_i = \frac{e^{\eta_i}}{\sum_{j=1}^{K}e^{\eta_j}} \tag{6.26}$$

The canonical response function expressed in 6.26 is named **Softmax Function**. To complete the derivation of the GLM, we use Assumption 3 by positing $\eta_i = \mathbf{w}_i^T\mathbf{x}$, where $\mathbf{w}_1, \dots, \mathbf{w}_{K-1}$ are the trainable parameter vectors, one for each class, excluding the K-th one for which, for convenience, we place $\mathbf{w}_K = 0$ to have $\eta_K = \mathbf{w}_K^T\mathbf{x} = 0$ as already seen in 6.24.

One can easily see that $\sum_{i=1}^{K} p(t = i|\mathbf{x}, \mathbf{w}) = \sum_{i=1}^{K} \frac{e^{\eta_i}}{\sum_{j=1}^{K} e^{\eta_j}} = 1$ and that $\frac{e^{\eta_i}}{\sum_{j=1}^{K} e^{\eta_j}} \leq 1$. This is why 6.26 is called **Softmax Function**: it is a "softened max", which does not give a net prediction of the class, but gives a probability.

Consequently, the parameter of the multinomial distribution (which corresponds to the conditional distribution of the output variable) is given by:

$$p(t = i|\mathbf{x}, \mathbf{w}) = \phi_i = \frac{e^{\eta_i}}{\sum_{j=1}^{K} e^{\eta_j}} = \frac{e^{\mathbf{w}_i^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x}}} \tag{6.27}$$

The hypothesis function, following the GLM approach we saw earlier, is given by:

$$\begin{aligned} h_{\mathbf{W}} &= E[T(t)|\mathbf{x}, \mathbf{W}] \\ &= \begin{bmatrix} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_{K-1} \end{bmatrix} \\ &= \begin{bmatrix} \frac{e^{\mathbf{w}_1^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x}}} \\ \frac{e^{\mathbf{w}_2^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x}}} \\ \vdots \\ \frac{e^{\mathbf{w}_{K-1}^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x}}} \end{bmatrix} \end{aligned} \tag{6.28}$$

Thus, the probability of a particular class $p(t = i|\mathbf{x}, \mathbf{W})$ is given by the related softmax function $\frac{e^{\mathbf{w}_i^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x}}}$.

To find the optimal parameter set for softmax regression, we can again exploit maximum likelihood estimation. Assuming we have a dataset of N i.i.d. samples, we can write the likelihood function as:

$$\begin{aligned} l(\mathbf{W}) &= \sum_{i=0}^{N-1} \log p(t^{(i)}|\mathbf{x}^{(i)}, \mathbf{W}) \\ &= \sum_{i=0}^{N-1} \log \phi_1 \phi_2 \dots \phi_K \\ &= \sum_{i=0}^{N-1} \log \prod_{l=1}^{K} \left(\frac{e^{\mathbf{w}_l^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x}}} \right)^{I\{t^{(i)}=l\}} \end{aligned} \tag{6.29}$$

and computing the gradient, we can train the parameters $\mathbf{W}$ using an iterative method such as gradient descent.

Chapter 7

Generative Models for Classification

As we mentioned in Chapter 5, generative models attempt to make inferences directly on the joint distribution of features and classes. In this way, by applying Bayes' theorem, it is then possible to compute the posterior distribution and estimate the class by setting a decision threshold on the latter. The probability that the class relative to the feature vector $\mathbf{x}$ is the k-th can be calculated as:

$$p(t = C_k|\mathbf{x}) = \frac{p(\mathbf{x}|t = C_k)p(t = C_k)}{\sum_{j=1}^{K} p(\mathbf{x}|t = C_j)p(t = C_j)} \tag{7.1}$$

For simplicity of notation, below we will use $p(t = C_k|\mathbf{x}) = p(C_k|\mathbf{x})$.

If we refer to the binary case and write the previous equation with respect to the class C_1:

$$\begin{aligned} p(C_1|\mathbf{x}) &= \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_1)p(C_1) + p(\mathbf{x}|C_2)p(C_2)} \\ &= \frac{\cancel{p(\mathbf{x}|C_1)p(C_1)}}{\cancel{p(\mathbf{x}|C_1)p(C_1)}\left(1 + \frac{p(\mathbf{x}|C_2)p(C_2)}{p(\mathbf{x}|C_1)p(C_1)}\right)} \\ &\underbrace{=}_{\text{by setting } a=\log \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_2)p(C_2)}} \frac{1}{1 + e^{-a}} = \sigma(a) \end{aligned} \tag{7.2}$$

Thus we see that, even if we start from the joint distribution, the conditional probability of the output class is still given by an expression that depends on the sigmoid. This is in line with what we have already seen with GLMs and is reasonable, since although generative models start from the joint distribution, they still use the posterior distribution for prediction. The difference with discriminative models, however, lies in the fact that the posterior is modeled from the class conditional distribution and the prior: this allows us to explicitly integrate our possible knowledge about the process into the model.

In the multiclass case, if we apply the exponential identity $e^{a_k} = e^{\log(p(\mathbf{x}|C_k)p(C_k))}$ to

the expression 7.1 we obtain:

$$p(C_k|\mathbf{x}) = \frac{e^{a_k}}{\sum_{j=1}^{K} e^{a_j}} \tag{7.3}$$

that is, for the multiclass case, even if we start from the joint distribution, the posterior distribution of the class is given by the softmax function. Again, this result is expected, since it corresponds to that obtained for the softmax regression, *i.e.,* the discriminative model we introduced to handle multiclass problems.

Let us now see what are some possible structural choices for the class-conditional distributions $p(\mathbf{x}|C_k)$ and for the prior $p(C_k)$. The choices we make with respect to these distributions are crucial, since they lead to different generative models.

7.1 Gaussian Class Conditional Densities

Under the assumption that our features take on continuous values (*i.e.,* $\in \mathbb{R}$), a reasonable choice for the family of distributions with which to encode the class-conditional distribution is the Gaussian. The family of models obtained with this choice are referred to as **Gaussian Discriminant Analysis (GDA)** models. In the following, we will introduce two of them: **Linear Discriminant Analysis (LDA)** and **Quadratic Discriminant Analysis (QDA)**.

7.1.1 Linear Discriminant Analysis (LDA)

Let's first assume that the class-conditional distribution is Gaussian with the following form:

$$p(\mathbf{x}|C_k) = \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)} \tag{7.4}$$

In the equation, it is important to note that we have assumed that the averages $\boldsymbol{\mu}_k$ are different for each class, while the covariance Σ is the same for all classes. We will see in a moment how this last assumption will allow us to make simplifications in the derivation of the model.

> The averages $\boldsymbol{\mu}_k$ are vectors, since the Gaussian is multivariate (the feature vector $\mathbf{x} \in \mathbb{R}^{D\times 1}$).

We now try to compute the posterior distribution in the binary case for class C_1 assuming that the class-conditional distribution has a Gaussian form. From the 7.2 we have that:

$$p(C_1|\mathbf{x}) = \sigma(a) = \sigma\left(\log \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_2)p(C_2)}\right) \tag{7.5}$$

so, unfolding the sigmoid argument we have:

$$\log\left(\frac{\cancel{\frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}}}e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_1)^T\sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_1)}}{\cancel{\frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}}}e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_2)^T\sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_2)}}\right)+\log\frac{p(C_1)}{p(C_2)} \tag{7.6}$$

where the simplification was possible because we assumed that the covariances of the two classes are equal. By continuing, we get:

$$\begin{aligned}
&= -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_1)^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_1)+\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_2)^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu}_2)+\log\frac{p(C_1)}{p(C_2)}\\
&= -\frac{1}{2}(\cancel{\mathbf{x}^T\Sigma^{-1}\mathbf{x}}-\mathbf{x}^T\Sigma^{-1}\boldsymbol{\mu}_1-\boldsymbol{\mu}_1^T\Sigma^{-1}\mathbf{x}+\boldsymbol{\mu}_1^T\Sigma^{-1}\boldsymbol{\mu}_1)\\
&+\frac{1}{2}(\cancel{\mathbf{x}^T\Sigma^{-1}\mathbf{x}}-\mathbf{x}^T\Sigma^{-1}\boldsymbol{\mu}_2-\boldsymbol{\mu}_2^T\Sigma^{-1}\mathbf{x}+\boldsymbol{\mu}_2^T\Sigma^{-1}\boldsymbol{\mu}_2)\\
&= -\frac{1}{2}(-2\boldsymbol{\mu}_1^T\Sigma^{-1}\mathbf{x}+\boldsymbol{\mu}_1^T\Sigma^{-1}\boldsymbol{\mu}_1)+\frac{1}{2}(-2\boldsymbol{\mu}_2^T\Sigma^{-1}\mathbf{x}+\boldsymbol{\mu}_2^T\Sigma^{-1}\boldsymbol{\mu}_2)+\log\frac{p(C_1)}{p(C_2)}\\
&= \boldsymbol{\mu}_1^T\Sigma^{-1}\mathbf{x}-\boldsymbol{\mu}_2^T\Sigma^{-1}\mathbf{x}-\frac{1}{2}\boldsymbol{\mu}_1^T\Sigma^{-1}\boldsymbol{\mu}_1+-\frac{1}{2}\boldsymbol{\mu}_2^T\Sigma^{-1}\boldsymbol{\mu}_2+\log\frac{p(C_1)}{p(C_2)}\\
&= \mathbf{w}^T\mathbf{x}+w_0
\end{aligned} \tag{7.7}$$

where we defined:

$$\begin{aligned}
\mathbf{w} &= \Sigma^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_2)\\
w_0 &= -\frac{1}{2}\boldsymbol{\mu}_1^T\Sigma^{-1}\boldsymbol{\mu}_1+\frac{1}{2}\boldsymbol{\mu}_2^T\Sigma^{-1}\boldsymbol{\mu}_2+\log\frac{p(C_1)}{p(C_2)}
\end{aligned} \tag{7.8}$$

Where you can easily verify that w_0 is a scalar.

We therefore have that:

$$h_{\mathbf{w}}(\mathbf{x}) = p(C_1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}+w_0) \tag{7.9}$$

We again obtained a form that involves the sigmoid for the hypothesis function, similarly to what we saw in discriminative models (for Logistic Regression). Therefore, we have again a linear decision function: we had in fact already seen with Logistic Regression that imposing a threshold on the sigmoid is equivalent to imposing a threshold on its argument (which is a linear combination of the input with respect to the parameters). It is important to note that the simplifications implemented in the 7.7 were possible thanks to the assumption that the covariances of the two class-conditional distributions are equal.

The generative model expressed in 7.9 with the parameters defined according to 7.8 is called **Linead Discriminant Analysis (LDA)**.

In LDA, the sigmoid function has the role, as in Logistic Regression, of making the output a valid probability, while the term $\log\frac{p(C_1)}{p(C_2)}$ represents our a priori knowledge about the probability of the classes: if there is no such knowledge, it is reasonable to

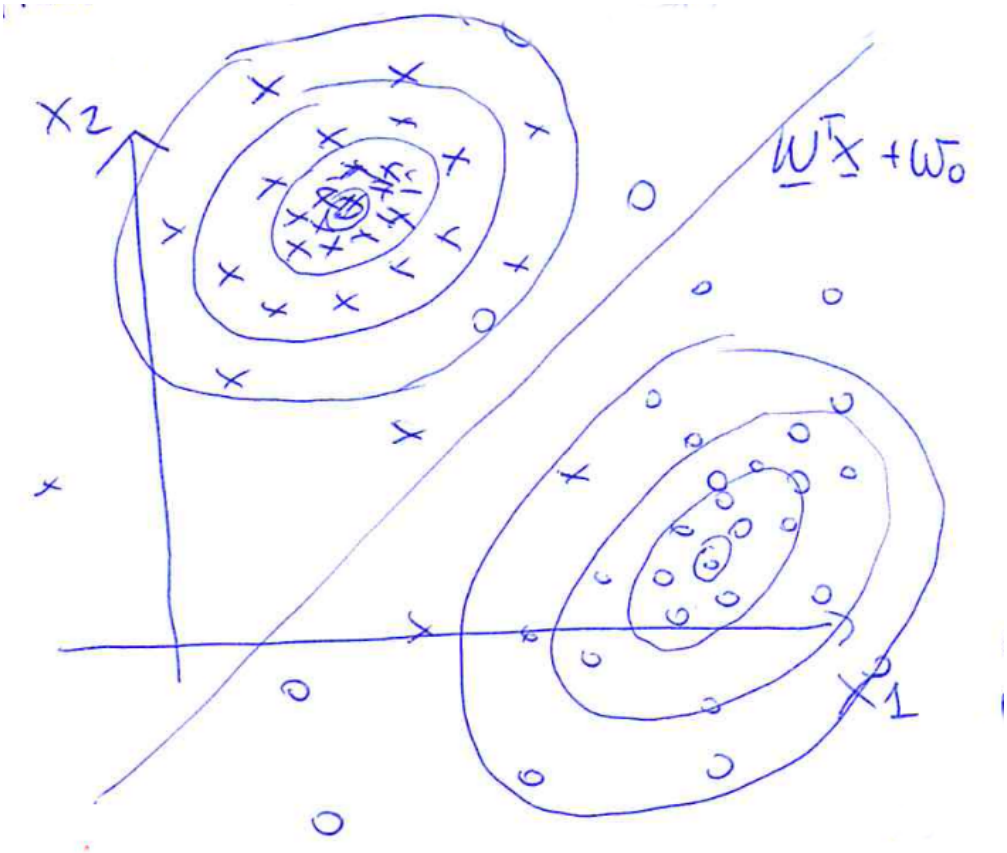


Figure 7.1: Decision boundary and class-conditional Gaussian distributions for the binary case. The plot depicts and example with two features.

assume $p(C_1) = P(C_2)$, so the term $\log \frac{p(C_1)}{p(C_2)}$ goes to zero and has no effect on the prediction.

An example of class conditional distributions with the associated decision hyperplane is shown in Figure 7.1.

> N.B. The fact that the separator is a line in the two-feature case is related to the simplification of the quadratic term $\mathbf{x}^T\Sigma^{-1}\mathbf{x}$, having chosen the covariances of the two equal class-conditional distributions.

Instead, it is possible to observe the role of the sigmoid in the graph in Figure 7.2. In this figure we see how the probability of class C_1 is encoded as the distance between the sigmoid and the feature plane.

As with Logistic Regression, a threshold can be imposed on the posterior to make the class decision:

$$p(C_1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + w_0) \geq 0.5 \implies \mathbf{w}^T\mathbf{x} + w_0 \geq \sigma^{-1}(0.5) = \log\left(\frac{0.5}{1-0.5}\right) = 0 \quad (7.10)$$

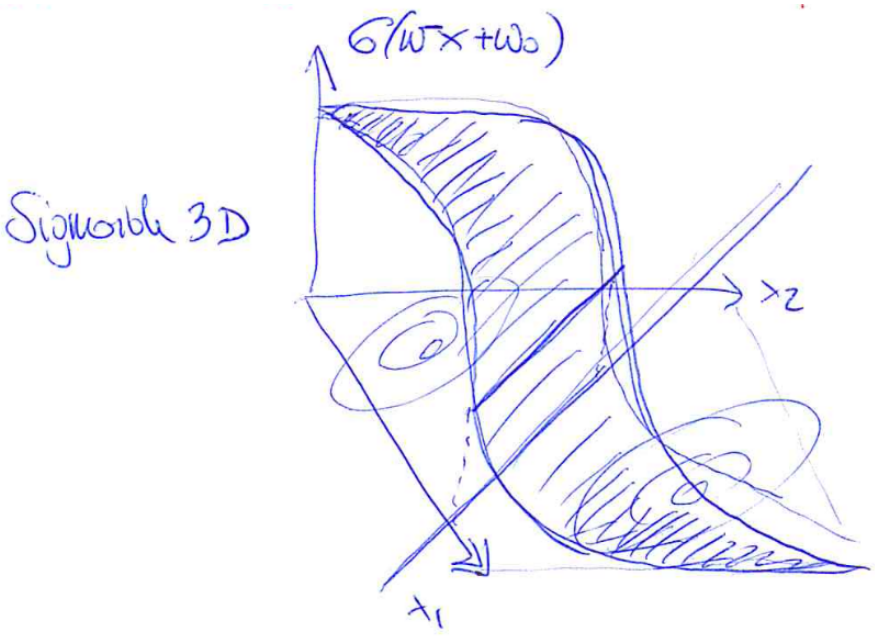


Figure 7.2: Role of the sigmoid function for the example depicted in Figure 7.1.

LDA for multiclass problems

In the multiclass case the posterior is given by:

$$p(C_k|\mathbf{x}) = \frac{e^{a_k}}{\sum_{j=1}^{K} e^{a_j}} \tag{7.11}$$

with

$$e^{a_k} = e^{\log(p(\mathbf{x}|C_k)p(C_k))} \tag{7.12}$$

By still assuming that the Gaussian class-conditional distributions all have the same covariance, it is possible to show that we obtain a linear separator in this case as well:

$$\begin{aligned} a_k &= \mathbf{w}_k^T \mathbf{x} + w_{k,0} \\ \mathbf{w}_k &= \Sigma^{-1} \boldsymbol{\mu}_k \\ w_{k,0} &= -\frac{1}{2} \boldsymbol{\mu}_k^T \Sigma^{-1} \boldsymbol{\mu}_k + \log p(C_k) \end{aligned} \tag{7.13}$$

In this case, the predicted class is the one for which:

$$a_i > a_j \ \forall i \neq j \ j = 1, 2, \ldots, K \tag{7.14}$$

while the probability of that prediction is given by the softmax function:

$$p(C_i|\mathbf{x}) = \frac{e^{a_i}}{\sum_{j=1}^{K} e^{a_j}} \tag{7.15}$$

7.1.2 Quadratic Discriminant Analysis (QDA)

The assumption of having equal covariances for all Gaussian class-conditional distributions allows us to have a linear separator in the feature space. If we remove this constraint, we obtain a much more flexible model, with a separator that is no longer linear and therefore allows to handle more complex distributions. The model we obtain in this case is called **Quadratic Discriminant Analysis (QDA)**. With the QDA, given its greater flexibility, it is necessary to pay attention to overfitting: the number of parameters to be determined in fact grows quadratically with the number of features and if the hypothesis of Gaussian class-conditional distribution is not verified in reality we will experience worse performance.

7.1.3 MLE per LDA

In the previous sections, we derived the hypothesis function for the LDA model. We now apply the MLE framework to estimate the optimal values for the parameters $\mathbf{w}$. We first note that, from 7.8, $\mathbf{w}$ and w_0 are computed by using the means of the Gaussians, the covariance of the two class-conditional distributions and the parameters of the priors on the classes. Considering the binary case, the prior on the classes can be modeled by a Bernoulli distribution with parameter ϕ:

$$\begin{aligned} p(C_1) &= \phi \\ p(C_2) &= 1 - \phi \end{aligned} \tag{7.16}$$

With MLE then our aim is to find the optimal values for μ_1, μ_2, $Sigma$ and ϕ.

Since LDA is a generative model, we start with the joint distribution for the single sample and express it as the product of the class-conditional and the prior:

$$\begin{aligned} p(\mathbf{x}^{(i)}, C_1) &= p(C_1)p(\mathbf{x}^{(i)}|C_1) = \phi N_1(\mathbf{x}^{(i)}|\boldsymbol{\mu}_1, \Sigma) \\ p(\mathbf{x}^{(i)}, C_2) &= p(C_2)p(\mathbf{x}^{(i)}|C_2) = (1-\phi) N_2(\mathbf{x}^{(i)}|\boldsymbol{\mu}_2, \Sigma) \end{aligned} \tag{7.17}$$

We now compute the joint distribution with respect to the entire dataset (always assuming we have N samples i.i.d.), combining the cases for the two classes and denoting by $\mathbf{t}$ the vector of labels and $\mathbf{X}$ the design matrix:

$$p(\mathbf{t}, \mathbf{X}|\phi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \Sigma) = \prod_{i=0}^{N-1} [\phi N_1(\mathbf{x}^{(i)}|\boldsymbol{\mu}_1, \Sigma)]^{t^{(i)}} [(1-\phi) N_2(\mathbf{x}^{(i)}|\boldsymbol{\mu}_2, \Sigma)]^{(1-t^{(i)})} \tag{7.18}$$

We compute, as usual for MLE, the log-likelihood (which, in this case, is computed on

the joint and not on $p(\mathbf{t}|Xb, \mathbf{w})$ as for Linear and Logistic Regression):

$$\begin{aligned} l(\phi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \Sigma) &= \log p(\mathbf{t}, \mathbf{X}|\phi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \Sigma) \\ &= \sum_{i=0}^{N-1} \Big[t^{(i)}(\log \phi + \log N_1(\mathbf{x}^{(i)}|\boldsymbol{\mu}_1, \Sigma)) \\ &\quad + (1 - t^{(i)})(\log(1-\phi) + \log N_2(\mathbf{x}^{(i)}|\boldsymbol{\mu}_2, \Sigma)) \Big] \end{aligned} \tag{7.19}$$

To derive the optimal values for the trainable parameters, as usual we derive the log-likelihood with respect to the parameter of interest and set it equal to zero. We start with ϕ:

$$\begin{aligned} \frac{\partial l(\phi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \Sigma)}{\partial \phi} &= \sum_{i=0}^{N-1} \left(t^{(i)} \frac{1}{\phi} - (1 - t^{(i)}) \frac{1}{1-\phi} \right) = 0 \\ &\implies \frac{1}{\phi} \sum_{i=0}^{N-1} t^{(i)} - \frac{1}{1-\phi} \sum_{i=0}^{N-1} 1 + \frac{1}{1-\phi} \sum_{i=0}^{N-1} t^{(i)} = 0 \\ &\implies \sum_{i=0}^{N-1} t^{(i)} - \cancel{\phi \sum_{i=0}^{N-1} t^{(i)}} - \phi \underbrace{\sum_{i=0}^{N-1} 1}_{N} + \cancel{\phi \sum_{i=0}^{N-1} t^{(i)}} = 0 \\ &\implies \phi = \frac{\sum_{i=0}^{N-1} t^{(i)}}{N} = \frac{N_1}{N} \end{aligned} \tag{7.20}$$

where by N_1 we denoted the samples for which $t^{(i)} = 1$ ($t^{(i)}$ is equal to 1 or 0 depending on the class to which the sample belongs, so the summation result will be equal to the number of samples belonging to the class labeled as 1).

Let us now see for $\boldsymbol{\mu}_1$ (the case for $\boldsymbol{\mu}_2$ is analogous):

$$\begin{aligned}
\nabla_{\boldsymbol{\mu}_1} l(\phi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \Sigma) = \nabla_{\boldsymbol{\mu}_1} \Bigg[\sum_{i=0}^{N-1} \Bigg[t^{(i)} (\underbrace{\cancel{\log \phi}}_{=0} + \log N_1(\mathbf{x}^{(i)} | \boldsymbol{\mu}_1, \Sigma)) \\
+ (1 - t^{(i)}) (\underbrace{\cancel{\log(1-\phi)}}_{=0} + \underbrace{\cancel{\log N_2(\mathbf{x}^{(i)} | \boldsymbol{\mu}_2, \Sigma))}}_{=0} \Bigg] \Bigg] = 0 \\
\implies \nabla_{\boldsymbol{\mu}_1} \left[\sum_{i=0}^{N-1} t^{(i)} \log N_1(\mathbf{x}^{(i)} | \boldsymbol{\mu}_1, \Sigma)) \right] = 0 \\
\implies \nabla_{\boldsymbol{\mu}_1} \left[-\frac{1}{2} \sum_{i=0}^{N-1} t^{(i)} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_1)^T \Sigma^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_1) + cost \right] = 0 \\
\implies \nabla_{\boldsymbol{\mu}_1} \Bigg[-\frac{1}{2} \sum_{i=0}^{N-1} t^{(i)} \Big((\mathbf{x}^{(i)})^T \Sigma^{-1} \mathbf{x}^{(i)} - (\mathbf{x}^{(i)})^T \Sigma^{-1} \boldsymbol{\mu}_1 \\
- \boldsymbol{\mu}_1^T \Sigma^{-1} \mathbf{x}(i) + \boldsymbol{\mu}_1^T \Sigma^{-1} \boldsymbol{\mu}_1 \Big) + \underbrace{\cancel{cost}}_{=0} \Bigg] = 0
\end{aligned} \tag{7.21}$$

Multiplying everything by Σ and deriving we get:

$$\begin{aligned}
\nabla_{\boldsymbol{\mu}_1} \left[-\frac{1}{2} \sum_{i=0}^{N-1} t^{(i)} \left(\underbrace{\cancel{\mathbf{x}(i)^T \mathbf{x}(i)}}_{=0} \underbrace{-\mathbf{x}(i)^T \boldsymbol{\mu}_1 - \boldsymbol{\mu}_1^T \mathbf{x}(i)}_{=-2(\mathbf{x}(i))^T} \underbrace{+\boldsymbol{\mu}_1^T \boldsymbol{\mu}_1}_{2\boldsymbol{\mu}_1} \right) \right] = 0 \\
\implies \sum_{i=0}^{N-1} t^{(i)} \mathbf{x}^{(i)} = \sum_{i=0}^{N-1} t^{(i)} \boldsymbol{\mu}_1 \\
\boldsymbol{\mu}_1 = \frac{1}{\sum_{i=0}^{N-1} t^{(i)}} \sum_{i=0}^{N-1} t^{(i)} \mathbf{x}^{(i)} = \frac{1}{N_1} \sum_{i=0}^{N-1} t^{(i)} \mathbf{x}^{(i)}
\end{aligned} \tag{7.22}$$

and similarly for $\boldsymbol{\mu}_2$:

$$\boldsymbol{\mu}_1 = \frac{1}{N_2} \sum_{i=0}^{N-1} (1 - t^{(i)}) \mathbf{x}^{(i)} \tag{7.23}$$

Then the optimal values for $\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_2$ are computed as the weighted sample averages that corresponds to the class 1 and 0, respectively.

Finally, we compute the optimal value for Σ We first eliminate the terms that depend

on ϕ since their gradient with respect to Σ is equal to 0). Then, we get:

$$\begin{aligned}\nabla_\Sigma l(\phi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \Sigma) = \nabla_\Sigma \Bigg[\sum_{i=0}^{N-1} \Big[& t^{(i)} \log N_1(\mathbf{x}^{(i)} | \boldsymbol{\mu}_1, \Sigma) \\ & + (1 - t^{(i)}) \log N_2(\mathbf{x}^{(i)} | \boldsymbol{\mu}_2, \Sigma) \Big] \Bigg] = 0 \\ \implies \sum_{i=0}^{N-1} t^{(i)} \nabla_\Sigma \Bigg[& \log \underbrace{\frac{1}{(2\pi)^{D/2}}}_{=0} + \log \frac{1}{|\Sigma|^{1/2}} - \frac{1}{2} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_1)^T \Sigma^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_1) \Bigg] + \\ \sum_{i=0}^{N-1} (1 - t^{(i)}) \nabla_\Sigma \Bigg[& \log \underbrace{\frac{1}{(2\pi)^{D/2}}}_{=0} + \log \frac{1}{|\Sigma|^{1/2}} - \frac{1}{2} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_2)^T \Sigma^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_2) \Bigg] = 0\end{aligned} \tag{7.24}$$

since $\nabla_\Sigma |\Sigma| = |\Sigma| (\Sigma^{-1})^T$ we get:

$$\begin{aligned}\sum_{i=0}^{N-1} \Bigg[& \Big(\cancel{\frac{1}{2}} t^{(i)} \frac{1}{|\Sigma|} |\Sigma| (\Sigma^{-1})^T \Big) + \Big(\cancel{\frac{1}{2}} (1 - t^{(i)}) \frac{1}{|\Sigma|} |\Sigma| (\Sigma^{-1})^T \Big) \\ & \cancel{\frac{1}{2}} t^{(i)} \nabla_\Sigma \Big((\mathbf{x}^{(i)} - \boldsymbol{\mu}_1)^T \Sigma^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_1) \Big) \cancel{\frac{1}{2}} (1 - t^{(i)}) \nabla_\Sigma \Big((\mathbf{x}^{(i)} - \boldsymbol{\mu}_2)^T \Sigma^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_2) \Big) \Bigg] = 0\end{aligned} \tag{7.25}$$

furthermore, since $\nabla_\Sigma A^T \Sigma^{-1} A = -\Sigma^{-1} A A^T \Sigma^{-1}$ and since Σ^{-1} is symmetric:

$$\begin{aligned}\implies & \sum_{i=0}^{N-1} \Big[t^{(i)} \Sigma^{-1} + (1 - t^{(i)}) \Sigma^{-1} \Big] - \sum_{i=0}^{N-1} t^{(i)} \Sigma^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_1)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_1)^T \Sigma^{-1} \\ & - \sum_{i=0}^{N-1} (1 - t^{(i)}) \Sigma^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_2)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_2)^T \Sigma^{-1} = 0 \\ \implies & \sum_{i=0}^{N-1} \Sigma^{-1} - \Sigma^{-1} \underbrace{\sum_{i=0}^{N-1} \Big(t^{(i)} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_1)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_1)^T \Big)}_{S_1} \Sigma^{-1} \\ & - \Sigma^{-1} \underbrace{\sum_{i=0}^{N-1} \Big(t^{(i)} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_1)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_1)^T \Big)}_{S_2} \Sigma^{-1} = 0 \\ \implies & N \Sigma^{-1} - \Sigma^{-1} (S_1) \Sigma^{-1} - \Sigma^{-1} (S_2) \Sigma^{-1} = 0 \\ \implies & \Sigma = \frac{1}{N} S_1 + \frac{1}{N} S_2\end{aligned} \tag{7.26}$$

S_1 and S_2 are the sample variances, in vector form, computed with the samples of class 1 and 0. The MLE solution for the covariance is then a weighted average of the sample

variances of the samples that belong to the two classes.

7.2 Class-Conditional Distributions for Discrete Features

The GDA models introduced in the previous section have class-conditional distributions of Gaussian type. This choice, as mentioned above, is appropriate for continuous features (or features that can be considered continuous), but is not suitable to handle cases where the features are discrete.

We start by considering the case of binary features. Specifically, we consider feature vectors that are D dimensional, but where each element can only take on the values 0 and 1, *i.e.,* $\mathbf{x} = \{x_0, x_1, \ldots, x_{D-1}\}$ with $x_i \in \{0, 1\}$, or, more compactly, $\mathbf{x} \in \{0, 1\}^D$. The class conditional distribution $p(\mathbf{x}|C_k)$, though discrete, has $2^D - 1$ values, which correspond to all possible combinations of presence/absence of each feature.

Let us take an example with the case of the spam classifier. In this case $t \in$ spam, not spam$\}$, while $\mathbf{x} \in \{0, 1\}^D$ is a vector in which each element encodes the presence or absence of a certain vocabulary word in the email text. If we assume that we have a vocabulary of 50000 words, our feature vector will have 50000 binary features, *i.e.,* $\mathbf{x} \in \{0, 1\}^{50000}$ and, consequently, the class-conditional distribution will be characterized by $2^{50000} - 1$ probability to be determined for the "spam" case:

$$p(x_0, x_1, \ldots, x_{49998}, x_{49999}|t = \text{"spam"}) \tag{7.27}$$

and as many for the "non-spam" case.

Determining all these values to characterize the joint distribution and then calculating the posterior over the classes is clearly complex. Therefore, to simplify the problem, it is common to use the **Naive Bayes Assumption**, which assumes that the features are **Conditionally Independent** given the class t. With respect to the example of the spam classifier, this means that, although the words "buy" and "deal" are not, in general, independent (observing the word "buy" intuitively increases the probability of observing the word "deal"), they become so when we observe that the email is actually "spam" or "not spam". In other words, if we know that the email received is "spam", observing the word "buy" does not give me any information with respect to the probability of observing the word "deal".

It is clear that this is a simplifying hypothesis that does not fully reflect reality: even if we know that the email is "spam", the observation of a certain word can potentially provide some information about the others. However, the hypothesis we make using Naive Bayes is that the amount of information lost with this assumption is negligible. Clearly it is important to understand, with respect to the problem at hand, whether this assumption is reasonable or too strong.

With the Naive Bayes Assumption, the class-conditional probabilities are simplified

as follows:

$$\begin{aligned} & p(x_0, x_1, \ldots, x_{49998}, x_{49999} | t = \text{"spam"}) \\ & = p(x_0 | t = \text{"spam"}) p(x_1 | t = \text{"spam"}) \ldots p(x_{49999} | t = \text{"spam"}) \\ & = \prod_{j=0}^{D-1} p(x_j | t = \text{"spam"}) \end{aligned} \tag{7.28}$$

Each of the class-conditional distributions of each word can now be modeled by a Bernoulli distribution (the features can only take binary values), so:

$$p(x_j | t = \text{"spam"}) = \phi_{j|1}^{x_j} (1 - \phi_{j|1})^{1 - x_j} \tag{7.29}$$

where by $\phi_{j|1}$ we denote the parameter of the Bernoulli distribution relative to feature j and having observed class 1 (we have two different parameters for the two cases where the observed class is 1 or 0). At this point, to determine the class-conditional distribution, we only need to determine $\phi_{j|1}$ and $\phi_{j|0}$ for the D features, for a total of $2D$ values instead of $2(2^D - 1)$. To these we must then add the ϕ_t parameter for the priors on the two classes, thus obtaining $2D + 1$ parameters with the Naive Bayes Assumption.

We now use the MLE framework to estimate these parameters. Assuming, as always, that we have N sample i.i.d., the likelihood can be written as:

$$\begin{aligned} & p(\mathbf{t} | \mathbf{X}, \phi_{0|1}, \ldots, \phi_{D-1|1}, \phi_{0|0}, \ldots, \phi_{D-1|0}, \phi_t) \propto \prod_{i=0}^{N-1} p(\mathbf{x}^{(i)}, t^{(i)}) \\ & \propto \prod_{i=0}^{N-1} p(\mathbf{x}^{(i)} | t^{(i)}) p(t^{(i)}) \\ & \propto \prod_{i=0}^{N-1} \left[\prod_{j=0}^{D-1} \phi_{j|t^{(i)}}^{x_j^{(i)}} (1 - \phi_{j|t^{(i)}})^{1 - x_j^{(i)}} \right] p(t^{(i)}) \end{aligned} \tag{7.30}$$

where we left the proportionality since the denominator of Bayes' theorem cancels out during maximization, and we did not make explicit the Bernoulli distribution for the prior on $t^{(i)}$ to keep the notation compact. Calculating the log-likelihood we obtain:

$$\begin{aligned} & l(\boldsymbol{\phi}, \phi_t) = \log(p(\mathbf{t} | \mathbf{X}, \phi_{0|1}, \ldots, \phi_{D-1|1}, \phi_{0|0}, \ldots, \phi_{D-1|0}, \phi_t)) \\ & \propto \sum_{i=0}^{N-1} \left[\sum_{j=0}^{D-1} \left(x_j^{(i)} \log \phi_{j|t^{(i)}} + (1 - x_j^{(i)}) \log(1 - \phi_{j|t^{(i)}}) \right) + \log p(t^{(i)}) \right] \end{aligned} \tag{7.31}$$

Where $\boldsymbol{\phi}$ is the vector containing all the training parameters.

To compute the parameter $\phi_{j|1}$ we derive the log-likelihood with respect to it and

set it equal to zero (and similarly we can do this for $\phi_{j|0}$ and ϕ_t):

$$\begin{aligned}\frac{\partial l(\boldsymbol{\phi}, \phi_t)}{\partial \phi_{j|1}} &= \sum_{i=0}^{N_1-1} \left[x_j^{(i)} \frac{1}{\phi_{j|1}} - (1 - x_j^{(i)}) \frac{1}{1 - \phi_{j|1}} \right] = 0 \\ &\implies \phi_{j|1} = \frac{\sum_{i=0}^{N_1-1} x_j^{(i)}}{N_1} = \frac{\sum_{i=0}^{N-1} I\{x_j^{(i)} = 1 \text{ and } t^{(i)} = 1\}}{\sum_{i=0}^{N-1} I\{t^{(i)} = 1\}}\end{aligned} \tag{7.32}$$

where by N_1 we denoted the number of samples for which $t^{(i)} = 1$ (since the derivative with respect to $\phi_{j|1}$ will be different from zero only for these).

Similarly for $\phi_{j|0}$ and ϕ_t we can show that the optimal values are given by:

$$\phi_{j|0} = \frac{\sum_{i=0}^{N_0-1} x_j^{(i)}}{N_0} = \frac{\sum_{i=0}^{N-1} I\{x_j^{(i)} = 1 \text{ and } t^{(i)} = 0\}}{\sum_{i=0}^{N-1} I\{t^{(i)} = 0\}} \tag{7.33}$$

$$\phi_t = \frac{\sum_{i=0}^{N-1} I\{t^{(i)} = 1\}}{N} = \frac{N_1}{N} \tag{7.34}$$

Once the parameters are determined, it is possible to estimate the predicted class for an input $\mathbf{x}$ by computing the posterior over the classes using Bayes' theorem:

$$\begin{aligned}p(t = 1|\mathbf{x}) &= \frac{p(\mathbf{x}|t = 1)p(t = 1)}{p(\mathbf{x})} \\ &= \frac{\left[\prod_{j=0}^{D-1} p(x_j|t = 1)\right] p(t = 1)}{\left[\prod_{j=0}^{D-1} p(x_j|t = 1)\right] p(t = 1) + \left[\prod_{j=0}^{D-1} p(x_j|t = 0)\right] p(t = 0)}\end{aligned} \tag{7.35}$$

and the class is assigned by checking if $p(t = 1|\mathbf{x}) > p(t = 0|\mathbf{x})$.

7.2.1 Naive Bayes for Multinomial Features*

In the previous section we considered the case where features take on binary values (0 and 1). How does the formulation with Nauie Bayes changes in the case of multinomial features, *i.e.,* features that can take more than two values?

To model the class-conditional probability in this case, we need to use the **multinomial distribution**. Recall that we have already seen this distribution with the softmax regression, where in that case, however, it was used to model the posterior over the classes known the featues and not the distribution of features known the class.

Multinomial Distribution The multinomial distribution allows to model discrete random variables that can take more than two values. In this case, the random variable is encoded with **one-hot vectors**, i.e., vectors of length equal to the number of values that the variable can take and in which each element represents one of these values, ordered with respect to the indices. All elements of the vector are set to zero, except for the one related to the value assumed by the variable, which is set to 1. For example, given a random variable X that can take 6 values, the possible encodings of its one-hot vector are:

$$\begin{aligned}
\mathbf{x} &= [1,0,0,0,0,0]^T \\
\mathbf{x} &= [0,1,0,0,0,0]^T \\
\mathbf{x} &= [0,0,1,0,0,0]^T \\
\mathbf{x} &= [0,0,0,1,0,0]^T \\
\mathbf{x} &= [0,0,0,0,1,0]^T \\
\mathbf{x} &= [0,0,0,0,0,1]^T
\end{aligned} \tag{7.36}$$

With the constraint that each element of the vector can only take on values 0 and 1, it is possible to summarize the condition of the one-hot vector with the following relationship:

$$\sum_{l=1}^{L} x_l = 1 \tag{7.37}$$

where x_l are the various components of the vector and L is the number of possible values that the random variable can take.
If we denote by μ_l the probability of realization of each possible value of the feature, we can write the pdf of the multinomial as follows:

$$p(\mathbf{x}|\boldsymbol{\mu}) = \prod_{l=1}^{L} \mu_l^{x_l} \tag{7.38}$$

with

$$\sum_{l=1}^{L} \mu_l = 1 \tag{7.39}$$

and

$$\mu_l \geq 0 \ \forall l \tag{7.40}$$

As we saw in the softmax regression case, one of the parameters is redundant in that it can be expressed through the others: $1 - \sum_{l=1}^{L-1} \mu_l$.
The expected value of each x_l is given by:

$$\begin{aligned} E[\mathbf{x}|\boldsymbol{\mu}] &= \sum_{\mathbf{x}} p(\mathbf{x}|\boldsymbol{\mu})\mathbf{x} = p\left(\begin{bmatrix}1\\0\\\vdots\\0\end{bmatrix}|\boldsymbol{\mu}\right)\begin{bmatrix}1\\0\\\vdots\\0\end{bmatrix} + \cdots + p\left(\begin{bmatrix}0\\0\\\vdots\\1\end{bmatrix}|\boldsymbol{\mu}\right)\begin{bmatrix}0\\0\\\vdots\\1\end{bmatrix} \\ &= \mu_1\begin{bmatrix}1\\0\\\vdots\\0\end{bmatrix} + \cdots + \mu_L\begin{bmatrix}0\\0\\\vdots\\1\end{bmatrix} = \begin{bmatrix}\mu_1\\\mu_2\\\vdots\\\mu_L\end{bmatrix} \end{aligned} \tag{7.41}$$

The posterior on the classes (assuming the case of binary classification and that the features can all take the same number of values), with multinomial features, can then be written as:

$$p(t^{(i)}|\mathbf{X}^{(i)}, \mathcal{M}) \propto p(\mathbf{X}^{(i)}|t^{(i)}, \mathcal{M})p(t^{(i)}) \tag{7.42}$$

where we used the matrix notation for the features of the single sample because each feature is itself a one-hot vector of L possible values:

$$\mathbf{X}^{(i)} = \begin{bmatrix} x_{0,1}^{(i)} & x_{0,2}^{(i)} & \cdots & x_{0,L}^{(i)} \\ x_{1,1}^{(i)} & x_{1,2}^{(i)} & \cdots & x_{1,L}^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ x_{D-1,1}^{(i)} & x_{D-1,2}^{(i)} & \cdots & x_{D-1,L}^{(i)} \end{bmatrix} \tag{7.43}$$

while $\mathcal{M}$ represents the matrix with the parameters of the multinomial distribution for each feature:

$$\mathcal{M} = \begin{bmatrix} \mu_{0,1} & \mu_{0,2} & \cdots & \mu_{0,L} \\ \mu_{1,1} & \mu_{1,2} & \cdots & \mu_{1,L} \\ \vdots & \vdots & \ddots & \vdots \\ \mu_{D-1,1} & \mu_{D-1,2} & \cdots & \mu_{D-1,L} \end{bmatrix} \tag{7.44}$$

Under the Naive Bayes hypothesis, the features are independent known the class, so:

$$\begin{aligned} p(\mathbf{X}^{(i)}|t^{(i)},\mathcal{M}) &= p(\mathbf{x}_0^{(i)},\mathbf{x}_1^{(i)},\ldots,\mathbf{x}_{D-1}^{(i)}|t^{(i)},\mathcal{M}) \\ &= p(\mathbf{x}_0^{(i)}|t^{(i)},\mathcal{M})p(\mathbf{x}_1^{(i)}|t^{(i)},\mathcal{M})\ldots p(\mathbf{x}_{D-1}^{(i)}|t^{(i)},\mathcal{M}) \\ &= \prod_{j=0}^{D-1} p(\mathbf{x}_j^{(i)}|t^{(i)},\mathcal{M}) \end{aligned} \tag{7.45}$$

so, considering N sample i.i.d., the likelihood becomes:

$$\begin{aligned} p(\mathbf{t}|\mathbf{X},\mathcal{M}) &\propto p(\mathbf{X}|\mathbf{t},\mathcal{M})p(\mathbf{t}) \\ &\propto \prod_{i=0}^{N-1}\left[\prod_{j=0}^{D-1}\prod_{l=1}^{L}\mu_{j,l|t^{(i)}}^{x_{j,l}^{(i)}}\right]p(\mathbf{t}) \end{aligned} \tag{7.46}$$

To estimate the optimal value for the generic $\mu_{j,l|1}$ relative to the class-conditional distribution conditioned on observing class 1 (similarly can be computed for class 0), we compute, as usual, the log-likelihood and maximize the derivative with respect to the parameter of interest:

$$\begin{aligned} \frac{\partial \log p(\mathbf{t}|\mathbf{X},\mathcal{M})}{\partial \mu_{j,l|1}} &= \frac{\partial}{\partial \mu_{j,l|1}} \log \prod_{i=0}^{N-1}\left[\prod_{j=0}^{D-1}\prod_{l=1}^{L}\mu_{j,l|t^{(i)}}^{x_{j,l}^{(i)}}\right] + \underbrace{\cancel{\log p(t^{(i)})}}_{\text{does not depend on } \mu_{j,l|1}} \\ &= \frac{\partial}{\partial \mu_{j,l|1}} \log \left[\underbrace{\cancel{\prod_{j=0}^{D-1}}}_{\text{only the j-th feature depends on } \mu_{j,l|1}} \prod_{i=0}^{N-1}\prod_{l=1}^{L}\mu_{j,l|t^{(i)}}^{x_{j,l}^{(i)}}\right] \end{aligned} \tag{7.47}$$

since only the parameters related to the class-conditional distributions conditioned on class 1 depend on $\mu_{j,l|1}$ we obtain:

$$\begin{aligned} &= \frac{\partial}{\partial \mu_{j,l|1}} \log \left[\mu_{j,1|1}^{x_{j,0}^{(0)}}\ldots\mu_{j,L|1}^{x_{j,L}^{(0)}}\ldots\mu_{j,1|1}^{x_{j,1}^{(N_1-1)}}\ldots\mu_{j,L|1}^{x_{j,L}^{(N_1-1)}}\cancel{\mu_{j,1|0}^{x_{j,0}^{(N_1)}}\ldots\mu_{j,L|0}^{x_{j,L}^{(N_1)}}\ldots\mu_{j,1|0}^{x_{j,1}^{(N-1)}}\ldots\mu_{j,L|0}^{x_{j,L}^{(N-1)}}}\right] \\ &= \frac{\partial}{\partial \mu_{j,l|1}} \log \left[\prod_{l=1}^{L}\mu_{j,l|1}^{\sum_{i=0}^{N_1-1}x_{j,l}^{(i)}}\right] \end{aligned} \tag{7.48}$$

where we have assumed that the samples are organized so that the first N_1 belong to class 1 and the rest to class 0. If we denote by $N_{j,l|1}$ the number of samples for which $\{x_{j,l}=1 \text{ and } t^{(i)}=1\}$ we have:

$$\frac{\partial \log p(\mathbf{t}|\mathbf{X},\mathcal{M})}{\partial \mu_{j,l|1}} = \frac{\partial}{\partial \mu_{j,l|1}} \log \left[\prod_{l=1}^{L}\mu_{j,l|1}^{N_{j,l|1}}\right] = \frac{\partial}{\partial \mu_{j,l|1}} \sum_{l=1}^{L} N_{j,l|1}\log \mu_{j,l|1} \tag{7.49}$$

When maximizing the previous formula (or minimizing the negative log-likelihood), however, we must take into account that we have a constraint on the parameters $\mu_{j,l|1}$ that imposes that their sum over the L feature values is 1. This constraint must be taken into account during optimization to avoid the trivial solution $\mu_{j,l|1} = 0$. Therefore, the optimization problem becomes:

$$\min_{\mu_{j,l|1}} \sum_{l=1}^{L} N_{j,l|1} \log \mu_{j,l|1} \tag{7.50}$$

$$\text{s.t.} \sum_{l=1}^{L} \mu_{j,l|1} = 1 \tag{7.51}$$

The optimization problem above is constrained. To solve it, we need to introduce an additional tool: the Lagrange multiplier method (see Chapter 8). We write the Lagrangian function associated with the problem 7.50:

$$L(\mu_{j,l|1}, \lambda) = \sum_{l=1}^{L} N_{j,l|1} \log \mu_{j,l|1} + \lambda(\sum_{l=1}^{L} \mu_{j,l|1} - 1) \tag{7.52}$$

having only an equality constraint, it suffices to compute the gradient with respect to $\mu_{j,l|1}$ and with respect to λ and set them to zero:

$$\frac{\partial L(\mu_{j,l|1}, \lambda)}{\partial \mu_{j,l|1}} = 0 \implies N_{j,l|1} \frac{1}{\mu_{j,l|1}} + \lambda = 0 \tag{7.53}$$

$$\frac{\partial L(\mu_{j,l|1}, \lambda)}{\partial \lambda} = 0 \implies \sum_{l=1}^{L} \mu_{j,l|1} - 1 = 0 \tag{7.54}$$

Summing for $l = 1, \ldots, L$ both terms of the first equation and substituting $\sum_{l=1}^{L} \mu_{j,l|1} = 1$ given by the second equation (the one related to the constraint) we get:

$$\sum_{l=1}^{L} N_{j,l|1} = -\lambda \underbrace{\sum_{l=1}^{L} \mu_{j,l|1}}_{=1} \implies \lambda = -\sum_{l=1}^{L} N_{j,l|1} = N_1 \tag{7.55}$$

By resubstituting the value of λ in the first equation of 7.53 we have:

$$N_{j,l|1} \frac{1}{\mu_{j,l|1}} + N_1 = 0 \implies \mu_{j,l|1} = \frac{N_{j,l|1}}{N_1} = \frac{\sum_{i=0}^{N-1} I(\{x_{j,l}^{(i)} = 1 \text{ and } t^{(i)} = 1\})}{\sum_{i=0}^{N-1} \{t^{(i)} = 1\}} \tag{7.56}$$

7.2.2 Laplace Smoothing

Taking the example of the spam classifier, we saw how each feature represents the presence or absence of a word. The parameter of the Bernoulli distribution associated with

a particular feature j is given by:

$$\phi_{j|1} = \frac{\sum_{i=0}^{N_1-1} x_j^{(i)}}{N_1} = \frac{\sum_{i=0}^{N-1} I\{x_j^{(i)} = 1 \text{ and } t^{(i)} = 1\}}{\sum_{i=0}^{N-1} I\{t^{(i)} = 1\}} \tag{7.57}$$

and with respect to class 0 by:

$$\phi_{j|0} = \frac{\sum_{i=0}^{N_0-1} x_j^{(i)}}{N_0} = \frac{\sum_{i=0}^{N-1} I\{x_j^{(i)} = 1 \text{ and } t^{(i)} = 0\}}{\sum_{i=0}^{N-1} I\{t^{(i)} = 0\}} \tag{7.58}$$

If, however, a given word is never present in the dataset, the numerators of the previous equations are equal to 0 and the related parameters $\phi_{j|1}$ and $\phi_{j|0}$ cancel. During inference, when we compute the probability of a class given the input vector we get an undefined expression (the numerator and denominator products cancel due to the presence of a null term):

$$\begin{aligned} p(t=1|\mathbf{x}) &= \frac{p(\mathbf{x}|t=1)p(t=1)}{p(\mathbf{x})} \\ &= \frac{\left[\prod_{j=0}^{D-1} p(x_j|t=1)\right] p(t=1)}{\left[\prod_{j=0}^{D-1} p(x_j|t=1)\right] p(t=1) + \left[\prod_{j=0}^{D-1} p(x_j|t=0)\right] p(t=0)} = \frac{0}{0} \end{aligned} \tag{7.59}$$

This problem emerges due to the limitations of Maximum Likelihood Estimation (MLE), which assigns null probability to words it has never seen in the dataset (MLE always risks overfitting the data). To overcome this problem, we could modify the expressions for parameter estimation as follows:

$$\phi_{j|1} = \frac{\sum_{i=0}^{N-1} I\{x_j^{(i)} = 1 \text{ and } t^{(i)} = 1\} + 1}{\sum_{i=0}^{N-1} I\{t^{(i)} = 1\} + 2} \tag{7.60}$$

and with respect to class 0 by:

$$\phi_{j|0} = \frac{\sum_{i=0}^{N_0-1} x_j^{(i)}}{N_0} = \frac{\sum_{i=0}^{N-1} I\{x_j^{(i)} = 1 \text{ and } t^{(i)} = 0\} + 1}{\sum_{i=0}^{N-1} I\{t^{(i)} = 0\} + 2} \tag{7.61}$$

In fact, we have artificially increased the number of times a word is observed, *i.e.,* we are assuming that a word is observed at least once given that a certain class has been observed. In this way, the numerator of the previous expressions cannot be zero. This process is known as **Laplace Smoothing** which in its general form (thus considering that each feature is represented by a multinomial random variable) predicts that the parameter estimates for the Multinomial distributions of each feature are modified as follows:

$$\mu_{j,l|1} = \frac{\sum_{i=0}^{N-1} I(\{x_{j,l}^{(i)} = 1 \text{ and } t^{(i)} = 1\}) + 1}{\sum_{i=0}^{N-1} \{t^{(i)} = 1\} + K} \tag{7.62}$$

where K represents the number of values that the j-th feature can take.

7.2.3 Bag of Words (BoW)

If we always refer to the case of spam filtering, the encoding that we chose in the previous sections to represent the words in the text of the email states that each word is represented by indicating its presence or absence with a binary random variable. This choice is very common in problems that involve the processing of textual input (*e.g.,* for applications of document classification, natural language processing, sentiment analysis, ...). In general, in these applications, since the inputs can have different lengths (email texts can be made up of a variable number of words), it is necessary to find strategies to have a fixed length representation of the input. This process of **vectorization** of the input is what we implemented when we introduced Naive Bayes for the case of the spam classifier: we decided to represent each email with a vector of fixed length (equal to the size of the vocabulary of words), in which each element encodes the presence or absence of the relevant vocabulary word. This approach is called **Bag of Words (BoW)**.

Clearly, this representation is a simplification and discards a lot of information, such as syntactic rules, sequence and relationship between expressed concepts, etc... However, in many cases this encoding provides good results. Moreover, we will now see some modifications to the BoW technique to make it more representative.

Choice of the Vocabulary The choice of words that compose our vocabulary has an impact both with respect to the expressiveness of the coding and to the computational cost. Larger vocabularies correspond to longer feature vectors and therefore a larger number of parameters must be determined. Conversely, small vocabularies have fewer parameters, but may not be sufficiently representative.

There are techniques of unsupervised learning (some of which we will see later) to create vocabularies that are "optimized" with respect to the available data. For the moment, however, we assume that we identify the P words in our vocabulary as the most common (relative to a threshold) in a set of documents that we will use as a training set for vocabulary identification. In this way, the vocabulary will not contain all the words of a certain language, but only the most significant ones with respect to their frequency in the training documents (see Figure 7.3).

TF Weighting Not all vocabulary words carry the same weight with respect to determining the features of a text. For example, some words might be repeated many times because they describe the concept expressed by a document and are therefore very important (consider, for example, the words "train" or "linear" in a machine learning article).

We could then give a higher importance to words that are repeated many times within a document. In practice, our feature vector will not only encode the presence or absence

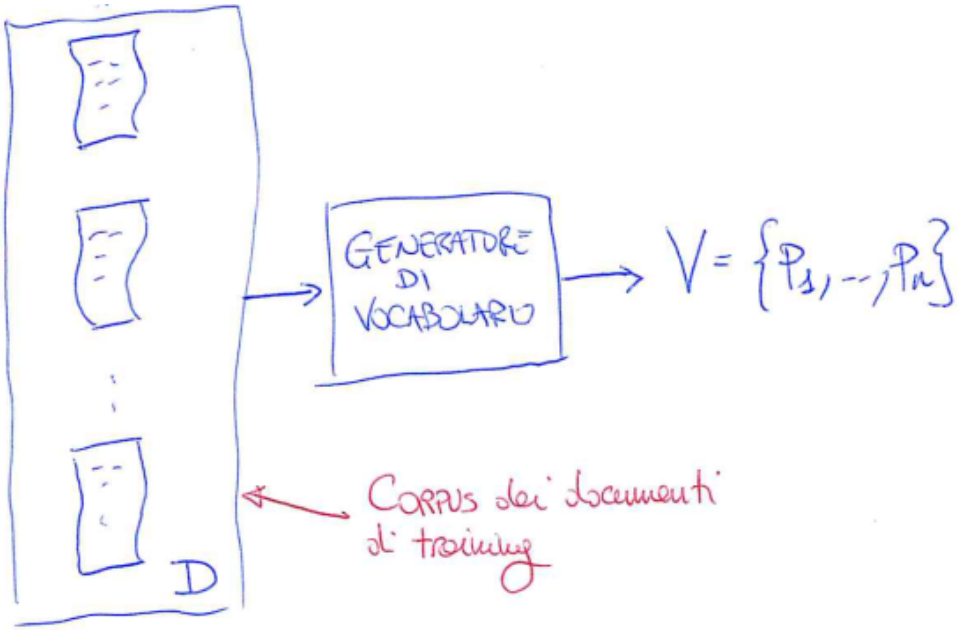


Figure 7.3: Creation of a BoW vocabulary by considering the most frequent words in a training set of documents..

of a word, but also provide information with respect to how frequently they appear in the document. In fact, we are "weighing" the words in the input representing that document with respect to their frequency, and this approach is called **Term Frequency (TF) weighting**. We must, however, avoid the case where some documents assign more weight to certain words than others just because they are longer. For this reason, TF weighting is normalized with respect to the frequency of the most common term in the document:

$$TF(j,d) = 0.5 + 0.5 \frac{\overbrace{\#\{j \text{ in } d\}}^{\text{frequency of the word j in document d}}}{\underbrace{\max\{\#\{j \text{ in } d\}, \forall j \in d\}}_{\text{frequency of the most common term in d}}} \tag{7.63}$$

IDF Weighting In addition to weighing a word with its frequency in the document, it is reasonable to assume that words that are very common in many documents are not very representative (consider articles or propositions, for example). We can then decrease the weight of words that are very common in the entire training set of documents. This type of weighting is known as **Inverse Document Frequency (IDF)**:

$$IDF(j,D) = \log \frac{\overbrace{|D|}^{\text{Number of documents in the training set}}}{\underbrace{|\{d \in D : t \in d\}|}_{\text{Number of documents in the training set where the word j appears}}} \tag{7.64}$$

In general, the term TF and IDF are combined in the **TF-IDF Weighting**.

Feature Expansion To try to partially capture the syntactic and semantic content persisting in the document, one possibility is to create features that represent combinations of words, e.g., pairs of very common words. In general, combinations of n word are called **n-gram**. This process is analogous to the polynomial expansion we saw in the Linear Reegression case. Clearly, with this approach, the expressiveness of the feature vector increases, but the number of parameters also rises and we are therefore more subject to the risk of overfitting. It is therefore appropriate to use model selection techniques to determine the level of expansion to be used.

Chapter 8

Lagrange Multiplier Method

Most of the problems we have seen so far involved optimizing unconstrained objective functions. How can we instead handle optimization functions that have constraints with respect to the input domain?

Consider, in general, the following problem:

$$\begin{aligned} \min \text{(or max)} \; & f(\mathbf{x}) \\ \text{subject to} \; & h_1(\mathbf{x}) \\ & h_2(\mathbf{x}) \\ & \vdots \\ & h_M(\mathbf{x}) \end{aligned} \tag{8.1}$$

where $\mathbf{x} \in \mathbb{R}^D$ (where D is the dimension of the feature vector) and the functions f and $h_1, \ldots, h_M$ are continuous and with continuous partial second derivatives (*i.e.,* belong to C^2).

We begin with a simple example: consider the problem of maximizing a function of two variables $f(x_1, x_2)$ subject to the constraint on x_1 and x_2 that we can generically write in the form $h(x_1, x_2) = 0$. A first approach to the problem might be to use the constraint equation to express one variable as a function of the other, *e.g.,* $x_2 = g(x_1)$, substitute in the expression of f which would become $f(x_1, g(x_1))$, maximize with respect to x_1 and use the optimal value to find x_2^* as well. However, it is not necessarily straightforward to express one variable analytically as a function of the other via the constraint. Also, this approach treats the two variables differently and thus breaks the symmetry that is naturally present in the constraint.

An alternative consists in introducing a parameter λ called **Lagrange Multiplier**. It is possible to understand the role of this parameter starting from some geometric considerations. The constraint $h(x_1, x_2) = 0$ in fact defines a curve in the space $D - 1$ (in this case 1-dimensional), and differentiating with respect to x_1 we obtain (using the

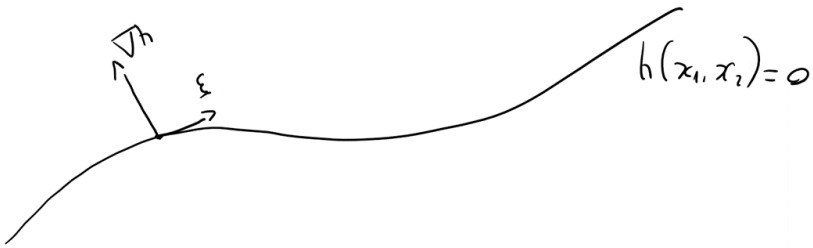


Figure 8.1: The gradient ∇h and the tangent $\boldsymbol{\epsilon}$ to the curve are orthogonal.

chain rule for derivatives):

$$\frac{\partial h}{\partial x_1} + \frac{\partial h}{\partial x_2}\frac{dx_2}{dx_1} = 0 \tag{8.2}$$

from which it follows that the tangent to the curve $h(x_1, x_2) = 0$ is the vector $\boldsymbol{\epsilon}(x_1, x_2) = \left(1, \frac{dx_2}{dx_1}\right)^T$, while the gradient is given by $\nabla h = \left(\frac{\partial h}{\partial x_1}, \frac{\partial h}{\partial x_2}\right)$. Consequently, the above equation dictates that:

$$(\nabla h)^T \cdot \boldsymbol{\epsilon} = 0 \tag{8.3}$$

that is, the gradient and tangent to the curve are orthogonal to each other. This property can also be seen by considering a displacement $\boldsymbol{\epsilon}$ along the curve $h(\mathbf{x} + \boldsymbol{\epsilon})$ (with $\mathbf{x} = \{x_1, x_2\}$ in this example). Calculating the Taylor expansion around the point $\mathbf{x}$ we get:

$$h(\mathbf{x} + \boldsymbol{\epsilon}) \approx h(\mathbf{x}) + \boldsymbol{\epsilon}^T \nabla h(\mathbf{x}) \tag{8.4}$$

Since both $h(\mathbf{x})$ and $h(\mathbf{x} + \boldsymbol{\epsilon})$ lie on the curve then $h(\mathbf{x} + \boldsymbol{\epsilon}) = h(\mathbf{x})$. For infinitesimal displacements, *i.e.,* $||\boldsymbol{\epsilon}|| \to 0$ we have $\boldsymbol{\epsilon}^T \nabla h(\mathbf{x}) = 0$ and since $\boldsymbol{\epsilon}$ is a displacement along the curve and therefore parallel (tangent) to it, then $\nabla h(\mathbf{x})$ is perpendicular to the curve (see Figure 8.1).

We now consider displacements along the surface to increase or decrease the value of the function f. If a displacement along the curve $h(x_1, x_2) = 0$ results in an increase or decrease of f, it means that that displacement has a component along the gradient ∇f and hence $(\nabla f)^T \boldsymbol{\epsilon} \neq 0$(see Figure 8.2). At a point of maximum (or minimum) of f, however, it must necessarily occur that $(\nabla f)^T \boldsymbol{\epsilon} = 0$, because if it didn't it would mean that we can still move along the curve to increase or decrease f. Consequently, on an extremal point we have:

$$(\nabla f)^T \boldsymbol{\epsilon} = 0 \tag{8.5}$$

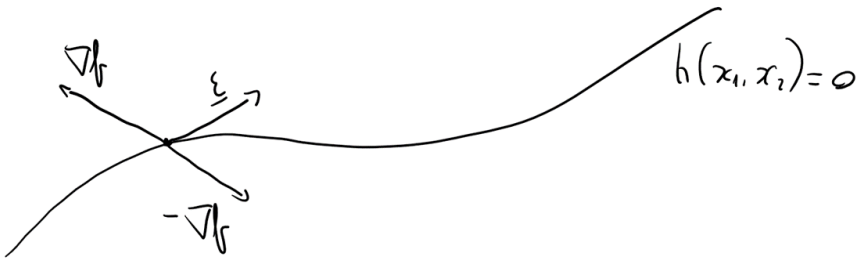


Figure 8.2: The gradient ∇f and the tangent $\boldsymbol{\epsilon}$ to the curve are not orthogonal when the motion has a component along the gradient of the function f.

Now, $\boldsymbol{\epsilon}$ is orthogonal to both the gradient of f and the gradient of h at an extremal point. Hence, ∇f and ∇h must be parallel, i.e., there exists a $\lambda \in \mathbb{R}$ such that:

$$\nabla f + \lambda \nabla h = 0 \tag{8.6}$$

where $\lambda \neq 0$ is called **Lagrange Multiplier**. Figure 8.4 depicts this condition in the case of a minimization problem. In the figure, the f contour lines are such that $c_1 > c^* > c_2 > c_3$. The tangent to the curve at a maximum point must be orthogonal to ∇f, otherwise we could move along the curve increasing or decreasing f. We can see how the point of minimum obtained for (x_1^*, x_2^*) is relative to a value of the function c^* that is greater than the minimum we would arrive at without constraints. However, because a constraint is present, we are bounded to choose points in the curve $h(x_1, x_2) = 0$ to find the minimum (or maximum) of f.

In summary, points that satisfy the following equations:

$$h(x_1, x_2) = 0 \tag{8.7}$$

$$\nabla f + \lambda \nabla h = 0 \text{ for some } \lambda \tag{8.8}$$

are extremal points of f subject to the constraint $h(x_1, x_2) = 0$.

It is convenient at this point to introduce the **Lagrangian** associated with the constrained problem we defined earlier as follows:

$$L(x_1, x_2, \lambda) = f(x_1, x_2) + \lambda h(x_1, x_2) \tag{8.9}$$

and it can be seen that by calculating the gradient of the Lagrangian and setting it equal

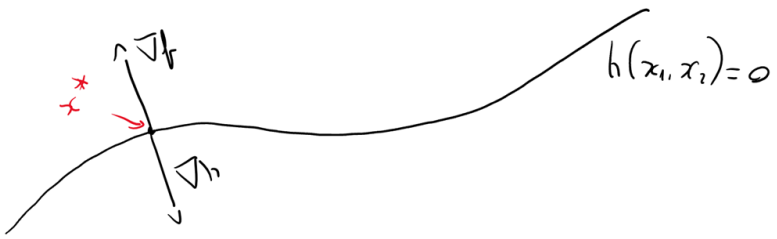


Figure 8.3: The gradient ∇f and the gradient ∇h are parallel in an extrema point.

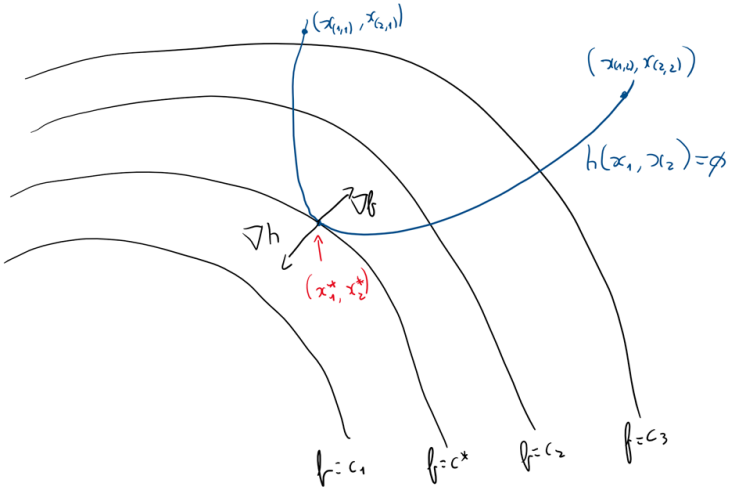


Figure 8.4: In the case of a minimization problem, the optimal solution is the point that lies in the $h(x_1, x_2) = 0$ curve for which the gradient ∇h is orthogononal to the gradient ∇f.

to zero we get the set of equations we identified in 8.7:

$$\nabla L = \begin{pmatrix} \frac{\partial f}{\partial x_1} + \lambda \frac{\partial h}{\partial x_1} \\ \frac{\partial f}{\partial x_2} + \lambda \frac{\partial h}{\partial x_2} \\ h \end{pmatrix} = (\nabla f + \lambda \nabla h, h) \tag{8.10}$$

The method involving constructing the Lagrangian function and setting its gradient to zero is known as **method of Lagrangian multipliers**.

Let's take an example: we want to determine the stationary point (maximum or minimum) of the function $f(x_1, x_2) = 1 - 2x_1^2 - 2x_2^2$ subject to the constraint $h(x_1, x_2) = 2x_1 + 2x_2 - 1 = 0$. We begin by constructing the Lagrangian function:

$$L(\mathbf{x}\lambda) = 1 - 2x_1^2 - 2x_2^2 + \lambda(2x_1 + 2x_2 - 1) \tag{8.11}$$

If we calculate the graidient we get:

$$\begin{aligned} -4x_1 + 2\lambda &= 0 \\ -4x_2 + 2\lambda &= 0 \\ 2x_1 + 2x_2 - 1 &= 0 \end{aligned} \tag{8.12}$$

and solving we get that the stationary point is $(x_1^*, x_2^*) = (\frac{1}{4}, \frac{1}{4})$ and the Lagrange multiplier is $\lambda = 1/2$

So far we have considered only equality constraints. What happens in the case of inequality constraints, *e.g.,* $h(x_1, x_2) \geq 0$?

Consider Figure 8.5, and suppose we want to maximize the function $f(x_1, x_2)$ subject to the constraint $h(x_1, x_2) \geq 0$. In this case there are two possible scenarios, depending on whether the point of maximum lies in the region $h(\mathbf{x}) > 0$ or in the boundary of the region $h(\mathbf{x}) = 0$.

- In the first case, the constraint is called **inactive** and the function $h(\mathbf{x})$ does not play any role in determining the point of maximum since we are free to move in that area and the condition of stationarity is simply given by $\nabla f(\mathbf{x}) = 0$. This situation can still be represented by the Lagrangian function and corresponds to the situation $\lambda = 0$.

- In the second case, the constraint is called **active**, because it is conditioning the computation of the maximum (or minimum). The solution is then the same as we identified in the case of the equality constraint (so $\lambda \neq 0$).

However, in the case of the **active** constraint, we must also pay attention to the sign of λ and distinguish 4 cases:

- Scenario 1: Constraint $h(\mathbf{x}) \geq 0$ and maximization problem. In this case, ∇h is oriented towards the interior of the constraint region and ∇f and ∇h must

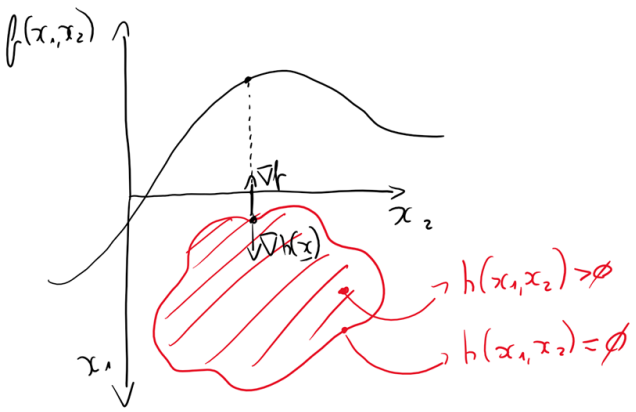


Figure 8.5: Lagrange multiplier method with inequality constraints.

have opposite sign (anti-parallel), otherwise we could move inside the constraint to increase the value of f and thus the maximum would not lie on the boundary. Consequently, since we have $\nabla f + \lambda \nabla h = 0 \implies \nabla f = -\lambda \nabla h$, we must necessarily have $\lambda > 0$.

- Scenario 2: Constraint $h(\mathbf{x}) \leq 0$ and maximization problem. In this case, ∇h is oriented outward from the constraint region and ∇f and ∇h must have the same (parallel) sign, otherwise we could move inside the constraint to increase the value of f and thus the maximum would not lie on the boundary. Consequently, since $\nabla f = -\lambda \nabla h$, we must necessarily have $\lambda < 0$.

- Scenario 3: Constraint $h(\mathbf{x}) \geq 0$ and minimization problem. In this case, ∇h is oriented towards the interior of the constraint region and ∇f and ∇h must have the same (parallel) sign, otherwise we could move inside the constraint to decrease the value of f and thus the minimum would not be on the boundary. Consequently, since we have $\nabla f = -\lambda \nabla h$, we must necessarily have $\lambda < 0$.

- Scenario 4: Constraint $h(\mathbf{x}) \leq 0$ and minimization problem. In this case, ∇h is oriented outward from the constraint region and ∇f and ∇h must have opposite sign (anti-parallel), otherwise we could move inside the constraint to decrease the value of f and thus the minimum would not lie on the boundary. Consequently, since $\nabla f = -\lambda \nabla h$, we must necessarily have $\lambda > 0$.

Whether the constraint is active or inactive, we still have that the relation $\lambda h(\mathbf{x}) = 0$ holds. Thus, we can summarize the set of conditions for stationarity as follows: considering the Lagrangian associated with the f-maximization problem with constraint

$h(\mathbf{x}) \geq 0$:

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda h(\mathbf{x}) \tag{8.13}$$

the stationarity conditions are:

$$\begin{aligned} \nabla_{\mathbf{x}} L(\mathbf{x}, \lambda) &= 0 \\ \frac{\partial L(\mathbf{x}, \lambda)}{\partial \lambda} &= 0 \\ \lambda h(\mathbf{x}) &= 0 \\ h(\mathbf{x}) &\geq 0 \\ \lambda &\geq 0 \end{aligned} \tag{8.14}$$

where the last two equations will change depending on the 4 scenarios identified above. The conditions expressed in the 8.14 are known as **Karush-Kuhn-Tucker (KKT) conditions**.

In general, we may be dealing with optimization problems with multiple constraints, both equality and inequality. It is easy to extend the previous considerations to the more general case. Suppose we want to maximize a function $f(\mathbf{x})$ subject to inequality constraints $h_j(\mathbf{x}) \geq 0$ with $j = 1, \ldots, J$ and equality constraints $r_k(\mathbf{x}) = 0$ with $k = 1, \ldots, K$. By introducing the Lagrangian multipliers λ_j and μ_k, we can write the Lagrangian function as follows:

$$L(\mathbf{x}, \lambda_1, \ldots, \lambda_J, \mu_1, \ldots, \mu_K) = f(\mathbf{x}) + \sum_j^J \lambda_j h_j(\mathbf{x}) + \sum_k^K \mu_k r_k(\mathbf{x}) \tag{8.15}$$

and the KKT conditions are:

$$\begin{aligned} \nabla_{\mathbf{x}} L(\mathbf{x}, \lambda_1, \ldots, \lambda_J, \mu_1, \ldots, \mu_K) &= 0 \\ \frac{\partial L(\mathbf{x}, \lambda_1, \ldots, \lambda_J, \mu_1, \ldots, \mu_K)}{\partial_{\lambda_j}} &= 0 \ , \forall j = 1, \ldots, J \\ \frac{\partial L(\mathbf{x}, \lambda_1, \ldots, \lambda_J, \mu_1, \ldots, \mu_K)}{\partial_{\mu_k}} &= 0 \ , \forall k = 1, \ldots, K \\ \lambda_j h_j(\mathbf{x}) &= 0 \ , \forall j = 1, \ldots, J \\ r_k(\mathbf{x}) &= 0 \ , \forall k = 1, \ldots, K \\ h_j(\mathbf{x}) &\geq 0 \ , \forall j = 1, \ldots, J \\ \lambda_j &\geq 0 \ , \forall j = 1, \ldots, J \end{aligned} \tag{8.16}$$

Chapter 9

Model Selection and Model Assessment

In the previous chapters, we introduced various supervised learning algorithms with the aim of estimating, from the data, a model capable of mapping two random variables, *i.e.,* t (the output variable) and $\mathbf{x}$ (the feature vector). These two variables are characterized by a joint distribution $p(t, \mathbf{x})$ which is unknown and of which we have information only through a set of samples (the dataset) that are realizations of it. In detail, we have access to a dataset, called **training set** $\mathcal{D}_{training}$ consisting of samples $\{(\mathbf{x}^{(0)}, t^{(0)}), (\mathbf{x}^{(1)}, t^{(1)}), \ldots, (\mathbf{x}^{(N_1-1)}, t^{(N_1-1)})\}$ which are realizations of the joint distribution, *i.e.,* $(\mathbf{x}^{(i)}, t^{(i)}) \sim p(\mathbf{x}, t)$.

Using the training data, we try to estimate a model that relates $\mathbf{x}$ and t and approximates well the unknown relationship that exists between the two variables. We saw that, in some case, it is sufficient for us to directly estimate the function $t = f(\mathbf{x})$ (as in the case of discriminant functions), other times we instead try to model the posterior distribution of the output variable $p(t|\mathbf{x})$ (as in the case of discriminative models). We have also seen that we can directly make inference about the joint distribution $p(\mathbf{x}, t)$ with approaches based on generative models.

Regardless of the type of strategy we use, the goal is always the same: to minimize a cost function computed on the training set data. In general, the cost function, or **Loss function**, expresses a measure of similarity between the predicted value $\hat{t}$ and the true value of supervision t. We have, for example seen the quadratic loss for Linear regression:

$$L(t, \hat{t}) = L(t, h_{\mathbf{w}}(\mathbf{x})) = L(t, \mathbf{w}^T\mathbf{x}) = (\hat{t} - t)^2 \tag{9.1}$$

or the sigmoid-based loss for Logistic Regression:

$$L(t, \hat{t}) = L(t, h_{\mathbf{w}}(\mathbf{x})) = L(t, \sigma(\mathbf{w}^T\mathbf{x})) = t \log(\sigma(\mathbf{w}^T\mathbf{x})) + (1 - t) \log(1 - \sigma(\mathbf{w}^T\mathbf{x})) \tag{9.2}$$

It is of interest to calculate the error on all samples in the training set and obtain

the **Training Error**. For example, in the case of regression, we have already seen in Chapter 2, equation 2.10, that this can be calculated as:

$$E_{training} = \frac{1}{2}\sum_{i=0}^{n_1-1}(h_{\mathbf{w}_{MLE}}(x^{(i)}) - t^{(i)})^2 \tag{9.3}$$

and which corresponds to the loss function used to find the optimal parameters of the Linear regression model (see equation 4.11). Alternatively, we can also consider the mean:

$$E_{training} = \frac{1}{2N_1}\sum_{i=0}^{n_1-1}(h_{\mathbf{w}_{MLE}}(x^{(i)}) - t^{(i)})^2 \tag{9.4}$$

However, having a low training error is, in general, not what we really care about. What is important to us is that the model provides a prediction as close as possible to the true value when it processes samples never seen during training, *i.e.,* , test samples. We are, in particular, interested in understanding whether the error computed on a never-seen dataset, the **Test Set** $\mathcal{D}_{test}$, is indeed low: we therefore consider the **Test Error** and, specifically, consider the average loss function computed on the test dataset:

$$E_{test} = E[L(t, h_{\mathbf{w}_{MLE}}(\mathbf{x}))|\mathcal{D}_{training}] \tag{9.5}$$

where conditioning on $\mathcal{D}_{training}$ indicates that the training set is fixed and used to train the model with which we computed the predictions on $\mathcal{D}_{test}$.

A good model, in fact, guarantees a low test error and not too much different from the training one. If in fact the error on the training set is much lower than the error on the test set we say we have **overfitting**. Vice versa, if the error on training sets is high regardless of the error on the test set (although we also expect this to be very high), it means that the model is not even able to explain the training samples and, therefore, to capture the characteristics of the data generation process: in this case we speak of **underfitting**.

A Machine Learning model acts on the trainable parameters $\mathbf{w}$ and, once optimized, achieves a certain test error. What can we do if we are not satisfied with this error? In general, it is possible to act on the structure and definition of the hypothesis function $h_{\mathbf{w}}(\mathbf{x})$: the ML algorithm is responsible to find the optimal set of $\mathbf{w}$, while we can act on the characteristics of the hypothesis function. We can, for example:

- Choose the structure of the hypothesis function. For example, we may choose to use a hypothesis function that predicts a linear relationship between input and output with respect to parameters (*i.e.,* Linear regression);
- Choose whether and which basis function to use;
- Choose what importance to give to the regularization hyper-parameter (as we will see in the case of Ridge Regression);

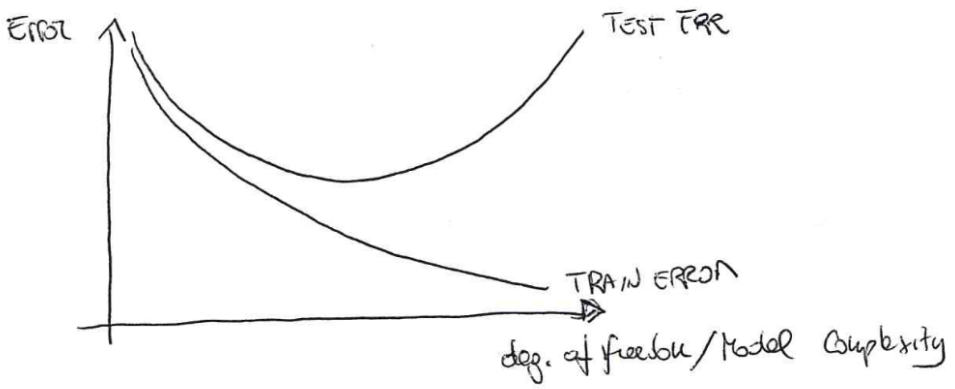


Figure 9.1: Training and test error curves varying the model complexity (degree of freedom).

- Choose whether to perform a polynomial expansion and of what degree;
- Choose the form of the class conditional distributions for generative models;
- Choose the number of neighbors to consider in K-NN classifier;
- ...

Each of these choices has an impact on the **Complexity of the model** or, alternatively, on its **Degree of freedom (DoF)**, *i.e.,* , its representational power. If we notice that a model gives very high errors on both training and test sets, it means that it is rigid (low complexity). To increase its flexibility we can, for example, use more basis functions or reduce the effect of regularization. However, increasing the complexity too much could result in models that follow the noise of the training data, leading to overfitting. To better understand this concept, we can draw the training and test error curves as the complexity of the model varies (see Figure 9.1)

From the figure we can see that, as the degrees of freedom of the model increase, the error on the training decreases (the model has more flexibility and is free to follow the training data). However, although initially also the test error decreases, by increasing the complexity, at some point it will tend to grow since the model begins to follow the noise and no longer focuses on the pattern hidden among the data.

It makes sense to ask ourselves what the best strategy is for identifying the best degree of complexity. As we have seen, to do this it is necessary to evaluate the performance of the model on data never seen during training.

A first option would be to train the model on $\mathcal{D}_{training}$ with a specific configuration of the degrees of freedom (hyper-parameters and design choices), test it on $\mathcal{D}_{test}$, and reiterate if we are not satisfied with the performance. In this way, however, the test dataset does in fact play a role in determining the hyper-parameters: we actively use it to determine them, and so they necessarily take values that "work" well with respect to



Figure 9.2: Training, Validation and Test splits.

the test data. We have, in fact, "exhausted" all of our data and have no more "unseen" samples to test the generalization capabilities of the final model. For this reason, we need to consider a third set, known as **Validation Set**, to be used to find the best configuration with respect to the complexity of the model (see Figure 9.2).

The procedure that is used to choose the configuration of the degrees of freedom of a model (and thus its complexity) is known as **Model Selection**. The procedure that instead aims at evaluating the performance and the generalization capabilities of the model once both the **trainable parameters** and the **hyper-parameters** that control its complexity have been optimized (the latter using the model selection procedure), is known as **Model Assessment**.

Before introducing the strategies we can use to perform model selection and model assessment, let us give some more details about the concept of model complexity and, in particular, the trade-off between rigidity and flexibility.

9.1 Bias-Variance Decomposition

As we said in the previous section, when we define the machine learning model to be used, we make choices and assumptions that affect its structure and thus the result we obtain. As mentioned above, our ultimate goal is to obtain a model that works well on data that have never been used either to find the trainable parameters, or for the hyper-parameters and model choices that determine the degrees of freedom (complexity): the test data.

The purpose of the algorithm is then to estimate the "true unknown" model by using the training data. When we design the algorithm, as we already mentioned, we make some choices that influence its flexibility. For example, if we choose to use linear regression, we are deciding that the "unknown" model is characterized by a linear relationship between input and output. These assumptions clearly entails an error with respect to the true model, an error that will be higher the more the assumption deviates from reality. If the assumptions are too strong, we can think of relaxing them and increase the complexity of the model. However, in this case we risk, as we already saw, to incur into overfitting. To characterize these scenarios, we define two concepts:

- **Bias**: the error deriving from assumptions that are too strong and "unrealistic" (or in any case not completely true) that the lerning algorithm makes. More generally,

its the error the arises from using a model with a reduced number of degrees of freedom.

- **Variance**:the error associated with the sensitivity of the learning algorithm to noise in the training data.

A model with high bias and low variance is a rigid model with few degrees of freedom, while a flexible model (with many degrees of freedom) has high variance and low bias.

Having a high error due to bias leads us to a condition called **Underfitting**, as the error is generated by the amount of relevant information contained in the data that is discarded due to the low representative power of the model. Instead, a high error because of the variance is said **Overfitting** and it depends from having modeled also the noise of the training set (that, in practice, translates in a model that "learns to memorize" the training data). In this second case, we are, in fact, modeling information that is not relevant for inference.

Let us try to formalize these aspects. Assume that we want to estimate a response variable t from features $\mathbf{x}$. We denote by $f(\mathbf{x})$ the "unknown" model that relates the two variables and assume that we have access not directly to $y = f(\mathbf{x})$, but to a noisy version of it $t = f(\mathbf{x}) + \epsilon$, where $\epsilon \sim N(0, \sigma_\epsilon^2)$. Since $f(\mathbf{x})$ is a deterministic function, we have that:

$$E[t] = f(\mathbf{x}) \tag{9.6}$$

$$Var[t] = \sigma_\epsilon^2 \tag{9.7}$$

We can now calculate the error we have when we use an estimate $h_{\mathbf{w}^*}(\mathbf{x})$ (our hypothesis function that we choose to estimate the unknown process) to predict the output associated with a test sample $\mathbf{x}_0$ (here, we assume that we have already completed the training and found the optimal parameters $\mathbf{w}^*$:

$$Err(\mathbf{x}_0) = E[(t - h_{\mathbf{w}^*}(\mathbf{x}))^2 | \mathbf{x} = \mathbf{x}_0] = E[t^2] - 2E[h_{\mathbf{w}^*}(\mathbf{x}_0)]E[t] + E[h_{\mathbf{w}^*}(\mathbf{x}_0)^2] \tag{9.8}$$

By noticing that:

$$Var[t] = E[t^2] - E[t]^2 \implies E[t^2] = Var[t] + E[t]^2 = \sigma_\epsilon^2 + f(\mathbf{x}_0)^2 \tag{9.9}$$

$$E[h_{\mathbf{w}^*}(\mathbf{x}_0)^2] = Var[h_{\mathbf{w}^*}(\mathbf{x}_0)] + E[h_{\mathbf{w}^*}(\mathbf{x}_0)]^2 \tag{9.10}$$

we obtain:

$$\begin{aligned} Err(\mathbf{x}_0) &= \sigma_\epsilon^2 + f(\mathbf{x}_0)^2 - 2E[h_{\mathbf{w}^*}(\mathbf{x}_0)]f(\mathbf{x}_0) + Var[h_{\mathbf{w}^*}(\mathbf{x}_0)] + E[h_{\mathbf{w}^*}(\mathbf{x}_0)]^2 \\ &= \underbrace{\sigma_\epsilon^2}_{\text{Noise or irreducible error}} + \underbrace{(E[h_{\mathbf{w}^*}(\mathbf{x}_0)] - f(\mathbf{x}_0))^2}_{\text{BIAS}^2} + \underbrace{Var[h_{\mathbf{w}^*}(\mathbf{x}_0)]}_{\text{VARIANCE}} \end{aligned} \tag{9.11}$$

The error is therefore composed of a part related to the noise on t, which we cannot

reduce (it is in fact called **irreducible error**), a part related to the Bias and one related to the Variance. We can act on these two terms, since they depend on the hypothesis function $h_{\mathbf{w}^*}(\mathbf{x}_0)$ we choose.

9.1.1 Example: K-NN Regressor

In the previous chapters we have introduced the K-NN classifier. There is also a version of this algorithm for regression problems, where the estimated output value is computed as the average of the output values associated with the K points in the neighborhood of the test sample in the feature space:

$$h(\mathbf{x}_0) = \frac{1}{K} \sum_{t \in \mathcal{N}_{\mathbf{x}_0}} t \tag{9.12}$$

by again assuming $t = f(\mathbf{x}_0) + \epsilon$, with $\epsilon \sim N(0, \sigma_\epsilon^2)$ and by noticing that:

$$E[h(\mathbf{x}_0)] = \frac{1}{K} \sum_{t \in \mathcal{N}_{\mathbf{x}_0}} E[t] = \frac{1}{K} \sum_{t \in \mathcal{N}_{\mathbf{x}_0}} f(\mathbf{x}_0) \tag{9.13}$$

and that:

$$\begin{aligned} Var[h(\mathbf{x}_0)] &= Var[\frac{1}{K} \sum_{t \in \mathcal{N}_{\mathbf{x}_0}} t] \\ &= \frac{1}{K^2} \sum_{t \in \mathcal{N}_{\mathbf{x}_0}} Var[t] \\ &= \frac{1}{K^2} \sum_{t \in \mathcal{N}_{\mathbf{x}_0}} Var[f(\mathbf{x}_0)] + Var[\epsilon] \\ &= \frac{1}{K^2} \sum_{t \in \mathcal{N}_{\mathbf{x}_0}} Var[\epsilon] \\ &= \frac{1}{K^2} K \sigma_\epsilon^2 = \frac{\sigma_\epsilon^2}{K} \end{aligned} \tag{9.14}$$

we obtain:

$$Err(\mathbf{x}_0) = \sigma_\epsilon^2 + \underbrace{[\frac{1}{K} \sum_{t \in \mathcal{N}_{\mathbf{x}_0}} f(\mathbf{x}_0) - f(\mathbf{x}_0)]^2}_{\text{Bias}} + \underbrace{\frac{\sigma_\epsilon^2}{K}}_{\text{Variance}} \tag{9.15}$$

Hence, by increasing K, *i.e.,* the number of neighbors to be considered, the variance decreases and the bias increases: this is reasonable, since the average to predict the output of the test sample is calculated on a larger number of samples and potentially farther and farther from $\mathbf{x}_0$ in the feature space. Consequently, the prediction will be less affected by the noise present on the training samples, but, with K too large we risk to predict always the same value.

9.1.2 Example: Linear Regression

We now analyze the Bias and the Variance in the case of Linear Regression. As we know, the hypothesis function, once the optimal parameters have been estimated using MLE, can be expressed as follows (assuming we do not use basis functions):

$$h_{\mathbf{w}_{MLE}|\mathbf{X},\mathbf{t}}(\mathbf{x}_0) = \mathbf{w}_{MLE}^T \mathbf{x}_0 = \mathbf{x}_0^T \mathbf{w}_{MLE} \tag{9.16}$$

and by substituting in place of $\mathbf{w}_{MLE}^T$ the expression given by the normal equation:

$$h_{\mathbf{w}_{MLE}|\mathbf{X},\mathbf{t}}(\mathbf{x}_0) = \mathbf{x}_0^T (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{t} \tag{9.17}$$

We start by calculating the Bias term and we do so by assuming that the "true" model $f(\mathbf{x}_0)$ is indeed linear, *i.e.,* $f(\mathbf{x}_0) = \mathbf{x}_0^T\mathbf{w}$

$$\begin{aligned}
\text{Bias} &= f(\mathbf{x}_0) - E_D[\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T \underbrace{\mathbf{t}}_{=f(\mathbf{X})+\epsilon=\mathbf{X}\mathbf{w}+\epsilon}] \\
&= \mathbf{x}_0^T\mathbf{w} - E_D[\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T(\mathbf{X}\mathbf{w}+\epsilon)] \\
&= \mathbf{x}_0^T\mathbf{w} - E_D[\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{X}\mathbf{w} + \mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\epsilon)] \\
&= \mathbf{x}_0^T\mathbf{w} - \underbrace{E_D[\mathbf{x}_0^T\cancel{(\mathbf{X}^T\mathbf{X})^{-1}}\cancel{\mathbf{X}^T\mathbf{X}}\mathbf{w}]}_{=E_D[f(\mathbf{x}_0)]=\mathbf{x}_0^T\mathbf{w}} + \mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\underbrace{E[\epsilon]}_{=0} \\
&= \mathbf{x}_0^T\mathbf{w} - \mathbf{x}_0^T\mathbf{w} = 0
\end{aligned} \tag{9.18}$$

We can see that, if the true unknown model is indeed linear, then the Linear Regression model has zero bias, *i.e.,* , it is **Unbiased**.

Let's now calculate the variance term, again assuming that the true model is linear.

$$\begin{aligned}
\text{Variance} = Var[h_{\mathbf{w}_{MLE}}] &= E[(h_{\mathbf{w}_{MLE}}(\mathbf{x}_0) - \underbrace{E[h_{\mathbf{w}_{MLE}}]}_{=\mathbf{x}_0^T\mathbf{w} \text{ since the model is unbiased}})^2] \\
&= E[(\mathbf{x}_0^T\mathbf{w}_{MLE} - \mathbf{x}_0^T\mathbf{w})^2] \\
&= E[(\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{t} - \mathbf{x}_0^T\mathbf{w})^2] \\
&= E[(\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T(\mathbf{X}\mathbf{w} + \epsilon) - \mathbf{x}_0^T\mathbf{w})^2] \\
&= E[(\cancel{\mathbf{x}_0^T\mathbf{w}} + \mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\epsilon - \cancel{\mathbf{x}_0^T\mathbf{w}})^2] \\
&= E[(\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\epsilon)^2] \\
&= E[(\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\epsilon)(\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\epsilon)^T] \\
&= E[\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\epsilon\epsilon^T(\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T)^T] \\
&= \mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^TE[\epsilon\epsilon^T](\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T)^T \\
&= \mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\sigma_\epsilon^2\mathbf{I}(\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T)^T \\
&= \sigma_\epsilon^2\mathbf{x}_0^T\cancel{(\mathbf{X}^T\mathbf{X})^{-1}}\cancel{\mathbf{X}^T\mathbf{X}}(\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1})^T \\
&= \sigma_\epsilon^2\mathbf{x}_0^T(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{x}_0
\end{aligned} \tag{9.19}$$

The Variance is therefore related to the spread of the noise σ_ϵ^2 and is inversely proportional to the variance of the data. This is intuitive, since the more different the data are in the training set, the more experience the model has in training. Consequently, the effect of noise will be reduced. This result is closely related to what we had obtained by calculating the variance on the estimated weights with MLE:

$$Var[\mathbf{w}_{MLE}] = \sigma_\epsilon^2(\mathbf{X}^T\mathbf{X})^{-1} \tag{9.20}$$

9.2 Approaches for Model Selection and Model Assessment

9.2.1 Model Selection

From the previous sections we have understood that the choice of the flexibility or complexity of the model is fundamental. To do so, we have the possibility to act on several aspects. In particular, the DoFs of a model can depend on:

- **Type of algorithm and hypothesis function** $h_{\mathbf{w}}(\mathbf{x})$. For example, we could choose the hypothesis function of Linear Regression, Ridge Regression, Neural Networks, etc.... Each hypothesis function carries with it assumptions that impact the flexibility of the model;
- **Hyper-parameters of regularization**. For example, the λ hyper-parameter of Ridge Regression or Lasso Regression that we will see in the next chapter;

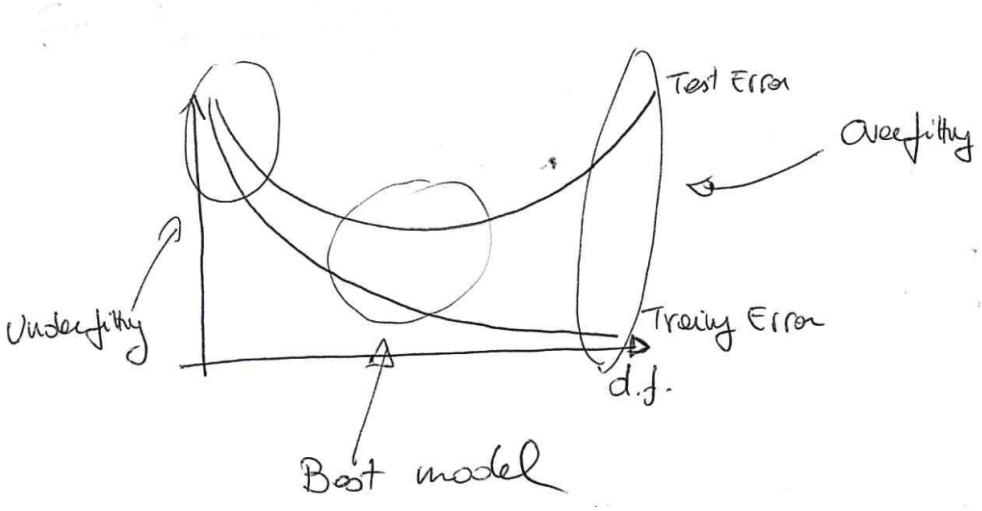


Figure 9.3: Configurations of DoF that result in underfitting, overfitting and Best Model.

- **Hyper-parameters specific to the chosen algorithm**. For example, the number of K neighbors of K-NN classifier;
- **Degree of polynomial expansion of features**;
- **Number and type of Basis functions**;
- **Choice of the number of parameters w of the algorithm**. In linear regression, for example, by increasing the basis functions we are in fact increasing the number of parameters to be learned and thus the flexibility of the model. With neural networks, as we will see, we will have even more flexibility with respect to the choice of vector size $\mathbf{w}$.
- ...

The best DoF configuration for the model is the one that guarantees the lowest error on the data set used, *i.e.,* the point shown as "Best Model" in the graph in Figure 9.3. However, as we said, we cannot use the test set to find the best configuration, otherwise we would no longer have any useful data to verify the generalization capabilities of the final model (the one with both parameters $\mathbf{w}$, and hyper-parameters and design choices fixed). Instead, we must use a third set of data, called the **validation set**, to identify the best degree of complexity for our problem.

We have three possible strategies for identifying a sub-set of **Validation**: **Fixed Validation set**, **Cross-Validation (K-Fold)** and **Leave-One-Out** (see Figure 9.4).

- **Fixed Validation Set**: In this case we split the entire dataset into 3 fixed subsets: one for training, one for validation and one for testing. The typical split is $60\% - 20\% - 20\%$ and is normally done after a shuffle of the data to try to compensate

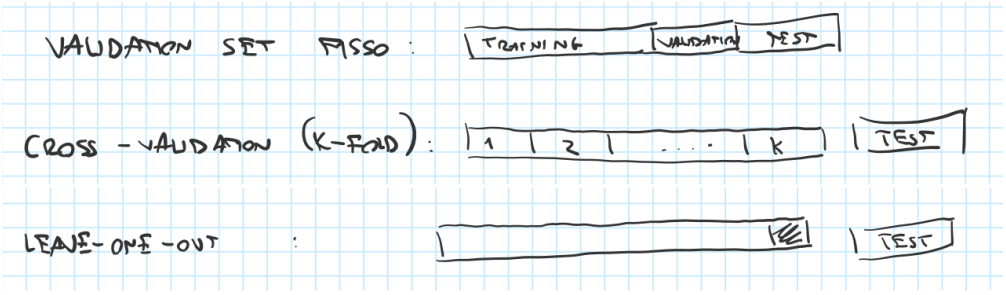


Figure 9.4: Fixed Validation set, Cross-Validation and Leave-one-out.

for situations where the data were collected sequentially and are therefore not in i.i.d. At this point, to identify the best DoF configurations, the strategy is as follows: we choose a certain complexity configuration (*e.g.,* , a certain value for the hyper-parameter λ), we train the model on the training set (60% of the data) with that configuration, we test the performance on the validation set, and we iterate for all possible DoF configurations. In the end, the best model will be the one that guarantees the lowest error on the validation set. A pseudocode of this procedure is shown in the algorithm 3, in the case of exploring the best values for the hyper-parameter λ. The fixed validation set strategy is appropriate in case of

Algorithm 3 Model Selection with Fixed Validation set

1: Input: $\mathcal{D}_{training}$ and $\mathcal{D}_{validation}$
2: **for all** $\lambda_j \in \{10^{-3}, 10^{-2}, 10^{-1}, 10^{0}, 10^{1}, 10^{2}, 10^{3}\}$ **do**
3: Train model on $\mathcal{D}_{training}$ with λ_j
4: Test the trained model on $\mathcal{D}_{validation}$
5: **end for**
6: Select as the best λ_j the one that attains the lowest error on $\mathcal{D}_{validation}$

datasets with a large number of samples, because, in these cases, the statistical representativeness of the various sub-sets is reasonable and the risk of having, for example, samples on the training set very different from those of the validation set is reduced.

- **K-Fold Cross Validation**: In case of not very large datasets, in order to avoid having very different samples on the various sub-sets, one possibility is to iteratively use various parts of the training set both for the actual training of the parameters and as validation. In this case, we initially divide the dataset into 2 subsets, typically 70% for training and 30% for testing. Next, the training set is divided into K subset (K fold). Given a certain configuration of DoF, we use $K-1$ of those folds to train the model and the last one as validation to compute the error. We repeat this operation considering all the possible combinations of the folds (we make therefore in total K trainings for that particular configuration of DoF). The

overall error for that particular DoF configuration is calculated as the average of the errors obtained in the various validation folds. This procedure is repeated for all configurations considered and the best one is chosen as the one that obtains the lowest average error. This procedure is expressed in the algorithm 4, again in the case of exploring the best values for the hyper-parameter λ.

Algorithm 4 Model Selection with K-Fold Cross-validation

Input: $\mathcal{D}_{training}$
$\lambda^* = n.a.$
$AvgErr = 0$
  for all $\lambda_j \in \{10^{-3}, 10^{-2}, 10^{-1}, 10^0, 10^1, 10^2, 10^3\}$ **do**
    $CurrErr = 0$
    for all $k \in \{0, 1, \ldots, K\}$ **do**
      Train model on the $K-1$ folds excluding the k-th with λ_j
      $ValErr =$ Test the trained model on the k-th fold
      $CurrErr+ = ValErr$
    end for
    $CurrAvgErr = CurrErr/K$
    if $CurrAvgErr < AvgErr$ **then**
      $AvgErr = CurrAvgErr$
      $\lambda^* = \lambda_j$
    end if
  end for
return λ^*

- **Leave-One-Out**: This strategy is used in the case of very small datasets and is a special case of cross-validation in which each fold contains a single sample of the dataset: the training is done iteratively on $N-1$ samples and the test on the last remaining sample. The procedure is shown in the Algorithm 5.

Whatever is the strategy used, once identified the best configuration of DoF, it is common to re-train the model with that configuration putting together the data of the training and of the validation set (in the case of fixed validation set) or using all the training set (in the case of cross-validation and leave-one-out). This is done since, as we will see in the next sub-section, a model trained with more data exhibits, in general, better generalization capabilities. Consequently, once chosen the optimal hyper-parameters with the procedures of model selection, it is convenient to use all the training data available to train the final model.

Algorithm 5 Model Selection with Leave-One-Out

Input: $\mathcal{D}_{training}$ with N samples
$\lambda^* = n.a.$
$AvgErr = 0$
  for all $\lambda_j \in \{10^{-3}, 10^{-2}, 10^{-1}, 10^0, 10^1, 10^2, 10^3\}$ **do**
    $CurrErr = 0$
    for all $i \in \{0, 1, \ldots, N-1\}$ **do**
      Train model on the $N-1$ samples excluding the i-th with λ_j
      $ValErr =$ Test the trained model on the i-th sample
      $CurrErr+ = ValErr$
    end for
    $CurrAvgErr = CurrErr/N$
    if $CurrAvgErr < AvgErr$ **then**
      $AvgErr = CurrAvgErr$
      $\lambda^* = \lambda_j$
    end if
  end for
return λ^*

N.B. In the case where multiple elements controlling DoFs are explored at the same time (*e.g.,* , multiple hyper-parameters) it is necessary to perform a **Grid Search**, *i.e.,* the hyper-parameters should be explored in a grid (with nested for loops). This is because, for example, the optimal parameter λ given a certain degree of polynomial expansion may not be the same for another degree. We therefore need to explore all possible combinations of the various elements that affect DoFs.

N.B. 2 The exploration space of hyper-parameters is theoretically infinite (λ takes values in $\mathbb{R}$) and thus the computational cost of exploring it properly could be very high. A possible strategy could involve a first step in which values are explored at very large intervals (*e.g.,* $\{10^{-3}, 10^{-2}, 10^{-1}, 10^0, 10^1, 10^2, 10^3\}$). Next, once we have identified the interval that guarantees the smallest error, we can perform a more granular search in that area (*e.g.,* $[10^{-2}, 10^{-1}]$).

Role of the number or training samples

From the point of view of the impact on the trend of the training and the test errors, also the number of training samples plays a fundamental role and can therefore be object of evaluation through model selection. Typically, having fixed the complexity of the model (therefore the various hyper-parameters, the design choices, etc...), if this is trained with a reduced number of samples we expect a very high error on the test and very low on the training: the loss function used during training depends on few samples and it is, therefore, easier for the model to set its parameters $\mathbf{w}$ to guarantee a low error. However,

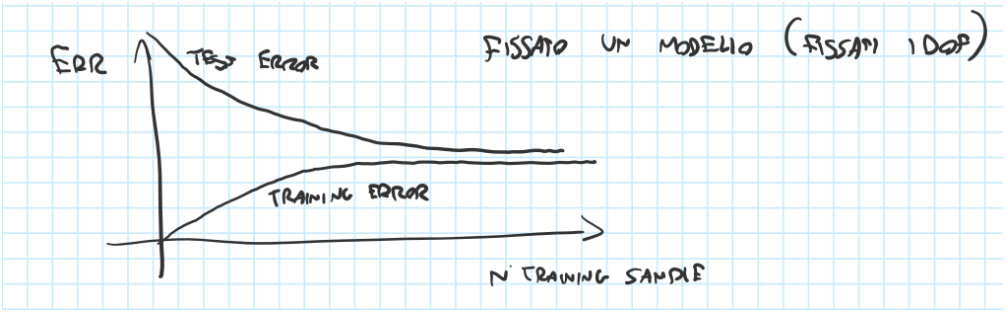


Figure 9.5: Training and test error by varying the number of training samples

with little experience is clear that the model perform poorly on data never seen. By increasing the number of samples of the training set, the model has more and more elements to take into account during the optimization and it will be more difficult to reduce the estimation error for all of them. On the other hand, with more experience it is easire to understand the structure of the data generation process and therefore to generalize better on samples never seen: the test error therefore will decrease (you see Figure 9.5). It is therefore clear that it is always worthwhile to increase the number of samples in the training set. However, since acquiring the dataset is expensive, this type of study is carried out to get an idea of the real advantage that we could obtain by acquiring more data. In fact, training the model using more and more portions of the training set is equivalent to simulate the situation in which we increase the size of our dataset.

9.2.2 Model Assessment

Once we have trained the model and identified the best DoF configuration, we can verify the performance of our model on never observed data, *i.e.,* the **Assessment** phase. For this phase we can use the same techniques seen for the model selection, that is **Fixed Test Set**, **K-Fold Cross Validation** and **Leave-one-out**. In the case of fixed test set, it is sufficient to evaluate the performance of the final model on the split of the dataset reserved for testing. In this case, as previously said, it is better to use the model trained on all the data that do not belong to the test set (using both the training and validation data).

If we choose instead to use Cross validation or Leave-One-Out for model assessment because of the small size of the dataset, we must:

- Divide the whole dataset into folds (or consider all the samples of the dataset in the case of Leave-One-Out);
- Retrain the model for each possible combination of folds using the optimal hyper-parameters found in the model selection phase;
- Perform the test on the remaining fold

The final error will be the average of the errors obtained on the various folds. The pseudocode in this case is analogous to the one used for model selection, with the difference that we do not have the outer for loop iterating on the hyper-parameters, because in model assessment we are only interested in the performance. An example, in the case of model assessment with cross-validation is shown in the algorithm 6.

Algorithm 6 Model Assessment with K-Fold Cross-validation

Input: $\mathcal{D}_{total}$
  $AvgErr = 0$
  for all $k \in \{0, 1, \ldots, K\}$ **do**
    Train model on the $K - 1$ folds excluding the k-th (use the optimal DoF configuration found during the model selection)
    $ValErr =$ Test the trained model on the k-th fold
    $CurrErr+ = ValErr$
  end for
  $AvgErr = CurrErr/K$
return $AvgErr$

Chapter 10

Ridge and Lasso Regression

10.1 Ridge Regression

In the previous chapter, when we talked about bias-variance decomposition, we saw how the variance of the linear regression depends proportionally on the variance of the noise of the measurements $\mathbf{t}$ and inversely on the variance of the data:

$$Var[h_{\mathbf{w}_{MLE}}(\mathbf{x}_o)] = \sigma_\epsilon^2 \mathbf{x}_0^T (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{x}_0 \tag{10.1}$$

it is possible to show that this term can be approximated in the following way:

$$Var[h_{\mathbf{w}_{MLE}}(\mathbf{x}_o)] \sim \sigma_\epsilon^2 \frac{D}{N} \tag{10.2}$$

where D is the number of effective parameters (typically equal to the number of features) and N is the number of samples in the training set. This relation highlights how as the number of samples of the training set increases, the variance of the estimator decreases and consequently the overfitting is contained. This is intuitive, since the model has more experience at its disposal during the training (even if the term $(\mathbf{X}^T\mathbf{X})^{-1}$ expresses this concept better, since it says that it is not enough to have more samples, but it is necessary that the samples represent as many input configurations as possible). Conversely, as the number of features (parameters) increases, the variance increases. This is also reasonable, as the model becomes more complex and the risk of overfitting increases. This phenomenon is often referred to as the **Curse of Dimensionality**.

We have also seen how the variance of the estimate for a test sample is closely related to the variance of the parameters learned via MLE: both, in fact depend on $(\mathbf{X}^T\mathbf{X})^{-1}$:

$$Var[\mathbf{w}_{MLE}] = \sigma_\epsilon^2 (\mathbf{X}^T\mathbf{X})^{-1} \tag{10.3}$$

$$Var[h_{\mathbf{w}_{MLE}}(\mathbf{x}_o)] = \sigma_\epsilon^2 \mathbf{x}_0^T (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{x}_0 \tag{10.4}$$

it is evident then that if $\mathbf{X}^T\mathbf{X}$ is a singular matrix or at least "close" to the singularity, the uncertainty about the parameters and the estimate grows. When does this condition occur?

In general, a square matrix U is non-invertible or singular if one of the following equivalent conditions is true:

- $\det U = |U| = 0$;
- At least one eigenvalue is null;
- U is **Rank-deficient**, *i.e.,* one or more of its columns (or rows) is linear combination of the others.

In our case, we are interested in checking whether the design matrix X is singular. Since we can reasonably assume that a sample cannot be a linear combination of other samples (this is if the dataset has been properly collected), the last condition of the previous list reduces to the following:

- X is singular if it is **Column-Rank-deficient**, *i.e.,* , one or more of its columns is linear combination of the others. In particular, it is singular if one of its **features** is a linear combination of the others.

This concept is closely related to **Collinearity** and **Multicollinearity**: a feature is multicollinear to others if its correlation with the latter is high. This means that it can be expressed "approximately" as a linear combination of other features. Not surprisingly, if multicollinearity is present, we observe an increase in uncertainty related to the parameters.

In practice a feature will never be a perfect linear combination of others (unless it was constructed as such), but strong collinearity results in "almost" linear dependence leading to "almost" rank-deficiency. As a result, $(\mathbf{X}^T\mathbf{X})^{-1}$ will become very large, greatly increasing the variance on the parameters $\mathbf{w}_{MLE}$.

To address collinearity and multicollinearity we can use the correlation matrix and the Variance Inflation Factor (VIF). However, the use of VIF is not optimal, because it is not always clear what is a value of VIF that can act as a cut-off to determine multicollinearity. Also, to compute VIF, we need to train a model for each feature, which can become computationally prohibitive in case of datasets with many features and many samples.

Is there another solution to handle cases where $\mathbf{X}^T\mathbf{X} \to 0$ and thus overfitting?

N.B. Multicollinearity $\to$ Rank-deficiency $\to$ $\mathbf{X}^T\mathbf{X} \to 0$ $\to$ $Var[h_{\mathbf{w}_{MLE}}(\mathbf{x}_o)] \to \infty$ $\to$ High Variance $\to$ Overfitting.

Recall that, from an intuitive point of view, if a feature has a strong collinearity with another, during optimization the value of the loss function (contour line) with respect to

the weight w associated with that feature will be constant over a wide range of values. There will thus be many good choices of w with respect to minimization, which increases the uncertainty in determining it. Intuitively, in fact, if a feature is collinear, it does not add information in order to predict $\hat{t}$, so almost any value will do.

In the degenerate case where a feature is a perfect linear combination of others, we actually get a minimization problem with inifinite solutions with respect to the parameter associated with that feature: the variance of $\mathbf{w}_{MLE}$ then becomes infinite.

This, during optimization, results in very high values for the parameters, in modulus. Indeed, suppose that a parameter w_i of a feature i becomes very large, and that this feature is collinear to another feature j. Since the parameter w_j associated to the latter is free to assume values over a very wide range without changing the loss, to compensate the value w_i (in order to have reasonable predictions $\hat{t}$ and close to the supervision label), also w_j will assume high values with opposite sign.

From this last consideration we can deduce that, in order to compensate the collinearity and avoid to have high variance values for $\mathbf{w}_{MLE}$, we can think to constrain the norm of the parameters: this operation is called **Regularization**. The idea is to optimize the value of the trainable parameters with the constraint that their norm must be small. In this way, if a feature is collinear, the constraint will prevent the weight relative to it from growing and indeed will cause it to cancel, since it has no impact with respect to prediction. The regularization performs intrinsically a **Feature selection**: by cancelling or reducing a lot the parameters related to collinear features or to those that are not significant with respect to the prediction, it is implicitly reducing the degrees of freedom of the model.

Following this intuition we can introduce a regularized version of Linear regression, which takes the name of **Ridge Regression**:

$$\begin{aligned} \mathbf{w}_{MLE,RIDGE} &= \underset{\mathbf{w}\in\mathbb{R}^D}{\operatorname{argmin}} \sum_{i=0}^{N-1} (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2 \\ \text{subject to } &\sum_{j=1}^{D-1} w_j^2 \leq l \end{aligned} \tag{10.5}$$

N.B. In the above formula w_0 is not included!

N.B. 2 The features $\mathbf{x}$ must be centered in zero, since the **Ridge Estimator** are not equivariant with respect to scaling operations.

Using Lagrange multipliers, the previous problem can be written as follows:

$$\mathbf{w}_{MLE,RIDGE} = \underset{\mathbf{w}\in\mathbb{R}^D}{\operatorname{argmin}} \sum_{i=0}^{N-1} (t^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)})^2 + \lambda \sum_{j=1}^{D-1} w_j^2 \tag{10.6}$$

where it is possible to prove that $\forall l\ \exists \lambda \geq 0$ such that the solution to problem 10.5 is the same as 10.6.

We now derive the analytical solution for this problem.

N.B. 3 In the case of linear regression, we saw that the loss function:

$$L(\mathbf{w}) = \frac{1}{2}\sum_{i=0}^{N-1}(t^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)})^2 \tag{10.7}$$

can be written in the following vector form:

$$L(\mathbf{w}))\frac{1}{2}(\mathbf{t} - \mathbf{X}\mathbf{w})^T(\mathbf{t} - \mathbf{X}\mathbf{w}) \tag{10.8}$$

we can see this by considering that:

$$\mathbf{X} \in \mathbb{R}^{N\times D} = \begin{pmatrix} \mathbf{x}^{(0)})^T \\ \mathbf{x}^{(1)} \\ \vdots \\ \mathbf{x}^{(N-1)} \end{pmatrix}, \mathbf{w} \in \mathbb{R}^{D\times 1} = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_{D-1} \end{pmatrix}, \mathbf{t} \in \mathbb{R}^{N\times 1} = \begin{pmatrix} t^{(0)} \\ t^{(1)} \\ \vdots \\ t^{(N-1)} \end{pmatrix} \tag{10.9}$$

thus

$$\mathbf{t} - \mathbf{X}\mathbf{w} = \begin{pmatrix} t^{(0)} - (\mathbf{x}^{(0)})^T\mathbf{w} \\ t^{(1)} - (\mathbf{x}^{(1)})^T\mathbf{w} \\ \vdots \\ t^{(N-1)} - (\mathbf{x}^{(N-1)})^T\mathbf{w} \end{pmatrix} = \begin{pmatrix} t^{(0)} - \mathbf{w}^T\mathbf{x}^{(0)} \\ t^{(1)} - \mathbf{w}^T\mathbf{x}^{(1)} \\ \vdots \\ t^{(N-1)} - \mathbf{w}^T\mathbf{x}^{(N-1)} \end{pmatrix} \tag{10.10}$$

and:

$$\begin{aligned} &\frac{1}{2}(\mathbf{t} - \mathbf{X}\mathbf{w})^T(\mathbf{t} - \mathbf{X}\mathbf{w}) = \\ &= \frac{1}{2}\left((t^{(0)} - \mathbf{w}^T\mathbf{x}^{(0)}) \dots (t^{(N-1)} - \mathbf{w}^T\mathbf{x}^{(N-1)}\right) \begin{pmatrix} t^{(0)} - \mathbf{w}^T\mathbf{x}^{(0)} \\ \vdots \\ t^{(N-1)} - \mathbf{w}^T\mathbf{x}^{(N-1)} \end{pmatrix} \\ &= \frac{1}{2}\left((t^{(0)} - \mathbf{w}^T\mathbf{x}^{(0)})^2 + \dots + (t^{(N-1)} - \mathbf{w}^T\mathbf{x}^{(N-1)})^2\right) \\ &= \frac{1}{2}\sum_{i=0}^{N-1}(t^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)})^2 \end{aligned} \tag{10.11}$$

As with linear regression, the loss function of ridge regression can be written in vector

form:

$$\begin{aligned} L(\mathbf{w}) &= \frac{1}{2}(\mathbf{t}-\mathbf{X}\mathbf{w})^T(\mathbf{t}-\mathbf{X}\mathbf{w}) + \frac{1}{2}\lambda\mathbf{w}^T\mathbf{w} \\ &= \frac{1}{2}(\mathbf{t}^T-\mathbf{w}^T\mathbf{X}^T)^T(\mathbf{t}-\mathbf{X}\mathbf{w}) + \frac{1}{2}\lambda\mathbf{w}^T\mathbf{w} \\ &= \frac{1}{2}\left[\mathbf{t}^T\mathbf{t} - \mathbf{t}^T\mathbf{X}\mathbf{w} - \mathbf{w}^T\mathbf{X}^T\mathbf{t} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}\right] + \frac{\lambda}{2}\mathbf{w}^T\mathbf{w} \\ &= \frac{1}{2}\left[-2\mathbf{w}^T\mathbf{X}^T\mathbf{t} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w} + \mathbf{t}^T\mathbf{t}\right] + \frac{\lambda}{2}\mathbf{w}^T\mathbf{w} \end{aligned} \tag{10.12}$$

$$\begin{aligned} \nabla_{\mathbf{w}} L(\mathbf{w}) &= \nabla_{\mathbf{w}}\left[\frac{1}{2}\left[-2\mathbf{w}^T\mathbf{X}^T\mathbf{t} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w} + \mathbf{t}^T\mathbf{t}\right] + \frac{\lambda}{2}\mathbf{w}^T\mathbf{w}\right] = 0 \\ &\implies -\mathbf{X}^T\mathbf{t} + \mathbf{X}^T\mathbf{X} + \lambda\mathbf{w} = 0 \implies (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\mathbf{w} = \mathbf{X}^T\mathbf{t} \\ &\implies \mathbf{w}_{MLE,RIDGE} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{t} \end{aligned} \tag{10.13}$$

The MLE solution for ridge regression is similar to that for linear regression with the exception of the term $\lambda\mathbf{I}$ which adds up to $\mathbf{X}^T\mathbf{X}$. Since $\lambda \geq 0$, we are adding to the elements on the diagonal of $\mathbf{X}^T\mathbf{X}$ a positive term. Thus, even in the case where $\mathbf{X}^T\mathbf{X}$ is singular, $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ would not be singular. To understand this, assume for simplicity that $\mathbf{X}^T\mathbf{X}$ is in diagonal form and thus has eigenvalues along the diagonal. if $\mathbf{X}^T\mathbf{X}$ is singular, we will find at least one null eigenvalue along the diagonal. If $\lambda > 0$ we will add a positive term to all elements of the diagonal of $\mathbf{X}^T\mathbf{X}$ and thus have no more null eigenvalues.

10.1.1 Effective Number of Parameters

In the case of Linear Regression, we have seen that the number of parameters (and hence the degrees of freedom) is equal to the number D of features (or of the basis functions used). In general, however, the **effective** number of parameters could be different due to, for example, collinearity of the features. To calculate the actual number of parameters, in the case of linear regression we can exploit the **Hat Matrix**:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T \tag{10.14}$$

$\mathbf{H}$, as we have already mentioned in previous chapters, is idempotent and symmetrical.

Recall that a matrix is idempotent if $\mathbf{H}^T\mathbf{H} = \mathbf{H}$. To prove the idempotency of $\mathbf{H}$ it is sufficient to observe that:

$$\begin{aligned}\mathbf{H}^T\mathbf{H} &= (\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T)^T\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T \\ &= (\mathbf{X}^T)^T \underbrace{((\mathbf{X}^T\mathbf{X})^T)^{-1}}_{(\mathbf{A}^{-1})^T=(\mathbf{A}^T)^{-1}} \mathbf{X}^T\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T \\ &= \mathbf{X}\cancel{(\mathbf{X}^T\mathbf{X})^{-1}}\cancel{\mathbf{X}^T\mathbf{X}}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T = H \end{aligned} \tag{10.15}$$

while for symmetry we note that:

$$\mathbf{H}^T = (\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T)^T = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T = H \tag{10.16}$$

As we saw earlier, the hat matrix relates the estimates $\hat{\mathbf{t}}$ to the supervision labels $\mathbf{t}$ and since it is idempotent we speak in this case of **Projection**. We have also seen that $\mathbf{H}$ has the following properties:

- $h_{i,i}$ measures the distance between the feature value $\mathbf{x}^{(i)}$ of the i-th sample and the average of the features over all N samples.
- $h_{i,i}$ is between 0 and 1.
- The sum of all $h_{i,i}$ (i.e., the sum of the diagonal of $\mathbf{H}$, *i.e.,* the trace of $\mathbf{H}$) corresponds to the number of parameters in the regression (the dimensionality of $\mathbf{w}_{ML}$), i.e., D.

This last property can be understood by considering that $\mathbf{X} \in \mathbb{R}^{N\times D}$. Since usually $D \leq N$, if $\mathbf{X}$ has full rank it must necessarily be equal to $rank(\mathbf{X}) = min(N, D) = D$. If we assume that we have no collinearity, $\mathbf{X}^T\mathbf{X}$ is nonsingular and the trace of $\mathbf{H}$ is:

$$\begin{aligned} trace(\mathbf{H}) &= trace(\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T) \underbrace{=}_{trace(AB)=trace(BA)} trace(\mathbf{X}^T\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}) \\ &= trace(\mathbf{I}_{D\times D}) = D \end{aligned} \tag{10.17}$$

In the case where instead a feature was a linear combination of others, we could exclude the associated column from $\mathbf{X}$ and the rank would then be $P < D$, resulting in fewer degrees of freedom.

We now calculate the actual DoFs in the case of ridge regression. The hat matrix for ridge regression is given by:

$$\hat{\mathbf{t}} = \mathbf{X}\mathbf{w}_{MLE,RIDGE} = \underbrace{\mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T}_{\mathbf{H}_\lambda}\mathbf{t} \tag{10.18}$$

To compute the trace of $\mathbf{H}_\lambda$ we need to compute its eigenvalues. To do this we resort

to **Singular Value Decomposition (SVD)**. SVD is a technique that allows a matrix $\mathbf{X}$ to be expressed by the product of 3 matrices:

$$\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^T \tag{10.19}$$

where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices of order $N \times D$ and $D \times D$ respectively.

- $\mathbf{U}$ contains the eigenvectors of $\mathbf{X}\mathbf{X}^T$;
- $\mathbf{V}$ contains the eigenvectors of $\mathbf{X}^T\mathbf{X}$;
- $\mathbf{D}$ is a diagonal matrix whose elements on the diagonal are the **Singular Values** of $\mathbf{X}$. The singular values other than 0 of $\mathbf{X}$ are the square roots of the eigenvalues of both $\mathbf{X}\mathbf{X}^T$ and $\mathbf{X}^T\mathbf{X}$.

We can now compute the trace of $\mathbf{H}_\lambda$ by applying SVD to $\mathbf{X}$. The term $\mathbf{X}^T\mathbf{X}$ that appears in the expression of $\mathbf{H}_\lambda$ then becomes:

$$\mathbf{X}^T\mathbf{X} = \mathbf{V}\mathbf{D}\mathbf{U}^T\mathbf{U}\mathbf{D}\mathbf{V}^T \tag{10.20}$$

since $\mathbf{U}$ and $\mathbf{V}$ are orthogonal we have that $\mathbf{V}^T = \mathbf{V}^{-1}$ and $\mathbf{U}^T = \mathbf{U}^{-1}$, so:

$$\mathbf{X}^T\mathbf{X} = \mathbf{V}\mathbf{D}\underbrace{\mathbf{U}^T\mathbf{U}}_{=\mathbf{I}}\mathbf{D}\mathbf{V}^T = \mathbf{V}\mathbf{D}^2\mathbf{V}^T \tag{10.21}$$

hence $\mathbf{H}_\lambda$ can be expressed as:

$$\begin{aligned}
\mathbf{H}_\lambda &= \mathbf{U}\mathbf{D}\mathbf{V}^T(\mathbf{V}\mathbf{D}^2\mathbf{V}^T + \lambda\mathbf{I})^{-1}\mathbf{V}\mathbf{D}\mathbf{U}^T \\
&= \mathbf{U}\mathbf{D}\mathbf{V}^T(\mathbf{V}\mathbf{D}^2\mathbf{V}^T + \underbrace{\mathbf{V}\lambda\mathbf{V}^T}_{\mathbf{V}\mathbf{V}^T=\mathbf{I}})^{-1}\mathbf{V}\mathbf{D}\mathbf{U}^T \\
&= \mathbf{U}\mathbf{D}\mathbf{V}^T(\mathbf{V}(\mathbf{D}^2 + \lambda\mathbf{I})\mathbf{V}^T)^{-1}\mathbf{V}\mathbf{D}\mathbf{U}^T
\end{aligned} \tag{10.22}$$

since $(\mathbf{A}\mathbf{B}\mathbf{C})^{-1} = (\mathbf{B}\mathbf{C})^{-1}\mathbf{A}^{-1} = \mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1}$ we have:

$$\begin{aligned}
\mathbf{H}_\lambda &= \mathbf{U}\mathbf{D}\mathbf{V}^T(\underbrace{\mathbf{V}^T}_{\mathbf{V}^T=\mathbf{V}^{-1}})^{-1}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}(\mathbf{V})^{-1}\mathbf{V}\mathbf{D}\mathbf{U}^T \\
&= \mathbf{U}\mathbf{D}\underbrace{\mathbf{V}^T\mathbf{V}}_{=\mathbf{I}}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\underbrace{\mathbf{V}^T\mathbf{V}}_{=\mathbf{I}}\mathbf{D}\mathbf{U}^T \\
&= \mathbf{U}\mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{D}\mathbf{U}^T
\end{aligned} \tag{10.23}$$

We note at this point that the expression $\mathbf{U}\mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{D}\mathbf{U}^T$ is the diagonalization of $\mathbf{H}_\lambda$ with respect to $\mathbf{U}$.

N.B. A matrix $\mathbf{T}$ is diagonalizable if there exists a matrix $\mathbf{P}$ such that:

$$\mathbf{P}^{-1}\mathbf{T}\mathbf{P} = \mathbf{C} = \begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \ddots & & & \\ 0 & 0 & \dots & \lambda_n \end{pmatrix} \tag{10.24}$$

in our case we have $\mathbf{T} = \mathbf{H}_\lambda$, $\mathbf{P} = \mathbf{U}$ and $\mathbf{C} = \mathbf{D}(\mathbf{D}^2+\mathbf{I})^{-1}\mathbf{D}$ is the diagonalization of $\mathbf{H}_\lambda$ with respect to $\mathbf{U}$ in fact:

$$\begin{aligned} &\mathbf{U}^{-1}\mathbf{H}_\lambda\mathbf{U} = \mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{D} \\ &\implies \mathbf{H}_\lambda = \mathbf{U}\mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{D}\mathbf{U}^{-1} \\ &\implies \mathbf{H}_\lambda = \mathbf{U}\mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{D}\mathbf{U}^{T} \end{aligned} \tag{10.25}$$

Consequently, $\mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{D}$ contains the eigenvalues of $\mathbf{H}_\lambda$ and each element on its diagonal has the form $\frac{d_i^2}{d_i^2+\lambda}$. The trace of $\mathbf{H}_\lambda$ is then given by the sum of these elements:

$$trace(\mathbf{H}_\lambda) = \sum_{j=0}^{D-1} \frac{d_j^2}{d_j^2 + \lambda} \tag{10.26}$$

The eigenvalues of $\mathbf{H}_\lambda$ are therefore the scaled version via λ of the eigenvalues of $\mathbf{X}^T\mathbf{X}$. By increasing λ we are pushing the eigenvalues of $\mathbf{H}_\lambda$ to zero and consequently reducing its degrees of freedom.

- If $\lambda \to \infty$ then the DoF $\to 0$;
- If $\lambda \to 0$ then the DoF $\to D$.

Thus, regularization results in a reduction in the degrees of freedom of the model and, thus, in the risk of overfitting.

10.1.2 Bias and Variance of the Ridge Regression model

Let's calculate the expected value and variance of $\mathbf{w}_{MLE,RIDGE}$

N.B. 2 We are not computing the bias and variance of the prediction on a test sample, but on the optimal parameters $\mathbf{w}_{MLE,RIDGE}$. The two are clearly related, as we saw earlier, but the computation is slightly different.

Assuming that the true process has a linear dependence between input and output, we have that $E[\mathbf{t}|\mathbf{X}] = \mathbf{X}\mathbf{w}$ in the case of linear regression (and also ridge regression,

once we learn the optimal parameters). Hence:

$$\begin{aligned} E[\mathbf{w}_{MLE,RIDGE}|\mathbf{X}] &= E[(\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{t}|\mathbf{X}] \\ &= (\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^T E[\mathbf{t}|\mathbf{X}] = (\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{X}\underbrace{\mathbf{w}}_{\text{the true unknown model}} \\ &= (\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}(\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I}-\lambda\mathbf{I})\mathbf{w} \\ &= [\mathbf{I}-\lambda(\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}]\mathbf{w} \\ &= \mathbf{w}-\lambda(\mathbf{X}^T\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{w} \end{aligned} \tag{10.27}$$

Thus the bias of the ridge regression estimator is proportional to λ: the higher λ, the higher the bias, while for $\lambda \to 0$ we return to the unbiased case of linear regression.

As for the variance, it can be shown that:

$$trace(Var[\mathbf{w}_{MLE,RIDGE}|\mathbf{X}]) = \sigma_\epsilon^2 \sum_{j=0}^{D-1} \frac{d_j^2}{d_j^2+\lambda} \tag{10.28}$$

while for linear regression:

$$trace(Var[\mathbf{w}_{MLE,LR}|\mathbf{X}]) = \sigma_\epsilon^2 \sum_{j=0}^{D-1} \frac{1}{d_j^2} \tag{10.29}$$

Hence, with $\lambda \geq 0$ we have that $trace(Var[\mathbf{w}_{MLE,LR}|\mathbf{X}]) > trace(Var[\mathbf{w}_{MLE,RIDGE}|\mathbf{X}])$: ridge regression thus has systematically less uncertainty about its estimated optimal parameters thatn linear regression.

10.2 Lasso Regression

In ridge regression high values of λ tend to make the parameters associated with the various features very small. If, on the other hand, we really want to "zero out" some degrees of freedom, we can use **Lasso Regression**, which produces a **sparse** solution, *i.e.,* a model with many parameters at zero. In practice, the only difference with ridge regression is in the constraint on the solution problem, which is no longer related to the square of the parameters, but to their modulus:

$$\mathbf{w}_{MLE,LASSO} = \underset{\mathbf{w}\in\mathbb{R}^D}{\operatorname{argmin}} \sum_{i=0}^{N-1} (t^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)})^2 \tag{10.30}$$

$$\text{subject to } \sum_{j=1}^{D-1} |w_j| \leq l \tag{10.31}$$

Using Lagrange multipliers as per ridge regression, we can write:

$$\mathbf{w}_{MLE,LASSO} = \underset{\mathbf{w}\in\mathbb{R}^D}{\operatorname{argmin}} \sum_{i=0}^{N-1} (t^{(i)} - \mathbf{w}^T\mathbf{x}^{(i)})^2 + \lambda \sum_{j=1}^{D-1} |w_j| \tag{10.32}$$

In this case, however, there is no closed-form solution and **quadratic programming** techniques must be used.

To conclude, it should be pointed out that often, when we talk about ridge or lasso regression it is usual to say that we are applying **L2 or L1 penalties** to the optimization problem. This is because ridge regression constrains the problem with the L2 norm of the parameters $\sum_{j=0}^{D-1}(w_j)^2$, while lasso with the L1 norm $\sum_{j=0}^{D-1}|w_j|$.

> The L2 and L1 penalties can be applied to virtually any machine learning algorithm, including logistic regression.

Chapter 11

Support Vector Machine (SVM)

In the case of Logistic Regression, the hypothesis function was given by:

$$p(t = 1|\mathbf{x}, \mathbf{w}) = h_{\mathbf{w}}(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1 + \exp^{-\mathbf{w}^T\mathbf{x}}} \tag{11.1}$$

The trend of the sigmoid as its argument varies (*i.e.,* $\mathbf{w}^T\mathbf{x}$) is shown in Figure 11.1. Clearly, if $p(t = 1|\mathbf{x}, \mathbf{w}) \to 1$, we are more confident about the prediction $t = 1$, whereas if $p(t = 1|\mathbf{x}, \mathbf{w})$ is close to 0.5, the confidence about the prediction decreases. Since we know that $h_{\mathbf{w}}(\mathbf{x}) \geq 0.5 \iff \mathbf{w}^T\mathbf{x} \geq 0$, it is clear that the larger the value of $\mathbf{w}^T\mathbf{x}$, the larger the probability $p(t = 1|\mathbf{x}, \mathbf{w})$ and thus also our confidence with respect to the prediction $t = 1$. For this reason, if we consider a training set with N samples, our goal is to obtain parameters $\mathbf{w}$ that guarantee $\mathbf{w}^T\mathbf{x} >> 0$ if $t = 1$, while $\mathbf{w}^T\mathbf{x} << 0$ if $t = 0$. For reasons that will become clear shortly, suppose that the output variable takes values in the set $t \in \{1, -1\}$ instead of $t \in \{1, 0\}$.

We know that $\mathbf{w}^T\mathbf{x} = 0$ identifies the decision boundary or decision hyper-plane, which marks the boundary between points in the feature space that belong to either class 1 or class -1. Intuitively, the further away a sample is from the hyper-plane, the more confident we will be with respect to its prediction.

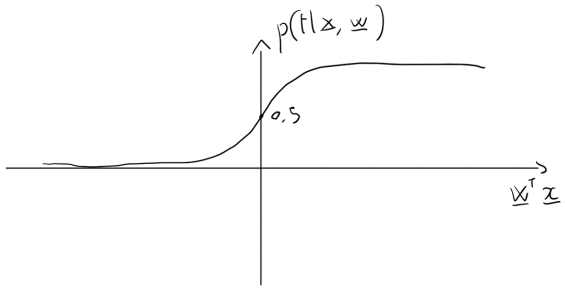


Figure 11.1: Hypothesis function for Logistic Regression

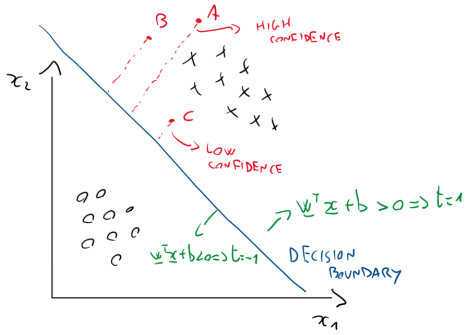


Figure 11.2: Relation between the confidence of the predictions and the distance from the decision hyper-plane (boundary).

> N.B. We talk about hyper-plane, in general, because if the features are in a D dimensional space, the separator is identified by the equation $\mathbf{w}^T\mathbf{x} = 0$, which determines an object in the $D-1$ dimensional space. In the two-feature case, we have a line as separator, in fact $w_0 + w_1x_1 + w_2x_2 = 0$, while in the 3-feature case we have a plane.

We can see this better by looking at Figure 11.2. Point A is associated with a value $\mathbf{w}^T\mathbf{x}_A >> 0$ and its distance from the hyper-plane is high: the risk of the point passing to the other side of the hyper-plane due to small perturbations is very low and this therefore determines a higher confidence than predicting $t = 1$ for that point. Conversely, point C is close to the hyper-plane, so $\mathbf{w}^T\mathbf{x}_C \approx 0$: small changes in the feature values could result in a transition to the other side of the hyper-plane. The confidence with respect to prediction for this point is therefore lower.

It is then reasonable to look for the hyper-plane (determined by the trainable parameters $\mathbf{w}$) that is as far away as possible from the points in the training dataset. How can we determine this? There are clearly many possible hyper-planes separating the two classes (see Figure 11.3). But which one is the best? To determine it we need a quantitative measure of the distance between the points and the hyper-plane: this measure is called **Margin**.

11.1 Functional and Geometric Margin

With respect to the classification problem, the possibility of computing the probability of the posterior distribution on the output variable was possible by resorting to discriminative models (such as Logistic Regression). If we are not interested in modeling the distribution explicitly, we can formulate the problem similarly to what we did for discriminant functions, where the classifier is characterized by a function that maps the term $\mathbf{w}^T\mathbf{x} + b$ (N.B. b is the bias term, often denoted w_0) into the values 1 and -1. In

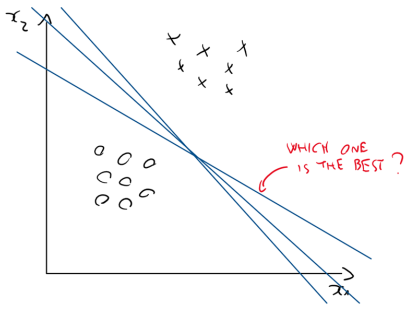


Figure 11.3: There are many possible hyper-planes that separate the two classes.

this case, instead of using the sigmoid, we saw that one possibility consists in using a function $g(\cdot)$ as follows:

$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T\mathbf{x} + b) \tag{11.2}$$

$$\text{where } g(\mathbf{w}^T\mathbf{x} + b) = \begin{cases} +1 \text{ if } \mathbf{w}^T\mathbf{x} + b \geq 0 \\ -1 \text{ if } \mathbf{w}^T\mathbf{x} + b < 0 \end{cases} \tag{11.3}$$

As we have at the beginning of the chapter, confidence with respect to the prediction of a sample $(x^{(i)}, t^{(i)})$ is related to how much $\mathbf{w}^T\mathbf{x}^{(i)} + b$ is greater than zero (in the case $t^{(i)} = 1$) or less than zero (in the case $t^{(i)} = -1$). By taking advantage of the fact that we have encoded labels with $+1$ and -1, we can write:

$$\hat{\gamma}^{(i)} = t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \tag{11.4}$$

The quantity $\hat{\gamma}^{(i)}$ is called the **Functional Margin** for the i-th sample. Thus, if $t^{(i)} = -1$, for the functional margin to be large it is necessary that $\mathbf{w}^T\mathbf{x} << 0$. Conversely, in the case $t^{(i)} = 1$, for the functional margin to be large, we must have $\mathbf{w}^T\mathbf{x} >> 0$.

Having defined the functional margin for one sample, the functional margin associated with the hyper-plane, defined by the parameters $\mathbf{w}$ and b, can be reasonably assumed to be the smallest functional margin among all those for the various samples in the dataset:

$$\hat{\gamma} = \min_{i=0,\ldots,N-1} \hat{\gamma}^{(i)} \tag{11.5}$$

Unfortunately, the functional margin has a property that makes it unsuitable for the purpose of determining the optimal parameters $\mathbf{w}$ and b. If we replace $\mathbf{w}$ and b with $2\mathbf{w}$ and $2b$, the prediction of the function of g does not change, since it is only associated with the sign of $\mathbf{w}^T\mathbf{x}$. However, in this way the functional margin is increased by a factor of 2. If we optimize the parameters by maximizing this margin, we could indefinitely

increase $\mathbf{w}$ and b to increase $\hat{\gamma}$ without an effect on the predictions. This, in addition to generating a lot of uncertainty in determining the optimal parameters of the hyper-plane, may also generate solutions without practical meaning.

Let's introduce a margin with a more defined geometric meaning. To do so, we compute the margin of a point as its distance with respect to the hyper-plane. We have already seen these calculations in the 5.5 section when we talked about decision boundary for discriminant functions. Taking up those calculations and considerations, we can calculate the margin $\gamma^{(i)}$ of a point $\mathbf{x}^{(i)}$ by considering that:

- The distance between the point $\mathbf{x}^{(i)}$ and the hyperplane is a vector proportional to the versor of the hyperplane $\mathbf{w}/||\mathbf{w}||$ and is therefore defined as $\gamma^{(i)}\frac{\mathbf{w}}{||\mathbf{w}||}$. The scalar term that governs the length of the vector is precisely the margin.
- The vector $\mathbf{x}^{(i)}$ can be computed as the sum of the vector $\mathbf{x}_{\perp}^{(i)}$ relative to the projection point on the hyperplane and the projection vector $\gamma^{(i)}\frac{\mathbf{w}}{||\mathbf{w}|}$, *i.e.,* :

$$\mathbf{x}^{(i)} = \mathbf{x}_{\perp}^{(i)} + \gamma^{(i)}\frac{\mathbf{w}}{||\mathbf{w}||} \tag{11.6}$$

- The point identified by the vector $\mathbf{x}_{\perp}^{(i)}$, belongs to the hyper-plane, for which $\mathbf{w}^T\mathbf{x}^{(i)} + b = 0$ holds.

Putting these aspects together, we get:

$$\mathbf{x}_{\perp}^{(i)} = \mathbf{x}^{(i)} - \gamma^{(i)}\frac{\mathbf{w}}{||\mathbf{w}||} \tag{11.7}$$

Hence:

$$\begin{aligned}\underbrace{\mathbf{w}^T\mathbf{x}_{\perp}^{(i)} + b}_{=0} &= \mathbf{w}^T(\mathbf{x}^{(i)} - \gamma^{(i)}\frac{\mathbf{w}}{||\mathbf{w}||}) + b \\ &\implies \mathbf{w}^T\mathbf{x}^{(i)} - \gamma^{(i)}\frac{||\mathbf{w}||^2}{||\mathbf{w}||} + b = 0 \\ &\implies \gamma^{(i)} = \frac{\mathbf{w}^T\mathbf{x}^{(i)} + b}{||\mathbf{w}||} = \left(\frac{\mathbf{w}}{||\mathbf{w}||}\right)^T\mathbf{x}^{(i)} + \left(\frac{b}{||\mathbf{w}||}\right)\end{aligned} \tag{11.8}$$

This relation holds in the case where the sample is on the "positive" side of the hyperplane. For the negative case it is sufficient to change the sign of $\gamma^{(i)}$ and, thus, in general, for the sample $(x^{(i)}, t^{(i)})$ we can write:

$$\gamma^{(i)} = t^{(i)}\left(\left(\frac{\mathbf{w}}{||\mathbf{w}||}\right)^T\mathbf{x}^{(i)} + \left(\frac{b}{||\mathbf{w}||}\right)\right) \tag{11.9}$$

The quantity $\gamma^{(i)}$ is called **Geometrical Margin** of the sample $(x^{(i)}, t^{(i)})$. This margin does not suffer from the functional margin problem. In fact, the geometric margin is

Invariant to rescaling, since, if we change $\mathbf{w}$ and b to $2\mathbf{w}$ and $2b$, $\gamma^{(i)}$ will remain the same. This property is very useful, as we can impose any constraint on the weights without changing anything with respect to the optimization problem.

We also note that the geometric margin is the normalized functional margin with respect to $||\mathbf{w}||$ and thanks to the invariance to rescaling property, we can impose $||\mathbf{w}|| = 1$ to make them equivalent. This will allow us to optimize with respect to one or the other without impacting the solution, which, as we will see in a moment, will become particularly handy.

As we did for the margin functional, we consider as the geometric margin of the hyper-plane defined by $\mathbf{w}$ and b the smallest margin among all those of the various samples in the dataset:

$$\gamma = \min_{i=0,\dots,N-1} \gamma^{(i)} \tag{11.10}$$

11.2 Hard Margin Classifier

Assume for the moment that the points in the training set are linearly separable, *i.e.,* it is possible to find a hyper-plane that perfectly divides the points in the set into belonging to class 1 and -1. To find the best hyper-plane, it is reasonable to consider the geometric margin (hence the margin of the sample closest to the hyper-plane) and maximize it. We then define the following optimization problem:

$$\max_{\gamma,\mathbf{w},b} \gamma \tag{11.11}$$

$$\text{subject to } t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq \gamma \ , \ i = 0, \dots, N-1$$

$$||\mathbf{w}|| = 1 \tag{11.12}$$

The two constraints ensure that all margins are at least equal to γ (the smallest) and that the geometric and functorial margin are equal $||\mathbf{w}|| = 1$.

Unfortunately, it is not easy to solve this optimization problem, since the constraint $||\mathbf{w}|| = 1$ is **Non-Convex**. To understand why this constraint is non-convex, we introduce some concepts of convex optimization.

11.2.1 Convex optimization notes

We begin by giving the definition of **Affine Set**:

Definition 11.1. Affine Set: A set C is said to be **Affine** if the line passing through two distinct points of C is itself in C, *i.e.,* if $\forall \mathbf{x}, \mathbf{y} \in C$ and any $\theta \in \mathbb{R}$ we have that $\theta\mathbf{x} + (1-\theta)\mathbf{y} \in C$.

Among the properties of affine sets we have that:

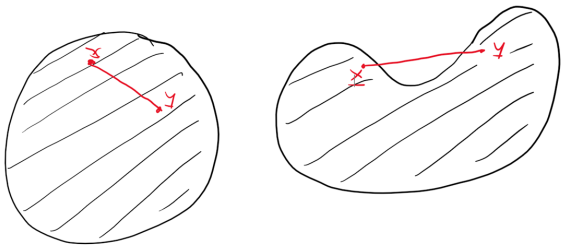


Figure 11.4: On the left, an example of a convex set. On the right a non-convex set.

- A point expressed as a combination of points of an affine set $\theta_1\mathbf{x}_1 + \cdots + \theta_k\mathbf{x}_k$ with $\theta_1 + \cdots + \theta_k = 1$ is called **Affine Combination** of the points $\mathbf{x}_1, \ldots, \mathbf{x}_k$.
- An affine set contains every affine combination of its points.

An example of an affine set is the set of solutions of a linear system of equations, $C = \{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{b}\}$, in fact, assuming that $\mathbf{x}, \mathbf{y} \in C$, *i.e.,* $\mathbf{A}\mathbf{x} = \mathbf{b}$ and $\mathbf{A}\mathbf{y} = \mathbf{b}$, then for every θ:

$$\mathbf{A}(\theta\mathbf{x} + (1-\theta)\mathbf{y}) = \theta\mathbf{A}\mathbf{x} + (1-\theta)\mathbf{A}\mathbf{y} = \theta\mathbf{b} + (1-\theta)\mathbf{b} = \mathbf{b} \tag{11.13}$$

so $\theta\mathbf{x} + (1-\theta)\mathbf{y} \in C$.

Definition 11.2. Convex Set: A set C is said to be **Convex** if the segment between two distinct points of C is itself in C, *i.e.,* if $\forall \mathbf{x}, \mathbf{y} \in C$ and any $\theta : 0 \leq \theta \leq 1$ we have that $\theta\mathbf{x} + (1-\theta)\mathbf{y} \in C$.

As with affine sets, the following properties apply:

- A point expressed as a combination of points of an affine set $\theta_1\mathbf{x}_1 + \cdots + \theta_k\mathbf{x}_k$ with $\theta_1 + \cdots + \theta_k = 1$ and $\theta_1, \ldots, \theta_k \geq 0$ is called a **Convex Combination** of points $\mathbf{x}_1, \ldots, \mathbf{x}_k$.
- A convex set contains every convex combination of its points.

Examples of a convex and non-convex set are shown in Figure 11.4.

Definition 11.3. Convex Function: A function $f : \mathbb{R}^d \to \mathbb{R}$ is called **Convex** if $Dom(f)$ is a convex set and if $\forall \mathbf{x}, \mathbf{y} \in Dom(f)$ and $\theta : 0 \leq \theta \leq 1$ we have:

$$f(\theta\mathbf{x} + (1-\theta)\mathbf{y}) \leq \theta f(\mathbf{x}) + (1-\theta)f(\mathbf{y}) \tag{11.14}$$

An example of a convex function is shown in Figure 11.5, while a non-convex function is shown in Figure 11.6.

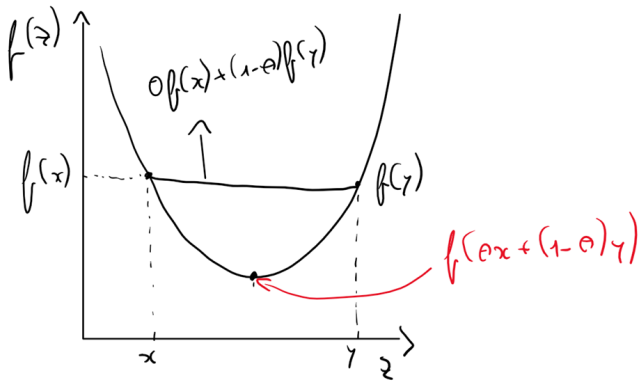


Figure 11.5: An example of a convex function.

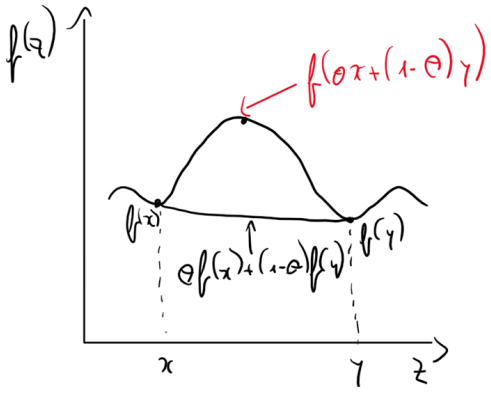


Figure 11.6: An example of a non-convex function.

It makes sense to ask if there are any results that, in addition to the condition expressed in the definition, allow us to identify whether a function is convex or not:

Theorem 11.4. *Suppose that f is differentiable (i.e., ∇f exists at every point of $Dom(f)$). f is convex if and only if $Dom(f)$ is convex and:*

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^T(\mathbf{y} - \mathbf{x}) \tag{11.15}$$

is verified $\forall \mathbf{x}, \mathbf{y} \in Dom(f)$

We prove the $\implies$ part of the theorem in the one-dimensional case.

Proof: "$\implies$". We assume f convex and consider the one-dimensional case, i.e., $Dom(f) \in \mathbb{R}^1$. Since f is convex, then it holds that $\forall \mathbf{x}, \mathbf{y} \in Dom(f)$:

$$f(\theta\mathbf{x} + (1-\theta)\mathbf{y}) = f(\theta\mathbf{x} + \mathbf{y} - \theta\mathbf{y}) = f(\mathbf{y} + \theta(\mathbf{x} - \mathbf{y})) \leq \theta f(\mathbf{x}) + (1-\theta)f(\mathbf{y}) \tag{11.16}$$

dividing by θ we obtain:

$$f(\mathbf{x}) \geq \frac{f(\mathbf{y} + \theta(\mathbf{x} - \mathbf{y})) - f(\mathbf{y})}{\theta} \tag{11.17}$$

and for the limit $\theta \to 0$ we get the relation 11.15.

Another way to check if a function is convex is to take advantage of the following theorem:

Theorem 11.5. *Suppose that f is twice differentiable (i.e., the Hessian $\nabla^2 f$ exists at every point of $Dom(f)$). f is convex if and only if $Dom(f)$ is convex and its Hessian is* ***semidefinite positive****, i.e., $\forall \mathbf{x} \in Dom(f)$:*

$$\nabla^2 f(\mathbf{x}) \succcurlyeq 0 \tag{11.18}$$

Now that we have defined what a convex set and a convex function are, let's talk about **Convex Optimization Problems**. To do so, we consider the following constrained optimization problem:

$$\begin{aligned} &\min_{\mathbf{x}} f(\mathbf{x}) \\ &\text{subject to } g_i(\mathbf{x}) \leq 0,\ i = 0, \ldots, L-1 \\ &\qquad\qquad h_j(\mathbf{x}) = 0,\ j = 0, \ldots, K-1 \end{aligned} \tag{11.19}$$

We define some useful concepts for optimization purposes:

Definition 11.6. Feasible Point: A point $\mathbf{x}$ is **Feasible** if it satisfies all the constraints of the optimization problem. The optimization problem is said to be **Feasible** if there exists at least one point $\mathbf{x}$ that is feasible.

Definition 11.7. Optimal Value: The optimal value of the generic optimization problem introduced earlier is defined as:

$$p^* = \inf\{f(\mathbf{x}) : g_i(\mathbf{x}) \leq 0,\, i = 0, \ldots, L-1, h_j(\mathbf{x}) = 0,\, j = 0, \ldots, K-1\} \tag{11.20}$$

where $p^* = \infty$ if the problem is **infeasible** and $p^* = -\infty$ if it is **unbounded below**.

Definition 11.8. Optimal Feasible Point: A feasible $\mathbf{x}$ point is **optimal** if $f(\mathbf{x}) = p^*$. A feasible $\mathbf{x}$ point is **locally optimal** if $\exists R > 0$ such that:

$$f(\mathbf{x}) = \inf_{\mathbf{z}}\{f(\mathbf{z}) : g_i(\mathbf{z}) \leq 0,\, i = 0, \ldots, L-1, h_j(\mathbf{z}) = 0,\, j = 0, \ldots, K-1\ ,\ ||\mathbf{z} - \mathbf{x}||_2 \leq R\} \tag{11.21}$$

Definition 11.9. Convex Optimization problem: An optimization problem is convex if:

- $f(\mathbf{x})$ is convex;
- $g_i(\mathbf{x})$ is a convex set $\forall i = 0, \ldots, L-1$;
- $h_i(\mathbf{x})$ is an affine set $\forall i = 0, \ldots, K-1$

Convex optimization problems are much easier to handle. In fact, there is the following important theorem that, for these kind of problem, guarantees that the local optimum point is also global:

Theorem 11.10. *Every local optimum point of a convex optimization problem is a* ***Global*** *optimum point.*

Proof.

Suppose $\mathbf{x}^*$ *is a local optimum of a convex optimization problem, i.e.,* $\mathbf{x}^*$ *is feasible and* $\exists R > 0$ *such that:*

$$f(\mathbf{x}^*) = \inf_{\mathbf{z}}\{f(\mathbf{z}) : \mathbf{z} \textit{ is feasible},\, ||\mathbf{z} - \mathbf{x}^*||_2 \leq R\} \tag{11.22}$$

Suppose now that $\mathbf{x}^*$ *is not globally optimal, that is, there exists a* $\mathbf{y}$ *feasible such that* $f(\mathbf{y}) < f(\mathbf{x}^*)$*. Clearly* $||\mathbf{y} - \mathbf{x}^*||_2 > R$ *otherwise, since* $\mathbf{x}^*$ *is a local optimum we would necessarily have* $f(\mathbf{y}) > f(\mathbf{x}^*)$*. Consider the point* $\mathbf{z}$ *defined in the following way:*

$$\mathbf{z} = (1 - \theta)\mathbf{x}^* + \theta\mathbf{y} \tag{11.23}$$

With θ *defined as follows:*

$$\theta = \frac{R}{2||\mathbf{y} - \mathbf{x}^*||_2} \tag{11.24}$$

and therefore, since $||\mathbf{y} - \mathbf{x}^*||_2 > R$ *we have that* $0 \leq \theta \leq 1/2$. *If we calculate the distance between* $\mathbf{z}$ *and* $\mathbf{x}^*$ *we get:*

$$\begin{aligned} ||\mathbf{z} - \mathbf{x}^*||_2 &= ||(1-\theta)\mathbf{x}^* + \theta\mathbf{y} - \mathbf{x}^*||_2 = ||\theta(\mathbf{y} - \mathbf{x}^*)||_2 \\ &= \theta||\mathbf{y} - \mathbf{x}^*||_2 = \frac{R}{2} \end{aligned} \tag{11.25}$$

Hence, we necessarily have that $f(\mathbf{z}) > f(\mathbf{x}^*)$, *since* $\mathbf{z}$ *falls in the area where* $\mathbf{x}^*$ *is local optimum. Taking advantage of the fact that* f *is convex (since by hypothesis we have a convex optimization problem) we have:*

$$\begin{aligned} f(\mathbf{z}) = f(\theta\mathbf{y} + (1-\theta)\mathbf{x}^*) &\underbrace{\leq}_{\text{since } f \text{ is convex}} \theta f(\mathbf{y}) + (1-\theta)f(\mathbf{x}^*) \\ &\underbrace{\leq}_{\text{since } f(\mathbf{y}) < f(\mathbf{x}^*)} \theta f(\mathbf{x}^*) + (1-\theta)f(\mathbf{x}^*) = f(\mathbf{x}^*) \end{aligned} \tag{11.26}$$

It then follows from the latter relation that $f(\mathbf{x}^*) \geq f(\mathbf{z})$ *which contradicts the assumption that* $\mathbf{x}^*$ *is a local optimum.*

11.2.2 Hard-margin classifier as a convex optimization problem

It is now clear then that the problem we introduced earlier, *i.e.,* :

$$\max_{\gamma, \mathbf{w}, b} \gamma \tag{11.27}$$

$$\text{subject to } t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq \gamma \ , \ i = 0, \ldots, N-1$$

$$||\mathbf{w}||_2 = 1 \tag{11.28}$$

is non-convex, because one of its constraints is a non-convex set, *i.e.,* $||\mathbf{w}||_2 = 1$. This can be easily seen by drawing this set in the two-dimensional case (see Figure 11.7). The points that belong to this set are all those on the edge of the circle. If we take two of these points, the segment connecting them clearly does not belong to the set, since it falls inside the circle. If we had instead a constraint like $||\mathbf{w}||_2 \leq 1$, the set would be convex, since it also includes all the points inside the circle.

Can we modify the problem 11.27 to make it convex? We start by removing the non-convex constraint. To do this, we recall that if $||\mathbf{w}||_2 = 1$ the functional margin and the geometric margin are equivalent. We also know that the two are related by $\gamma = \hat{\gamma}/||\mathbf{w}||_2$, so we can remove the constraint and optimize with respect to the functional margin:

$$\max_{\hat{\gamma}, \mathbf{w}, b} \frac{\hat{\gamma}}{||\mathbf{w}||_2} \tag{11.29}$$

$$\text{subject to } t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq \hat{\gamma} \ , \ i = 0, \ldots, N-1$$

Unfortunately, in removing the non-convex constraint we obtained a non-convex

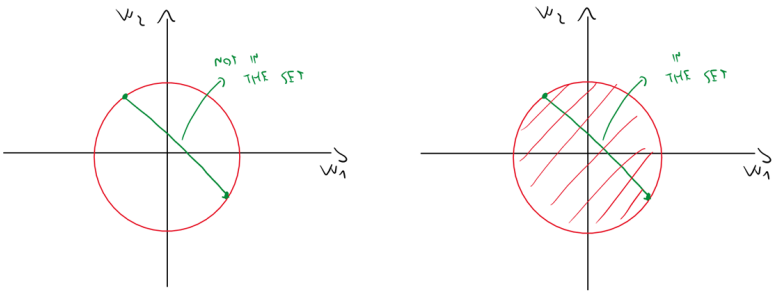


Figure 11.7: On the left the set $||\mathbf{w}||_2 = 1$, which includes all the points on the border of the circle (in red). It is non convex, since if we take two distinct of the set, the segment between them does not belong to the set. On the other hand, on the right, we depict the set $||\mathbf{w}||_2 \leq 1$, which includes all the points inside the circle. In this case the set is convex, since for each pair of points within the set, the segment between them belongs to the set as well.

function, *i.e.*, $\frac{\hat{\gamma}}{||\mathbf{w}||_2}$ (this is a function of type $\frac{1}{\sqrt{x^2}}$, shown in Figure 11.8).

To solve this problem, recall that we can change $\mathbf{w}$ and b without altering the geometric margin and thus the optimization problem. We exploit this property and choose the parameters so that the functional margin is equal to 1. The problem then becomes:

$$\begin{aligned} &\max_{\hat{\gamma},\mathbf{w},b} \frac{1}{||\mathbf{w}||_2} \\ &\text{subject to } t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1 \ , \ i = 0, \ldots, N-1 \end{aligned} \tag{11.30}$$

At this point, we note that maximizing $\frac{1}{||\mathbf{w}||_2}$ is equivalent to minimizing $||\mathbf{w}||_2^2$:

$$\begin{aligned} &\min_{\hat{\gamma},\mathbf{w},b} ||\mathbf{w}||_2^2 \\ &\text{subject to } t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1 \ , \ i = 0, \ldots, N-1 \end{aligned} \tag{11.31}$$

The function $||\mathbf{w}||_2^2$ is clearly convex (it is a paraboloid), thus our optimization problem is now convex and can be easily solved by **Quadratic Programming** techniques.

The optimization problem in this form is known as the **Primal Form** of the **Hard Margin Classifier**. it is interesting at this point to introduce the concept of a **Dual** problem, which allows us to describe it in a more efficient form.

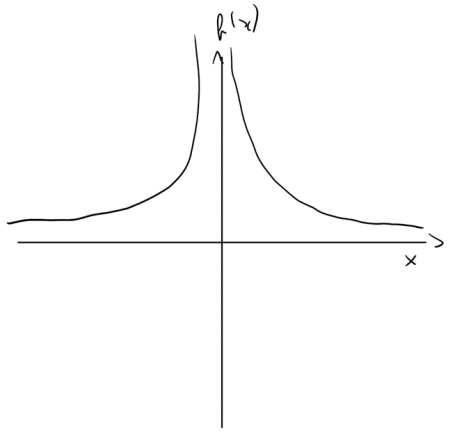


Figure 11.8: Function $\frac{1}{\sqrt{x}}$.

11.2.3 Lagrange Duality

The form of the optimization problem we saw earlier is known as **Primal Form**. In general, an optimization problem where we make explicit the function to be optimized and its constraints is called in **Primal Form**:

$$\min_{\hat{\gamma},\mathbf{w},b} f(\mathbf{w}) \tag{11.32}$$

$$\text{subject to } g_i(\mathbf{w}) \leq 0 \ , \ i = 0, \ldots, L-1$$

$$h_j(\mathbf{w}) = 0 \ , \ j = 0, \ldots, K-1 \tag{11.33}$$

To introduce the dual form, we rewrite the problem using the **Generalized Lagrangian**:

$$L(\mathbf{w}, \alpha, \beta) = f(\mathbf{w}) + \sum_{i=0}^{L-1} \alpha_i g_i(\mathbf{w}) + \sum_{j=0}^{K-1} \beta_j h_j(\mathbf{w}) \tag{11.34}$$

Let's introduce quantity:

$$\theta_P(\mathbf{w}) = \max_{\alpha_i, \beta_j, \alpha_i \geq 0} L(\mathbf{w}, \alpha, \beta) \tag{11.35}$$

where, P indicates that we are considering the form **Primal**. We can easily see that, when the constraints are satisfied, $\theta_P(\mathbf{w})$ is in fact equal to $f(\mathbf{w})$, while it is equal to ∞ if they are not. In fact, if at least one constraint is not satisfied, one of the terms in the summations of the Lagrangian is non-zero. Since we want to maximize the Lagrangian, we can indefinitely increase α_i or β_j to increase the value of the function. If, on the other hand, all constraints are satisfied, all elements of the summation are zero and changing

α_i or β_j has no effect on the value of the Lagrangian. We therefore have:

$$\theta_P(\mathbf{w}) = \begin{cases} f(\mathbf{w}) \text{ if } \mathbf{w} \text{ satisfies the constraints} \\ \infty \text{ otherwise} \end{cases} \tag{11.36}$$

If we therefore minimize $\theta_P(\mathbf{w})$, we are in fact working with primal problem defined by 11.32:

$$\min_{\mathbf{w}} \theta_P(\mathbf{w}) = \min_{\mathbf{w}} \max_{\alpha_i, \beta_j, \alpha_i \geq 0} L(\mathbf{w}, \alpha, \beta) \tag{11.37}$$

Let's define the **Dual problem** as follows:

$$\max_{\alpha_i,\beta_j,\alpha_i\geq 0} \theta_D(\alpha,\beta) = \max_{\alpha_i,\beta_j,\alpha_i\geq 0} \min_{\mathbf{w}} L(\mathbf{w},\alpha,\beta) \tag{11.38}$$

The dual problem is then defined by reversing the order of the minimum and maximum with respect to the primal problem. If we denote by p^* and d^* the optimal solutions of the primal and dual problems, respectively, *i.e.,* $p^* = \min_{\mathbf{w}} \theta_P(\mathbf{w})$ and $d^* = \max_{\alpha_i,\beta_j,\alpha_i\geq 0} \theta_D(\alpha,\beta)$, it is possible to show that the following relation holds:

$$d^* = \max_{\alpha_i,\beta_j,\alpha_i\geq 0} \min_{\mathbf{w}} L(\mathbf{w},\alpha,\beta) \leq \min_{\mathbf{w}} \max_{\alpha_i,\beta_j,\alpha_i\geq 0} L(\mathbf{w},\alpha,\beta) = p^* \tag{11.39}$$

In addition, if the following conditions are met:

- the optimization problem is convex, *i.e.,* $f(\mathbf{w})$ is convex, all constraints $g_i \leq 0$ are convex sets, and all constraints $h_j = 0$ are affine sets;
- the constraints $g_i \leq 0$ are **Strictly Feasible**, *i.e.,* $\exists \mathbf{w} : g_i(\mathbf{w}) \leq 0 \; \forall i = 0,\ldots,K-1$.

then $\exists \mathbf{w}^*$, α_i^*, β_j^* so that $\mathbf{w}^*$ is the solution to the primal problem $\min_{\mathbf{w}} \theta_P(\mathbf{w})$ and α_i^*, β_j^* are the solution to the dual problem $\max_{\alpha_i,\beta_j,\alpha_i\geq 0} \theta_D(\alpha,\beta)$ and it holds $d^* = p^*$. In addition, $\mathbf{w}^*$, α_i^*, β_j^* satisfy the KKT:

$$\begin{aligned}
\nabla_{\mathbf{w}} L(\mathbf{w}^*,\alpha_i^*,\beta_j^*) &= 0 \\
\frac{\partial L(\mathbf{w}^*,\alpha_i^*,\beta_j^*)}{\partial_{\alpha_i}} &= 0 \;,\; \forall i = 0,\ldots,L-1 \\
\frac{\partial L(\mathbf{w}^*,\alpha_i^*,\beta_j^*)}{\partial_{\beta_j}} &= 0 \;,\; \forall j = 0,\ldots,K-1 \\
\alpha_i^* g_i(\mathbf{w}^*) &= 0 \;,\; \forall i = 0,\ldots,L-1 \\
h_j(\mathbf{w}^*) &= 0 \;,\; \forall j = 0,\ldots,K-1 \\
g_i(\mathbf{w}^*) &\leq 0 \;,\; \forall i = 0,\ldots,L-1 \\
\alpha_i &\geq 0 \;,\; \forall i = 0,\ldots,L-1
\end{aligned} \tag{11.40}$$

We take up the max-margin classifier problem at this point. We have seen how the problem in primal form is expressed in the following form (where we assume to scale the parameters $\mathbf{w}$ to have a functional margin equal to $1/2$ for computational convenience):

$$\begin{aligned}
&\min_{\hat{\gamma},\mathbf{w},b} \frac{1}{2}||\mathbf{w}||_2^2 \\
&\text{subject to } t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1 \;,\; i = 0,\ldots,N-1
\end{aligned} \tag{11.41}$$

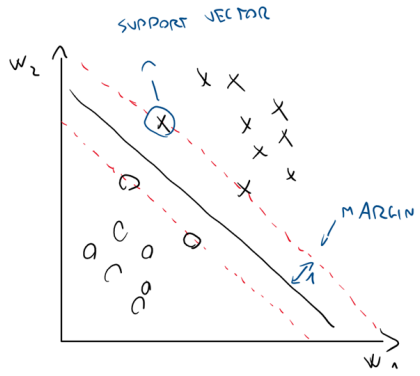


Figure 11.9: Example of support vectors.

We can rewrite the constraints as follows:

$$g_i(\mathbf{w}) = -t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) + 1 \leq 0 \ , \ i = 0, \dots, N-1 \tag{11.42}$$

Note that we have of such constraint for each training sample. We know that $f(\mathbf{w})$ in this case is convex and the sets identified by $g_i(\mathbf{w})$ are also convex. Moreover, as we said at the beginning of the chapter, the max margin classifier is defined with the assumption that the classes are linearly separable, *i.e.,* , that there exists a hyper-plane that perfectly divides the points of class +1 from those belonging to class -1. This means that there necessarily exists a set of parameters $\mathbf{w}^*$ that satisfies all constraints. So the condition of **Strictly Feasibility** is also verified.

Consequently, the solution to the primal and dual problems are equivalent, *i.e.,* $p^* = d^*$ and the KKTs are satisfied. In particular, the **Dual Complementarity** condition $\alpha_i^* g_i(\mathbf{w}^*) = 0$ is satisfied. We can therefore work indifferently with the primal or dual formulation.

It is important to note that the dual complementarity condition implies that the α_i^* are different from zero in the case of an active constraint and thus $g_i(\mathbf{w}^*) = 0$, *i.e.,* $t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) = 1$. This condition is true for all and only the points that are on the **margin**, *i.e.,* at distance 1 from the hyperplane (see 11.9). These samples are called **Support Vector** because, as we will see shortly, they are the only ones that contribute to the prediction of the class relative to a test sample once the parameters are trained.

Let's derive the dual formulation for the max margin classifier. We start by constructing the Lagrangian:

$$L(\mathbf{w}, b, \alpha_i) = \frac{1}{2}||\mathbf{w}||^2 - \sum_{i=0}^{N-1} \alpha_i (t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) - 1) \tag{11.43}$$

To find the dual form we proceed by minimizing with respect to $\mathbf{w}$ and b:

$$\begin{aligned}\nabla_{\mathbf{w}} L(\mathbf{w}, b, \alpha_i) &= \nabla_{\mathbf{w}} \left[\frac{1}{2} \underbrace{||\mathbf{w}||^2}_{=\mathbf{w}^T\mathbf{w}} - \sum_{i=0}^{N-1} \alpha_i (t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) - 1) \right] \\ &= \mathbf{w} - \sum_{i=0}^{N-1} \alpha_i t^{(i)} \mathbf{x}^{(i)} = 0 \\ \implies \mathbf{w} &= \sum_{i=0}^{N-1} \alpha_i t^{(i)} \mathbf{x}^{(i)} \end{aligned} \tag{11.44}$$

$$\begin{aligned}\nabla_b L(\mathbf{w}, b, \alpha_i) &= \nabla_b \left[\frac{1}{2} \underbrace{||\mathbf{w}||^2}_{=\mathbf{w}^T\mathbf{w}} - \sum_{i=0}^{N-1} \alpha_i (t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) - 1) \right] \\ \implies & \sum_{i=0}^{N-1} \alpha_i t^{(i)} = 0 \end{aligned} \tag{11.45}$$

Substituting the 11.44 into the 11.43 we have:

$$\begin{aligned} L(\mathbf{w}, b, \alpha_i) &= \frac{1}{2} \left(\sum_{i=0}^{N-1} \alpha_i t^{(i)} \mathbf{x}^{(i)} \right)^T \left(\sum_{j=0}^{N-1} \alpha_j t^{(j)} \mathbf{x}^{(j)} \right) - \sum_{i=0}^{N-1} \alpha_i t^{(i)} \left(\sum_{j=0}^{N-1} \alpha_j t^{(j)} \mathbf{x}^{(j)} \right)^T \mathbf{x}^{(i)} \\ &- \sum_{i=0}^{N-1} \alpha_i t^{(i)} b + \sum_{i=0}^{N-1} \alpha_i \\ &= \frac{1}{2} \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} \alpha_i \alpha_j t^{(i)} t^{(j)} (\mathbf{x}^{(i)})^T \mathbf{x}^{(j)} - \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} \alpha_i \alpha_j t^{(i)} t^{(j)} (\mathbf{x}^{(i)})^T \mathbf{x}^{(j)} \\ &- b \underbrace{\sum_{i=0}^{N-1} \alpha_i t^{(i)}}_{=0 \text{ see } 11.45} \sum_{i=0}^{N-1} \alpha_i \\ &= \sum_{i=0}^{N-1} \alpha_i - \frac{1}{2} \sum_{i,j=0}^{N-1} \alpha_i \alpha_j t^{(i)} t^{(j)} (\mathbf{x}^{(i)})^T \mathbf{x}^{(j)} \end{aligned} \tag{11.46}$$

So the dual problems becomes:

$$\begin{aligned}\max_{\alpha_i} \theta_D(\alpha_i) &= \max_{\alpha_i} L(\mathbf{w}, b, \alpha_i) \\ &= \max_{\alpha_i} \sum_{i=0}^{N-1} \alpha_i - \frac{1}{2} \sum_{i,j=0}^{N-1} \alpha_i \alpha_j t^{(i)} t^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\ \text{s.t.} \alpha_i &\geq 0 \ \forall i = 0, \ldots, N-1 \\ \sum_{i=0}^{N-1} \alpha_i t^{(i)} &= 0\end{aligned} \tag{11.47}$$

Once the optimal α_i^* are found, we can compute the optimal $\mathbf{w}^*$ using 11.44:

$$\mathbf{w}^* = \sum_{i=0}^{N-1} \alpha_i^* t^{(i)} \mathbf{x}^{(i)} \tag{11.48}$$

while for the optimal b^* it is sufficient to note that for the samples on the margin it holds $\mathbf{w}^* \mathbf{x}^{(i)} + b^* = 1$, so if we consider the smallest of the samples belonging to class $+1$ and the largest of those in class -1 we have:

$$\begin{aligned}\min_{i:t^{(i)}=1} \mathbf{w}^* \mathbf{x}^{(i)} + b^* = 1 = \max_{i:t^{(i)}=-1} -\mathbf{w}^* \mathbf{x}^{(i)} - b^* \\ \implies b^* = -\frac{\max_{i:t^{(i)}=-1} \mathbf{w}^* \mathbf{x}^{(i)} + \min_{i:t^{(i)}=1} \mathbf{w}^* \mathbf{x}^{(i)}}{2}\end{aligned} \tag{11.49}$$

The class of a test sample $\mathbf{x}_{TEST}$ can then be estimated by calculating:

$$\begin{aligned}(\mathbf{w}^*)^T \mathbf{x}_{TEST} + b^* &= \left(\sum_{i=0}^{N-1} \alpha_i^* t^{(i)} \mathbf{x}^{(i)} \right)^T \mathbf{x}_{TEST} + b^* \\ &= \sum_{i=0}^{N-1} \alpha_i^* t^{(i)} \langle \mathbf{x}^{(i)}, \mathbf{x}_{TEST} \rangle + b^*\end{aligned} \tag{11.50}$$

From this expression we can note two things:

- the class estimate of the sample $\mathbf{x}_{TEST}$ depends on its scalar product with training samples;
- Since $\alpha_i^* \neq 0$ only for the support vectors (*i.e.,* the samples on the edge), the number of products to be computed is very small (only a few samples will be on the edge). For this reason, the dual formulation is very efficient in inference.

Both the primal and dual formulations of the max (or hard) margin classifier, are also known as hard-margin versions of the **Support Vector Machine (SVM)**.

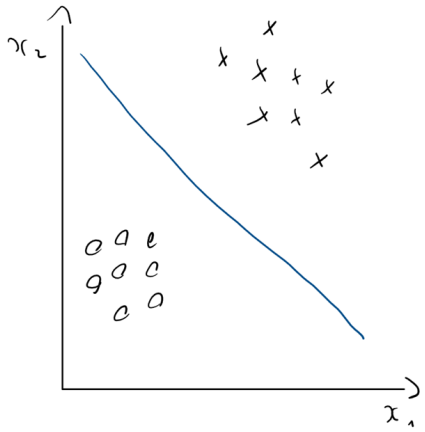


Figure 11.10: Separable case.

11.3 Kernels

In the previous section, we introduced the max margin classifier under the assumption of separable classes, *i.e.,* assuming that there exists a hyper-plane that perfectly separates the samples belonging to the two classes (see Figure 11.10). Can we continue to use the max margin classifier in the case of non-separable classes? (*e.g.,* see the plot on the left in Figure 11.11).

To do so, we can think of applying a function $\boldsymbol{\phi}(\mathbf{x})$ to the features to project them into a space where the samples are linearly separable (see Figure reffig:kernel2). We have already seen a similar strategy when we discussed Linear Regression: in that case, the **Basis function** were used to perform a **Feature mapping** in order to generate new features to capture relationships (even non-linear) between the original input attributes. In the case of the max-margin classifier, the principle is identical, with the added advantage of being able to use these transformations to map samples into a linearly separable space.

We have seen that, in its dual version, the SVM is characterized by scalar products between samples. Consequently, if we transform the features with a $\boldsymbol{\phi}(\cdot)$ mapping, it is sufficient to replace, in the formulations introduced in the previous sections the terms $\langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle$ with $\langle \boldsymbol{\phi}(\mathbf{x}^{(i)}), \boldsymbol{\phi}(\mathbf{x}^{(j)}) \rangle$.

In SVM terminology, given a function of **Feature mapping** $\boldsymbol{\phi}(\cdot)$, the scalar product between two transformed samples is called **Kernel**:

$$K(\mathbf{x}, \mathbf{y}) = \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{y}) \tag{11.51}$$

and the special case $K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$ occurs when $\boldsymbol{\phi}(\mathbf{x}) = \mathbf{x}$. Substituting the Kernel in

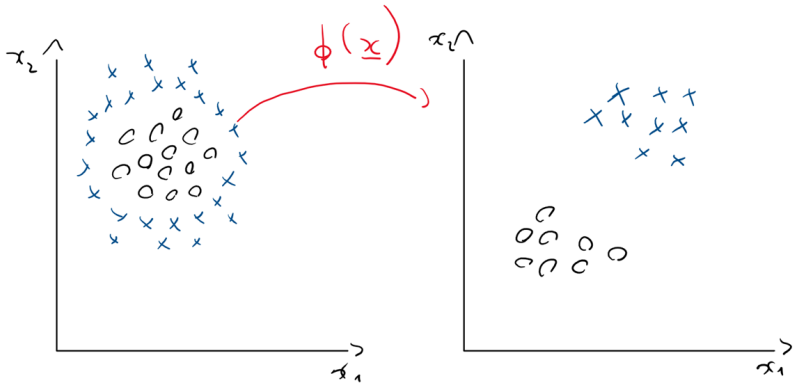


Figure 11.11: Non-separable case and feature transformation to obtaina separable scenario.

place of scalar products in the definition of the dual SVM problem we get:

$$\max_{\alpha_i} \sum_{i=0}^{N-1} \alpha_i - \frac{1}{2} \sum_{i,j=0}^{N-1} \alpha_i \alpha_j t^{(i)} t^{(j)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \tag{11.52}$$

$$\text{s.t.} \alpha_i \geq 0 \ \forall i = 0, \ldots, N-1$$

$$\sum_{i=0}^{N-1} \alpha_i t^{(i)} = 0$$

The Kernel thus offers a simple way to handle cases of non-separability and to transform input attributes by capturing even non-linear relationships between them.

To compute the Kernel function, in the first instance we can simply compute $\boldsymbol{\phi}(\mathbf{x})$ for two samples and take their inner product. However, it can be shown that it is often much more efficient to compute the Kernel function directly, instead of first computing the transformation $\boldsymbol{\phi}(\cdot)$ for the two samples separately. Consider, for example, the Kernel function:

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y})^2 \tag{11.53}$$

with $\mathbf{x}$ and $\mathbf{y} \in \mathbb{R}^d$. To see how this Kernel can be expressed as a scalar product of $\boldsymbol{\phi}(\mathbf{x})$ and $\boldsymbol{\phi}(\mathbf{y})$ we note that:

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y})^2 = \left(\sum_{i=0}^{D-1} x_i y_i \right) \left(\sum_{j=0}^{D-1} x_j y_j \right) = \left(\sum_{i=0}^{D-1} \sum_{j=0}^{D-1} x_i x_j y_j y_i \right)$$

$$= \sum_{i,j=0}^{D-1} (x_i x_j)(y_i y_j) = \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{y}) \tag{11.54}$$

where the feature mapping is given by (in the case $d = 3$):

$$\boldsymbol{\phi}(\mathbf{x}) = \begin{pmatrix} x_1x_1 \\ x_1x_2 \\ x_1x_3 \\ x_2x_1 \\ x_2x_2 \\ x_2x_3 \\ x_3x_1 \\ x_3x_2 \\ x_3x_3 \end{pmatrix} \tag{11.55}$$

We can then see that directly computing the Kernel function $K(\mathbf{x}, \mathbf{y})$ requires $O(d)$ operations, while computing first $\boldsymbol{\phi}(\mathbf{x})$ and then performing the scalar product of it requires $O(d^2)$. In general, a Kernel of this type is called **Polynomial Kernel**:

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y} + c)^p \tag{11.56}$$

This Kernel always corresponds to a mapping function in a feature space $\begin{pmatrix} d+p \\ p \end{pmatrix}$. However, computing the kernel function directly has a cost that depends linearly only on d, $O(d)$, though the feature space is $O(d^p)$.

Intuitively, it is possible to see the Kernel as a function that measures the similarity between two feature maps. In fact, if the scalar product of $\boldsymbol{\phi}(\mathbf{x})$ and $\boldsymbol{\phi}(\mathbf{y})$ is null, it means that the two feature maps are orthogonal and therefore very different from each other. On the other hand, when the value of Kernel is high it means that the two feature maps are very "close" or "similar" to each other.

It therefore makes sense to design Kernel functions (and therefore feature maps) that measure how similar two samples $\mathbf{x}$ and $\mathbf{y}$ are. In addition to the linear and polynomial Kernel, a popular Kernel that follows this principle is the **Gaussian Kernel** or **RBF (Radial Basis Function)**:

$$K(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{||\mathbf{x}-\mathbf{y}||^2}{2\sigma^2}\right) \tag{11.57}$$

where σ^2 is a hyper-parameter of the Kernel. It is possible to show that this Kernel corresponds to a feature map on an infinite dimensional space.

In general, we could then design any similarity or distance function between two samples that seems reasonable to us. To know whether this function is a valid Kernel, however, we must show that there exists some feature map $\boldsymbol{\phi}\cdot$ such that $K(\mathbf{x}, \mathbf{y}) = \boldsymbol{\phi}(\mathbf{x})^T\boldsymbol{\phi}(\mathbf{y})$.

To introduce the properties that tells us whether a function is a valid Kernel or

not, let's start by assuming that $K(\cdot,\cdot)$ is a valid kernel that corresponds to a certain feature map $\boldsymbol{\phi}(\cdot)$ and introducing the notion of **Kernel Matrix**. In particular, given a set of N samples $\{\mathbf{x}^{(0)},\dots,\mathbf{x}^{(N-1)}\}$, we can define the Kernel matrix $\mathbf{K}$ in which each element contains the kernel function evaluated with respect to samples i and j, *i.e.,* , $\mathbf{K}_{i,j} = K(\mathbf{x}^{(i)},\mathbf{x}^{(j)})$. If the function K is a valid Kernel, then the associated kernel matrix must have elements of the type $K(\mathbf{x}^{(i)},\mathbf{x}^{(j)}) = \boldsymbol{\phi}(\mathbf{x}^{(i)})^T\boldsymbol{\phi}(\mathbf{x}^{(j)}) = \boldsymbol{\phi}(\mathbf{x}^{(j)})^T\boldsymbol{\phi}(\mathbf{x}^{(i)}) = K(\mathbf{x}^{(j)},\mathbf{x}^{(i)})$. This means that a first condition for the kernel matrix to represent a valid kernel is that it needs to be **Symmetric**.

Furthermore, considering a generic vector $\mathbf{z}\in\mathbb{R}^d$ we can see that:

$$\begin{aligned}\mathbf{z}^T\mathbf{K}\mathbf{z} &= \sum_i\sum_j z_i K_{i,j} z_j \\ &= \sum_i\sum_j z_i \boldsymbol{\phi}(\mathbf{x}^{(i)})^T\boldsymbol{\phi}(\mathbf{x}^{(j)}) z_j \\ &= \sum_i\sum_j z_i \sum_k \phi_k(\mathbf{x}^{(i)})^T\phi_k(\mathbf{x}^{(j)}) z_j \\ &= \sum_k\sum_i\sum_j z_i \phi_k(\mathbf{x}^{(i)})^T\phi_k(\mathbf{x}^{(j)}) z_j \end{aligned} \tag{11.58}$$

If we expand the two summations in the last formula with respect to i and j in the simple case of two-dimensional vectors we have (for a generic index k of the outermost summation):

$$\begin{aligned}\sum_i^2\sum_j^2 z_i\phi_k(\mathbf{x}^{(i)})^T\phi_k(\mathbf{x}^{(j)})z_j &= z_1\phi_k(\mathbf{x}^{(1)})\phi_k(\mathbf{x}^{(1)})z_1 \\ &\quad + z_1\phi_k(\mathbf{x}^{(1)})\phi_k(\mathbf{x}^{(2)})z_2 + z_2\phi_k(\mathbf{x}^{(2)})\phi_k(\mathbf{x}^{(1)})z_1 \\ &\quad + z_2\phi_k(\mathbf{x}^{(2)})\phi_k(\mathbf{x}^{(2)})z_2 \\ &= z_1^2\phi_k^2(\mathbf{x}^{(1)}) + 2z_1z_2\phi_k(\mathbf{x}^{(1)})\phi_k(\mathbf{x}^{(2)}) + z_2^2\phi_k^2(\mathbf{x}^{(2)}) \\ &= (z_1\phi_k(\mathbf{x}^{(1)}) + z_2\phi_k(\mathbf{x}^{(2)}))^2 \end{aligned} \tag{11.59}$$

Extending to the d-dimensional case the 11.58 becomes:

$$\mathbf{z}^T\mathbf{K}\mathbf{z} = \sum_k\left(\sum_i z_i\phi_k(\mathbf{x}^{(i)})\right)^2 \geq 0 \tag{11.60}$$

The fact that $\mathbf{z}^T\mathbf{K}\mathbf{z}\geq 0$ implies that the matrix $\mathbf{K}$ is **positive semi-definite**, *i.e.,* $\mathbf{K}\succcurlyeq 0$.

Consequently, if a function K is a valid kernel, then the related kernel matrix $\mathbf{K}$ is symmeric and positive semi-definite. It turns out that these are also sufficient conditions, as the following theorem postulates:

Theorem 11.11. ***Mercer Theorem****: Given a function $K:\mathbb{R}^d\times\mathbb{R}^d\to\mathbb{R}$, for it to be*

*a **valid Kernel**, it is a necessary and sufficient condition that, for each set of points* $\{\mathbf{x}^{(0)}, \ldots, \mathbf{x}^{(N-1)}\}$ *with* $N < \infty$*, the related Kernel matrix* $\mathbf{K}$ *is symmetric and positive semi-definite.*

11.4 Support Vector Machine (SVM)

Kernels allow the feature space to be transformed to achieve separability. However, it is not necessarily always possible to find a transformation that guarantees perfect separability in the new space. In Figure 11.12, an example is shown where outliers are present. In this scenario, it is very unlikely to find a hyper-plane that perfectly separates the two classes. This case is different from that in Figure 11.11, where the classes were non-separable, but the pattern (or structure) emerged clearly from the data: therefore, the classes could be separated in the original feature space by using a non-linear separator (*e.g.,* , in the 2D case in the figure, a circle). To take advantage of the hard-margin formulation, in that case we exploited kernels (*e.g.,* the RBF kernel) to perform feature mapping and transform the features into a space where the samples are linearly separable. In Figure 11.12 instead the isolated points "mixed" in the classes do not follow a pattern, instead, they scatter in those areas of the space due to noise. It is therefore not reasonable (nor often possible) to find a kernel that handles these cases.

Moreover, even in the case of separability, outliers could significantly affect the hyper-plane. In Figure 11.13, for example, the red dot does not prevent us from finding a separating hyper-plane according to the max-margin formulation, but forces the latter to a non-optimal shape with respect to all other points. Intuitively, we would like this outlier to be ignored during parameter training. Specifically, we would like our classifier to guarantee the maximum margin between the samples of the two classes while trying to ignore data noise and the presence of outliers. In other words, we need a **Regularization** mechanism.

To do so, we modify the max-margin formulation as follows:

$$\min_{\hat{\gamma},\mathbf{w},b} \frac{1}{2}||\mathbf{w}||_2^2 + C\sum_{i=0}^{N-1}\xi_i \tag{11.61}$$

$$\text{subject to } t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1 - \xi_i \ , \ i = 0, \ldots, N-1 \tag{11.62}$$

$$\xi_i \geq 0 \ , \ i = 0, \ldots, N-1$$

This formulation is called **Soft Margin Classifier** and is the most popular and general formulation of the SVM. The variables ξ_i are called **Slack Variables** and their role is to allow the margin of each sample to be less than 1. If the slack variable is $0 \geq \xi_i \geq 1$, the margin of the sample will be between 0 and 1: in this case we are allowing the sample to be closer to the hyper-plane, without forcing the latter to move to ensure margin greater than or equal to 1 also for this sample. If instead the slack variable is greater

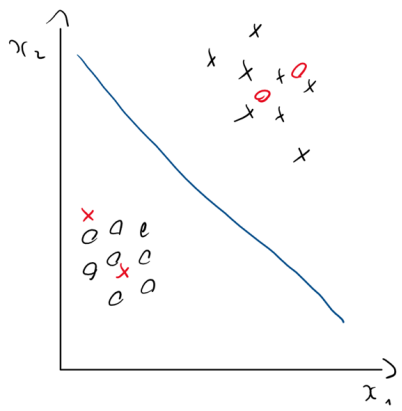


Figure 11.12: Non separable case that it is difficult to address by using a kernel.

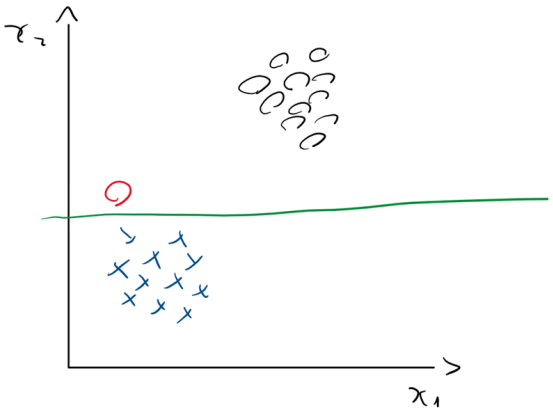


Figure 11.13: Separable case with outliers.

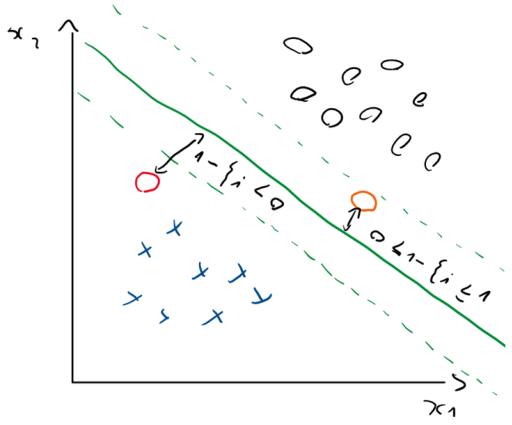


Figure 11.14: The role of slack variables.

than 1, the margin becomes less than zero: in this case we are allowing a sample to be "misclassified", *i.e.,* we allow the possibility that, as an instance, a sample of class 1 is in the region of the feature space related to class -1. In this way we are able to make the selection of optimal parameters less sensitive to the possible presence of noise and outliers (see Figure 11.14).

The term $C\sum_{i=0}^{N-1}\xi_i$ is needed to weigh the effect of slack variables and avoid too high values for them, which could reduce the margin even for those samples that are not outliers. The hyper-parameter C then controls the relative importance between the term $\frac{1}{2}||\mathbf{w}||_2^2$ and the slack variables:

- if $C \to \infty$, the slack variables assume a large weight in the optimization and thus, to reduce the value of the overall loss, we are forced to make $\xi \to 0$. On the other hand, the term $\frac{1}{2}|||\mathbf{w}||_2$ is free to take larger values, which corresponds to reducing the margin. In fact, we are returning to the hard-margin case, and the optimizer will do its best to ensure a hyper-plane that guarantees separability for all samples. Intuitively, we are not applying regularization, which increases the risk of modeling the noise: hence, we may run into **Overfitting** more easily.

- if $C \to 0$ the term $\frac{1}{2}|||\mathbf{w}||_2^2$ takes on a greater weight and the optimizer will try to increase the margin as much as possible by exploiting the slack variables. These are, indeed, more "free" to take on large values. In this case we are applying a strong regularization, and the risk is therefore to run into high **Bias** or **Underfitting**.

It is interesting to observe that, by writing $\xi_i \geq 1 - t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b)$ and remembering that $\xi \geq 0$, we can express the slack variables as:

$$\xi_i = \max\{0, 1 - t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b)\} \tag{11.63}$$

The problem 11.61 then becomes:

$$\min_{\hat{\gamma},\mathbf{w},b} \underbrace{\frac{1}{2}||\mathbf{w}||_2^2}_{\text{Regularization term}} + C \underbrace{\sum_{i=0}^{N-1} \max\{0, 1 - t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b)\}}_{\text{Loss term}} \tag{11.64}$$

The term $\sum_{i=0}^{N-1} \max\{0, 1 - t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b)\}$ takes the name **Hinge Loss** and in this form the parallelism with the loss function of *e.g.,* Ridge Regression is evident:

$$\min_{\mathbf{w}} \frac{1}{2} \sum_{i=0}^{N-1} (t^{(i)} - \hat{t^{(i)}})^2 + \lambda||\mathbf{w}||_2^2 \tag{11.65}$$

It can be observed that λ and C both play the role of hyper-parameters controlling regularization, and are conceptually related by an inverse proportionality relation $\lambda \propto \frac{1}{C}$.

Finally, as with the hard margin, it is possible to derive the dual form also for the soft margin case by exploiting the concepts we have seen in the previous sections.

Chapter 12

Neural Networks

In the previous chapters we have, for the most part, introduced supervised learning algorithms in which the hypothesis functions assumed a linear relationship with respect to the trainable parameters $\mathbf{w}$. This is true, for example, for linear regression, but also for logistic regression which, although it uses a nonlinear function (the sigmoid), it does so only to squash the output of the linear combination between 0 and 1. We have also seen how the use of **Basis Function** or **Kernel** allows us to model nonlinear relationships between input features. Instead, in this chapter we introduce **Neural Networks**, which allow us to learn models that are **non-linear** with respect to both the input and the parameters $\mathbf{w}$.

Neural networks draw their inspiration from the **Perceptron** algorithm. In graphical form, this algorithm can be represented as in Figure 12.1. Assuming $\mathbf{x} \in \mathbb{R}^D$, the perceptron hypothesis function can be viewed as connecting D nodes, each corresponding to one of the features, to a summation node with arcs weighted by w_j weights. The output of this node is then processed by a node implementing the decision function:

$$h(z) = \begin{cases} +1 \text{ if } z \geq 0 \\ -1 \text{ if } z < 0 \end{cases} \tag{12.1}$$

which outputs the estimated class, *i.e.,* +1 or -1. The decision function is a non-linear function, but as with logistic regression it is not used to model a non-linear dependence with respect to $\mathbf{w}$, but simply to make a decision for classification purposes on the output of the model, characterized by a linear combination of the inputs with respect to the parameters (as expressed by the summation node).

We have seen how the perceptron suffers from some limitations, including the fact that convergence is guaranteed only in the case of perfectly separable classes.

Neural networks significantly extend the structure shown in Figure 12.1 of the perceptron to provide models capable of representing even significantly nonlinear processes: for this reason, neural networks are also known as **Multi Layer Perceptron (MLP)**. As we will see, neural networks are also very flexible, as they are not constrained to solve

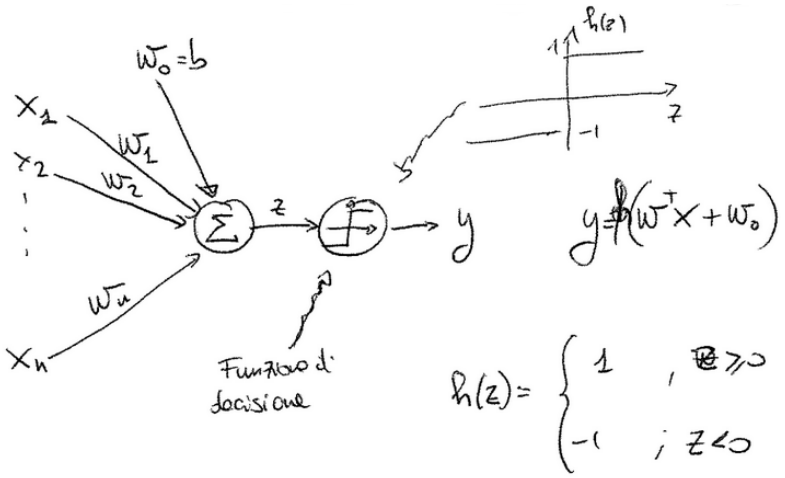


Figure 12.1: Graphical representation of the Perceptron algorithm.

regression or classification problems, but can handle a wide family of learning problems.

12.1 Loss Function and Stochastic Gradient Descent (SGD)

To begin the discussion, we can view neural networks (NNs) as a model capable of learning a generic and **nonlinear** hypothesis function $h_{\mathbf{w}}(\mathbf{x})$ with respect to the parameters $\mathbf{w}$. Differently from the other algorithms we saw, NNs are not constrained to particular loss functions, but are flexible with respect to custom choices specific to the task at hand (provided, as we shall see, that these loss functions are derivable). As an example, let's consider regression and suppose that for our problem it is reasonable to minimize the loss function of quadratic regression already seen in the case of linear regression:

$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=0}^{N-1}(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - t^{(i)})^2 \tag{12.2}$$

It is again important to state that this choice of loss is not the only one possible for neural networks, but we are free to change it depending on whether the task is regression or classification or on some custom choices. However, it is clearly reasonable in most regression problems to use the cost function already seen for linear regression also for neural networks. In fact, the latter guides the training to make the predictions as close as possible to the supervised values.

It is, on the other hand, important to point out that $h_{\mathbf{w}}(\mathbf{x}^{(i)})$ does not assume a linear relation with respect to the parameters, but is characterized by a series of non-linearities as we will see in the next sections.

Regardless of the particular form of $h_{\mathbf{w}}(\mathbf{x}^{(i)})$, parameter training in the case of neural networks uses iterative techniques as we have seen for other approaches. One of the most popular optimizers for NNs is **Gradient Descent** in its **Stochastic Gradient Descent**

(SGD) version due to its higher computational efficiency. In particular, instead of calculating the gradient at each iteration considering all the samples in the training set, SGD considers, at each iteration, a single sample randomly selected from those available. The pseudocode in this case is the following: The most popular version of SGD involves

Algorithm 7 SGD

Input: Learning rate α, Max iter n_{iter}
Init $\mathbf{w}$ randomly
  for all $i = 0, \dots, n_{iter} - 1$ **do**
    Sample $\{\mathbf{x}^{(i)}, t^{(i)}\}$ randomly from $\mathcal{D}_{training}$
    Update $\mathbf{w}$ with $\mathbf{w} = \mathbf{w} - \alpha \nabla_{\mathbf{w}} \frac{1}{2}(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - t^{(i)})^2$
  end for

updating parameters using not a single sample but a randomly selected **Batch** of B samples: It is possible to show that the SGD version has convergence properties similar

Algorithm 8 SGD

Input: Learning rate α, Max iter n_{iter}
Init $\mathbf{w}$ randomly
  for all $i = 0, \dots, n_{iter} - 1$ **do**
    Sample B samples $\{\mathbf{x}^{(j)}, t^{(j)}\}$ randomly from $\mathcal{D}_{training}$ (without replacement)
    Update $\mathbf{w}$ with $\mathbf{w} = \mathbf{w} - \frac{\alpha}{B} \sum_{j=0}^{B-1} \nabla_{\mathbf{w}} \frac{1}{2}(h_{\mathbf{w}}(\mathbf{x}^{(j)}) - t^{(j)})^2$
  end for

to those of GD and is much more computationally efficient.

12.2 Multi layer Neural Networks

At a high level, the NNs learn hypothesis functions consisting of sequences of non-linear combinations of the input features. To understand their structure, we begin with a simple example in which we consider a single feature. We consider the problem of house price prediction using house size as input.

Using linear regression, the solution obtained does not exclude the possibility of having prices less than zero, which is clearly not possible. We fix this by including a simple nonlinear function that sets the output to 0 when the combination of parameter and input feature is less than zero. Our simple hypothesis function then becomes:

$$h_w(x) = \max\{wx + b, 0\} \tag{12.3}$$

In the context of neural networks, the function $\max\{0, x\}$ is known as **Rectified Linear Unit (ReLU** (see Figure 12.2). In general, such nonlinear functions that map values from $\mathbb{R}$ to $\mathbb{R}$ and are known as **Activation Functions**. The block that combines the input feature with the w parameters and applies the activation function is the **Neuron**

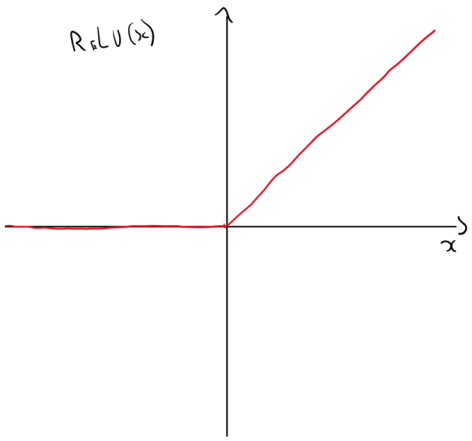


Figure 12.2: ReLU function.



Figure 12.3: Single neuron NN.

of the neural network. In this simple example, we thus have a single-neuron NN (see Figure 12.3.

We can at this point extend the previous network by combining multiple features and passing the output of multiple neruons to a subsequent layer of neurons that combines them. Suppose that among the input attributes we have, in addition to the "size" of the house, the "number of bedrooms", the "zip code", and the "presence of transportation services". It is reasonable to think of constructing an intermediate feature "Family Size" from the "number of bedrooms" and the "size" of the house. Similarly, we can think of having an intermediate feature "Presence of Pedestrian Streets" which depends on the "zip code", and another named "Quality of Neighborhood Services" which depends on the "zip code" and the "presence of transportation services". These three intermediate features can then be further combined at a later level to estimate the price of the house (see Figure 12.4). Referring to the figure, we have 4 **Input Neurons** (which are

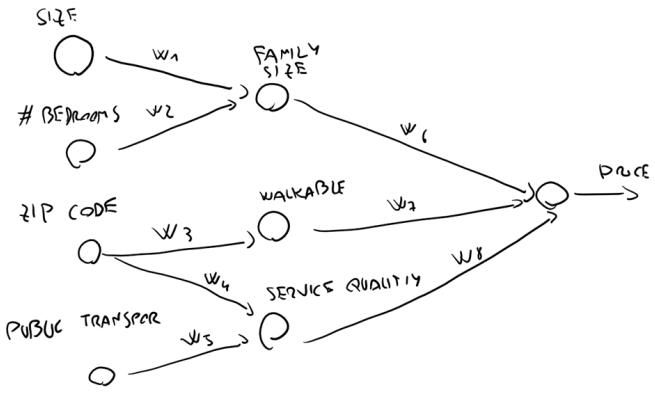


Figure 12.4: A more complex NN example.

not counted in the neural networks because they are not associated with parameters), 3 **Hidden Neurons** related to the intermediate features "Family size", "Presence of pedestrian streets" and "Quality of Neighborhood Services" and 1 **Output Neuron** that provides the estimate of the price of the house. In formulas, we can write the relationships linking the various hidden neurons as follows:

$$\begin{aligned} a_1 &= \text{ReLU}(w_1 x_1 + w_2 x_2 + b_1) \\ a_2 &= \text{ReLU}(w_3 x_3 + b_2) \\ a_3 &= \text{ReLU}(w_4 x_3 + w_5 x_4 + b_3) \end{aligned} \tag{12.4}$$

while for the output neuron we have:

$$o = h_{\mathbf{w}}(\mathbf{x}) = w_6 a_1 + w_7 a_2 + w_8 a_3 + b_o \tag{12.5}$$

where a_1 corresponds to the intermediate feature neuron "Family Size", a_2 to "Presence of Pedestrian Streets" and a_3 to "Quality of Neighborhood Services" and b_1, b_2, b_3 and b_0 are the bias parameters associated with each neuron.

In general, by **Input Neurons** we refer to the nodes that represent the input features. The **Hidden Neurons**, on the other hand, are the intermediate layer neurons characterized by nonlinear activation functions. Each intermediate layer of these neurons is named **Hidden Layer** and a neural network can consist of an arbitrary number of layers. All neurons in the hidden layers always have nonlinear activation functions.

The final layer, called the **Output Layer**, contains one or more **Output Neurons**, each of which provides an estimate of the associated output variable. The activation function of the output neurons is not necessarily nonlinear and depends on the task under consideration. For example, if the output variable is not necessarily a positive number, it is reasonable that the output neuron does not use a ReLU type function,

but simply performs a linear combination of the neurons of the previous hidden layer (which, instead, necessarily apply non-linear activation functions).

12.2.1 Two Layer MLP

In the previous example we chose which input features were appropriate to combine together to obtain new features with a certain meaning. However, having this kind of knowledge a priori is not always possible and it is certainly complex to insert it in the structure of our network when the number of attributes is very large. What is instead reasonable to do is to let the network to discover such combinations. In particular, we "connect" each input node to each neuron of the first hidden layer and then each neuron of the hidden layer to all neurons of the next layer (hidden or output). Since the weight of a connection is related to the value of its parameter w_j, in this way we leave to the training phase of the neural network the task of determining the importance of certain connections and the way to combine the various features. For this reason, the layers of neural networks are also called **Fully Connected Layers**: indeed, each of their neurons is connected to all those of the next layer.

So if we consider the simple two-layer network in Figure 12.5 (one hidden and one output, since by convention the input layer is not considered), the output of the first neuron a_1 can be calculated as follows:

$$a_1 = \text{ReLU}((w_1^{[1]})_1 x_1 + (w_1^{[1]})_2 x_2 + (w_1^{[1]})_3 x_3 + b_1) = \text{ReLU}((\mathbf{w}_1^{[1]})^T \mathbf{x} + b_1) \tag{12.6}$$

where the convention $(w_i^{[l]})_j$ denotes the parameter related to the edge starting from node (neuron) j of the previous layer (the input in our example) and arriving at neuron i of layer l. In vector form, $\mathbf{w}_i^{[l]}$ denotes the vector of parameters that combine all nodes of the previous layer to compute the activation of neuron i of layer l. In our example, $\mathbf{w}_i^{[1]} \in \mathbb{R}^3$ since we have 3 input features. Extending to all neurons in the hidden layer we have:

$$\begin{aligned} a_1 &= \text{ReLU}((\mathbf{w}_1^{[1]})^T \mathbf{x} + b_1) \\ a_2 &= \text{ReLU}((\mathbf{w}_2^{[1]})^T \mathbf{x} + b_2) \\ a_3 &= \text{ReLU}((\mathbf{w}_3^{[1]})^T \mathbf{x} + b_3) \\ a_4 &= \text{ReLU}((\mathbf{w}_4^{[1]})^T \mathbf{x} + b_4) \end{aligned} \tag{12.7}$$

While for the output neuron;

$$o = (\mathbf{w}_o^{[2]})^T \mathbf{a} + b_o \tag{12.8}$$

where $\mathbf{a}$ denotes the vector of hidden layer activations and b_o the bias parameter.

More generally, a neural network with two layers, *i.e.,* a hidden layer with m neurons

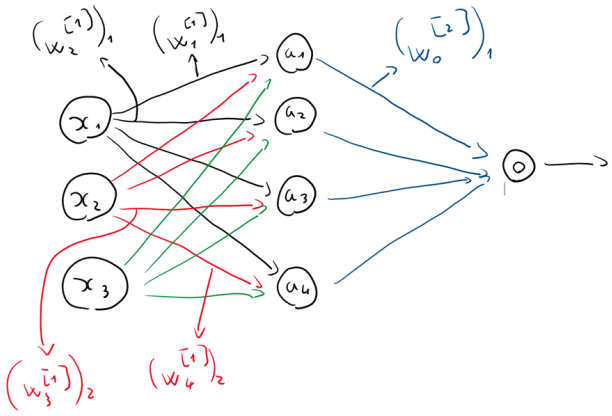


Figure 12.5: Simple two-layer NN.

and an output one with one neuron, and an input $\mathbf{x} \in \mathbb{R}^D$ is defined as:

$$z_j = (\mathbf{w}_j^{[1]})^T \mathbf{x} + b_j^{[1]} \ \forall j = 1, \ldots, m \tag{12.9}$$

where $\mathbf{w}_j^{[1]} \in \mathbb{R}^D$ and $b_j^{[1]} \in \mathbb{R}$. The activation of neuron j is calculated by applying the activation function to the linear combination of features z_j:

$$\begin{aligned} a_j &= \text{ReLU}(z_j) \\ \mathbf{a} &= [a_1, \ldots, a_m]^T \in \mathbb{R}^m \end{aligned} \tag{12.10}$$

so the final output (and thus the overall hypothesis function) is obtained as follows:

$$h_{\mathbf{w}}(\mathbf{x}) = (\mathbf{w}^{[2]})^T \mathbf{a} + b^{[2]} \tag{12.11}$$

where $\mathbf{w}^{[2]} \in \mathbb{R}^m$ e $b^{[2]} \in \mathbb{R}$.

Vectorization of the Two layer MLP

It is convenient to express the previous relations in vector form. In particular, we group all the parameter vectors used to calculate the activations of the m neurons of the hidden layer into a single matrix $\mathbf{W}^{[1]} \in \mathbb{R}^{m \times D}$:

$$\mathbf{W}^{[1]} = \begin{bmatrix} (\mathbf{w}_1^{[1]})^T \\ (\mathbf{w}_2^{[1]})^T \\ \vdots \\ (\mathbf{w}_m^{[1]})^T \end{bmatrix} \in \mathbb{R}^{m \times D} \tag{12.12}$$

hence, we can now write $\mathbf{z} = [z_1, \ldots, z_m]^T \in \mathbb{R}^m$ as:

$$\underbrace{\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{bmatrix}}_{\mathbf{z} \in \mathbb{R}^{m \times 1}} = \underbrace{\begin{bmatrix} (\mathbf{w}_1^{[1]})^T \\ (\mathbf{w}_2^{[1]})^T \\ \vdots \\ (\mathbf{w}_m^{[1]})^T \end{bmatrix}}_{\mathbf{W}^{[1]} \in \mathbb{R}^{m \times D}} \underbrace{\begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_D - 1 \end{bmatrix}}_{\mathbf{z} \in \mathbb{R}^{D \times 1}} + \underbrace{\begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ \vdots \\ b_m^{1]} \end{bmatrix}}_{\mathbf{b}^{[1]} \in \mathbb{R}^{m \times 1}} \tag{12.13}$$

o, equivalently:

$$\mathbf{z} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} \tag{12.14}$$

To compute the activations by applying the ReLU it is sufficient to perform an element-wise operation on the vector $\mathbf{z}$:

$$\mathbf{a} = \text{ReLU}(\mathbf{a}) \tag{12.15}$$

At this point, by defining for the output layer the matrix $\mathbf{W}^{[2]} = [(\mathbf{w}^{[2]})^T] \in \mathbb{R}^{1 \times m}$, we can compactly express the equations for the two-layer network as follows:

$$\mathbf{a} = \text{ReLU}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) \tag{12.16}$$

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{W}^{[2]}\mathbf{a} + b^{[2]} \tag{12.17}$$

12.3 Multi Layer Perceptron

Taking advantage of vector notation, it is possible to extend the formulation for the neural network with two layers to the case with an arbitrary number of layers. In particular, if we denote by $\mathbf{W}^{[1]}, \mathbf{W}^{[2]}, \ldots, \mathbf{W}^{[r]}$ the matrices with the parameters relating to the various layers and by $\mathbf{b}^{[1]}\mathbf{b}^{[2]}, \ldots, b^{[r]}$ the bias vectors, we can write the multi-layer nerual network as follows:

$$\begin{aligned} \mathbf{a}^{[1]} &= \text{ReLU}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) \\ \mathbf{a}^{[2]} &= \text{ReLU}(\mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}) \\ \mathbf{a}^{[3]} &= \text{ReLU}(\mathbf{W}^{[3]}\mathbf{a}^{[2]} + \mathbf{b}^{[3]}) \\ &\ldots \\ \mathbf{a}^{[r-1]} &= \text{ReLU}(\mathbf{W}^{[r-1]}\mathbf{a}^{[r-2]} + \mathbf{b}^{[r-1]}) \\ h_{\mathbf{w}}(\mathbf{x}) &= \mathbf{W}^{[r]}\mathbf{a}^{[r-1]} + b^{[r]} \end{aligned} \tag{12.18}$$

(N.B. the last bias is a scalar since the output layer has only one neuron).

Each of the matrices has a dimension $\mathbf{W}^{[k]} \in \mathbb{R}^{m_k \times m_{k-1}}$ where m_k is the number of neurons in the hidden layer k and m_{k-1} is the number of neurons in the previous layer.

For the bias vectors, we have $b^{[k]} \in \mathbb{R}^{m_k}$. The total number of neurons in the network is $m_1 + m_2 + \cdots + m_r$ and the total number of parameters, considering an input vector of D size is $(D+1)m_1 + (m_1+1)m_2 + \cdots + (m_{r-1}+1)m_r$ (the +1 denote the biases).

In addition to the ReLU there are other possible activation functions, including the **Sigmoid**:

$$\sigma(z) = \frac{1}{1+e^{-z}} \tag{12.19}$$

and the **Tanh**:

$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \tag{12.20}$$

In hidden layers we cannot use linear activations. This is because, assuming we use an activation $\sigma(z) = z$, in the case of the two-layer NN, for example, we would have:

$$\begin{aligned} h_{\mathbf{w}}(\mathbf{x}) &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} \\ &= \mathbf{W}^{[2]}\sigma(\mathbf{z}^{[1]}) \\ &= \mathbf{W}^{[2]}\mathbf{z}^{[1]} \\ &= \mathbf{W}^{[2]}\mathbf{W}^{[1]}\mathbf{x} = \tilde{\mathbf{W}}\mathbf{x} \end{aligned} \tag{12.21}$$

that is, it is possible to collapse the two layers into one by combining the two matrices. In other words, applying a linear function to another linear function results in an additional linear function with respect to the input features and, as a result, we lose the representative caability of NNs to model nonlinear processes.

We saw how neural networks are able to combine features through non-linear operations to compute feature maps useful for the classification or regression task. This type of operation was also performed by basis functions and kernels. What is the difference between the two cases?
In the case of kernels and basis functions, the nonlinear transformation functions are **hand engineered**, *i.e.,* we manually designed those functions and, therefore, we say we perform **Feature Engineering**. In neural networks, on the other hand, it is the network itself that learns the most appropriate feature map with respect to the optimization problem under consideration and is therefore able to achieve greater efficiency. Besides, the possibility to increase the number of neurons and of hidden layers, allows to learn feature maps of arbitrary complexity. The neural network not only learns the classifier or the regressor to estimate the output variable, but also the optimal feature maps for the task.
In this sense, it is possible to demonstrate the following fundamental theorem for neural networks

Theorem 12.1. ***Universal Approximation Theorem:*** *A neural network with a single hidden layer and an adequate number of neurons can approximate any continuous function of arbitrary complexity.*

12.4 Backpropagation

In the previous section we saw how to compute the hypothesis function for neural networks. The operation that involves the calculation of the hypothesis function starting from an input sample $\mathbf{x}$ is known as **Forward operation**: this is because the input is forwarded through the various layers.

At this point we are able to calculate the loss function for a certain sample. For example, in the case of regression:

$$L(\mathbf{w}) = \frac{1}{2}(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - t^{(i)})^2 \tag{12.22}$$

We have also seen that to train the network and find the optimal parameters, we can exploit SGD and, in particular, use the procedure described in the Algorithm 8. To do so, we are missing one last ingredient: computing the gradient $\nabla_{\mathbf{w}} L(\mathbf{w})$.

For this purpose, neural networks exploit an extremely efficient procedure known as **Backpropagation** or **Auto-differentiation**. The efficiency of backpropagation is guaranteed by the following theorem:

Theorem 12.2. *Suppose a differentiable network, or circuit, of size N computes a real-valued function. Then, the gradient of this function can be computed in $O(N)$, with a circuit of operations of size $O(N)$.*

A differentiable circuit is defined as a composition of differentiable arithmetic operations, such as sums and subtractions, and differentiable functions (such as ReLU, exp, log, ...). The operations involved during the MLP forward compute a real valued function (the hypothesis function) and are characterized by a set of differentiable operations (sums and molitplications) and differentiable functions (ReLU). Consequently, the previous theorem holds and assures us that the gradient computation has a similar order of cost as the forward operation requires.

Let us now see how we can compute the gradient by exploiting backpropagation. To do so, we first review the concept of **Chain Rule** for derivatives.

12.4.1 Chain Rule

Suppose we have a function L that depends on a set of variables $w_0, w_1, \ldots, w_{D-1}$ via a set of functions $g_1, \ldots, g_k$ of $\mathbf{w}$:

$$g_i = g_i(w_0, \ldots, w_{D-1}) \ , \forall i = 1, \ldots, k \tag{12.23}$$

$$L = L(g_1(\mathbf{w}), g_2(\mathbf{w}), \ldots, g_k(\mathbf{w})) \tag{12.24}$$

Then, the gradient of L with respect to $\mathbf{w}$ can be computed by the following relation (**Chain Rule**):

$$\frac{\partial L}{\partial w_j} = \sum_{i=1}^{k} \frac{\partial L}{\partial g_i} \frac{\partial g_i}{\partial w_j} \tag{12.25}$$

Let's take an example. Consider the function:

$$f(x, y) = \frac{x^2 + y^2}{x^2 + (x + y)^2} \tag{12.26}$$

and compute the derivative with respect to x using the chain rule. We start by expressing f as consisting of two simpler functions:

$$f(x, y) = g_1(x, y) g_2(x, y) \tag{12.27}$$

$$\text{where } g_1(x, y) = x^2 + y^2$$

$$\text{and } g_2(x, y) = \frac{1}{x^2 + (x + y)^2} \tag{12.28}$$

By applying the chain rule:

$$\begin{aligned}\frac{\partial f(x,y)}{\partial x} &= \sum_{i=1}^{2} \frac{\partial f(x,y)}{\partial g_i(x,y)} \frac{\partial g_i(x,y)}{\partial x} \\ &= \frac{\partial f(x,y)}{\partial g_1(x,y)} \frac{\partial g_1(x,y)}{\partial x} + \frac{\partial f(x,y)}{\partial g_2(x,y)} \frac{\partial g_2(x,y)}{\partial x} \\ &= g_2(x,y)2x + g_1(x,y) \frac{\partial g_2(x,y)}{\partial x} \end{aligned} \tag{12.29}$$

we continue by expressing $g_2(x,y)$ as the result of the composition of two functions, *i.e.,* $\frac{1}{\cdot}$ and $g_3(x,y)$: $g_2(x,y) = \frac{1}{g_3(x,y)}$.

$$\begin{aligned}\frac{\partial f(x,y)}{\partial x} &= g_2(x,y)2x + g_1(x,y) \left[\frac{\partial g_2(x,y)}{\partial g_3(x,y)} \frac{\partial g_3(x,y)}{\partial x} \right] \\ &= g_2(x,y)2x + g_1(x,y) \left[-\frac{1}{g_3(x,y)^2} \frac{\partial g_3(x,y)}{\partial x} \right] \end{aligned} \tag{12.30}$$

We can now also express $g_3(x,y)$ as consisting of two functions: $g_3(x,y) = g_4(x,y) + g_5(x,y)$, where $g_4(x,y) = x^2$ and $g_5(x,y) = (x+y)^2$. Hence, $\frac{\partial g_3(x,y)}{\partial x}$ becomes:

$$\begin{aligned}\frac{\partial g_3(x,y)}{\partial x} &= \frac{\partial g_3(x,y)}{\partial g_4(x,y)} \frac{\partial g_4(x,y)}{\partial x} + \frac{\partial g_3(x,y)}{\partial g5_(x,y)} \frac{\partial g_5(x,y)}{\partial x} \\ &= 2x + 2(x+y). \end{aligned} \tag{12.31}$$

By substituting the latter relation into the 12.30 we have:

$$\begin{aligned}\frac{\partial f(x,y)}{\partial x} &= g_2(x,y)2x + g_1(x,y) \left[-\frac{1}{g_3(x,y)^2} (2x + 2(x+y)) \right] \\ &= \frac{1}{x^2 + (x+y)^2} 2x + (x^2 + y^2) \left[-\frac{1}{x^2 + (x+y)^2} (2x + 2(x+y)) \right] \end{aligned} \tag{12.32}$$

12.4.2 Backpropagation of a NN with one neuron

In the following, for simplicity, we denote by $\hat{t}^{(i)} = h_{\mathbf{w}}(\mathbf{x}^{(i)})$ the network forward result (which computes the hypothesis function) and consider the loss for a single sample (corresponding to an iteration of SGD with single-sample update) described by a feature vector $\mathbf{x} \in \mathbb{R}^D$:

$$L(\mathbf{w}) = \frac{1}{2}(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - t^{(i)})^2 = \frac{1}{2}(\hat{t}^{(i)} - t^{(i)})^2 \tag{12.33}$$

Let's start by derving the backpropagation step in the simple case of a single-neuron NN. The forward phase, in this configuration, is described by the following expressions:

$$\begin{aligned} z &= (\mathbf{w}_1^{[1]})^T \mathbf{x} + b^{[1]} \\ \hat{t} &= \text{ReLU}(z) \\ L(\mathbf{w}_1^{[1]}) &= \frac{1}{2}(\hat{t} - t)^2 \end{aligned} \tag{12.34}$$

To update the parameters with SGD we have to calculate the gradient of the loss with respect to the trainable parameters $\mathbf{w}_1^{[1]} = [(w_1^{[1]})_0, (w_1^{[1]})_1, \ldots, (w_1^{[1]})_{D-1}]$, *i.e.*, $\nabla_{\mathbf{w}_1^{[1]}} L(\mathbf{w}_1^{[1]})$. To do so, we consider the gradient with respect to the generic $(w_1^{[1]})_i$ and use the chain rule:

$$\frac{\partial L}{\partial (w_1^{[1]})_i} = \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial (w_1^{[1]})_i} \tag{12.35}$$

and by noticing that $\hat{t}$ also depends on $(w_1^{[1]})_i$ through $\mathbf{z}$ we get:

$$\frac{\partial L}{\partial (w_1^{[1]})_i} = \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial z} \frac{\partial z}{\partial (w_1^{[1]})_i} = (\hat{t} - t) \cdot \underbrace{1\{z \geq 0\}}_{\text{Derivative of ReLU}} \cdot x_i \tag{12.36}$$

This holds for the single parameter $(w_1^{[1]})_i$, while for the entire vector of parameters, we can easily extend the previous relation into the vector case:

$$\nabla_{\mathbf{w}_1^{[1]}} L(\mathbf{w}_1^{[1]}) = (\hat{t} - t) \cdot 1\{z \geq 0\} \cdot \mathbf{x} \tag{12.37}$$

and similarly we can compute the gradient with respect to $b^{[1]}$:

$$\frac{\partial L}{\partial b^{[1]}} = \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial z} \frac{\partial z}{\partial b^{[1]}} = (\hat{t} - t) \cdot 1\{z \geq 0\} \cdot 1 \tag{12.38}$$

12.4.3 Backpropagation of a NN with two layers

We now proceed to compute the gradient in the case of a two-layer neural network. We start again by performing the forward step and computing the loss with respect to the

single sample $(\mathbf{x}, t)$ (we consider the non-vector form for now):

$$\begin{aligned} z_j &= (\mathbf{w}_j^{[1]})^T \mathbf{x} + b_j^{[1]} \ \forall j = 1, \dots, m \\ a_j &= \text{ReLU}(z_j) \\ \mathbf{a} &= [a_1, \dots, a_m]^T \in \mathbb{R}^m \\ \hat{t} &= (\mathbf{w}^{[2]})^T \mathbf{a} + b^{[2]} \\ L(\mathbf{w}) &= \frac{1}{2}(\hat{t} - t)^2 \end{aligned} \tag{12.39}$$

where $\mathbf{w}$ indicates all the parameters of the network, including those of the hidden layer and of the output layer.

By means of the chain rule, we now compute the partial derivative of the loss with respect to a parameter of the output layer $(\mathbf{w}^{[2]})_i$:

$$\frac{\partial L}{\partial (w^{[2]})_i} = \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial (w^{[2]})_i} = (\hat{t} - t) a_i \tag{12.40}$$

where a_i is the activation of the i-th neuron of the hidden layer calculated during the forward step.

For the parameters of the hidden layer, the calculation is slightly more complex, since the gradient propagates through several functions before arriving at $(w_j^{[1]})_i$. By again applying the chain rule until we arrive at the function that depends directly on the parameter we have:

$$\begin{aligned} \frac{\partial L}{\partial (w_j^{[1]})_i} &= \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_j} \frac{\partial a_j}{\partial z_j} \frac{\partial z_j}{\partial (w_j^{[1]})_i} \\ &= (\hat{t} - t)(w^{[2]})_j \cdot 1\{z_j \geq 0\} x_i \end{aligned} \tag{12.41}$$

The efficiency of backpropagation emerges from the presence of several terms in the chain rule that are shared among the computations for the various parameters. In particular, $(\hat{t} - t)$, $(w^{[2]})_j$ and $1\{z_j \geq 0\}$ are shared between different gradient computations. The back-propagation for the two-layer NN can then be rearranged more efficiently as shown in the 9 Algorithm:

Vectorized Backpropagation for the Two-layer NN

The backpropagation can be written in a more compact (and thus more computationally efficient) vectorized form. Before we begin, we resume the forward step in vector form

Algorithm 9 Backpropagation for two-layer NN

1: Perform the forward step and compute $z_1, \ldots, z_m$, $a_1, \ldots, a_m$ and $\hat{t}$
2: Compute $\delta_o = \frac{\partial L}{\partial \hat{t}} = (\hat{t} - t)$
3: Compute $\delta_j = \frac{\partial L}{\partial z_j}$, $\forall j = 1, \ldots, m$ as:
4: $\quad \delta_j = \frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial a_j} \frac{\partial a_j}{\partial z_j} = (\hat{t} - t)(w^{[2]})_j \cdot 1\{z_j \geq 0\}$
5: Compute $\frac{\partial L}{\partial (w_j^{[1]})_i}$, $\frac{\partial L}{\partial b_j^{[1]}}$, $\frac{\partial L}{\partial (w^{[2]})_i}$ and $\frac{\partial L}{\partial b^{[2]}}$ by:
6: $\quad \frac{\partial L}{\partial (w_j^{[1]})_i} = \frac{\partial L}{\partial z_j} \frac{\partial z_j}{\partial (w_j^{[1]})_i} = \delta_j x_i$
7: $\quad \frac{\partial L}{\partial b_j^{[1]}} = \frac{\partial L}{\partial z_j} \frac{\partial z_j}{\partial b_j^{[1]}} = \delta_j$
8: $\quad \frac{\partial L}{\partial (w^{[2]})_j} = \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial (w^{[2]})_j} = \delta_o a_j$
9: $\quad \frac{\partial L}{\partial b^{[2]}} = \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial b^{[2]}} = \delta_o$

for the two-layer NN:

$$\mathbf{a}^{[1]} = \text{ReLU}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) \tag{12.42}$$

$$\hat{t} = h_{\mathbf{w}}(\mathbf{x}) = \mathbf{W}^{[2]}\mathbf{a}^{[1]} + b^{[2]} \tag{12.43}$$

$$L = \frac{1}{2}(\hat{t} - t) \tag{12.44}$$

To compute the gradient we first note that the derivative of a scalar L with respect to a matrix $\mathbf{W} \in \mathbb{R}^{m \times D}$ gives, as a result, a matrix of size $m \times D$ in which each element is the derivative of L with respect to the element i, j of the matrix, *i.e.*, $\frac{\partial L}{\partial \mathbf{W}_{i,j}}$.

In vector form, the parameters of the output layer are described by the matrix $\mathbf{W}^{[2]} \in \mathbb{R}^{1 \times m}$ (which contains only one row). The derivative of the loss with respect to this matrix is then:

$$\frac{\partial L}{\partial \mathbf{W}^{[2]}} = \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial \mathbf{W}^{[2]}} \tag{12.45}$$

By indicating with $\delta^{[2]} = \frac{\partial L}{\partial \hat{t}} = (\hat{t} - t)$ and recalling that $\frac{\partial \hat{t}}{\partial \mathbf{W}^{[2]}} = \frac{\partial (\mathbf{W}^{[2]}\mathbf{a}^{[1]} + b^{[2]})}{\partial \mathbf{W}^{[2]}} = (\mathbf{a}^{[1]})^T$ we get:

$$\frac{\partial L}{\partial \mathbf{W}^{[2]}} = \delta^{[2]}\mathbf{a}^T \tag{12.46}$$

For the derivative with respect to $\mathbf{W}^{[1]}$ we instead have:

$$\frac{\partial L}{\partial \mathbf{W}^{[1]}} = \frac{\partial L}{\partial \mathbf{z}} \frac{\partial \mathbf{z}^{[1]}}{\partial \mathbf{W}^{[1]}} \tag{12.47}$$

where:

$$\frac{\partial L}{\partial \mathbf{z}^{[1]}} = \frac{\partial L}{\partial \hat{t}} \frac{\partial \hat{t}}{\partial \mathbf{a}^{[1]}} \frac{\partial \mathbf{a}^{[1]}}{\partial \mathbf{z}^{[1]}} \tag{12.48}$$

with:

$$\begin{aligned}\frac{\partial L}{\partial \hat{t}} &= \delta^{[2]} \\ \frac{\partial \hat{t}}{\partial \mathbf{a}^{[1]}} &= (\mathbf{W}^{[2]})^T \\ \frac{\partial \mathbf{a}^{[1]}}{\partial \mathbf{z}^{[1]}} &= \text{ReLU'}(\mathbf{z}^{[1]}) = 1\{\mathbf{z}^{[1]} \geq 0\}\end{aligned} \tag{12.49}$$

$\text{ReLU'}(\mathbf{z}^{[1]})$ refers to the element-wise derivative of ReLU. Thus, we obtain:

$$\frac{\partial L}{\partial \mathbf{z}^{[1]}} = \underbrace{\delta^{[2]}}_{\in \mathbb{R}} \underbrace{(\mathbf{W}^{[2]})^T}_{\in \mathbb{R}^{m \times 1}} \odot \underbrace{1\{\mathbf{z}^{[1]} \geq 0\}}_{\in \mathbb{R}^{m \times 1}} = \underbrace{\boldsymbol{\delta}^{[1]}}_{\in \mathbb{R}^{m \times 1}} \tag{12.50}$$

where the symbol $\odot$ denotes the element-wise product. At this point, we can easily finish calculating the derivative with respect to $\mathbf{W}^{[1]}$:

$$\underbrace{\frac{\partial L}{\partial \mathbf{W}^{[1]}}}_{\in \mathbb{R}^{m \times D}} = \frac{\partial L}{\partial \mathbf{z}^{[1]}} \frac{\partial \mathbf{z}^{[1]}}{\partial \mathbf{W}^{[1]}} = \underbrace{\boldsymbol{\delta}^{[1]}}_{\in \mathbb{R}^{m \times 1}} \underbrace{\mathbf{x}^T}_{\in \mathbb{R}^{1 \times D}} \tag{12.51}$$

and, as we expected, the dimension of the matrix containing the derivatives is indeed $m \times D$. All these steps are summarized in the Algorithm 10

Algorithm 10 Vectorized backpropagation for two-layer NN

1: Perform the forward step and compute $\mathbf{z}^{[1]} \in \mathbb{R}^m$, $\mathbf{a}^{[1]} \in \mathbb{R}^m$ and $\hat{t}$
2: Compute $\delta^{[2]} = \frac{\partial L}{\partial \hat{t}} = (\hat{t} - t)$
3: Compute $\boldsymbol{\delta}^{[1]} = \frac{\partial L}{\partial \mathbf{z}}$, $\forall j = 1, \ldots, m$ as:
4: $\quad \boldsymbol{\delta}^{[1]} = \delta^{[2]}(\mathbf{W}^{[2]})^T \odot 1\{\mathbf{z}^{[1]} \geq 0\}$
5: Compute
6: $\quad \frac{\partial L}{\partial \mathbf{W}^{[2]}} = \delta^{[2]}(\mathbf{a}^{[1]})^T \in \mathbb{R}^{1 \times m}$
7: $\quad \frac{\partial L}{\partial b^{[2]}} = \delta^{[2]}$
8: $\quad \frac{\partial L}{\partial \mathbf{W}^{[1]}} = \boldsymbol{\delta}^{[1]}\mathbf{x}^T \in \mathbb{R}^{m \times D}$
9: $\quad \frac{\partial L}{\partial \mathbf{b}^{[1]}} = \boldsymbol{\delta}^{[1]} \in \mathbb{R}^m$

12.5 Backpropagation for MLP

The derivation of backpropagation in the case of an NN with an arbitrary number of layers can be easily obtained by extending the two-layer case. We start by resuming the

forward step equations:

$$\begin{aligned}
\mathbf{a}^{[1]} &= \mathrm{ReLU}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) \\
\mathbf{a}^{[2]} &= \mathrm{ReLU}(\mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}) \\
\mathbf{a}^{[3]} &= \mathrm{ReLU}(\mathbf{W}^{[3]}\mathbf{a}^{[2]} + \mathbf{b}^{[3]}) \\
&\dots \\
\mathbf{a}^{[r-1]} &= \mathrm{ReLU}(\mathbf{W}^{[r-1]}\mathbf{a}^{[r-2]} + \mathbf{b}^{[r-1]}) \\
\hat{t} = h_{\mathbf{w}}(\mathbf{x}) &= \mathbf{W}^{[r]}\mathbf{a}^{[r-1]} + b^{[r]} \\
L &= \frac{1}{2}(\hat{t} - t)^2
\end{aligned} \tag{12.52}$$

The backpropagation can be computed as shown in Algorithm 11.

Algorithm 11 Backpropagation for MLP

1: Perform the forward step and compute (and store)$\mathbf{z}^k$, $\mathbf{a}^k$ $\forall k = 1, \dots, r-1$ and $\hat{t}$
2: Compute $\delta^{[r]} = \frac{\partial L}{\partial \hat{t}} = (\hat{t} - t)$
3: **for all** $k = r - 1$ to 1 do: **do**
4: $\boldsymbol{\delta}^{[k]} = ((\mathbf{W}^{[k+1]})^T \delta^{[k+1]}) \odot 1\{\mathbf{z}^{[k]} \geq 0\}$
5: $\frac{\partial L}{\partial \mathbf{W}^{[k+1]}} = \boldsymbol{\delta}^{[k+1]}(\mathbf{a}^{[k]})^T$
6: $\frac{\partial L}{\partial \mathbf{b}^{[k+1]}} = \boldsymbol{\delta}^{[k+1]}$
7: **end for**
8: Compute
9: $\frac{\partial L}{\partial \mathbf{W}^{[1]}} = \delta^{[1]}\mathbf{x}^T$
10: $\frac{\partial L}{\partial \mathbf{b}^{[1]}} = \delta^{[1]}$

12.6 Loss functions for NN and Backpropagation

As we anticipated at the beginning of the chapter, the NNs are flexible with respect to the task that they must resolve. In particular, it is possible to use an arbitrary loss function that is reasonable with respect to the task at hand (as long as it is derivable).

We have seen the equations for forward and backward assuming a quadratic loss for the regression:

$$\begin{aligned}
\mathbf{a}^{[1]} &= \mathrm{ReLU}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) \\
&\dots \\
\mathbf{a}^{[r-1]} &= \mathrm{ReLU}(\mathbf{W}^{[r-1]}\mathbf{a}^{[r-2]} + \mathbf{b}^{[r-1]}) \\
\hat{t} = h_{\mathbf{w}}(\mathbf{x}) &= \mathbf{W}^{[r]}\mathbf{a}^{[r-1]} + b^{[r]} \\
L &= \frac{1}{2}(\hat{t} - t)^2
\end{aligned} \tag{12.53}$$

How do the forward step and the backward step change in case we decide to use other loss functions?

Clearly, the structure of the hidden layers remains the same regardless of the loss function used. On the other hand, the expressions for the estimation $\hat{t} = h_{\mathbf{w}}(\mathbf{x})$ and the expression of $L(\mathbf{w})$ can change. Let's look at some examples:

12.6.1 Binary Classification Loss

Let's look at the case of binary classsification. In this scenario, the hypothesis function (and thus the output of the last layer) is given by:

$$\begin{aligned}\tilde{t} &= \mathbf{W}^{[r]}\mathbf{a}^{[r-1]} + b^{[r]} \\ \hat{t} &= h_{\mathbf{w}}(\mathbf{x}) = \sigma(\tilde{t}) = \frac{1}{1 + e^{(\mathbf{W}^{[r]}\mathbf{a}^{[r-1]} + b^{[r]})}}\end{aligned} \tag{12.54}$$

While for the loss function we can use the **Binary Cross Entropy Loss** that we saw for Logistic Regression (negative log-likelihood):

$$L = -t\log(\hat{t}) - (1-t)\log(1-\hat{t}) = -t\log(\sigma(\tilde{t})) - (1-t)\log(1-\sigma(\tilde{t})) \tag{12.55}$$

Then, differently from the regression case, the output of the last layer is processed by the sigmoid function to obtain valid probability values for estimating class membership. For backpropagation, we compute $\frac{\partial L}{\partial \tilde{t}}$ (the other terms for propagation through the hidden layers are identical):

$$\begin{aligned}\frac{\partial L}{\partial \tilde{t}} &= -t\frac{1}{\sigma(\tilde{t})}\sigma'(\tilde{t}) + (1-t)\frac{1}{1-\sigma(\tilde{t})}\sigma'(\tilde{t}) \\ &= -t\frac{\sigma'(\tilde{t})}{\sigma(\tilde{t})} + \frac{\sigma'(\tilde{t})}{1-\sigma(\tilde{t})} - t\sigma'(\tilde{t})\frac{\sigma'(\tilde{t})}{1-\sigma(\tilde{t})} \\ &= \frac{-t\sigma'(\tilde{t}) + t\sigma'(\tilde{t})\sigma(\tilde{t}) + \sigma'(\tilde{t})\sigma(\tilde{t}) - t\sigma(\tilde{t})\sigma'(\tilde{t})}{\sigma(\tilde{t})(1-\sigma(\tilde{t}))} \\ &= \frac{\sigma'(\tilde{t})(\sigma(\tilde{t}) - t)}{\sigma(\tilde{t})(1-\sigma(\tilde{t}))} = \frac{\sigma(\tilde{t})(1-\sigma(\tilde{t}))(\sigma(\tilde{t}) - t)}{\sigma(\tilde{t})(1-\sigma(\tilde{t}))} = \sigma(\tilde{t}) - t = \hat{t} - t\end{aligned} \tag{12.56}$$

We can therefore observe that even in the case of binary classification, the $\delta^{[r]} = \frac{\partial L}{\tilde{t}}$ that we propagate backwards to compute the gradients is always equal to the difference between the output provided by the last layer and the supervision label, as for the regression case. N.B. by output of the last layer we mean $\hat{t}$ and not $\tilde{t}$, *i.e.,* we refer to the output after applying the sigmoid.

12.6.2 Softmax Classification Loss

Let's now consider the multiclass case. The value of the output layer is, in this scenario, processed by the softmax function and the loss is calculated through the **Cross Entropy Loss** for the multiclass case.

In the chapter 6.2.3 we saw the expression for likelihood in the multiclass classification case:

$$\begin{aligned} l(\mathbf{W}) &= \sum_{i=0}^{N-1} \log p(t^{(i)}|\mathbf{x}^{(i)}, \mathbf{W}) \\ &= \sum_{i=0}^{N-1} \log \phi_1 \phi_2 \dots \phi_K \\ &= \sum_{i=0}^{N-1} \log \prod_{l=1}^{K} \left(\frac{e^{\mathbf{w}_l^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x}}} \right)^{I\{t^{(i)}=l\}} \end{aligned} \tag{12.57}$$

The loss function for the single sample is the negative log-likelihood calculated from the previous expression, *i.e.,* (not considering the summation over N samples since we are calculating the gradient with respect to only one sample):

$$L(\mathbf{W}) = -\sum_{l=1}^{K} I\{t = l\} \log \frac{e^{\mathbf{w}_l^T \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{x}}} \tag{12.58}$$

The forward step is:

$$\begin{aligned} \tilde{t}_l &= \mathbf{W}_l^{[r]} \mathbf{a}^{[r-1]} + b_l^{[r]} \\ \hat{t}_l &= \frac{e^{\tilde{t}_l}}{\sum_{j=1}^{K} e^{\tilde{t}_j}} \end{aligned} \tag{12.59}$$

and the loss function:

$$L = -\sum_{l=1}^{K} t_l \log \frac{e^{\tilde{t}_l}}{\sum_{j=1}^{K} e^{\tilde{t}_j}} \tag{12.60}$$

where $I\{t = l\} = t_l$. In the multiclass case, the last layer has a number of outputs equal to the number of classes, since the network has to estimate a probability for each possible class. Thus, for each output node there is a vector of parameters and a dedicated bias.

For backpropagation, we compute $\frac{\partial L}{\partial \tilde{t}_i}$. To do so, we need to compute the derivative with respect to all elements $\hat{t}_l = \frac{e^{\tilde{t}_l}}{\sum_{j=1}^{K} e^{\tilde{t}_j}}$ in the summation. We must then distinguish the cases $l = i$ and $l \neq i$:

- Case $l = i$:

$$\begin{aligned} \frac{\partial \hat{t}_l}{\partial \tilde{t}_l} &= \frac{e^{\tilde{t}_l}}{\sum_{j=1}^{K} e^{\tilde{t}_j}} - \frac{1}{(\sum_{j=1}^{K} e^{\tilde{t}_j})^2} e^{\tilde{t}_l} e^{\tilde{t}_l} \\ &= \frac{e^{\tilde{t}_l}}{\sum_{j=1}^{K} e^{\tilde{t}_j}} \left(1 - \frac{e^{\tilde{t}_l}}{\sum_{j=1}^{K} e^{\tilde{t}_j}} \right) = \hat{t}_l (1 - \hat{t}_l) \end{aligned} \tag{12.61}$$

- Case $l \neq i$:

$$\begin{aligned}\frac{\partial \hat{t}_l}{\partial \tilde{t}_i} &= -\frac{e^{\tilde{t}_l} e^{\tilde{t}_i}}{(\sum_{j=1}^{K} e^{\tilde{t}_j})^2} \\ &= -\frac{e^{\tilde{t}_i}}{\sum_{j=1}^{K} e^{\tilde{t}_j}} \frac{e^{\tilde{t}_l}}{\sum_{j=1}^{K} e^{\tilde{t}_j}} = -\hat{t}_i \hat{t}_l \end{aligned} \tag{12.62}$$

Hence:

$$\begin{aligned}\frac{\partial L}{\partial \tilde{t}_i} &= \frac{\partial}{\partial \tilde{t}_i}(-\sum_{l=1}^{K} t_l \log \hat{t}_l) \\ &= \sum_{l \neq i}^{K} (t_l \frac{1}{\hat{t}_l} \hat{t}_l \hat{t}_i) - t_i \frac{1}{\hat{t}_i} \hat{t}_i (1 - \hat{t}_i) \\ &= \sum_{l \neq i}^{K} (t_l) \hat{t}_i - t_i + t_i \hat{t}_i = \hat{t}_i \underbrace{\sum_{l=1}^{K} (t_l)}_{=1} - t_i = \hat{t}_i - t_i \end{aligned} \tag{12.63}$$

Thus the amount that will be propagated depends again on the difference between the estimated and supervised value.

12.7 Model Selection and Regularization for NN

When we design the architecture of a neural network, its degrees of freedom (or complexity) depend on several hyper-parameters including:

- Number of neurons for each hidden layer;
- Number of Hidden layer;
- Activation function (*e.g.,* ReLU, Tanh, Sigmoid);

The choice of these hyper-parameters to obtain a good trade-off between underfitting and overfitting can be done by exploiting, as for the other algorithms, model selection techniques. However, applying strategies such as cross validation to neural networks may be computationally impractical due to the time required for the various trainings.

Another possibility is to select hyper-parameter values that are considered reasonable (based on the experience) and apply regularization techniques to control overfitting. Several exist for neural networks and we will briefly discuss two of them in the following:

12.7.1 L2 Regularization or Weight Decay

A first possibility is to modify the loss function by adding a term that controls the norm of the parameters, as we have done for example for Ridge Regression. This strategy, in

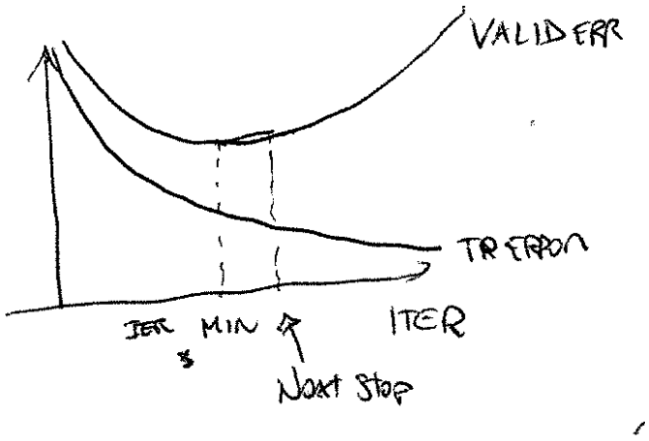


Figure 12.6: Early Stopping

the context of NNs, is also referred to as **Weight Decay**. The only difference to be taken into account in the case of neural networks is that, in the loss function, it is necessary to add a separate term for each hidden layer, with the corresponding hyper-parameter:

$$L_{WD}(\mathbf{w}) = L(\mathbf{w}) + \frac{\lambda_1}{2}(\mathbf{W}^{[1]})^T(\mathbf{W}^{[1]}) + \frac{\lambda_2}{2}(\mathbf{W}^{[2]})^T(\mathbf{W}^{[2]}) + \cdots + \frac{\lambda_r}{2}(\mathbf{W}^{[r]})^T(\mathbf{W}^{[r]}) \tag{12.64}$$

12.7.2 Early Stopping

Another simple way to regularize can be obtained by observing that, as the number of iterations of training through SGD increases, typically the training error decreases. On the other hand, the error on a validation set whose data are not used for training, initially decreases to reach a minimum point and then starts to increase again due to overfitting: as the training proceeds, in fact, the backpropagation will continue to adapt the parameters to explain the training data, up to the point in which, due to the great representative capacity of the NNs, it starts to model the noise.

Intuitively, then, it is reasonable to stop training when the validation error reaches its minimum. This technique is known as **Early Stopping** and consists simply of using a validation set to test the model performance periodically. By monitoring the error on this validation set, it is then possible to notice when it is starting to rise again and stop training (see Figure 12.6).

It is also possible to show that the number of iterations *tau* multiplied by the learning rate *alpha* is inversely proportional to the regularization hyper-parameter *lambda*: the more iterations of SGD, the less regularization we are applying. Conversely, a low number of iterations corresponds to an underfitting scenario:

$$\tau\alpha \propto \frac{1}{\lambda} \tag{12.65}$$

Chapter 13

Unsupervised Learning

In the previous chapters we considered supervised problems, that is, we assumed to have available a dataset in which each sample is associated with a supervision label: $\mathcal{D}_{training} = \{(\mathbf{x}^{(0)}, t^{(0)}), (\mathbf{x}^{(1)}, t^{(1)}), \ldots, (\mathbf{x}^{(N-1)}, t^{(N-1)})\}$.

Let's now consider **Unsupervised** scenarios: in this case, we have available samples for which only the input features are available and not the supervision labels. Our dataset is therefore of the type $\mathcal{D}_{training} = \{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N-1)}\}$.

We clearly cannot train a model to estimate an output quantity, since there is no supervisory information in the training dataset. However, we can look for **Patterns** and **Structures** within the data. These patterns can be used to solve some unsupervised tasks, including:

- **Clustering**: the goal of this task is to identify **Cluster** that group similar data together. Similarity is to be understood with respect to the feature vector describing the sample;
- **Dimensionality Reduction**: in this case, the goal is to find a transformation that maps the features from a high dimensionality space to a low dimensionality space while maintaining the fundamental characteristics, structure and original properties. In other words, we want to reduce the number of features without losing the intrinsic meaningful information "hidden" behind the original representation. Reducing dimensionality is useful both to remove redundant information and filter out noise, and to reduce the complexity of the problem;
- **Feature Extraction**: In this case, the goal is to extract meaningful features from raw data. Unlike dimensionality reduction, in this case the starting point is not features but unprocessed data, like the pixels of an image or the high frequency data set from an accelerometer.

We begin by introducing one of the most popular clustering algorithms: **K-Means**

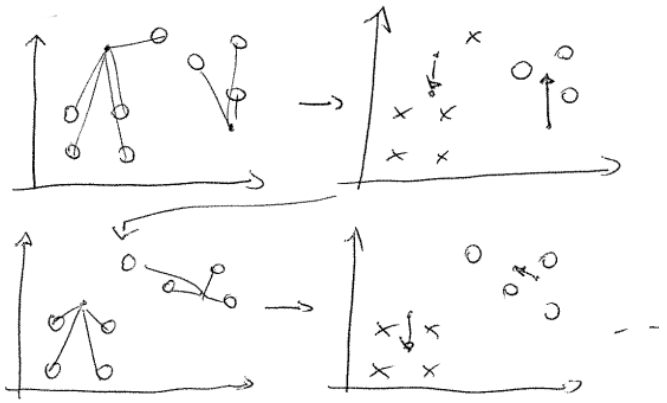


Figure 13.1: K-means iteration example.

13.1 K-Means

K-Means is a clustering algorithm. The goal is to identify K clusters among the data provided for training. So suppose we have N training samples $\{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N-1)}\}$ with $x^{(i)} \in \mathbb{R}^D$. K-Means identifies clusters by finding the **Centroids** that represent the center of the cluster. Membership in one cluster rather than another is determined by the proximity of the sample to its centroid.

In the training phase, K-Means attempts to identify centroids that are optimal with respect to the training data. To do so, it uses a two-step approach described in the Algorithm 12.

Algorithm 12 K-Means

Input: Training dataset $\mathcal{D}_{training} = \{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N-1)}\}$ with $\mathbf{x}^{(i)} \in \mathbb{R}^D$
Initialize cluster centroids $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^D$ randomly
Repeat until convergence
  for all $\mathbf{x}^{(i)} \in \mathcal{D}_{training}$ **do**
    $c^{(i)} = \underset{j}{\operatorname{argmin}} ||\mathbf{x}^{(i)} - \boldsymbol{\mu}_j||$
  end for
  for all $\boldsymbol{\mu}_j$ **do**
    $\boldsymbol{\mu}_j = \frac{\sum_{i=0}^{N-1} 1\{c^{(i)}=j\}\mathbf{x}^{(i)}}{\sum_{i=0}^{N-1} 1\{c^{(i)}=j\}}$
  end for

K-Means starts by randomly initializing the centroids. This can be done by placing the centroids in correspondence to K samples of the dataset taken randomly. In the first step K-Means calculates the assignment of the samples to the centroids, while in the next step it updates the "guess" on the centroids based on the previous assignments. This process is repeated iteratively until the centroids reach convergence (see Figure 13.1.

To see if we have convergence guarantee for K-Means we consider the **Distortion**

Function:

$$L(\boldsymbol{\mu}, \mathbf{c}) = \sum_{i=0}^{N-1} ||\mathbf{x}^{(i)} - \mu_{c^{(i)}}||^2 \tag{13.1}$$

which measures the distance between the samples and the centroids of the clusters. Intuitively, a good clustering result should guarantee a low value for the distortion function. It can be shown that K-means actually performs a gradient descent on the distortion function. In particular, the two inner loops of the algorithm first minimize $L(\boldsymbol{\mu}, \mathbf{c})$ with respect to $\mathbf{c}$ while keeping $\boldsymbol{\mu}$ fixed and then with respect to $\boldsymbol{\mu}$ while keeping $\mathbf{c}$ fixed. Consequently, the distortion function monotonically decreases and thus the algorithm converges.

However, the distortion function is a non-convex function: intuitively, updating the sample assignments to clusters can create discontinuities on $\boldsymbol{\mu}$ with respect to the previous iteration. The algorithm then is not guarateed to converge to a global otimum and could end into local minima. To overcome this problem, it is common to run several K-Means trainings with different centroid initializations and take the model that obtains the lowest value of the distortion function (which corresponds to the local minimum with the best value).

The number of clusters K is a hyper-parameter of K-Means and can be identified by model selection procedures.

The Euclidean distance used to compute centroids is not the only one possible for K-Means. In some situations, such as in the case of categorical features, other distances are more appropriate. The algorithm maintains its properties in the case of other distance functions as well.

The computational cost of the step 1 of K-means is $O(NK)$, as we need to compare each sample in the dataset with each centroid to compute the assignments. Step two, on the other hand, has a cost $O(N_j K)$, where N_j denotes the number of samples assigned to cluster j.

13.1.1 K-Medoids

In some situations, it may not be possible or reasonable to identify the cluster centroids as the average of the samples assigned to the cluster. In these cases, an alternative solution is to choose one of the samples of the dataset as the centroid of the cluster: in this case the centroids take the name of **Medoids** and the algorithm used to identify them is known as **K-Medoids**. The training procedure is very similar, with the difference that the medoid is computed by taking the sample closest to the arithmetic mean of the

samples assigned to that cluster (see Algorithm 13.

Algorithm 13 K-Medoids

Input: Training dataset $\mathcal{D}_{training} = \{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N-1)}\}$ with $\mathbf{x}^{(i)} \in \mathbb{R}^D$
Initialize cluster medoids $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^D$ by picking K random samples from $\mathcal{D}_{training}$
Repeat until convergence
  for all $\mathbf{x}^{(i)} \in \mathcal{D}_{training}$ **do**
    $c^{(i)} = \underset{j}{\operatorname{argmin}} ||\mathbf{x}^{(i)} - \boldsymbol{\mu}_j||$
  end for
  for all $j = 1, \ldots, K$ **do**
    $\bar{\mathbf{x}}_j = \frac{\sum_{i=0}^{N-1} 1\{c^{(i)}=j\}\mathbf{x}^{(i)}}{\sum_{i=0}^{N-1} 1\{c^{(i)}=j\}}$
    $\boldsymbol{\mu}_j = \underset{i}{\operatorname{argmin}} ||\mathbf{x}^{(i)} - \bar{\mathbf{x}}_j||$
  end for

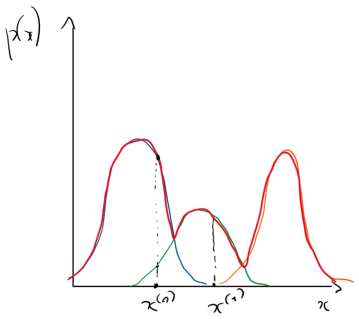


Figure 13.2: Example of a mixsture of gaussian in the one-dimensional case. The mixture (in red) is the result of the sum of three Gaussian (gree, blue and orange). The point $x^{(0)}$ is generated from the blue Gaussian with a higher probability than the green or the orange on (although these are non zero). Instead, for the sample $x^{(1)}$ the probability of being generated from the green Gaussian is the highest.

13.2 Gaussian Mixture Models (GMMs)

K-Means and K-Medoids performs a "hard" assignment of sample to clusters, in the sense that a sample belongs to the cluster to which it is assigned with probability 1.0 and to the other clusters with probability 0.0. Furthermore, these two algorithms often do not work well in situations where the samples of the dataset follow patterns that are not compatible with the metrics used to calculate similarity.

An approach that tries to solve these problems is known as **Mixture of Gaussians**, which instead tries to approximate K clusters with K Gaussians with possibly different means and variances. Each sample, then, has some probability of being generated by one of the K Gaussians. Clearly, since Gaussians have non-zero probability throughout their support, the assignment of samples to clusters is, in this case, **Soft**: we do expect that the probability that a sample was generated by a certain cluster is higher than that associated with the others, nevertheless the latter are still non-zero (see Figure 13.2).

The Mixture of Gaussian model describes the process of generating a $\mathbf{x}$ sample as the sum of K Gaussians with different mean and variance:

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k N(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k) \tag{13.2}$$

where the π_k is called the **Mixture Coefficient** and it must hold that $\sum_{k=1}^{K} \pi_k = 1$.

What we would be interested in is estimating the probability that a given sample belongs to a given cluster (or, equivalently, estimating the probability that a given sample was generated by a given Gaussian of the mixture). More explicitly, we would like to estimate $p(t = k|\mathbf{x}) = p(C_k|\mathbf{x})$, where C_k is the abbreviated notation for $t = k$.

In a supervised setting, *i.e.,* where we have a dataset

$$\mathcal{D}_{training} = \{(\mathbf{x}^{(0)}, t^{(0)}), (\mathbf{x}^{(1)}, t^{(1)}), \dots, (\mathbf{x}^{(N-1)}, t^{(N-1)})\} \tag{13.3}$$

we have already seen how to solve this problem: with the generative models, and in particular LDA and QDA. Indeed, we can compute $p(C_k|\mathbf{x})$ as:

$$p(C_k|\mathbf{x}) = \frac{p(\mathbf{x}|C_k)p(C_k)}{p(\mathbf{x})} \tag{13.4}$$

and estimate the parameters of the class conditional probabilities and priors with MLE on the joint probability:

$$\max_{\mu_1,\dots,\mu_k,\Sigma_1,\dots,\Sigma_k,\pi_k} \log p(\mathbf{X}, \mathbf{t}|\mu_1, \dots, \mu_k, \Sigma_1, \dots, \Sigma_k, \pi_k) \tag{13.5}$$

When we introduced LDA, we derived the MLE in the binary case and with covariances of the two Gaussians equal, *i.e.,* expressing the joint distribution as

$$p(\mathbf{X}, \mathbf{t}|\mu_1, \mu_2, \Sigma, \pi) = \prod_{i=0}^{N-1} (\pi N_1(\mathbf{x}^{(i)}|\mu_1, \Sigma))^{t^{(i)}} (1-\pi) N_2(\mathbf{x}^{(i)}|\mu_2, \Sigma)^{1-t^{(i)}} \tag{13.6}$$

We can also note an interesting aspect: if we try to calculate, from the joint distribution, the marginal distribution of $\mathbf{X}$ we obtain (for simplicity, always for the binary case and for the single sample):

$$\begin{aligned} p(\mathbf{x}^{(i)}|\mu_1, \mu_2, \Sigma, \pi) &= \sum_{t^{(i)}} p(\mathbf{x}^{(i)}, t^{(i)}|\mu_1, \mu_2, \Sigma, \pi) \\ &= \sum_{t^{(i)}} (\pi N_1(\mathbf{x}^{(i)}|\mu_1, \Sigma))^{t^{(i)}} (1-\pi) N_2(\mathbf{x}^{(i)}|\mu_2, \Sigma)^{1-t^{(i)}} \\ &= (\pi N_1(\mathbf{x}^{(i)}|\mu_1, \Sigma)) + (1-\pi) N_2(\mathbf{x}^{(i)}|\mu_2, \Sigma) \end{aligned} \tag{13.7}$$

If we extend the last expression to the case of K classes and with different variances for each Gaussian we get:

$$p(\mathbf{x}^{(i)}|\mu_1, \dots, \mu_k, \Sigma_1, \dots, \Sigma_k, \pi_k) = \sum_{k=1}^{K} \pi_k N(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k) \tag{13.8}$$

which is precisely the definition of Gaussian Mixture Model that we gave earlier. Working with GMMs is therefore analogous to working with the marginal distribution of a generative model.

But why do we have to work with the marginal and not with the joint? The reason lies in the fact that, in a **unsupervised** context, we do not have access to $t^{(i)}$ and consequently cannot use $p(\mathbf{X}, \mathbf{t})$ (we saw how its expression depends on $t^{(i)}$). Consequently, since we only know $\mathbf{x}^{(i)}$, we can only work with $p(\mathbf{X})$ and apply MLE to the it.

The sample assignments to the various Gaussians are therefore **unknown**, or rather **hidden** or **latent**: a certain sample was surely generated by one of the K Gaussians, but in the unsupervised context we do not know from which one. Consequently, the variable encoding membership in one of the Gaussians (or, equivalently, in one of the clusters or classes) is called **Latent Variable** and, to emphasize that it is unobserved, is denoted by the letter z and not by t.

> N.B. Although the latent variables are indicated with a different letter from the supervised labels, from the point of view of what they represent they are identical: both in fact, t_k and z_k, encode the belonging of a sample to one of the Gaussians/clusters/classes or, equivalently, they tell us that a sample has been generated by the k-th Gaussian. However, since they are not available in the unsupervised case, we denote them by a different letter to emphasize this aspect.

Before proceeding with MLE on $p(\mathbf{X})$, we introduce some quantities that will come in handy. As we did for all the cases where we had to handle categorical variables with K possible values, we express the variable $\mathbf{z}$ with the **one-hot vector** encoding (latent variables can take K values). The latent variable of the i-th sample is then be expressed as:

$$\mathbf{z}^{(i)} = [0, 0, \dots, 1, \dots, 0] \tag{13.9}$$

with $z_k^{(i)} \in \{0, 1\}$ and $\sum_{k=1}^{K} z_k^{(i)} = 1$ We describe the latent variable with a Multinomial distribution:

$$p(\mathbf{z}^{(i)}) = \prod_{k=1}^{K} \pi_k^{z_k^{(i)}} \tag{13.10}$$

with $p(z_k^{(i)} = 1) = \pi_k$, $\pi_k \geq 0$ and $\sum_{k=1}^{K} \pi_k = 1$. The 13.10 is also called the **Marginal Distribution** of $\mathbf{z}^{(i)}$. Since $z_k^{(i)}$ expresses the membership of a sample in one of the Gaussians, it is clear that we can write:

$$p(\mathbf{x}^{(i)}|z_k^{(i)}) = N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k) \tag{13.11}$$

then, expressing the previous relation with respect to the vector $\mathbf{z}^{(i)}$ we can write:

$$p(\mathbf{x}^{(i)}|\mathbf{z}^{(i)}) = \prod_{k=1}^{K} N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k)^{z_k^{(i)}} \tag{13.12}$$

which, in analogy to LDA, is known as **Latent Conditional Probability** (substituting C_k for z_k we in fact get the class conditional probability in the multiclass case). The

joint distribution can then be calculated as:

$$\begin{aligned} p(\mathbf{x}^{(i)}, \mathbf{z}^{(i)}) = p(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})p(\mathbf{z}^{(i)}) &= \prod_{k=1}^{K} N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k)^{z_k^{(i)}} \prod_{k=1}^{K} \pi_k^{z_k^{(i)}} \\ &= N_1(\cdot)^{z_1^{(i)}} N_2(\cdot)^{z_2^{(i)}} \dots N_K(\cdot)^{z_K^{(i)}} \pi_1^{z_1^{(i)}} \pi_2^{z_2^{(i)}} \dots \pi_K^{z_K^{(i)}} \\ &= \prod_{k=1}^{K} (\pi_k N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k))^{z_k^{(i)}} \end{aligned} \tag{13.13}$$

Again we note that, by replacing $z_k^{(i)}$ with $t_k^{(i)}$ we get the same expression we had obtained for LDA (you can see this in the binary case $K = 2$). We specify again that the use of z instead of t is only necessary to emphasize the fact that, being in an unsupervised context, the class are unobserved.

We can also easily see that by marginalizing with respect to $\mathbf{z}^{(i)}$ the joint distribution, we obtain the definition of mixture of Gaussian:

$$\begin{aligned} p(\mathbf{x}^{(i)}) = \sum_{\mathbf{z}^{(i)}} p(\mathbf{x}^{(i)}, \mathbf{z}^{(i)}) = \sum_{\mathbf{z}^{(i)}} p(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})p(\mathbf{z}^{(i)}) &= \sum_{\mathbf{z}^{(i)}} \prod_{k=1}^{K} N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k)^{z_k^{(i)}} \prod_{k=1}^{K} \pi_k^{z_k^{(i)}} \\ &= \sum_{\mathbf{z}^{(i)}} N_1(\cdot)^{z_1^{(i)}} N_2(\cdot)^{z_2^{(i)}} \dots N_K(\cdot)^{z_K^{(i)}} \pi_1^{z_1^{(i)}} \pi_2^{z_2^{(i)}} \dots \pi_K^{z_K^{(i)}} \\ &= N_1(\cdot)\pi_1 + N_2(\cdot)\pi_2 + \dots N_K(\cdot)\pi_K = \sum_{k=1}^{K} \pi_k N(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k) \end{aligned} \tag{13.14}$$

We also compute the posterior distribution of the latent variables (we do this for the k-th component of the variable):

$$\begin{aligned} p(z_k^{(i)}|\mathbf{x}^{(i)}) &= \frac{p(\mathbf{x}^{(i)}|\mathbf{z}^{(i)})p(\mathbf{z}^{(i)})}{p(\mathbf{x}^{(i)})} \\ &= \frac{N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k)\pi_k}{\sum_{k=1}^{K} \pi_k N(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k)} = \gamma(z_k^{(i)}) \end{aligned} \tag{13.15}$$

which is referred to as **Responsibility**.

At this point, if we were in a supervised setting, we could proceed as with LDA. Indeed, assuming we have a complete dataset in which we know both $\mathbf{x}$ and the associated latent variables $\mathbf{z}$, *i.e.,* $\mathcal{D}_{training} = \{(\mathbf{x}^{(0)}, \mathbf{z}^{(0)}), (\mathbf{x}^{(1)}, \mathbf{z}^{(1)}), \dots, (\mathbf{x}^{(N-1)}, \mathbf{z}^{(N-1)})\}$, we can compute the various parameters $\boldsymbol{\mu}_k, \Sigma_k, \pi_k$ by maximizing the log joint ditribution of

the entire dataset:

$$\begin{aligned}\max_{\boldsymbol{\mu}_k,\Sigma_k,\pi_k} \log p(\mathbf{X},\mathbf{Z}) &= \max_{\boldsymbol{\mu}_k,\Sigma_k,\pi_k} \log \prod_{i=0}^{N-1}\prod_{k=1}^{K}(\pi_k N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k,\Sigma_k))^{z_k^{(i)}} \\ &= \max_{\boldsymbol{\mu}_k,\Sigma_k,\pi_k} \sum_{i=0}^{N-1}\sum_{k=1}^{K} z_k^{(i)}\log\pi_k + z_k^{(i)}\log N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k,\Sigma_k) \end{aligned} \tag{13.16}$$

By proceeding with the maximization, we obtain the optimal closed-form values for the various parameters. As an example, we derive the value of π_k (the procedure will be analogous to what we have already seen when we derived π for the binary case of LDA):

$$\frac{\partial \log p(\mathbf{X},\mathbf{Z})}{\partial \pi_k} = 0 \implies \frac{\partial}{\partial \pi_k}\sum_{i=0}^{N-1}\sum_{k=1}^{K} z_k^{(i)}\log\pi_k = 0 \tag{13.17}$$

recalling that we have the constraint $\sum_{k=1}^{K}\pi_k = 1$ we can continue by introducing the lagrangian:

$$\begin{aligned}&\implies \frac{\partial}{\partial \pi_k}\left[\sum_{i=0}^{N-1}\sum_{k=1}^{K} z_k^{(i)}\log\pi_k + \lambda(\sum_{k=1}^{K}\pi_k - 1)\right] = 0 \\ &\implies \sum_{i=0}^{N-1}\frac{z_k^{(i)}}{\pi_k} + \lambda = 0 \end{aligned} \tag{13.18}$$

Adding up K both terms of the last expression we note that:

$$\sum_{i=0}^{N-1}\underbrace{\sum_{k=1}^{K} z_k^{(i)}}_{=1} = -\lambda\underbrace{\sum_{k=1}^{K}\pi_k}_{=1} \implies \lambda = -N \tag{13.19}$$

Resubstituting in the last expression of 13.18 we have:

$$\sum_{i=0}^{N-1}\frac{z_k^{(i)}}{\pi_k} = N \implies \pi_k = \frac{1}{N}\sum_{i=0}^{N-1} z_k^{(i)} \tag{13.20}$$

However, as we have already mentioned, in the unsupervised case we cannot work with the joint distribution since we only have the marginal available. Therefore, let's try to apply MLE to the latter:

$$\begin{aligned}\max_{\boldsymbol{\mu}_k,\Sigma_k,\pi_k} \log p(\mathbf{X}|\boldsymbol{\mu}_k,\Sigma_k,\pi_k) &= \max_{\boldsymbol{\mu}_k,\Sigma_k,\pi_k} \log \prod_{i=0}^{N-1}\sum_{k=1}^{K}\pi_k N(\mathbf{x}|\boldsymbol{\mu}_k,\Sigma_k) \\ &= \max_{\boldsymbol{\mu}_k,\Sigma_k,\pi_k} \sum_{i=0}^{N-1}\log\left(\sum_{k=1}^{K}\pi_k N(\mathbf{x}|\boldsymbol{\mu}_k,\Sigma_k)\right) \end{aligned} \tag{13.21}$$

In this case the expression is more problematic, because we have a logarithm of a sum. Let's try to see what happens if we optimize with respect to $\boldsymbol{\mu}_k$:

$$\frac{\partial \log p(\mathbf{X}|\boldsymbol{\mu}_k, \Sigma_k, \pi_k)}{\partial \boldsymbol{\mu}_k} = 0 \implies \frac{\partial}{\partial \boldsymbol{\mu}_k} \sum_{i=0}^{N-1} \log \left(\sum_{k=1}^{K} \pi_k N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k) \right) = 0$$

$$\implies \sum_{i=0}^{N-1} \underbrace{\frac{\pi_k N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k)}{\sum_{k=1}^{K} \pi_k N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k)}}_{\gamma(z_k^{(i)})} (-\cancel{\Sigma_k^{-1}}(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)) = 0$$

$$\implies \gamma(z_k^{(i)})(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k) = 0 \implies \boldsymbol{\mu}_k^* = \frac{\sum_{i=0}^{N-1} \gamma(z_k^{(i)}) \mathbf{x}^{(i)}}{\sum_{i=0}^{N-1} \gamma(z_k^{(i)})} = \frac{1}{N_k} \sum_{i=0}^{N-1} \gamma(z_k^{(i)}) \mathbf{x}^{(i)} \quad (13.22)$$

where N_k is the effective number of samples "soft-assigned" to the cluster K (effective because responsibilities are probabilities and therefore we are not adding "hard" assignments).

We note that to calculate the optimal $\boldsymbol{\mu}_k^*$ we need the responsibilities $\gamma(z_k^{(i)})$ which in turn depend on $\boldsymbol{\mu}_k$. Therefore, we cannot solve the problem in closed form. Furthermore, it can be shown that a similar situation occurs for the other parameters:

$$\Sigma_k = \frac{1}{N_k} \sum_{i=0}^{N-1} \gamma(z_k^{(i)})(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^T (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k) \quad (13.23)$$

$$\pi_k = \frac{N_k}{N} \quad (13.24)$$

However, to solve it, we can exploit an alternated optimization, in which we first compute the responsibilities by fixing the parameters $\boldsymbol{\mu}_k, \Sigma_k, \pi_k$ and then update the latter with the $\gamma(z_k^{(i)})$ we just computed. This procedure can be summarized in the Algorithm 14:

Algorithm 14 Expectation-Maximimization for GMMs

1: Initialize $\boldsymbol{\mu}_k, \Sigma_k, \pi_k$ randomly
2: Repeat until convergence
3: E-Step:
4: $\forall i = 0, \dots, N-1$ and $\forall k = 1, \dots, K$ compute $\gamma(z_k^{(i)}) = \frac{N(\mathbf{x}^{(i)}|\boldsymbol{\mu}_k, \Sigma_k)\pi_k}{\sum_{k=1}^{K} \pi_k N(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k)}$
5: M-Step:
6: $\forall k = 1, \dots, K$ compute $N_k = \sum_{i=0}^{N-1} \gamma(z_k^{(i)}$
7: $\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{i=0}^{N-1} \gamma(z_k^{(i)}) \mathbf{x}^{(i)}$
8: $\Sigma_k = \frac{1}{N_k} \sum_{i=0}^{N-1} \gamma(z_k^{(i)})(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^T (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)$
9: $\pi_k = \frac{N_k}{N}$

These two steps are called the **Expectation** step (E-Step) and the **Maximization** step (M-Step). In particular, the solution for GMMs is a special case of a more general algorithm known as **Expectation Maximization**.

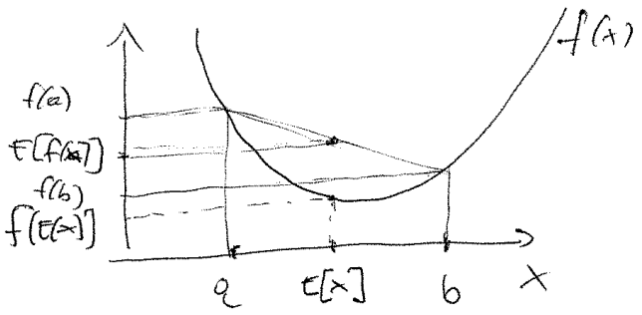


Figure 13.3: Example of the Jensen inequality in the monodimensional case. In this case X is a random variable with two values a and b, both associated with a 0.5 probability. Therefore, its expected value can be easily computed as $E[X] = \frac{a+b}{2}$. Similarly, the expecte value of $f(a)$ and $f(b)$ is $E[X] = \frac{f(a)+f(b)}{2}$

13.3 Expectation Maximization (EM-Algorithm)*

To derive the general form of the EM algorithm, two concepts must first be introduced.

13.3.1 Jensen Inequality

Given a function f, we know that it is convex if its domain is a convex set and if its Hessian matrix is positive semidefinite (in the case of a one-dimensional domain $f''(x) \geq 0$.

Theorem 13.1. ***Jensen Inequality*** *Let there be a convex function f and a random variable X. Then the following relation holds (see Figure 13.3):*

$$E[f(X)] \geq f(E[X]) \tag{13.25}$$

Moreover, if f is strictly convex, the equality $E[f(X)] = f(E[X])$ holds if and only if $X = E[X]$ with probability 1 (i.e., , if X is a constant).

In the case of a f concave function, we can see that $-f$ is convex and consequently, for concave functions, the Jensen Inequality continues to hold with inverted sign:

$$E[f(X)] \leq f(E[X]) \tag{13.26}$$

13.3.2 Kullback-Leibler Divergence (KL Divergence)

To define the Kullback-Leibler Divergence, we first take up the concept of **Entropy**. Given a random variable X, with pdf $p(x)$, we can compute its entropy as:

$$H[x] = -\sum_{x} p(x) \log p(x) \tag{13.27}$$

or, in the case of continuous random variable:

$$H[x] = -\int_x p(x) \log p(x) dx \tag{13.28}$$

It can be shown that entropy represents a lower bound on the number of units (*e.g.,* bits) of information required to encode $p(x)$.

What do we need entropy for in the context of KL-divergence? Sometimes, as in the case of the EM algorithm, we do not have access to or cannot express analytically a certain distribution $p(x)$. Consequently, one possible solution is to use an easier-to-manage function $q(x)$ to approximate the one of interest. To figure out if the function $q(x)$ contains the same information as $p(x)$ we compute the difference between their entropies (both computed with respect to $p(x)$):

$$\begin{aligned} &\underbrace{-\int_x p(x) \log q(x) dx}_{H[x] \text{ of } q(x)} - (\underbrace{-\int_x p(x) \log p(x) dx}_{H[x] \text{ of } p(x)}) \\ &= -\int_x p(x) \log \frac{q(x)}{p(x)} dx = \int_x p(x) \log \frac{p(x)}{q(x)} dx = KL(p||q) \end{aligned} \tag{13.29}$$

The last expression is the KL divergence between the two distributions.

It is important to note that $KL(p||q) \neq KL(q||p)$. We also note that:

$$\begin{aligned} KL(p||q) &= -\int_x p(x) \log \frac{q(x)}{p(x)} dx \\ &= E_{x \sim p(x)}[-\log \frac{q(x)}{p(x)}] \underbrace{\geq}_{\text{by Jensen Inequality}} -\log \int p(x) \frac{q(x)}{p(x)} dx = 0 \end{aligned} \tag{13.30}$$

That is, the KL divergence is always positive and is equal to 0 only when $p(x) = q(x)$.

13.3.3 EM Algorithm

We apply the MLE framework to the generic function $p(\mathbf{X}|\theta)$ that depends on the parameters to be trained θ (In the case of GMMs, for example, $\theta = \{\boldsymbol{\mu}_k, \Sigma_k, \pi_k\}$). To do

so, we compute the log-likelihood of $p(\mathbf{X}|\theta)$:

$$\begin{aligned}\log p(\mathbf{X}|\theta) &= \log \sum_{\mathbf{z}} p(\mathbf{X}, \mathbf{z}|\theta) \\ &= \log \sum_{\mathbf{z}} Q(\mathbf{z}) \frac{p(\mathbf{X}, \mathbf{z}|\theta)}{Q(\mathbf{z})} \\ &= \log E_{\mathbf{z}\sim Q(\mathbf{z})} \left[\frac{p(\mathbf{X}, \mathbf{z}|\theta)}{Q(\mathbf{z})} \right] \\ &\underbrace{\geq}_{\text{by Jensen Inequality}} E_{\mathbf{z}\sim Q(\mathbf{z})} \left[\log \frac{p(\mathbf{X}, \mathbf{z}|\theta)}{Q(\mathbf{z})} \right] = \sum_{\mathbf{z}} Q(\mathbf{z}) \log \frac{p(\mathbf{X}, \mathbf{z}|\theta)}{Q(\mathbf{z})}\end{aligned} \tag{13.31}$$

Thus, we have:

$$\log p(\mathbf{X}|\theta) \geq \sum_{\mathbf{z}} Q(\mathbf{z}) \log \frac{p(\mathbf{X}, \mathbf{z}|\theta)}{Q(\mathbf{z})} \tag{13.32}$$

Where $L(Q(\mathbf{z}), \theta) = \sum_{\mathbf{z}} Q(\mathbf{z}) \log \frac{p(\mathbf{X}, \mathbf{z}|\theta)}{Q(\mathbf{z}))}$ is known as **Evidence Lower Bound (ELBO)** and holds for any $Q(\mathbf{z})$.

Our interest is in ensuring the equality of this relation, so that in maximizing $L(Q(\mathbf{z}), \theta)$ we in fact maximize $\log p(\mathbf{X}|\theta)$. How should we choose $Q(\mathbf{z})$ for this purpose?

To understand this, we note that:

$$\begin{aligned}&\log p(\mathbf{X}|\theta) - \sum_{\mathbf{z}} Q(\mathbf{z}) \log \frac{p(\mathbf{X}, \mathbf{z}|\theta)}{Q(\mathbf{z})} \\ &= \underbrace{\sum_{\mathbf{z}} Q(\mathbf{z})}_{=1} \log p(\mathbf{X}|\theta) - \sum_{\mathbf{z}} Q(\mathbf{z}) \log \frac{p(\mathbf{X}, \mathbf{z}|\theta)}{Q(\mathbf{z})} \\ &= \sum_{\mathbf{z}} Q(\mathbf{z}) \log \frac{p(\mathbf{X}|\theta) Q(\mathbf{z})}{p(\mathbf{X}, \mathbf{z}|\theta)} = \sum_{\mathbf{z}} Q(\mathbf{z}) \log \frac{p(\mathbf{X}|\theta) Q(\mathbf{z})}{p(\mathbf{z}|\mathbf{X}, \theta) p(\mathbf{X}|\theta)} \\ &= \sum_{\mathbf{z}} Q(\mathbf{z}) \log \frac{Q(\mathbf{z})}{p(\mathbf{z}|\mathbf{X}, \theta)} = -\sum_{\mathbf{z}} Q(\mathbf{z}) \log \frac{p(\mathbf{z}|\mathbf{X}, \theta)}{Q(\mathbf{z})} = KL(Q(\mathbf{z})||p(\mathbf{z}|\mathbf{X}, \theta))\end{aligned} \tag{13.33}$$

hence, we get

$$\begin{aligned}\log p(\mathbf{X}|\theta) &= ELBO(Q(\mathbf{z}), \theta) + KL(Q(\mathbf{z})||p(\mathbf{z}|\mathbf{X}\theta)) \\ &= L(Q(\mathbf{z}), \theta) + KL(Q(\mathbf{z})||p(\mathbf{z}|\mathbf{X}\theta))\end{aligned} \tag{13.34}$$

Hence, the equality $\log p(\mathbf{X}|\theta) = ELBO(Q(\mathbf{z}), \theta)$ holds when $KL(Q(\mathbf{z})||p(\mathbf{z}|\mathbf{X}, \theta)) = 0$, that is, when we choose $Q(\mathbf{z}) = p(\mathbf{z}|\mathbf{X}, \theta)$.

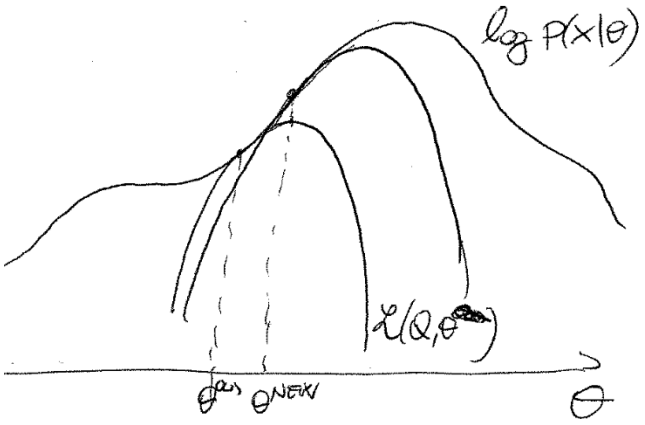


Figure 13.4: Example of the EM-iterations

Thus, given the same choice of parameters θ, if we impose $Q(\mathbf{z}) = p(\mathbf{z}|\mathbf{X}, \theta)$ we obtain equality in the relation 13.34, *i.e.,* $\log p(\mathbf{X}|\theta) = L(Q(\mathbf{z}), \theta)$. This constitutes the E-step of the algorithm, since we impose $Q(\mathbf{z})$ such that the ELBO, *i.e.,* , the expectation with respect to $Q(\mathbf{z})$ of the joint distribution, is equal to the log-likelihood $\log p(\mathbf{X}|\theta)$.

At this point, by fixing $Q(\mathbf{z})$ we can maximize $L(Q(\mathbf{z}), \theta)$ with respect to θ. What guarantees us that performing this operation results in a maximization of $\log p(\mathbf{X}|\theta)$ (which is the one we am interested in)? To answer this question, it suffices to recall that $L(Q(\mathbf{z}), \theta)$ is the lower bound of $\log p(\mathbf{X}|\theta)$, consequently, if we maximize it, we actually "push" $\log p(\mathbf{X}|\theta)$ toward larger values as well. However, as a side effect, because we are moving on $L(Q(\mathbf{z}), \theta)$ by changing θ, the KL divergence will no longer be zero. The EM algorithm then alternates between these two steps: the E-step nullifies the KL divergence, while the M-Step maximizes $L(Q(\mathbf{z}), \theta)$, which causes $\log p(\mathbf{X}|\theta))$ to grow but also the KL divergence to increase again (see Figure 13.4).

Furthermore, we can see that performing the M-step on the $L(Q(\mathbf{z}), \theta)$ is equivalent to working on the following expression:

$$\begin{aligned} L(Q(\mathbf{z}), \theta) &= \sum_{\mathbf{z}} \underbrace{Q(\mathbf{z})}_{=p(\mathbf{z}|\mathbf{X},\theta_{OLD}) \text{ after the E-step}} \log \frac{p(\mathbf{X}, \mathbf{z}|\theta)}{Q(\mathbf{z})} \\ &= \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{X}, \theta_{OLD}) \log p(\mathbf{X}, \mathbf{z}|\theta) - \underbrace{\sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{X}, \theta_{OLD}) \log p(\mathbf{z}|\mathbf{X}, \theta_{OLD})}_{\text{constant since it does not depend on } \theta} \end{aligned} \tag{13.35}$$

so the term to maximize is $\sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{X}, \theta_{OLD}) \log p(\mathbf{X}, \mathbf{z}|\theta)$. This operation is simple because the term appears in the logarithm without summation and we do not have the "problematic" situation that we had in working with the $\log p(\mathbf{X}|\theta)$. The EM algorithm is shown in the box 15

Algorithm 15 Expectation-Maximimization Algorithm

Initialize θ randomly
Repeat until convergence
    E-Step:
        Evaluate $p(\mathbf{z}|\mathbf{X}, \theta_{OLD})$
    M-Step:
        Compute θ_{NEW} as
            $\theta_{NEW} = \text{argmax}_\theta \sum_{\mathbf{z}} p(\mathbf{z}|\mathbf{X}, \theta_{OLD}) \log p(\mathbf{X}, \mathbf{z}|\theta)$

Chapter 14

PCA and ICA

14.1 Principal Component Analysis (PCA)

Let's now introduce an unsupervised technique for dimensionality reduction: **Principal Component Analysis (PCA)**.

Suppose in particular that we want to reduce features from a $\mathbb{R}^D$ space to a $\mathbb{R}^K$ space with $K << D$. The idea behind PCA is to identify a reduced dimensionality space that retains the information of the original space while discarding redundant and superfluous information.

To understand the intuition behind this algorithm, imagine we have a dataset $\mathcal{D} = \{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N-1)}\}$ containing surveys of pilots of radio-controlled helicopter. Each sample is characterized by two features, where the first expresses the skill of the pilot and the second expresses its degree of enjoyment. Intuitively, the two features have a high correlation, as those who have more fun driving helicopters are also likely to have higher skill. If we then look at Figure 14.1 it clearly emerges that there is a direction of greatest variation in the data, which is neither along x_1 nor along x_2, but along a combination of them $\mathbf{u}_1$. On the other hand, the second direction $\mathbf{u}_2$ does not carry relevant information since it is associated with the least significant portion of variation. PCA aims precisely at identifying these vectors in order to select those associated with more variation as the new representation of the dataset.

Given the strong correlation of the two features, we could say that they are collinear and thus reduce the size of the problem by simply removing one of them after checking the correlation matrix. PCA, however, follows another approach: it transforms the features into a space in which the original features are combined to make the hidden patterns between them explicit and, thus, highlight the "directions" of variation. In this way, we can choose those vectors that represent the directions of **maximum** variation and discard the others (which encode irrelevant or noise-related features).

To find the directions of greatest variation PCA tries to identify the vector $\mathbf{u}$ such that, when the data is projected along it, the variance of the projected data is maximum. If we again refer to Figure 14.1, we see in fact that if we project the samples along $\mathbf{u}_1$,

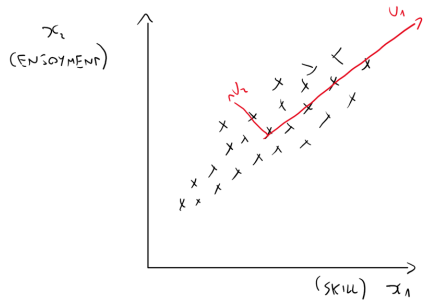


Figure 14.1: Direction of main variation in the helicopter example.

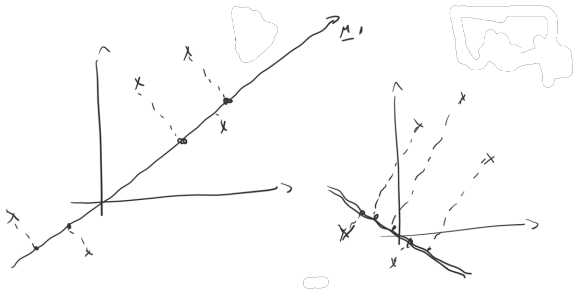


Figure 14.2: Projection of the sample along the direction of variation.

the variance of the projections is much larger than if we projected along $\mathbf{u}_2$.

Why do we want to find the vector for which the data projected onto it is associated with a high variance? The reason lies in the fact that the original data contains a certain amount of variance (and thus information). Consequently, it makes sense to look for a representation that retains the largest amount of this variance.

To formalize this, we start by normalizing the samples, *i.e.,* each feature is updated by subtracting the sample mean and dividing by the sample variance. Assuming that the vectors referring to the directions of variation pass through the origin, the variance of the projection of a dataset along these directions can be calculated simply by computing the scalar product between the sample itself and the unit vector $\mathbf{u}$ (*i.e.,* $||\mathbf{u}| = 1$) representing the direction of variation, *i.e.,* $\mathbf{x}^T\mathbf{u}$ (Figure 14.2). At this point we need only compute the variance of these projections with respect to the entire dataset and find the vector $\mathbf{u}$ that maximizes this variance:

$$\max_{\mathbf{u}} \frac{1}{N}\sum_{i=0}^{N-1}((\mathbf{x}^{(i)})^T\mathbf{u})^2 = \frac{1}{N}\sum_{i=0}^{N-1}(\mathbf{u})^T\mathbf{x}^{(i)}(\mathbf{x}^{(i)})^T\mathbf{u} = \mathbf{u}^T\underbrace{\left(\frac{1}{N}\sum_{i=0}^{N-1}\mathbf{x}^{(i)}(\mathbf{x}^{(i)})^T\right)}_{\Sigma}\mathbf{u} \quad (14.1)$$

$$\text{s.t. } ||\mathbf{u}|| = 1$$

where we see that the quantity in parentheses Σ of the last relation is the sample variance in vector form of the samples in the dataset. We continue by applying the method of Lagrange multipliers:

$$\max L(\mathbf{u}, \lambda) = \mathbf{u}^T \Sigma \mathbf{u} - \lambda(\mathbf{u}^T \mathbf{u} - 1) \tag{14.2}$$

Calculating the gradient with respect to $\mathbf{u}$ and the derivative with respect to λ we obtain:

$$\nabla_{\mathbf{u}} L(\mathbf{u}, \lambda) = \Sigma \mathbf{u} - \lambda \mathbf{u} = 0 \implies \Sigma \mathbf{u} = \lambda \mathbf{u} \tag{14.3}$$

$$\frac{\partial L(\mathbf{u}, \lambda)}{\partial \lambda} = \mathbf{u}^T \mathbf{u} = 1 \tag{14.4}$$

The first of the two relations indicates that $\mathbf{u}$ is the eigenvector of Σ with eigenvalue λ. Resubstituting in the Lagrangian and replacing $\mathbf{u}$ with $\mathbf{e}_\lambda$ to emphasize the fact that it is an eigenvector we get:

$$\begin{aligned} \max L(\mathbf{e}_\lambda, \lambda) &= \mathbf{e}_\lambda^T \underbrace{\Sigma \mathbf{e}_\lambda}_{=\lambda \mathbf{e}_\lambda} - \lambda(\mathbf{e}_\lambda^T \mathbf{e}_\lambda - 1) \\ &= \lambda \mathbf{e}_\lambda^T \mathbf{e}_\lambda - \lambda \mathbf{e}_\lambda^T \mathbf{e}_\lambda + \lambda = \lambda \end{aligned} \tag{14.5}$$

Thus, the direction of maximum variance is that encoded by the eigenvector associated with the largest eigenvalue of the sample variance matrix of the data. To compute this vector, we recall that the sample variance can be easily computed via the design matrix $\mathbf{X}$:

$$\Sigma = \frac{1}{N} \mathbf{X}^T \mathbf{X} \tag{14.6}$$

and its eigenvectors can be easily found via Singular Value Decomposition (SVD) of $\mathbf{X}$. In particular, by SVD we can write $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^T$, where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices and, in particular, $\mathbf{V}$ contains precisely the eigenvectors of $\mathbf{X}^T\mathbf{X}$. If we then take the eigenvector $\mathbf{v}_1$ of the matrix $\mathbf{V}$ corresponding to the largest eigenvalue, we can then project the entire dataset along the latter:

$$\begin{bmatrix} z_1^{(0)} \\ z_1^{(1)} \\ \vdots \\ z_1^{(N-1)} \end{bmatrix} = \begin{bmatrix} (\mathbf{x}^{(0)})^T \\ (\mathbf{x}^{(1)})^T \\ \vdots \\ (\mathbf{x}^{(N-1)})^T \end{bmatrix} \begin{bmatrix} v_{1,0} \\ v_{1,1} \\ \vdots \\ v_{1,D-1} \end{bmatrix} \tag{14.7}$$

Each $\mathbf{x}^{(i)} \in \mathbb{R}^D$ is then projected along the vector $\mathbf{v}_1$. Thus, the information contained in the D fetaures is condensed into the one component $z_j^{(i)}$.

Often, instead of projecting features along the direction of maximum variation, we may be interested in capturing not one but multiple directions of maximum variation.

To do this, it is sufficient to consider the k eigenvectors associated with the largest K eigenvalues. The features in the space resulting from the projection then become:

$$\underbrace{\begin{bmatrix} z_1^{(0)} & z_2^{(0)} & \dots & z_K^{(0)} \\ z_1^{(1)} & z_2^{(1)} & \dots & z_K^{(1)} \\ \vdots & & & \\ z_1^{(N-1)} & z_2^{(N-1)} & \dots & z_K^{(N-1)} \end{bmatrix}}_{\in \mathbb{R}^{N \times K}} = \underbrace{\begin{bmatrix} (\mathbf{x}^{(0)})^T \\ (\mathbf{x}^{(1)})^T \\ \vdots \\ (\mathbf{x}^{(N-1)})^T \end{bmatrix}}_{\in \mathbb{R}^{N \times D}} \underbrace{\begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \dots & \mathbf{v}_K \end{bmatrix}}_{\in \mathbb{R}^{D \times K}} \tag{14.8}$$

The steps of PCA are summarized in the algorithm 16.

Algorithm 16 PCA

1: Normalize the design matrix $\mathbf{X}$
2: Compute $\Sigma = \frac{1}{N}\mathbf{X}^T\mathbf{X}$
3: Compute the eigenvector of Σ by using the SVD
4: Take the k eigenvectors associated with the first k eigenvalues
5: Project the samples onto the k eigenvectors: $\mathbf{Z} = \mathbf{X}[\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_K]$

It should be noted, however, that PCA suffers from some limitations:

- In the new space $\mathbf{Z}$ we lost the interpretability of the features: we no longer have any way to figure out how each element of $\mathbf{Z}$ is related to the original features.
- The choice of the number of eigenvectors k to consider is not obvious and may have to be done with model selection;
- If the featues of the dataset are characterized by a high variance noise, since PCA tries to retain the original variance, it will keep the noise. PCA in fact is based on the assumption that the variance is what carries information, an assumption that is not valid in the case where the samples are very noisy.

14.2 Independent Component Analysis (ICA)

As with PCA, ICA seeks to find a new representation space for the data. However, the goal here is to identify the sources that generated the observed data and decouple them. To better understand this concept, it is very common to consider the "cocktail party problem": suppose we have D speakers and D microphones in a room. Clearly, each microphone records simultaneously the voice of all the people who are talking. The goal is then, starting from the acoustic signal acquired by the microphones, to separate the signals originated by the various speakers.

More formally, we denote by $x_j^{(i)}$ the sample acquired by the j-th microphone at time i and by $\mathbf{x}^{(i)}$ the D-dimensional vector of signals acquired by all microphones at time i. Similarly, we denote by $s_j^{(i)}$ the acoustic signal from the j-th speaker at time i and by $\mathbf{s}^{(i)}$ the D-dimensional vector of signals generated by all speakers at time i.

At a given time instant, we then observe a vector $\mathbf{x}^{(i)}$ obtained as a combination of the D sources:

$$\mathbf{x}^{(i)} = A\mathbf{s}^{(i)} \tag{14.9}$$

where $A \in \mathbb{R}^{D\times D}$ is called the **Mixing Matrix**. If we consider N observations collected in N time instants $\{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N-1)}\}$ we have:

$$\mathbf{X} = A\mathbf{S} \tag{14.10}$$

with

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(0)} & \mathbf{x}^{(1)} & \ldots & \mathbf{x}^{(N-1)} \end{bmatrix}, \mathbf{S} = \begin{bmatrix} \mathbf{s}^{(0)} & \mathbf{s}^{(1)} & \ldots & \mathbf{s}^{(N-1)} \end{bmatrix} \tag{14.11}$$

Our goal is to find a matrix $W = A^{-1}$ called the **Unmixing Matrix**, which allows us to separate the sources from the signals acquired by the microphones:

$$W = \begin{bmatrix} \mathbf{w}_0^T \\ \mathbf{w}_1^T \\ \vdots \\ \mathbf{w}_{D-1}^T \end{bmatrix} \tag{14.12}$$

with $\mathbf{w}_j \in \mathbb{R}^D$ which allows us to separate the j-th source $s_j^{(i)} = \mathbf{w}_j^T \mathbf{x}^{(i)}$

14.2.1 Ambiguities

Is the unmixing matrix W unique? If we have no information about the sources, there are some ambiguities that make more possible choices of W valid. In particular:

- Consider a permutation matrix $P \in \mathbb{R}^{D\times D}$, in which each row has only one 1 and all 0s. The purpose of the matrix is to perform a permutation of the rows of another matrix. For example:

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{14.13}$$

  If we have access only to $\mathbf{x}^{(i)}$ we have no way to distinguish W from PW. This is because a permutation of the original sources does not alter the signals captured by the microphones: if we indicate as source "1" a certain person and as source "2" another or vice versa, the signal captured by the microphones will be the same. However, this kind of ambiguity is not problematic in most cases, as we are often interested in isolating the sources, regardless of their order.

- Another ambiguity is related to scaling operations of W. For example, if instead of A we substitute αA (with α scalar) and instead of each $\mathbf{s}^{(i)}$ we substitute $\frac{1}{\alpha}\mathbf{s}^{(i)}$, the signal collected by the microphones is the same, in fact: $\mathbf{x}^{(i)} = \alpha A \frac{1}{\alpha}\mathbf{s}^{(i)} = A\mathbf{s}^{(i)}$. As with the previous ambiguity, however, this is often not a problem. In the cocktail party example, scaling the source is simply equivalent to altering its volume.

- The last ambiguity emerges in the case where the sources are **Gaussian**. In particular, suppose that $\mathbf{s}^{(i)} \sim N(0, \mathbf{I})$, where $\mathbf{I} \in \mathbb{R}^{D\times D}$ is the diagonal matrix of unit covariance. This means that the distribution of sources is, in the two-dimensional case for example, represented by circles centered in the origin. So if we observe signals $\mathbf{x}^{(i)} = A\mathbf{s}^{(i)}$, the distribution of $\mathbf{x}^{(i)}$ will itself be Gaussian: $\mathbf{x}^{(i)} \sim N(0, AA^T)$ since:

$$E_{\mathbf{s}^{(i)}\sim N(0,\mathbf{I})}[\mathbf{x}^{(i)}] = E[A\mathbf{s}^{(i)}] = AE[\mathbf{s}^{(i)}] = 0 \tag{14.14}$$

$$\begin{aligned} Cov[\mathbf{x}^{(i)}] &= E_{\mathbf{s}^{(i)}\sim N(0,\mathbf{I})}[\mathbf{x}^{(i)}(\mathbf{x}^{(i)})^T] = E[A\mathbf{s}^{(i)}(\mathbf{s}^{(i)})^T A^T] \\ &= AE[\mathbf{s}^{(i)}(\mathbf{s}^{(i)}]A^T = ACov[\mathbf{s}^{(i)}]A^T = AA^T \end{aligned} \tag{14.15}$$

  Consider now an orthogonal matrix (a rotation) R, such that $RR^T = I$ and let $A' = AR$. Consequently, if the sources were mixed with A' instead of A, we would observe $(\mathbf{x}^{(i)})' = A'\mathbf{s}^{(i)}$ and their distribution would always be a Gaussian with the same parameters as that obtained with A, $(\mathbf{x}^{(i)})' \sim N(0, AA^T)$, in fact:

$$\begin{aligned} Cov[(\mathbf{x}^{(i)})'] &= E_{\mathbf{s}^{(i)}\sim N(0,\mathbf{I})}[(\mathbf{x}^{(i)})'((\mathbf{x}^{(i)})')^T] = E[A'\mathbf{s}^{(i)}(\mathbf{s}^{(i)})^T A'^T] \\ &= E[AR\mathbf{s}^{(i)}(\mathbf{s}^{(i)})^T(AR)^T] = ARR^T A^T = AA^T \end{aligned} \tag{14.16}$$

  In the case of Gaussian sources, we are therefore unable to distinguish whether they were mixed with A or A'. There is thus an arbitrary rotation component that prevents us from recovering the sources unambiguously. Consequently, when using ICA we make the assumption of **non-Gaussian sources**.

To derive the solution of ICA, we first recall how the pdf of a random variable obtained as the result of applying a deterministic function to another random variable is expressed. For a certain time instant, we have that the vector of signals recorded by the microphones is given by $\mathbf{x} = g(\mathbf{s})$.The pdf of $\mathbf{x}$ is given by:

$$p_X(\mathbf{x}) = p_S(g^{-1}(\mathbf{x}))\left|\frac{\partial g^{-1}(\mathbf{x})}{\partial \mathbf{x}}\right| \tag{14.17}$$

In our case, $\mathbf{x} = A\mathbf{s}$, hence $g^{-1}(\mathbf{x}) = W\mathbf{x}$. Thus,

$$p_X(\mathbf{x}) = p_S(W\mathbf{x})\,|W| \tag{14.18}$$

At this point, we start by assuming that the sources are independent. Therefore, the

overall pdf (considering the sample at a certain time instant) is given by:

$$p_S(\mathbf{s}) = \prod_{j=0}^{D-1} p_S(s_j) \tag{14.19}$$

and the distribution of $\mathbf{x}$ is:

$$p_X(\mathbf{x}) = \left(\prod_{j=0}^{D-1} p_S(\mathbf{w}_j^T \mathbf{x}) \right) |W| \tag{14.20}$$

We now need to choose a reasonable function for $p_S(s_j)$. We know that it cannot be Gaussian to avoid ambiguities. So let's specify it starting with the cdf and choose it defined by a sigmoid: $F(\mathbf{s}) = \sigma(\mathbf{s})$. This function is reasonable as a CDF, since it grows from zero and tends to 1. The pdf is therefore simply the derivative of the cdf:

$$p_S(s_j) = F'(s_j) = \sigma'(s_j) = \sigma(s_j)(1 - \sigma(s_j)) \tag{14.21}$$

Let's continue by assuming we have collected samples for N time instants. We then apply MLE and compute the log-likelihood:

$$l(W) = \sum_{i=0}^{N-1} \left(\left[\sum_{j=0}^{D-1} \log \sigma'(\mathbf{w}_j^T \mathbf{x}^{(i)}) \right] + \log |W| \right) \tag{14.22}$$

and the gradient with respect to W is given by:

$$\nabla_W l(W) = \sum_{i=0}^{N-1} \left(\sum_{j=0}^{D-1} [\nabla_W \log \sigma'(\mathbf{w}_j^T \mathbf{x}^{(i)}) \right) + \nabla_W \log |W| \tag{14.23}$$

We do the calculation with respect to the j-th vector $\mathbf{w}_j$ starting with the first of the two terms in the summation:

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}_j^T} \log \sigma'(\mathbf{w}_j^T \mathbf{x}^{(i)}) &= \frac{1}{\sigma'(\mathbf{w}_j^T \mathbf{x}^{(i)})} \sigma''(\mathbf{w}_j^T \mathbf{x}^{(i)}) x_j^{(i)} \\ &= \frac{1}{\sigma'(\mathbf{w}_j^T \mathbf{x}^{(i)})} \sigma'(\mathbf{w}_j^T \mathbf{x}^{(i)})(1 - 2\sigma(\mathbf{w}_j^T \mathbf{x}^{(i)})) x_j^{(i)} \\ &= (1 - 2\sigma(\mathbf{w}_j^T \mathbf{x}^{(i)})) x_j^{(i)} \end{aligned} \tag{14.24}$$

The second derivative of the sigmoid is equal to:

$$\begin{aligned} \sigma''(s) &= \frac{\partial}{\partial s} \sigma(s)(1 - \sigma(s)) = \sigma(s)(1 - \sigma(s))^2 - \sigma^2(s)(1 - \sigma(s)) \\ &= \sigma(s)(1 - \sigma(s))(1 - 2\sigma(s)) = \sigma'(s)(1 - 2\sigma(s)) \end{aligned} \tag{14.25}$$

For the second term we have instead:

$$\nabla_W \log |W| = \frac{1}{|W|}|W|(W^{-1})^T = (W^{-1})^T \tag{14.26}$$

Combining these two terms and summing with respect to $j = 0, \ldots, D-1$ we can write:

$$\nabla_W l(W) = \sum_{i=0}^{N-1} \left(\begin{bmatrix} (1 - 2\sigma(\mathbf{w}_0^T \mathbf{x}^{(i)})) \\ (1 - 2\sigma(\mathbf{w}_1^T \mathbf{x}^{(i)})) \\ \vdots (1 - 2\sigma(\mathbf{w}_{D-1}^T \mathbf{x}^{(i)})) \end{bmatrix} (\mathbf{x}^{(i)})^T + (W^{-1})^T \right) \tag{14.27}$$

To find the optimal W matrix, it is then sufficient to use an iterative approach with the gradient just computed:

$$W = W + \alpha \nabla_W l(W) \tag{14.28}$$